C++ Örnekleriyle Nesne Yönelimli Programlama

Esnek Tasarım, Temiz Kod ve Mimari Standartlar

Elektronik Yük. Müh. İlhan ÖZKAN

Kapsamlı Uygulama Rehberi ve Ders Kitabı

2026 Baskısı

C++ Örnekleriyle Nesne Yönelimli Programlama

Yazar: Elektronik Yük. Müh. İlhan ÖZKAN

© Eylül 2026, İlhan ÖZKAN. Tüm hakları saklıdır.

Bu kitap kâr amacı güdülmeden hazırlanmıştır ve **referans gösterilerek** her yerde kullanılabilir.

Bu kitabın bir kısmı veya tamamı, **yazarın önceden yazılı izni alınarak** elektronik, mekanik, fotokopi veya herhangi bir kayıt sistemi ile çoğaltılabilir, yayımlanabilir veya depolanabilir.

ISBN: ***-***-*****-*-*

Baskı Tarihi: Aralık 2026
Baskı Yeri: Ankara, Türkiye

Önsöz

Yazılım geliştirme, yalnızca çalışan kod yazmaktan ibaret değildir. Gerçek anlamda nitelikli yazılım; anlaşılabilir, değiştirilebilir, sürdürülebilir ve genişletilebilir yapılar kurabilme becerisidir. Bugünün yazılım dünyasında bir programcının yalnızca programlama dili söz dizimini bilmesi yeterli değildir; algoritmik düşünmeyi, veri ile davranış arasındaki ilişkiyi, nesne yönelimli yaklaşımı, şablonları, hata yönetimini, tasarım kalıplarını ve mimari standartları da kavraması gerekir.

Bu kitap, yazılım geliştirme yolculuğunda sağlam bir temel oluşturmak isteyen öğrenciler, yeni başlayan geliştiriciler ve bilgilerini sistematik biçimde derinleştirmek isteyen uygulayıcılar için hazırlanmıştır. Eserin temel amacı; okuyucuyu düşük düzey programlama anlayışından başlayarak yapısal programlama, nesne yönelimli programlama, UML modelleme, SOLID ilkeleri, jenerik programlama, tasarım kalıpları ve temiz mimari yaklaşımına kadar adım adım ilerleyen bütüncül bir düşünce yapısına ulaştırmaktır.

Kitap boyunca yalnızca "nasıl kod yazılır?" sorusuna değil, aynı zamanda "neden böyle tasarlanmalıdır?" sorusuna da cevap aranmıştır. Çünkü yazılımda kalıcı başarı, ezberlenmiş kod parçalarından çok, doğru soyutlama yapabilme, sorumlulukları ayırabilme, değişime açık ama dağınıklıktan uzak mimariler kurabilme becerisine dayanır.Bu eser altı ana eksen üzerine kurulmuştur:

- ◇ İlk Bölümde, programlama dillerinin tarihsel gelişimi, değişkenler, fonksiyonlar, yapısal programlama paradigması ve yazılım geliştirme süreci ele alınmaktadır.
- ◇ İkinci Bölümde, sınıf ve nesne kavramlarından başlayarak kapsülleme, soyutlama, kalıtım, çok biçimlilik, arayüzler, yapıcılar, yıkıcılar, işleç çok yükleme ve C++ örnekleriyle nesne odaklı zihniyet açıklanmaktadır.
- ◇ Üçüncü Bölümde, hata yönetimi ve istisnalar (exceptions), yığın çözülmesi (stack unwinding), UML sınıf diyagramları, sınıflar arası ilişkiler ve nesne tasarlamada SOLID ilkeleri uygulamalı örneklerle incelenmektedir.
- ◇ Dördüncü Bölümde, jenerik programlama, fonksiyon ve sınıf şablonları, katlama ifadeleri (fold expressions), akıllı göstericiler (RAII prensibi) ve Standart Şablon Kütüphanesi (STL) konteynerleri ile algoritmaları detaylandırılmaktadır.
- ◇ Beşinci Bölümde, nesne imalatı, yapısal ve davranışsal tasarım kalıpları (Design Patterns) ile MVC mimari kalıbı incelenmektedir.
- ◇ Son Bölümde ise, temiz kod (Clean Code) prensipleri, yeniden düzenleme (refactoring), katmanlı mimari, bağımlılık enjeksiyonu, test edilebilirlik, güvenli kodlama standartları ve temiz mimari (Clean Architecture) ilkelerine yer verilmektedir.

Kitabın ayırt edici yönlerinden biri, kavramları yalnızca teorik tanımlarla sınırlı bırakmamasıdır. Her konu, mümkün olduğunca sade bir dille, günlük hayattan benzetmelerle ve C++ örnekleriyle desteklenmiştir. Böylece okuyucunun yalnızca kodu görmesi değil, kodun arkasındaki tasarım kararını da anlaması hedeflenmiştir.

Nesne yönelimli programlama ve modern yazılım mimarileri ilk bakışta zorlayıcı görünebilir. Ancak bu kavramlar doğru sırayla ve örneklerle ele alındığında, yazılım geliştirmenin yalnızca teknik bir uğraş değil, aynı zamanda disiplinli bir düşünme biçimi olduğu daha açık görülür.Bu kitabın hazırlanmasındaki temel motivasyon, yazılım geliştirmeye ilgi duyanlara sağlam, düzenli ve uygulanabilir bir başvuru kaynağı sunmaktır.

Bilginin paylaşıldıkça çoğaldığına inanarak hazırlanan bu çalışmanın; öğrenciler, eğitmenler ve yazılım geliştiriciler için faydalı olmasını dilerim.Kitabın güncel sürümüne ve kaynak kodlarına GitHub deposu üzerinden erişilebilir. Katkı, düzeltme ve geri bildirimleriniz kitabın gelecekteki sürümleri için son derece değerlidir.

Başarılar ve iyi çalışmalar dilerim.

Elektronik Yüksek Mühendisi, İlhan ÖZKAN
ilhanozkan[at]outlook.com
linkedin.com/in/ilhanozkan

İçindekiler

Önsöz **i**

1 Temel Yazılım Kavramları ve OOP'ye Giriş **1**
1.1 Düşük Düzey Programlama . . . 1
1.2 Genel Amaçlı Yüksek Düzey Diller . . . 2
1.3 Arapsaçı Kod . . . 3
1.4 Değişken . . . 4
1.5 Fonksiyon . . . 6
1.6 Yapısal Programlama . . . 7
1.7 Yazılım Geliştirme Süreci . . . 10
1.8 Yapısal Programlama Paradigması . . . 10
1.9 Nesne Yönelimli Programlama Paradigması . . . 11

2 Nesne Odaklı Zihniyet **13**
2.1 Sınıf ve Nesne Kavramı . . . 13
2.2 Nesnenin Durumu, Özellikleri ve Davranışları . . . 14
2.3 Sınıf ve Nesnelerin UML Sınıf Diyagramları ile Gösterilmesi . . . 14
2.4 Bilgi ve Veri Gizleme Nedir? . . . 15
2.5 Soyutlama ve Kapsülleme . . . 16
2.6 Nesnelerin İmal Edilmesi . . . 17
2.7 this Göstericisi . . . 19
2.8 Yapıcı . . . 19
2.8.1 Parametreli Yapıcı . . . 20
2.8.2 Kopyalama Yapıcısı . . . 20
2.9 Nesnelerin Yok Edilmesi ve Yıkıcı . . . 21
2.10 Fonksiyon Çok Yükleme . . . 22
2.11 İşleç Çok Yükleme . . . 23
2.11.1 Toplama İşlecinin Çok Yüklenmesi . . . 23
2.11.2 Atama İşlecinin Çok Yüklenmesi . . . 23
2.11.3 Tekli ve İkili İşleçlerin Çok Yüklenmesi . . . 25
2.12 Kalıtım Kavramı . . . 26
2.12.1 Ata ve Çocuk Sınıflar . . . 26
2.12.2 UML Sınıf Diyagramlarında Kalıtımın Gösterilmesi . . . 30
2.12.3 Kalıtımda Yapıcı ve Yıkıcılar . . . 30
2.12.4 Çocuk Sınıftan Nesne Üretilmesi . . . 32
2.12.5 Kalıtımda Erişim Belirleyiciler . . . 32
2.12.6 Çoklu Kalıtım . . . 34
2.12.7 Elmas Problemi . . . 35
2.12.8 Sanal Kalıtım . . . 37
2.13 Arkadaş Fonksiyon ve Sınıflar . . . 38
2.14 Soyut ve Somut Sınıf . . . 40
2.15 Statik Sınıf Üyeleri . . . 41
2.16 const Sınıf Üyeleri . . . 42
2.17 mutable Depolama Sınıfı . . . 42
2.18 Geçersiz Kılma . . . 43
2.19 Arayüzler . . . 44
2.20 Çok Biçimlilik . . . 45

2.21 Üçlü, Beşli ve Sıfır Kuralı . . . 47
2.21.1 Üçlü Kural (Rule of Three) . . . 47
2.21.2 Beşli Kural (Rule of Five) . . . 48
2.21.3 Sıfır Kuralı (Rule of Zero) . . . 48

3 İstisnalar, Esnek Tasarım ve SOLID İlkeleri 49
3.1 Hatalar ve İstisnalar . . . 49
3.1.1 Yapısal Programlamada Hata Yönetimi . . . 49
3.1.2 Nesne Yönelimli Programlamada İstisna Yönetimi . . . 50
3.1.3 Standart İstisna Tipleri . . . 51
3.1.4 İstisna Nesnesi ve Hata Mesajı: what() . . . 54
3.1.5 noexcept Anahtar Kelimesi . . . 55
3.1.6 Kullanıcı Tanımlı İstisna Tipleri . . . 56
3.1.7 İstisnayı Yeniden Fırlatma ve Zincirleme . . . 58
3.1.8 Yığın Çözülmesi ve Kaynak Yönetimi . . . 58
3.1.9 İstisna Kullanımında İyi Uygulamalar . . . 60
3.2 Unified Modeling Language . . . 60
3.2.1 UML Nedir . . . 60
3.2.2 Sınıf Diyagramları . . . 61
3.3 Sınıflar Arası İlişkiler . . . 62
3.3.1 Ortaklık (Association) . . . 63
3.3.2 Biriktirme (Aggregation) . . . 64
3.3.3 Bileşim (Composition) . . . 66
3.3.4 Bağımlılık (Dependency) . . . 67
3.3.5 Genelleştirme (Generalization) . . . 68
3.3.6 Gerçekleştirme (Realization) . . . 69
3.3.7 İlişkilerin Özeti . . . 70
3.4 Sınıf Tasarlama (SOLID) İlkeleri . . . 70
3.4.1 Tek Sorumluluk İlkesi (SRP) . . . 71
3.4.2 Açık-Kapalı İlkesi (OCP) . . . 72
3.4.3 Liskov İkame İlkesi (LSP) . . . 74
3.4.4 Arayüzleri Ayırma İlkesi (ISP) . . . 76
3.4.5 Bağımlılıkları Tersine Çevirme İlkesi (DIP) . . . 78
3.4.6 İlkelerin Özeti . . . 80

4 Şablonlar 81
4.1 Şablon Nedir? . . . 81
4.2 Foksiyon Şablonları . . . 81
4.3 Değişken Sayıda Argüman Alabilen Fonksiyon Şablonları . . . 82
4.4 Katlama İfadeleri . . . 83
4.5 Sınıf Şablonları . . . 85
4.6 Faydalı Şablonlar . . . 87
4.6.1 std::pair . . . 87
4.6.2 std::move . . . 88
4.6.3 std::swap . . . 88
4.6.4 std::forward . . . 88
4.6.5 Değişmez Değerler ve Diğerleri . . . 88
4.7 Akıllı Göstericiler . . . 89
4.7.1 Kaynak Edinimi İlk Değer Atamasıdır! Prensibi . . . 89
4.7.2 std::unique_ptr ile Tekil Sahiplik . . . 90
4.7.3 std::shared_ptr ile Paylaşılan Sahiplik . . . 90
4.7.4 std::weak_ptr ile Zayıf Takipçi . . . 91
4.7.5 Akıllı Gösterici Örnekleri . . . 92
4.8 Standart Şablon Kütüphanesi . . . 94

4.9 Konteyner Şablonları . . . 94
4.9.1 std::array . . . 95
4.9.2 std::vector . . . 98
4.9.3 std::map, std::multimap ve std::pair . . . 99
4.9.4 std::list ve std::forwardlist . . . 103
4.9.5 std::set ve std::multiset . . . 103
4.9.6 std::optional . . . 104
4.9.7 std::function ve std::bind . . . 105
4.9.8 std::tuple . . . 107
4.9.9 std::string . . . 107
4.10 Algoritma Şablonları . . . 109
4.11 Yineleme Şablonları . . . 111
4.12 Fonksiyon Nesneleri . . . 112

5 Tasarım Kalıpları 114
5.1 Kalıp Nedir? . . . 114
5.2 Nesne İmalatı ile İlgili Kalıplar . . . 115
5.2.1 Soyut Fabrika . . . 115
5.2.2 Fabrika Yöntemi . . . 117
5.2.3 Tekil Nesne . . . 119
5.2.4 Kurucu . . . 120
5.2.5 Prototip . . . 122
5.3 Yapısal Kalıplar . . . 124
5.3.1 Önyüz . . . 124
5.3.2 Adaptör . . . 127
5.3.3 Bileşik . . . 129
5.3.4 Süsleyici . . . 132
5.3.5 Sinek Sıklet . . . 135
5.3.6 Vekil . . . 137
5.3.7 Köprü . . . 139
5.4 Davranışla İlgili Kalıplar . . . 141
5.4.1 Sorumluluk Zinciri . . . 141
5.4.2 Komut . . . 143
5.4.3 Yineleyici . . . 146
5.4.4 Gözlemci . . . 148
5.4.5 Strateji . . . 151
5.4.6 Şablon Yöntem . . . 154
5.4.7 Ziyaretçi . . . 155
5.4.8 Hatıra . . . 158
5.4.9 Ara Bulucu . . . 161
5.4.10 Durum . . . 164
5.4.11 Yorumlayıcı . . . 169
5.5 MVC Mimari Kalıbı . . . 172

6 Temiz Kod ve Mimari Standartlar 175
6.1 Temiz Kod Nedir? . . . 175
6.1.1 Kodun Okunabilir Olması . . . 175
6.1.2 Kodun Niyetini Açık Etmesi . . . 176
6.1.3 Yorum Satırları Ne Zaman Kullanılmalı? . . . 176
6.2 Anlamlı İsimlendirme . . . 177
6.2.1 Değişken İsimlendirme . . . 177
6.2.2 Fonksiyon ve Yöntem İsimlendirme . . . 177
6.2.3 Sınıf İsimlendirme . . . 178
6.2.4 Sabit, Enum ve Arayüz İsimlendirme . . . 178

6.3 Küçük Fonksiyonlar ve Yöntemler . . . 178
6.3.1 Fonksiyonların Tek İş Yapması . . . 179
6.3.2 Parametre Sayısını Azaltma . . . 179
6.3.3 Yan Etkilerden Kaçınma . . . 180
6.3.4 Erken Çıkış ve Guard Clause Kullanımı . . . 180
6.4 Küçük Sınıflar ve Sorumluluk Ayrımı . . . 181
6.4.1 Sınıfın Boyutu Ne ile Ölçülür? . . . 181
6.4.2 Tanrı Sınıf Problemi . . . 181
6.4.3 Sınıfların Birlikte Değişen Davranışlara Göre Ayrılması . . . 182
6.4.4 Bağlaşım ve Uyum . . . 182
6.5 Kod Tekrarı ve Yeniden Kullanılabilirlik . . . 182
6.5.1 DRY İlkesi . . . 182
6.5.2 Gereksiz Soyutlama Tehlikesi . . . 183
6.5.3 Kalıtım Yerine Bileşim . . . 183
6.6 Hata Yönetimi ve İstisna Mimarisi . . . 183
6.6.1 Hata Kodları Yerine İstisnalar . . . 183
6.6.2 Anlamlı İstisna Sınıfları . . . 184
6.6.3 İstisnayı Yutmamak . . . 184
6.6.4 Kaynak Yönetimi . . . 185
6.6.5 Katmanlara Göre Hata Yönetimi . . . 185
6.7 Boş Değerler, Optional ve Geçersiz Durum Yönetimi . . . 185
6.7.1 Null Referans Problemi . . . 185
6.7.2 Optional Kullanımı . . . 186
6.7.3 Null Object Pattern . . . 186
6.7.4 Geçersiz Nesne Oluşturmamak . . . 186
6.8 Kapsülleme ve Değişmez Nesneler . . . 187
6.8.1 Alanları Doğrudan Açmamak . . . 187
6.8.2 Değişmez Nesneler . . . 189
6.8.3 Nesne Durumunun Tutarlılığı . . . 190
6.9 Katmanlı Mimari Temelleri . . . 191
6.9.1 Katmanlı Mimari Nedir? . . . 191
6.9.2 Katmanlar Arası Bağımlılık . . . 193
6.9.3 DTO, Entity ve Model Ayrımı . . . 193
6.9.4 Repository ve Service Kavramları . . . 194
6.10 Bağımlılık Yönetimi ve Bağımlılık Enjeksiyonu . . . 195
6.10.1 Bağımlılık Yönü . . . 195
6.10.2 Bağımlılık Enjeksiyonu (Dependency Injection) . . . 195
6.10.3 Denetimin Tersine Çevrilmesi (IoC) Kavramı . . . 196
6.11 Test Edilebilir Kod Yazmak . . . 197
6.11.1 Test Edilebilirlik Nedir? . . . 197
6.11.2 Birim Test Mantığı . . . 197
6.11.3 Mock ve Fake Nesneler . . . 198
6.11.4 Test Edilemeyen Kodun Belirtileri . . . 198
6.12 Kod Biçimlendirme Standartları . . . 199
6.12.1 Girintileme ve Boşluklar . . . 199
6.12.2 Dosya ve Klasör Düzeni . . . 199
6.12.3 Kod Stil Rehberi . . . 200
6.12.4 C++ ve Java İçin Stil Farkları . . . 200
6.13 Yeniden Düzenleme (Refactoring) ve Kod Kokuları . . . 200
6.13.1 Refactoring Nedir? . . . 200
6.13.2 Yaygın Refactoring Teknikleri . . . 200
6.13.3 Kod Kokuları . . . 201
6.13.4 Refactoring Öncesi Testin Önemi . . . 201

6.14 Performans ve Temiz Kod Dengesi . . . 201
6.14.1 Erken Optimizasyon . . . 202
6.14.2 Algoritmik Verimlilik . . . 203
6.14.3 Bellek ve Kaynak Yönetimi . . . 205
6.15 Güvenli Kodlama Standartları . . . 207
6.15.1 Girdi Doğrulama . . . 207
6.15.2 Taşma ve Sınır Hataları . . . 208
6.15.3 Hassas Verilerin Korunması . . . 209
6.15.4 Hata Mesajlarında Bilgi Sızdırmamak . . . 210
6.16 Dokümantasyon ve API Tasarımı . . . 211
6.16.1 Kod İçi Dokümantasyon . . . 211
6.16.2 Dış Dokümantasyon . . . 212
6.16.3 İyi API Tasarımı . . . 212
6.17 Sürüm Kontrolü ve Kod Kalitesi Süreci . . . 213
6.17.1 Git Kullanım Disiplini . . . 213
6.17.2 Kod İnceleme . . . 213
6.17.3 Sürekli Entegrasyon . . . 214
6.18 Temiz Mimari İlkelerine Giriş . . . 214
6.18.1 Mimari Neden Önemlidir? . . . 215
6.18.2 İş Kurallarını Merkeze Almak . . . 216
6.18.3 Dış Dünyayı Soyutlamak . . . 217
6.18.4 Temiz Mimari Katmanları . . . 219
6.19 Bölüm Sonu Uygulaması: Kütüphane Ödünç Verme Sistemi . . . 223

1. Bölüm
Temel Yazılım Kavramları ve OOP'ye Giriş

1.1 Düşük Düzey Programlama

1642 yılında *Blaise Pascal* tarafından icat edilen Pascaline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (*punch card*) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 yılında *Charles Babbage* fark makinesi ve analitik makineyi icat etmiştir. Analitik makine, buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Lovelace*, Babbage ile beraber çalışmıştır. Ada Lovelace, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu algoritmanın o dönemdeki "Analitik Makine" için devrimsel bir gelişmedir. Çünkü şartlı dallanma ve döngü mantığı içerir. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

İlk Programcı

Augusta Ada Lovelace, tarihteki **ilk programcı** olarak kabul edilir.

1854 yılında *George Boole*, ikili sayı sistemine temel olan ve günümüz bilgisayarlarının kullandığı bool cebiri üzerine çalışmalar yapmıştır.

1890 yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabildiği bir elektromekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 yılında *Konrad Zuze* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 yılında *Howard Aiken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 15,5 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1946 yılında Pensilvanya Üniversitesi Moore Elektrik Mühendisliği Okulu'nda *J. Presper Eckert* ve *John Mauchly* önderliğinde **ENIAC** (*Electronic Numerical Integrator and Computer*, Elektronik Sayısal Entegratör ve Hesaplayıcı) isimli sayısal elektronik bilgisayar kamuoyuna tanıtıldı (yapımı 1945'te tamamlanmıştı). Askeri amaçla üretildi ve top mermilerinin menzil tablolarını hesaplamak için kullanıldı. Yaklaşık 18000 elektronik tüp kullanan ENIAC; 150 kW gücünde idi ve yaklaşık 30 ton

ağırlığıyla 167 m^2 yer kaplıyordu. Saniyede 5 bin toplama işlemi yapabiliyordu ve MARK-I bilgisayarından yaklaşık bin kat daha hızlıydı. ENIAC'ın programı kablo ve anahtarlarla değiştiriliyordu; yani program bellekte saklanmıyordu.

1945 yılında *John von Neumann*, **EDVAC** (*Electronic Discrete Variable Automatic Computer*) için yazdığı raporda, programın da veri gibi bellekte saklandığı mimariyi tanımladı. **Von Neumann mimarisi** (*stored-program computer*) olarak bilinen bu mimari, günümüzdeki bilgisayarlarda hala kullanılmaktadır. EDVAC ise 1949'da teslim edildi ve 1951 yılında tam olarak çalışır hale geldi.

Yukarıda anlatılan bilgisayarlar, **emir kodu** (*instruction code*) veya emir kodunu işaret eden **hatırlatıcı isimlerle** (*mnemonic*) programlar yazılmaktadır. Emir kodu ve hatırlatıcı isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama** (*low-level programming*) denir. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan mühendisler bu programlama türünü kullanır.

Düşük Düzey Programlama Dili

Düşük düzey programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

1.2 Genel Amaçlı Yüksek Düzey Diller

Hatırlatıcı isimler veya emir kodu yerine bildiğimiz konuşma diline yakın **talimatlarla** (*statement*) metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (*high-level programming language*) olarak adlandırılır.

Talimat

Yüksek seviyeli bir dilde talimatlar (*statement*), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (*logical sequence*).

1954 yılında **FORTRAN** (*FORmula TRANslation*), *John Backus* liderliğinde IBM 704 için geliştirilirdi. FORTRAN, `READ`, `WRITE`, üç yollu `IF` gibi 32 farklı talimata sahiptir. İlk yaygın kullanılan, yüksek düzey bir programlama dilidir.

Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir [1]. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERONS FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
READ(5,501) IA,IB,IC
501 FORMAT(3I5)
IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H  B= ,I5,5H  C= ,I5,8H  AREA= ,F10.2,
$13H SQUARE UNITS)
STOP
END
```

[1] https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

Yüksek Düzey Programlama Dili

Emir kodlarını (*instruction code*) ve sembolik isimleri (*mnemonic*) içermeyen programlama dilleri, **yüksek düzey programlama dili** (*high-level programming language*) olarak adlandırılır. Yüksek düzey dillerde **talimatlarla** (*statement*) yazılan programlar yine kendine has derleyiciler tarafından makine diline yani emir kodlarına çevrilir. Her talimat, genellikle birden fazla emirden (*instruction*) oluşur.

1.3 Arapsaçı Kod

1955-1959 yılları arasında **FLOW-MATIC B0** (*Business Language Version 0*) Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir [2].

```
(0)  INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1)  COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2)  TRANSFER A TO D .
(3)  WRITE-ITEM D .
(4)  JUMP TO OPERATION 8 .
(5)  TRANSFER A TO C .
(6)  MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7)  WRITE-ITEM C .
(8)  READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9)  JUMP TO OPERATION 1 .
(10)  READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11)  JUMP TO OPERATION 1 .
(12)  SET OPERATION 9 TO GO TO OPERATION 2 .
(13)  JUMP TO OPERATION 2 .
(14)  TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
OTHERWISE GO TO OPERATION 15 .
(15)  REWIND B .
(16)  CLOSE-OUT FILES C ; D .
(17)  STOP . (END)
```

Görüldüğü üzere bu zamanlarda yazılan programlarda bir sürü `GO TO` ya da `JUMP TO` talimatı (*statement*) bulunmaktadır. Bu tür kodlarda hangi talimatın hangi sırada icra edileceğini yani kontrol akışını (*control flow*) tespit etmek oldukça zordur. Bu da programın **okunaklığını oldukça azaltır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (*spaghetti code*) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

Tüm Kaynak Kodlar Gözle Okunur

İşin doğası gereği **makine diline çevrilecek tüm kaynak kodlar gözle okunmalıdır!**

1958 yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş yüksek düzey programlama dilidir;

```
(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)
```

1959 yılında **COBOL** (*Common Business-Oriented Language*) programlama dili, CODASYL komitesi tarafından İngilizceye benzer yüksek düzey bir dil olarak geliştirilmiştir;

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.

DATA DIVISION.
```

[2] https://gist.github.com/marcgg/f9f2da31a973ba319809

```
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).

PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.
```

1958 yılında daha sonra **ALGOL** (*ALGOritmic Language*) adını alan **IAL** (*International Algebraic Language*) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok yapısal dile önderlik etmiştir. Bu dil ile birlikte **kod blokları** (*code block*), iç içe fonksiyon (*nested function*), değişken kapsamı (*scope*) ve dizi (*array*) kavramları programcının hayatına girmiştir. 1958–1969 arası `PROCEDURE` talimatı bu dile eklendi. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```
BEGIN
  FILE F (KIND=REMOTE);
  EBCDIC ARRAY E [0:11];
  REPLACE E BY "HELLO WORLD!";
  WRITE (F, *, E);
END.
```

1958 yılındaki FORTRAN diline çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, değer üreten fonksiyonlar için `FUNCTION`, `CALL` ve `RETURN` özellikleri eklenmiş ve **FORTRAN-II** adını almıştır. Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için `PROCEDURE` ve `BEGIN-END` arasında kod bloğu özellikleri eklenmiştir.

1966 yılındaki FORTRAN-IV diline `DOUBLE PRECISION`, `COMPLEX` ve `LOGICAL` **veri tipleri** (*data type*) ile Mantıksal `IF` talimatları (*statement*) eklenmiştir.

1964 yılında Dartmouth College'da *John G. Kemeny* ve *Thomas E. Kurtz* tarafından geliştirilen **BASIC** (*Beginner's All-purpose Symbolic Instruction Code*) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı yorumlanarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980'li yılların başında kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

1.4 Değişken

Değişkenler (*variable*) aslında cebirde kullandığımız değişkenlerdir. Cebir (*algebra*) işlemlerinde doğrudan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$x \in \mathbb{R},\ 3x^2 + 2x + 10 \tag{1.1}$$

$$c^2 = a^2 + b^2 \tag{1.2}$$

$$c = 2\pi r \tag{1.3}$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde x, a, b ve r **bağımsız değişken**, c ise **bağımlı değiş-**

kendir. Örneklerdeki 3,2,10,2 ve π ise **sabit** (*constant*) sayılardır.

Bilgisayarda Değişkenler

Değişkenler;
- ◇ Bellekte belirli bir yer kaplayan ve içinde veri saklayan yapılardır.
- ◇ Saklanan verinin bellekte kapladığı alan ve temsil edilme biçimi değişkenin **veri tipine** (*data type*) bağlıdır. Örneğin tam sayı ve kayan noktalı sayı farklı veri tipleridir.

Tam sayılar veri tiplerinden en çok kullanılanıdır ve bellekte **ikili** (*binary*) tabanda tutulur. Tam sayıyı ifade eden ikili sayının en anlamlı biti, işaret (*sign*) biti olarak adlandırılır ve bu bitin değeri `1` ise sayı negatif, `0` ise sayı pozitiftir. Tablo 1.1'de en çok kullanılan tam sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	1 bayt	-128 ile +127 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.768 ile +32.767 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 ile +2.147.483.647 arasında veya 0 ile 4.294.967.295 arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807 arasında veya 0 ile 18.446.744.073.709.551.615 arasında

Tablo 1.1: Tam Sayılar ve Sayı Sınırları

İşaretli ve İşaretsiz Tam Sayılar

n bitlik bir tamsayı işaretli (*signed*) olarak kullanılacaksa değer aralığı -2^{n-1} ile $2^{n-1}-1$ arasında olacaktır. Aynı bit sayısına sahip sayı, işaretsiz (*unsigned*) olarak kullanılacak ise 0 ile 2^n-1 aralığında değer alabilecektir.

Gerçek sayılar da en çok kullanılan veri tiplerindendir. Bu sayılar, **kayan noktalı sayı** (*floating point number*) olarak biçimlendirilir. Kayan noktalı sayı kavramı ilk olarak 1914 yılında Leonardo *Torres Quevedo* tarafından Charles Babage'ın analitik makinesine kullanılmak üzere yayınlanmıştır.

Türkçemizde gerçek sayılarda ondalık ayracı olarak virgül kullanılır (Örnek: 50,25). Aslında Türkçe olarak kayan virgüllü sayı dememiz gerekirdi. Ancak **kodu İngilizce yazacağımızdan ondalık ayracı olarak nokta kullanılmalıdır** (Örnek: 50.25). Bu nedenle bundan sonraki örneklerde ondalık ayracı olarak nokta kullanılacaktır.

$$12345 = 1234.5 \times 10^1 = 123.45 \times 10^2 = 12.345 \times 10^3 = 1.2345 \times 10^4 = 0.12345 \times 10^5 \tag{1.4}$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında sayı, 10'a bölünmüş, sağa kaydırıldığında ise sayı 10 ile çarpılmış olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı **kesir** (*fraction*) olarak adlandırılır.

Kayan Noktalı Sayılar Da İkili Tabanda Tutulur

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (*binary*) sayı tabanında tutulur**. Örneğin ondalık tabanda 50.25 sayısı ikili tabanda 1.1001001×2^5 olarak ifade edilir.

1938 yılında *Konrad Zuse*, ilk ikili (*binary*) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır. Günümüzde standartlaşmış ve çok kullanılan kayan noktalı sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir[3] (Tablo 1.2).

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 bayt	1.2×10^{-38} ile $3.4 \times 10^{+38}$ arasında **tek hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	2.2×10^{-308} ila $1.8 \times 10^{+308}$ arasında **çift hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 1.2: Kayan Noktalı Sayılar ve Sınırları

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi** (*data type*) belirler. Değişkenlerin veri tipine kısaca **değişken tipi** (*variable type*) adı da verilir. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya `bayt` denilirken, C, C++ ve C# dilinde `char` adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

1.5 Fonksiyon

Matematikte **fonksiyon** (*function*), A kümesinin her elemanını B kümesinin tam bir elemanına eşleyen **ilişkidir** (*relationship*).

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...	...	...

Tablo 1.3: İki İle Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade** (*expression*) edilir.

$$f(x) = 2x \tag{1.5}$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x \tag{1.6}$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in \mathbb{R}, f(x) = 2x + 3 \tag{1.7}$$

[3] IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1–84.

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür** (*return*).

$$a \in \mathbb{R}, \quad b \in \mathbb{Z}, \quad g(a,b) = 2a + b \tag{1.8}$$

Bu fonksiyon ise reel sayı kümesinden her a elemanı ve tam sayılar kümesinden her b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1.0 ve b=1.0 argümanlarını alarak yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (*return*).

Örneklerdeki x, a ve b parametre olarak adlandırılır. f(1.0) ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre** (*actual parameter*) ya da **argüman** (*argument*) olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0) \tag{1.9}$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler** (*call*).

1.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, birden fazla çağrı noktası olabilen `COROUTINE` ve `FUNCTION` kullanılıyor olsa da hala `GOTO` ve `JUMPTO` ifadeleri kullanılmakta ve kafa karışıklığına devam etmekteydi.

GOTO İfadelerinin Zararlı Kanul Edilmesi

1968 yılında *Edsger W. Dijkstra* tarafından, kontrol akışını anlamakta zorlandığımızdan `GOTO` ve `JUMPTO` ifadeleri **zararlı olarak dile getirilmiştir**.

1970 yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil yapısal programlamayı öğretmek için geliştirilmiştir. **Yapısal programlamada** (*structured programming*) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (`if`, `for`, `do`, `repeat`) birbirinden ayrılmıştır.

Yapısal Programlama

Yapısal programlama, fonksiyonların birbirini çağırmasıyla yapılır. Yapısal programlamada veri ile veriyi işleyen yapılar birbirinden ayrıldığından, arapsaçı programlamanın aksine kontrol akışını (*control flow*) anlamak oldukça kolaydır. Bu nedenle **yapısal programlamada okunaklık olukça yüksektir**.

Bunların yanında Pascal, kendi kendini çağıran **özyinelemeli** (*recursive*) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (*pointer*) desteklemektedir[4]. Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon noktalı `BEGIN-END.` bloğudur. Aşağıda bir sayısı konsoldan alan ve faktöriyelini hesaplayıp sonucu yazdıran bir program örneği verilmiştir;

```
1  PROGRAM FaktoriyelHesaplama;
2
3  FUNCTION Faktoriyel(n: integer): longint;
4  VAR {1.Yerel Degiskenler}
5    i: integer;
6    sonuc: longint;
7  BEGIN {2. Yerel Fonksiyon Algoritmasi}
8    sonuc := 1;
9    FOR i := 1 TO n DO
10     sonuc := sonuc * i;
11   Faktoriyel := sonuc;  {3. sonuc degeri geri dondurluyor}
```

[4] Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

```
END;

VAR {1. Ana Fonksiyon Verileri}
  sayi: integer;
BEGIN
  {2. Ana Fonksiyon Algoritmasi}
  WRITELN('Faktoriyeli hesaplanacak sayiyi giriniz: ');
  READLN(sayi);
  IF sayi < 0 THEN
    WRITELN('Negatif sayilarin faktoriyeli tanimsizdir.')
  ELSE
    WRITELN(sayi, '! = ', Faktoriyel(sayi));
  {3.Ana Fonksiyondan Cikiliyor}
END.
```

Kod 1.1: Faktöriyel Hesaplayan Pascal Programı

Geri değer döndürmeyen ve sadece belli bir mantığı gerçekleştiren fonksiyonlar, Pascal dilinde `PROCEDURE` olarak adlandırılır. Aslında bu terim ALGOL dilinden gelmektedir.

Yapısal Programlamanın Genel Çerçevesi

1. Programın başladığı yeri belirten bir **ana fonksiyon** (*main function*) tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız.
2. Her fonksiyonda ilk önce **veri yapıları** (*data structure*) tanımlanır. Bunlar;
   (a) En basit veri yapısı `char`, `integer`, `float` ve `double` gibi **ilkel veri tiplerinden** (*primitive data type*) olabilen **değişken** (*variable*),
   (b) İlkel veri tiplerinden meydana gelen **dizi** (*array*),
   (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (*structure*),
   (d) Ya da dinamik olarak yani çalışma anında (*run-time*) birbirine bağlanmış verilerden oluşan **bağlı liste** (*linked list*) olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları** (*control structure*) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe `if`, `if-else`, `switch`, `while`, `do-while`, `for`, `continue`, `break`, `goto` ve `return` **talimatlarından** (*statement*) oluşur.

C Programlama Dili,*Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. 1978 yılında *Brian Kernighan ve Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır.

C programlama dili; **yapısal programlama dilidir**, hızlı ve verimli programlar üretir, hatırlatıcı isimlerle (*mnemonic*) yazılan montaj (*assembly*) dili desteği ile **düşük düzey programlamayı** (*low-level programming*) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda verilen Pascal yapısal programının C dilinde kodlanmış hali aşağıda verilmiştir;

```
#include <stdio.h>

long faktoriyel(int n) {
  //1.yerel degiskenler
  int i;
  long sonuc = 1;
  //2. yerel fonksiyon algoritmasi
  for (i = 1; i <= n; i++) {
    sonuc = sonuc * i;
  }
  //3. sonuc degeri geri donduruluyor
  return sonuc;
```

```c
}

int main() { // Ana fonksiyon
  //1.yerel degiskenler
  int sayi;
  //2. Ana fonksiyon algoritması
  printf("Faktoriyeli hesaplanacak sayiyi giriniz: ");
  scanf("%d", &sayi);
  if (sayi < 0) {
    printf("Negatif sayilarin faktoriyeli tanimsizdir.\n");
  } else {
    printf("%d! = %ld\n", sayi, faktoriyel(sayi));
  }
  //3.Ana fonksiyondan cikiliyor
  return 0;
}
```

Kod 1.2: Faktöriyel Hesaplayan C Programı

Yapısal programlamanın en çok uygulandığı bir başka dil olan C dilinde ana fonksiyon, `main()` fonksiyonudur. Pascal dilinde `PROCEDURE` olarak tanımlanan geri değer döndürmeyen fonksiyonlar, C dilinde `void` olarak tanımlanır.

Programın icrası çoğunlukla fonksiyonun ilk talimatından başlar ve sırasıyla icra edilir. Yani kontrol akışı (*control flow*) ilk talimattan başlar ve sırasıyla düzgün devam eder. İşte **programın icrasını değiştirmeyen** bu tür talimatlara **ardışık işlem** (*sequential operation*) adı verilir.

Ardışık İşlemler

1. Değişkenlerin kimliklendirildiği **değişken tanımlamaları** (*identifier definition*):
   (a) `int yariCap=3;`
   (b) `const float PI=3.14;`
   (c) `float daireninAlani,daireninCevresi;`
2. **İfadelerden** (*expression*): (Sabit, değişken ve operatör içeren söz dizimlerine **ifade** adı verilir)
   (a) `daireninAlani=PI*yariCap*yariCap;`
   (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi **giriş-çıkış işlemleri** (*input-output operation*):
   (a) `printf("Girdiginiz sayilarin toplami: %d\n", toplam);`
   (b) `scanf("%d", &sayi2);`

Kontrol işlemleri (*control operation*) ise programın icra sırasını yani kontrol akışını (*control flow*) değiştiren kontrol yapılarıdır.

Kontrol İşlemleri

1. **Duruma göre seçimler** (*conditional choice*): `if`, `if-else` ve `switch` talimatları.
2. **İlişkisel döngü** (*relational loop*): `while`, `do-while` ve `for` talimatları.
3. **Dallanmalar** (*jump*): `continue`, `break`, `goto` ve `return` talimatları.

Fiziksel ve Mantıksal Satır

Kontrol işlemlerinde; kodlanan **mantıksal satırlar** (*logical sequence*) ile **fiziksel olarak icra edilen satırlar** (*physical sequence*) birbirinden farklıdır.

Yapısal programlamada, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağı üzerinde durulur. Seksenli yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür.

1.7 Yazılım Geliştirme Süreci

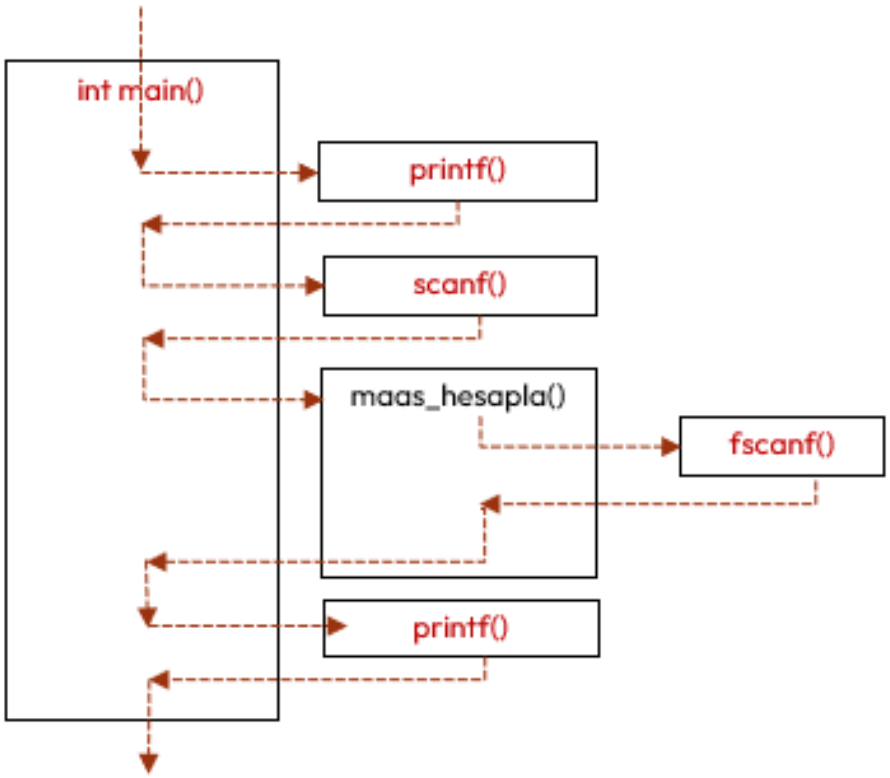
Bilgisayar programlanırken sadece karmaşık bir matematik formülünün hesaplanması problemini ele almaz. Bilgisayarların yaygınlaşması ve hayatımıza girmesiyle birlikte yaşantımızın bir parçası olan çoğu problemi çözer hale gelmiştir. Gümünüzde bilgisayarla çözülebilecek problemlerin karmaşıklığı o kadar artmıştır ki problemlerin tanımından yazılımın tamamlanmasına kadar olan sürecin yönetimi çok önemli hale gelmiştir. Bu süreç, **yazılım geliştirme süreci** (*software development process*) olup şu aşamalardan oluşur:

- **Gereksinim analizi** (*requirements analysis*),
- **Şartname** (*specification*),
- **Tasarım** (*design*),
- **Gerçekleştirim** (*implementation*),
- **Doğrulama ve geçerlilik** (*verification and validation*) ve
- **Bakım** (*maintenance*).

Programlama dili özellikle yazılım geliştirme sürecinin "gerçekleştirim aşamasında" (*implementation stage*) kullanılır; ancak dilin özellikleri tasarım kararlarını da etkileyebilir.

1.8 Yapısal Programlama Paradigması

Yapısal programlama paradigmasında yazılım ihtiyaçları, **birbirini çağıran fonksiyonlar** yazılarak çözülür (Şekil 1.1). Bu durum bir önceki başlıkta verilen Kod 1.1 ve Kod 1.2 örneklerinden de anlaşılabilir. Fonksiyonların algoritmasını oluşturan **talimatlar** (*statement*) art arda izlenerek programların neler yaptığı, bu talimatlar izlenerek **kontrol akışı** (*control flow*) anlaşılabilir. Bu paradigma yani fonksiyonların birbirini çağırmasıyla yapılan programlama yaklaşımına sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.



Şekil 1.1: Yapısal Programlama Paradigması

Yapısal programlamayla yazılım geliştirmede en sık karşılaşılan karmaşıklık, veri ile veriyi işleyen fonksiyonların birbirinden kopuk kurgulanmasıdır. Bu fonksiyon yaklaşımı küçük ölçekli projelerde hızlı sonuç verse de, sistem büyüdükçe durum yönetimi kontrolden çıkar:

- ◇ Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. **Değişim yönetimi** (*change management*) çok zordur ve uzun zaman alır.
- ◇ Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda **hataların izini sürmek** (*error handling*) işi zorlaşır.
- ◇ Yapısal programlamada, **gösterici** (*pointer*) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun **güvenli** (*safe code*) olarak çalıştığının incelenmesi çok zordur.
- ◇ Sürekli olarak benzer projelerde aynı kodları **tekrar yazmak** (*duplicate code*) gereklidir.
- ◇ Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi **başarısız** (*fail*) olmuştur.

Yazılım Ne Zaman Başarısız Olur?

Bir yazılım yalnızca çalışmadığında değil; kullanıcıya beklenen değeri sunmadığında, maliyet hedeflerini aşınca veya zamanında teslim edilemediğinde de başarısız kabul edilir. Bu nedenle bir yazılım projesi özellikle şu durumlarda başarısız sayılır:
- ◇ Kullanıcı ihtiyaçlarını ve beklentilerini karşılayamıyorsa,
- ◇ Öngörülen bütçenin üzerine çıkıyorsa,
- ◇ Planlanan teslim tarihine yetiştirilemiyorsa.

Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi. Smalltalk, 1972 yılında Xerox Park şirketinde *Alan Kay* önderliğinde *Dan Ingalls*, *Adele Goldberg*, *Ted Kaehler*, *Diana Merry*, *Scott Wallace* ve *Peter Deutsch* tarafından geliştirilmiş bir dildir. Smalltalk dilinde **nesneler** (*object*) temel yapı taşlarıdır.

Yapısal programlamada geliştirmede en sık karşılaşılan karmaşıklık, veri ile veriyi işleyen fonksiyonların birbirinden kopuk kurgulanmasıdır. Bu yaklaşım küçük ölçekli projelerde hızlı sonuç verse de, sistem büyüdükçe durum yönetimi kontrolden çıkar.

1.9 Nesne Yönelimli Programlama Paradigması

Nesne yönelimli programlama yani **OOP** (*Object-Oriented Programming*), durumu ve davranışı tek bir birim yani **nesne** (*object*) altında toplayarak bir önceki başlıktaki sorunların çoğunu çözer. Nesneler, **duruma** (*state*) sahiptir ve nesnenin o anki verisidir. Nesneler ayrıca textbfdavranışa (*behavior*) sahiptir ve bu davranışlar ile nesne verisini işler.

Nesne Yönelimli Programlama Paradigması

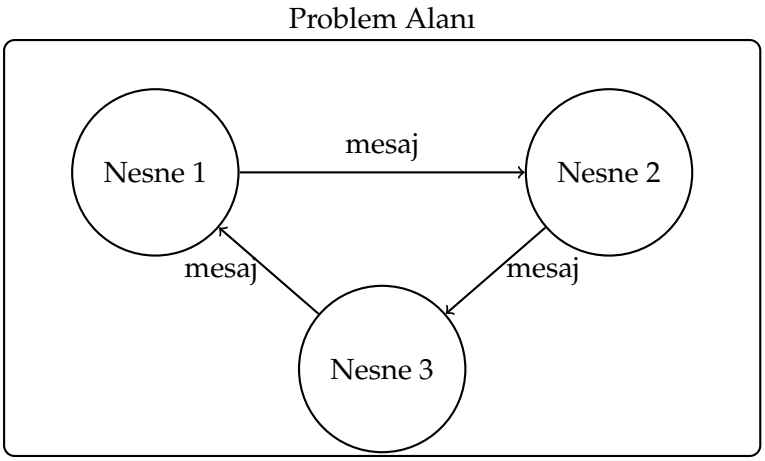
Yapısal programlamanın aksine **nesne yönelimli programlama** paradigmasında, nesneler programın temel yapı taşlarıdır. **Nesneler; durum ve davranışlara sahiptir**. Programlama, nesnelerin birbirlerine ileti göndermesi (*message-passing*) ile yapılır.

Buna; Üye, Kitap ve Kütüphane Görevlisi içeren bir Kütüphane Sistemi örnek verilebilir: Bu nesneler birbirleriyle mesajlaşarak/etkileşerek problemi modeller (Şekil 1.3).

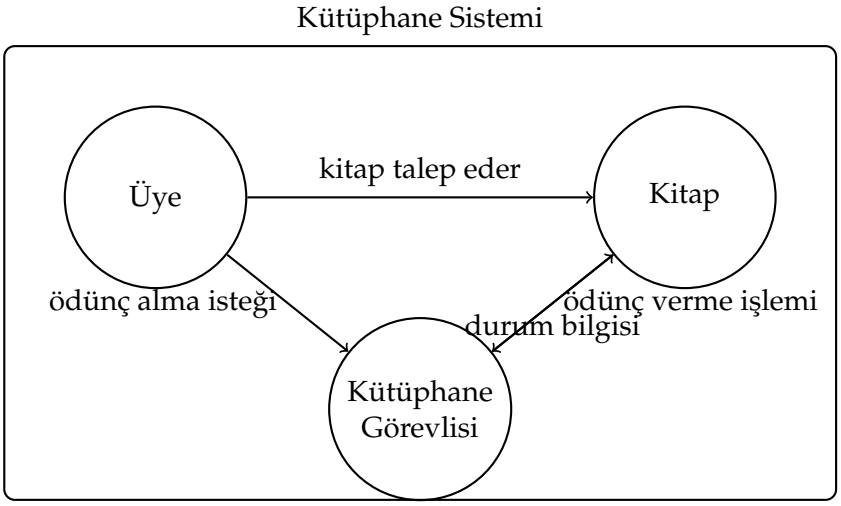
OOP ile yazılım geliştirme sürecinde **UML** (*Unified Modeling Language*) gibi grafiksel gösterimler tasarım aşamasında çoğunlukla kullanılır.

OOP dört temel kavram üzerine kuruludur:

- ◇ **Soyutlama (Abstraction):** Karmaşık iç çalışma mekanizmalarını dış dünyadan gizleyerek yalnızca gerekli arayüzleri sunmak.
- ◇ **Kapsülleme (Encapsulation):** Sınıf değişkenlerini korumaya almak (private) ve erişimi onaylanmış metotlar üzerinden sağlamak.



Şekil 1.2: Nesne Yönelimli Geliştirme Paradigması



Şekil 1.3: Kütüphane sisteminin nesneler ve aralarındaki etkileşimlerle modellenmesi

- ◇ **Kalıtım (Inheritance):** "Bir türüdür" ilişkisi kurarak ortak kod gruplarını üst sınıflara taşımak.
- ◇ **Çok Biçimlilik (Polymorphism):** Aynı mesaja farklı nesne tiplerinin kendi yapısına uygun özel yanıtlar verebilmesi.

OOP paradigmasında, yukarıdaki kavramları bir anda özümsemek gerektiğinden **öğrenme eğrisi diktir**.

2. Bölüm
Nesne Odaklı Zihniyet

2.1 Sınıf ve Nesne Kavramı

Nesne yönelimli programlamanın temelinde iki ana kavram bulunur: **sınıf** ve **nesne**.

Sınıf (*class*), nesnelerin hangi özelliklere ve davranışlara sahip olacağını tanımlayan bir şablondur. Bir sınıf doğrudan çalışan bir varlık değildir; nesne üretmek için kullanılan bir plandır.

Nesne (*object*) ise bu sınıftan üretilmiş somut varlıktır. Başka bir ifadeyle sınıf plan, nesne ise bu plandan üretilmiş gerçek örnektir.

Nesne-Sınıf İlişkisi

Nasıl ki bir mimari plandan birçok ev inşa edilebiliyorsa, bir sınıftan da birçok nesne üretilebilir. Mimari plan sınıfa, bu plandan yapılan evler ise nesnelere karşılık gelir.

Örneğin bir `Araba` sınıfı tanımlanabilir. Bu sınıftan `araba1`, `araba2` ve `araba3` gibi farklı nesneler üretilebilir. Her nesne aynı sınıftan türemiştir ancak her biri kendi verilerine yani durumuna sahiptir.

```
class Araba {
public:
  string marka;
  int hiz;

  void hizlan(int miktar) {
    this->hiz = this->hiz + miktar;
  }
};

// Kullanım:
int main() {
  Araba araba1;
  araba1.marka = "Toyota";
  araba1.hiz = 0;

  Araba araba2;
  araba2.marka = "BMW";
  araba2.hiz = 50;

  araba1.hizlan(20);
  araba2.hizlan(30);

  return 0;
}
```

Bu örnekte `Araba` bir sınıftır. `araba1` ve `araba2` ise bu sınıftan üretilmiş iki farklı nesnedir.

2.2 Nesnenin Durumu, Özellikleri ve Davranışları

Bir nesnenin sahip olduğu veriler özellik (*property*) olarak adlandırılır. Nesne özellikleri, sınıflarda **alan** (*field*) olarak tanımlanır. Örnekte, `marka` ile `hiz`, `Araba` sınıfının alanlarıdır. Bu alanların belirli bir andaki değerleri ise nesnenin **durumunu** (*state*) oluşturur.

Özellik ve Alan İlişkisi

Genel olarak sınıflarda tanımlanan alanlar, kontrollü olarak dış dünyaya açıldığında **özellik** (*property*) olarak adlandırılır. Her alan bir özelliğe dönüşmeyebilir. İleride anlatılacaktır.

Bir nesnenin yapabildiği işlemlere ise **davranış** denir. OOP dillerinde davranışlar **yöntem** (*method*) olarak adlandırılır. Yapısal programlamadaki fonksiyonlara (*function*) benzer ve onların gibi kodlanırlar. Fonksiyonlardan farklı olarak yöntemler bir nesneye ait davranışlardır (*behavior*).

```
class BankaHesabi {
public:
  string hesapNo;
  double bakiye;

  void paraYatir(double miktar) {
    if (miktar > 0) {
      this->bakiye += miktar;
    }
  }

  void paraCek(double miktar) {
    if (miktar > 0 && miktar <= this->bakiye) {
      this->bakiye -= miktar;
    }
  }

  double bakiyeOgren() {
    return this->bakiye;
  }
};

// Kullanım:
int main() {
  BankaHesabi hesap;
  hesap.bakiye=0.0;
  hesap.paraYatir(1000.0);

  return 0;
}
```

Bu örnekte bir banka hesabı nesnesi için:

- **Özellikler**: hesap numarası, bakiye, hesap sahibi
- **Durum**: bakiyenin o andaki değeri, hesabın aktif olup olmaması
- **Davranışlar**: para yatırmak, para çekmek, bakiye öğrenmek

Burada `hesapNo` ve `bakiye` nesnenin durumlarıdır. `paraYatir()`, `paraCek()` ve `bakiyeOgren()` ise nesnenin davranışlarıdır. Bu davranışlar, nesnenin durumlarından etkilenirler. Dolayısıyla her nesnenin durumu aynı olmayacağı için davranışları da farklı olur.

2.3 Sınıf ve Nesnelerin UML Sınıf Diyagramları ile Gösterilmesi

UML (*Unified Modeling Language*), yazılımları görsel olarak modellemek için kullanılan standart bir gösterim dilidir. Bir sonraki bölümde datylı anlatılacaktır. Sınıf diyagramları, sınıfların özelliklerini, davranışlarını ve aralarındaki ilişkileri göstermek için kullanılır. Bir UML sınıf diyagramı genellikle Sınıf adı, Özellikler ve Yöntemleri içeren üç bölümlü bir dikdörtgenden oluşur.

UML gösteriminde; genellikle şu sembollerle ifade edilir:



Şekil 2.1: Sınıfların UML Diyagramında Gösterilmesi

- ◇ Erişim belirleyiciler (*access modifier*): `+`: genel (*public*), `-`: özel (*private*),`#`: korumalı (*protected*)
- ◇ Soyut Sınıf ve Soyut Yöntemler: italik yazılır. İleride anlatılacaktır.
- ◇ `static` üyeler: Altı çizili yazılır. İleride anlatılacaktır.

2.4 Bilgi ve Veri Gizleme Nedir?

Bilgi gizleme (*information hiding*), bir sınıfın iç işleyişine ait ayrıntıların sınıf dışından saklanmasıdır. Başka bir ifadeyle, bir nesnenin içinde hangi verilerin tutulduğu, bu verilerin nasıl değiştirildiği veya bir işlemin içeride hangi adımlarla yapıldığı dış dünyaya açılmaz.

Kullanıcıya yalnızca gerekli olan dış arayüz sunulur. Bilgi gizleme kavramının sınıf içindeki verilerin tutulduğu alanlara (*field*) uygulanmış hâline çoğu zaman **veri gizleme** (*data hiding*) denir. Veri gizlemede amaç, sınıfın içindeki alan değişkenlerine doğrudan erişimi engellemektir. Çünkü nesnenin durumunu temsil eden veriler doğrudan değiştirilirse nesne geçersiz veya tutarsız bir duruma düşebilir.

Kod 2.2 örneğinde kullanım sırasında bir banka hesabının bakiyesi doğrudan dışarıdan değiştirilebilirse şu durum oluşabilir:

```
int main() {
  BankaHesabi hesap;
  hesap.bakiye = -5000; //!!
  hesap.paraYatir(1000.0);
  return 0;
}
```

Bu işlem teknik olarak mümkün olursa banka hesabı nesnesi geçersiz bir duruma düşer. Çünkü normal şartlarda bakiye, yalnızca belirli kurallar çerçevesinde değişmelidir. Bu nedenle `bakiye` gibi kritik verileri tutan alanlar `private` yapılır ve dışarıdan doğrudan erişime kapatılarak veri gizleme yapılır.

Veri gizleme (*data hiding*) ile sınıfın iç verilerini dış müdahaleye kapatarak nesnenin tutarsız duruma düşmesi engellenmesi işlemidir. Bu amaçla **erişim belirleyiciler** (*access modifier*) kullanılır. Nesne yönelimli programlamada yaygın olarak üç temel erişim belirleyici bulunur:

- ◇ **Özel** (`private`): Sadece sınıf içinden erişilebilir.
- ◇ **Genel** (`public`): Sınıf dışından da erişilebilir.
- ◇ **Korumalı** (`protected`): Sınıf içinden ve türetilmiş sınıflardan erişilebilir.

Aşağıda veri gizlemeye ilişkin bir örnek verilmiştir:

```
class Ogrenci {
private:
  std::string ad;
  double notOrtalamasi;

public:
  void notOrtalamasiAyarla(double deger) {
    if (deger >= 0 && deger <= 100) {
      this->notOrtalamasi = deger;
    } else {
      std::cout << "Not 0 ile 100 arasinda olmalidir!";
    }
```

```
  }
  double notOrtalamasiOgren() const {
    return this->notOrtalamasi;
  }
};
```

Bu örnekte `notOrtalamasi` alanının erişim belirleyicisi `private` yapılarak dışarıdan doğrudan değiştirilmesi engellenmiştir. `public` yöntemler aracılığıyla `notOrtalamasi` alanına güvenli erişim sağlanır. `notOrtalamasiAyarla()` yöntemi, değerin 0 ile 100 arasında olup olmadığını kontrol eder. Böylece negatif veya 100'den büyük geçersiz değerlerin atanması engellenir.

2.5 Soyutlama ve Kapsülleme

Soyutlama (*abstraction*), karmaşık bir yapının sadece gerekli yönlerini öne çıkarıp gereksiz ayrıntılarını gizleme işlemidir. Programlamada soyutlama, bir nesnenin dışarıdan nasıl kullanılacağını gösterirken içeride nasıl çalıştığını saklamaya yardımcı olur.

Kod 2.2 örneği verilen bir banka hesabı kullanıcısı para yatırabilir, para çekebilir ve bakiyesini öğrenebilir. Ancak bu işlemlerin sistem içinde hangi veri tabanı işlemleriyle yapıldığını bilmek zorunda değildir.

```
class BankaHesabi {
private:
  std::string hesapNo;
  double bakiye;

  void bakiyeKaydet() {
    // Veritabani islemleri
  }

public:
  std::string HesapNoGetir() const {
    return this->hesapNo;
  }

  void HesapNoKaydet(const std::string& yeniHesapNo) {
    if (yeniHesapNo != "") {
      this->hesapNo = yeniHesapNo;
    }
  }

  void paraYatir(double miktar) {
    if (miktar > 0) {
      this->bakiye += miktar;
    }

    bakiyeKaydet();
  }

  void paraCek(double miktar) {
    if (miktar > 0 && miktar <= this->bakiye) {
      this->bakiye -= miktar;
    }

    bakiyeKaydet();
  }

  double bakiyeOgren() const {
    return this->bakiye;
  }
};
```

Bu örnekte kullanıcı sadece `paraYatir()`, `paraCek()` ve `bakiyeOgren()` yöntemlerini kullanır. `bakiye` bilgisinin nasıl işlendiği sınıfın iç ayrıntısıdır.

Veri soyutlaması (*data abstraction*), bir nesnenin verisiyle ne yaptığına odaklanıp, bu verileri nasıl sakladığına dair ayrıntıları gizleme ilkesidir. Soyutlama sayesinde kullanıcı nesnenin sunduğu veriyi bilir; fakat bu verinin arka planda nasıl saklandığını bilmek zorunda değildir. Bunu sağlamak için `hesapNo` ve `bakiye` alanları dışarıya kapatılarak (`private`) için veri gizliliği (*data hiding*) yapılır. Ardından `hesapNo` alanına ait `HesapNoGetir()` ve `HesapNoKaydet()` yöntemleri eklenerek bu alan üzerinde veri soyutlaması yapılır.

Özellik

Nesnenin verisini **getiren** (*getter*) ve nesnenin verilerini **kaydeden** (*setter*) yöntemlerin ilgili sınıf **alanına** (*field*) uygulanmasıyla veri soyutlaması yapılır. Böylece nesnenin kendi verilerine nasıl erişileceği üzerinde hakları (*property*) olmuş olur. Dışarıdan istediğimiz gibi nesne verilerini değiştiremeyiz. İşte bu nedenle soyutlama yapılmış nesne durumlarına, **özellik** (*property*) adı verilir.

Bu sınıflardan oluşturulmuş bir banka hesabı nesnesi dışarıya şu işlemleri sunabilir:

```
hesap.paraYatir();
hesap.paraCek();
hesap.bakiyeOgren();
```

Kullanıcı bu işlemleri çağırarak hesabı kullanır. Ancak bakiyenin bellekte nasıl tutulduğunu, işlem geçmişinin nasıl kaydedildiğini veya verilerin hangi güvenlik kontrollerinden geçtiğini bilmesine gerek yoktur. Bunun için dışarıya kapalı `bakiyeKaydet()` yöntemi yazılarak **kontrol soyutlaması** (*control abstraction*) yapılmıştır.

Kapsülleme

Bir sınıf üzerinde hem veri soyutlaması ve hem de kontrol soyutlaması yapılması işlemine **kapsülleme** (*encapsulation*) adı verilir. Kapsülleme kısaca veri ile bu veriyi işleyen davranışların aynı sınıf içinde bir araya getirilmesidir.

2.6 Nesnelerin İmal Edilmesi

Bir sınıftan nesne imal etme işlemine **nesne oluşturma** (*instantiation*) denir. Nesne imal edildiği anda bellekte o nesne için yer ayrılır ve sınıfın **yapıcısı** (*constructor*) oluşan nesnenin durumlarına (verilerine) ilk değer vermek için çalıştırılır.

nesne imalatı doğrudan **yığın** (*stack*) bellekte yapılabilir. Bu durumda nesneye, derleme sırasında bellekte statik yer tahsisi yapılır. Küçük yazılımlar için bu kullanışlı olabilir. Buna örnek olarak Kod 2.5 örneğindeki sınıftan yığın bellekte nesne imal eden uygulama aşağıda verilmiştir;

```
int main() {
  BankaHesabi hesap; // hesap nesnesi yığın bellekte imal edildi

  hesap.HesapNoKaydet("TR123456789");
  hesap.paraYatir(1000);
  hesap.paraCek(250);

  std::cout << "Hesap No: " << hesap.HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap.bakiyeOgren() << std::endl;

  return 0;
}
```

Modern C++ uygulamalarında ve Java ve C# gibi nesne yönelimli dillerde nesne, **çalışma zamanında** (*run-time*) öbek (`heap`) bellekte imal edilir. nesne imal etmek için **`new` işleci** (*new operator*) kullanılır. Ancak imal edilen nesne kullanıldıktan sonra, **`delete` işleci** (*delete operator*) ile bu nesneye tahsis edilen bellek serbest bırakılır. Bu yapılmaz ise bellek sızıntısı (*memory leak*) oluşabilir.

Bu durumda yukarıdaki örnekte `BankaHesabi` sınıfından `hesap` kimlikli nesne, dinamik olarak

aşağıdaki gibi imal edilir:

```
int main() {
  BankaHesabi* hesap=new BankaHesabi; // hesap nesnesi öbek (heap) bellekte imal edildi

  hesap->HesapNoKaydet("TR123456789");
  hesap->paraYatir(1000);
  hesap->paraCek(250);

  std::cout << "Hesap No: " << hesap->HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap->bakiyeOgren() << std::endl;

  delete hesap; // hesap nesnesine tahsis edilen bellek yazılımcı tarafından kod
      yarazak manuel olarak serbest bırakıldı.
  return 0;
}
```

dinamik nesne imalatındaki bellek sızıntılarını önlemek için C++14 ile birlikte hayatımıza giren **akıllı göstericiler** (*smart pointers*) kullanılır. Üçüncü bölümde detaylı anlatılacaktır. Böylece yazılımcının unutabileceği `delete` işlecinden dolayı bellek sızıntılarının önüne geçilir. Bu durumda aynı örnek akıllı göstericiler ile aşağıdaki gibi yapılır:

```
int main() {
  std::unique_ptr<BankaHesabi> hesap = std::make_unique<BankaHesabi>(); // Akıllı gö
      sterici ile dinamik nesne imalatı

  hesap->HesapNoKaydet("TR123456789");
  hesap->paraYatir(1000);
  hesap->paraCek(250);

  std::cout << "Hesap No: " << hesap->HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap->bakiyeOgren() << std::endl;

  // Kapsamdan çıkılınca bellek otomatik olarak serbest bırakılır.
  return 0;
}
```

Yukarıdaki örneklerde bir adet hesap nesne imal edilmiştir. İstersek aynı sınıftan sınırsız sayıda nesne imal edebiliriz:

```
int main() {
  // Akıllı gösterici ile dinamik nesne imalatı:
  std::unique_ptr<BankaHesabi> hesap1 = std::make_unique<BankaHesabi>();

  std::cout << "hesap1 No: " << hesap1->hesapNoGetir() << std::endl; //
  std::cout << "hesap1 Bakiye: " << hesap1->bakiyeOgren() << std::endl; // 0

  std::unique_ptr<BankaHesabi> hesap2 = std::make_unique<BankaHesabi>();

  std::cout << "hesap2 No: " << hesap2->hesapNoGetir() << std::endl; //
  std::cout << "hesap2 Bakiye: " << hesap2->bakiyeOgren() << std::endl; // 0

  std::unique_ptr<BankaHesabi> hesap3 = std::make_unique<BankaHesabi>();

  std::cout << "hesap3 No: " << hesap3->hesapNoGetir() << std::endl; //
  std::cout << "hesap3 Bakiye: " << hesap3->bakiyeOgren() << std::endl; // 0

  return 0;
}
```

Burada şöyle bir sorun ortaya çıkar: İmal edilecek nesnelerin hepsinin `hesapNo` durumları `null` ve `bakiye` durumları `0.0` olur. İmal edilecek her nesnede bu durumların farklı farklı olması istenir. Yani nesneler farklı verilerle imal edilebilmelidir. Bu durumda nesne yapıcısı devreye girer.

2.7 this Göstericisi

C++'da her nesne, `this` adı verilen önemli bir gösterici aracılığıyla kendi adresine erişebilir. `this` Göstericisi (*this pointer*), tüm üye yöntemler için örtük bir değişkendir. Aşağıda karmaşık sayılar örneği verilmiştir;

```
#include <iostream>

class Karmasik {
  float gercek, sanal;
public:
  Karmasik(float pGercek=0.0, float pSanal=0.0) {
    this->gercek=pGercek;
    this->sanal=pSanal;
  }
  void yaz() { //yaz yöntemi
    std::cout<< this->gercek << "+" << this-> sanal << +"i" << std::endl;
  }
};

int main(void) {
  Karmasik c1(1.0,2.0);
  c1.yaz();
  Karmasik c2(3.0,5.0);
  c2.yaz();
}
```

2.8 Yapıcı

Yapıcı (*constructor*), nesnenin imal edildiği anda otomatik olarak çalışan özel bir yöntemdir. Yapıcının temel amacı nesneyi geçerli bir başlangıç durumuna getirmektir. Bu nedenle bir değer geri döndürmeler sadece nesne verisine başlangıç değeri vermek için kullanılırlar. Yapıcılar sınıf ismiyle aynı kimliğe sahip geri değer döndürmeyen yöntemlerdir.

```
class BankaHesabi {
private:

  std::string hesapNo;
  double bakiye;

  void bakiyeKaydet() {
    // Veritabani islemleri
  }

public:

  BankaHesabi() { //Yapıcı
    this->hesapNo="TR123456789";
    this->bakiye=1000.0;
  }

  std::string hesapNoGetir() const {
    return this->hesapNo;
  }

  void hesapNoKaydet(const std::string& yenihesapNo) {
    if (yenihesapNo != "") {
      this->hesapNo = yenihesapNo;
    }
  }

  void paraYatir(double miktar) {
    if (miktar > 0) {
      this->bakiye = this->bakiye + miktar;
    }
```

```
    bakiyeKaydet();
  }

  void paraCek(double miktar) {
    if (miktar > 0 && miktar <= this->bakiye) {
      this->bakiye = this->bakiye - miktar;
    }

    bakiyeKaydet();
  }

  double bakiyeOgren() const {
    return this->bakiye;
  }
};
```

Buradaki örnekte imal edilecek her nesnenin verisi, yine hep aynı başlangıç değeriyle (yani `hesapNo` verisi `"TR123456789"` ile ve `bakiye` verisi `1000.0`) olacaktır. Her nesneyi aynı verilerle imal etmek yerine `hesapNo` ve `bakiye` değerlerini parametre olarak alan parametreli yapıcı (*parameterized constructor*) tanımlanır.

§2.8.1 Parametreli Yapıcı

Parametreli yapıcı (*parameterized constructor*) ile imal edilecek her nesnenin farklı başlangıç verisine (duruma) sahip olarak imal edilmesi sağlanır.

```
class BankaHesabi {
  //...
  BankaHesabi(std::string pHesapNo, double pBakiye) { //Parametreli Yapıcı
    this->hesapNo=pHesapNo;
    this->bakiye=pBakiye;
  }
  //...
}
int main() {
  // Akıllı gösterici ile dinamik nesne imalatı:
  std::unique_ptr<BankaHesabi> hesap1 = std::make_unique<BankaHesabi>("TR111111111"
      ,4000);

  std::cout << "hesap1 No: " << hesap1->hesapNoGetir() << std::endl; //TR111111111
  std::cout << "hesap1 Bakiye: " << hesap1->bakiyeOgren() << std::endl; // 4000

  std::unique_ptr<BankaHesabi> hesap2 = std::make_unique<BankaHesabi>("TR222222222"
      ,8000);

  std::cout << "hesap2 No: " << hesap2->hesapNoGetir() << std::endl; //TR222222222
  std::cout << "hesap2 Bakiye: " << hesap2->bakiyeOgren() << std::endl; // 8000

  std::unique_ptr<BankaHesabi> hesap3 = std::make_unique<BankaHesabi>("TR333333333"
      ,12000);

  std::cout << "hesap3 No: " << hesap3->hesapNoGetir() << std::endl; //TR333333333
  std::cout << "hesap3 Bakiye: " << hesap3->bakiyeOgren() << std::endl; // 12000

  return 0;
}
```

§2.8.2 Kopyalama Yapıcısı

Kopyalama yapıcısı (*copy constructor*), var olan bir nesneden yeni bir nesne oluşturmak için kullanılan özel yapıcıdır. C++ derleyicisi, arka planda nesneyi üye-üye kopyalayan bir tane kopyalama yapıcısı üretir. Bu yapıcı özellikle nesne içinde dinamik kaynaklar tutulduğunda önemlidir. Yanlış kopyalama, iki nesnenin aynı kaynağı paylaşmasına ve program hatalarına yol açabilir.

```
class BankaHesabi {
  //...
  BankaHesabi(const BankaHesabi& digerHesap) {
    this->hesapNo = digerHesap.hesapNo;
    this->bakiye = digerHesap.bakiye;
  }
  //...
}
int main() {
  std::unique_ptr<BankaHesabi> hesap1 = std::make_unique<BankaHesabi>();

  std::unique_ptr<BankaHesabi> hesap2 = std::make_unique<BankaHesabi>(*hesap1);

  std::cout << "Hesap 1:" << std::endl;
  std::cout << "Hesap No: " << hesap1->HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap1->bakiyeOgren() << std::endl;

  std::cout << std::endl;

  std::cout << "Hesap 2:" << std::endl;
  std::cout << "Hesap No: " << hesap2->HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap2->bakiyeOgren() << std::endl;

  hesap2->HesapNoKaydet("TR987654321");
  hesap2->paraYatir(500);

  std::cout << std::endl;
  std::cout << "Degisikliklerden sonra:" << std::endl;

  std::cout << "Hesap 1:" << std::endl;
  std::cout << "Hesap No: " << hesap1->HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap1->bakiyeOgren() << std::endl;

  std::cout << std::endl;

  std::cout << "Hesap 2:" << std::endl;
  std::cout << "Hesap No: " << hesap2->HesapNoGetir() << std::endl;
  std::cout << "Bakiye: " << hesap2->bakiyeOgren() << std::endl;

  return 0;
}
```

Bu örneklerde tanımlanan yapıcı kopyalama yapıcısıdır. Bu yapıcı, diğer yapıcıyla aynı kimliğe (`yapici`) sahiptir ve parametre olarak aynı sınıftan imal edilmiş bir başka nesneyi parametre olarak alır.

2.9 Nesnelerin Yok Edilmesi ve Yıkıcı

Bir nesnenin yaşam süresi sona erdiğinde bellekte kapladığı alan serbest bırakılır. Belleği serbest bırakma öncesinde çalışan özel yönteme **yıkıcı** (*destructor*) denir. yıkıcı, genellikle dosya kapatma, ağ bağlantısını sonlandırma veya dinamik belleği serbest bırakma gibi temizlik işlemleri için kullanılır. Yıkıcı, sınıf kimliğiyle aynı olacak şekilde önünde `~` karakteri olan bir yöntem olarak tanımlanır. Geri dönüş tipi yoktur ve parametresi olmaz.

```
class BankaHesabi {
  //...
  // Yıkıcı / Destructor
  ~BankaHesabi() {
    std::cout << "BankaHesabi nesnesi yok ediliyor." << std::endl;
    std::cout << "Hesap No: " << this->hesapNo << std::endl;

    // Dosya kapatma, veritabani baglantisi kapatma,
    // dinamik bellek temizleme gibi islemler burada yapilir.
  }
```

```
  //...
}
int main() {
  //Bir başka kapsam için yeni blok tanımlanıyor:
  {
    BankaHesabi hesap1;

    hesap1.paraYatir(500);
    hesap1.paraCek(200);

    std::cout << "Hesap No: " << hesap1.HesapNoGetir() << std::endl;
    std::cout << "Bakiye: " << hesap1.bakiyeOgren() << std::endl;
  }
  // hesap1 bu bloktan çıkınca destructor otomatik çağrılır

  std::cout << "Program devam ediyor..." << std::endl;

  return 0;
}
/* Program Çıktısı:
BankaHesabi nesnesi olusturuldu.
Hesap No: TR123456789
Bakiye: 1300
BankaHesabi nesnesi yok ediliyor.
Hesap No: TR123456789
Program devam ediyor...
*/
```

2.10 Fonksiyon Çok Yükleme

Fonksiyon çok yükleme (*function overloading*), aynı kimliğe sahip birden fazla fonksiyonun farklı parametre listeleriyle tanımlanmasıdır. Derleyici veya yorumlayıcı, çağrılan fonksiyonun parametrelerine bakarak hangi sürümün çalıştırılacağını belirler. Çok yüklemede; kimlik aynı, parametre listesi farklı, yalnızca dönüş tipiyle ayrım yapılamaz.

yapısal programlama dilindeki gibi tanımlanan fonksiyonlar da çok yüklenebilir. Ancak çok yükleme, sınıf üyesi olan yöntemlerde daha çok kullanılır.

```
class Hesaplayici {
public:
  int topla(int a, int b) {
    return a + b;
  }

  int topla(int a, int b, int c) {
    return a + b + c;
  }

  std::string topla(const std::string& a, const std::string& b) {
    return a + b;
  }
};

int main() {
  Hesaplayici h;

  std::cout << "2 + 3 = " << h.topla(2, 3) << std::endl;
  std::cout << "2 + 3 + 4 = " << h.topla(2, 3, 4) << std::endl;
  std::cout << "Metin toplama: " << h.topla(std::string("Merhaba "), std::string("Dunya
      ")) << std::endl;

  return 0;
}
/* Program Çıktısı:
2 + 3 = 5
```

```
2 + 3 + 4 = 9
Metin toplama: Merhaba Dunya
*/
```

Bu örnekte `topla` adı üç farklı biçimde kullanılmıştır. Parametre sayısı veya türü değiştiği için bu geçerli bir çok yüklemedir.

2.11 İşleç Çok Yükleme

İşleç çok yükleme (*operator overloading*), var olan işleçlerin kullanıcı tanımlı sınıflar için yeniden anlamlandırılmasıdır. C++ dili işleç çok yüklemeyi **destekler**.

§2.11.1 Toplama İşlecinin Çok Yüklenmesi

Toplama (`+`) işleci sayılar için toplama yapar. Ancak `Nokta` sınıfından imal edilmiş iki farklı nesneyi toplamak için de özel olarak tanımlanabilir.

```
class Nokta {
private:
  int x;
  int y;

public:
  Nokta(int x, int y) {
    this->x = x;
    this->y = y;
  }

  Nokta operator+(const Nokta& diger) const {
    return Nokta(this->x + diger.x, this->y + diger.y);
  }

  void yazdir() const {
    std::cout << "(" << this->x << ", " << this->y << ")" << std::endl;
  }
};

int main() {
  Nokta n1(3, 4);
  Nokta n2(5, 6);

  Nokta toplam = n1 + n2;

  std::cout << "n1: ";
  n1.yazdir();

  std::cout << "n2: ";
  n2.yazdir();

  std::cout << "Toplam: ";
  toplam.yazdir();

  return 0;
}
```

§2.11.2 Atama İşlecinin Çok Yüklenmesi

Atama işleci, bir nesnenin değerinin başka bir nesneye aktarılmasını sağlar. Bazı dillerde özellikle dinamik kaynak kullanan sınıflarda atama işlecinin özel olarak tanımlanması gerekir.

```
class Nokta {
private:
  int x;
  int y;
```

```
public:
  Nokta(int x = 0, int y = 0) : x(x), y(y) {}

  Nokta(const Nokta& diger) {
    this->x = diger.x;
    this->y = diger.y;

    std::cout << "Kopyalama yapicisi calisti." << std::endl;
  }

  Nokta& operator=(const Nokta& diger) { //Atama işleci çok yüklemesi
    std::cout << "Atama isleci calisti." << std::endl;

    if (this != &diger) {
      this->x = diger.x;
      this->y = diger.y;
    }

    return *this;
  }

  void yazdir() const {
    std::cout << "(" << x << ", " << y << ")" << std::endl;
  }
};

int main() {
  Nokta n1(3, 4);
  Nokta n2(10, 20);

  std::cout << "Atamadan once:" << std::endl;

  std::cout << "n1 = ";
  n1.yazdir();

  std::cout << "n2 = ";
  n2.yazdir();

  n2 = n1;

  std::cout << "\nAtamadan sonra:" << std::endl;

  std::cout << "n1 = ";
  n1.yazdir();

  std::cout << "n2 = ";
  n2.yazdir();

  return 0;
}
/* Çıktı:

Atamadan once:
n1 = (3, 4)
n2 = (10, 20)
Atama isleci calisti.

Atamadan sonra:
n1 = (3, 4)
n2 = (3, 4)
*/
```

C++'ta kopyalama yapıcısı ile atama işlecinin farkını şu şekilde açıklayabiliriz:

```
Nokta n1(3, 4);
Nokta n3 = n1; //kopyalama yapıcısını çağırır
```

```
Nokta n2;
n2 = n1;      //Atama işlecini çağırır. Çünkü \n2 zaten vardır, sadece değeri değiş
    tirilmektedir.
```

§2.11.3 Tekli ve İkili İşleçlerin Çok Yüklenmesi

İşleçler genel olarak iki gruba ayrılabilir:

- ◇ **Tekli işleçler**: Tek işlenen (*operand*) üzerinde çalışır. Örneğin: `-x`, `x++`
- ◇ **İkili işleçler**: İki işlenen (*operand*) üzerinde çalışır. Örneğin: `x + y`, `x == y`

```
class SayiKutusu {
private:
  int deger;

public:
  // Yapıcı
  SayiKutusu(int deger) : deger(deger) {}

  // Tekli eksi işleci
  SayiKutusu operator-() const {
    return SayiKutusu(-this->deger);
  }

  // Ikili toplama işleci
  SayiKutusu operator+(const SayiKutusu& diger) const {
    return SayiKutusu(this->deger + diger.deger);
  }

  // x++ işleci
  // Sondan artirma / post-increment
  SayiKutusu operator++(int) {
    SayiKutusu eski = *this;
    this->deger++;
    return eski;
  }

  // --x işleci
  // Onden azaltma / pre-decrement
  SayiKutusu& operator--() {
    this->deger--;
    return *this;
  }

  void yazdir() const {
    std::cout << deger << std::endl;
  }

  int degerGetir() const {
    return deger;
  }
};

int main() {
  SayiKutusu kutu1(10);
  SayiKutusu kutu2(25);

  // Tekli işleç kullanimi
  SayiKutusu negatifKutu = -kutu1;

  // Ikili işleç kullanimi
  SayiKutusu toplamKutu = kutu1 + kutu2;

  std::cout << "kutu1: ";
  kutu1.yazdir();
```

```
  std::cout << "kutu2: ";
  kutu2.yazdir();

  std::cout << "-kutu1: ";
  negatifKutu.yazdir();

  std::cout << "kutu1 + kutu2: ";
  toplamKutu.yazdir();

  SayiKutusu x(10);

  std::cout << "Baslangic x: ";
  x.yazdir();

  SayiKutusu eski = x++;

  std::cout << "x++ sonucunda donen eski deger: ";
  eski.yazdir();

  std::cout << "x++ sonrasi x: ";
  x.yazdir();

  SayiKutusu yeni = --x;

  std::cout << "--x sonucunda donen yeni deger: ";
  yeni.yazdir();

  std::cout << "--x sonrasi x: ";
  x.yazdir();

  return 0;
}
/* Çıktı:
kutu1: 10
kutu2: 25
-kutu1: -10
kutu1 + kutu2: 35
Baslangic x: 10
x++ sonucunda donen eski deger: 10
x++ sonrasi x: 11
--x sonucunda donen yeni deger: 10
--x sonrasi x: 10
*/
```

2.12 Kalıtım Kavramı

Kalıtım (*inheritance*), bir sınıfın başka bir sınıfın özelliklerini ve davranışlarını devralmasıdır. Sınıflar arasında **bir türüdür** (*is a*) ilişkisi kurar ve kod tekrarını önler. **Değişim yönetimini** kolayca yapma üstünlüğünü verir.

Kalıtımın İki Önemli Özelliği

1. **Paylaşılan bir ortak kod** (*shared code*) vardır.
2. Paylaşılan **kodda yapılan değişiklik anında alt sınıflara yansır** (*snap change*).

§2.12.1 Ata ve Çocuk Sınıflar

Öğrenci, öğretmen ve müdürün olduğu bir okul sistemini örnek alarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak ve her bir sınıfta adi, cinsiyet gibi durumları tekrar tanımlamak gerekir.

```
class Ogrenci {
private:
```

```
  unsigned yas;
  char cinsiyet;
  unsigned ogrenciNo;
public:
  Ogrenci() {
    yas=18u;
    cinsiyet='E';
    ogrenciNo=0u;
  }
  void setYas(unsigned yeniYas) {
    if (yeniYas <18)
    std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
    else
    yas=yeniYas;
  }
  unsigned getYas() {
    return yas;
  }
  void setCinsiyet(char yeniCinsiyet) {
    if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' ||
        yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
    cinsiyet=yeniCinsiyet;
    else
    std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
  }
  char getCinsiyet() {
    return cinsiyet;
  }
  void yasiniSoyle() {
    std::cout << "Yaşım:" << yas << std::endl;
  }
  void cinsiyetSoyle() {
    if (cinsiyet=='E' || cinsiyet=='e')
    std::cout << "Cinsiyetim: ERKEK" << std::endl;
    else if (cinsiyet=='K' || cinsiyet=='k')
    std::cout << "Cinsiyetim: KADIN" << std::endl;
    else
    std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
  }
  void dersCalis() {
    std::cout << "Ders Çalışıyorum..." << std::endl;
  }
};

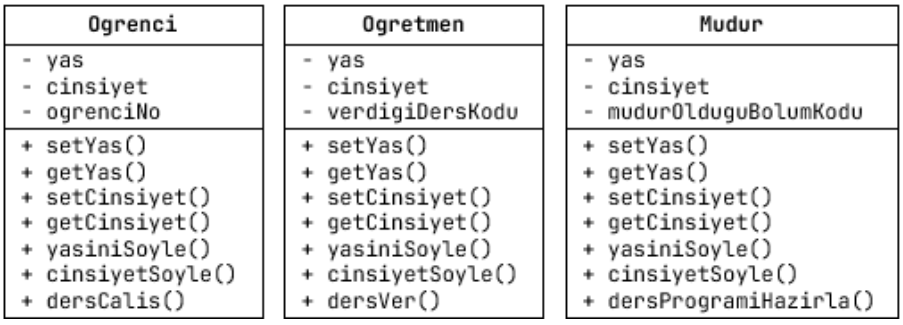
class Ogretmen {
  private:
  unsigned yas;
  char cinsiyet;
  unsigned verdigiDersKodu;
  public:
  Ogretmen() {
    yas=18u;
    cinsiyet='E';
    verdigiDersKodu=0u;
  }
  void setYas(unsigned yeniYas) {
    if (yeniYas <18)
    std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
    else
    yas=yeniYas;
  }
  unsigned getYas() {
    return yas;
  }
  void setCinsiyet(char yeniCinsiyet) {
    if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' ||
```

```
        yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
    cinsiyet=yeniCinsiyet;
    else
    std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
  }
  char getCinsiyet() {
    return cinsiyet;
  }
  void yasiniSoyle() {
    std::cout << "Yaşım:" << yas << std::endl;
  }
  void cinsiyetSoyle() {
    if (cinsiyet=='E' || cinsiyet=='e')
    std::cout << "Cinsiyetim: ERKEK" << std::endl;
    else if (cinsiyet=='K' || cinsiyet=='k')
    std::cout << "Cinsiyetim: KADIN" << std::endl;
    else
    std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
  }
  void dersVer() {
    std::cout << "Ders Veriyorum..." << std::endl;
  }
};

class Mudur {
  private:
  unsigned yas;
  char cinsiyet;
  unsigned muduruOlduguBolumKodu;
  public:
  Mudur() {
    yas=18u;
    cinsiyet='E';
    muduruOlduguBolumKodu=0u;
  }
  void setYas(unsigned yeniYas) {
    if (yeniYas <18)
    std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
    else
    yas=yeniYas;
  }
  unsigned getYas() {
    return yas;
  }
  void setCinsiyet(char yeniCinsiyet) {
    if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' ||
        yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
    cinsiyet=yeniCinsiyet;
    else
    std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
  }
  char getCinsiyet() {
    return cinsiyet;
  }
  void yasiniSoyle() {
    std::cout << "Yaşım:" << yas << std::endl;
  }
  void cinsiyetSoyle() {
    if (cinsiyet=='E' || cinsiyet=='e')
    std::cout << "Cinsiyetim: ERKEK" << std::endl;
    else if (cinsiyet=='K' || cinsiyet=='k')
    std::cout << "Cinsiyetim: KADIN" << std::endl;
    else
    std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
  }
  void dersProgramiHazirla() {
```

```
    std::cout << "Ders Programı Hazırlıyorum..." << std::endl;
  }
};
```

Bu sınıfların her birinde `yas` ve `cinsiyet` gibi ortak özellikler (*property*) veya her sınıfta aynı kodu içerecek şekilde `yasiniSoyle()` ve `cinsiyetSoyle()` gibi ortak davranışlar (*behavior*) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar.



Şekil 2.2: Okul Projesinde Sınıflar

Örnekteki gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir. Böylece proje zamanı kısalır ve daha yönetilebilir bir kod elde edilir. Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına **kalıtım** (*inheritance*) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar `Kisi` sınıfına toplanarak kalıtım uygulanmış şekilde kod yeniden düzenlenebilir.

```
class Kisi {
private:
  unsigned yas;
  char cinsiyet;
public:
  Kisi() {
    yas=18u;
    cinsiyet='E';
  }
  void setYas(unsigned yeniYas) {
    if (yeniYas <18)
      std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
    else
      yas=yeniYas;
  }
  unsigned getYas() {
    return yas;
  }
  void setCinsiyet(char yeniCinsiyet) {
    if (yeniCinsiyet == 'E' || yeniCinsiyet == 'e' || yeniCinsiyet == 'K' ||
        yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
      cinsiyet=yeniCinsiyet;
    else
      std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
  }
  char getCinsiyet() {
    return cinsiyet;
  }
  void yasiniSoyle() {
    std::cout << "Yaşım:" << yas << endl;
  }
  void cinsiyetSoyle() {
    if (cinsiyet=='E' || cinsiyet=='e')
      std::cout << "Cinsiyetim: ERKEK" << std::endl;
    else if (cinsiyet=='K' || cinsiyet=='k')
      std::cout << "Cinsiyetim: KADIN" << std::endl;
    else
      std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
```

```
  }
};

class Ogrenci: public Kisi { //Kisi sınıfından türemiş Ogrenci sınıfı
private:
  unsigned ogrenciNo;
public:
  Ogrenci() {
    ogrenciNo=0u;
  }
  void dersCalis() {
    std::cout << "Ders Çalışıyorum..." << std::endl;
  }
};

class Ogretmen:public Kisi { //Kisi sınıfından türemiş Ogretmen sınıfı
private:
  unsigned verdigiDersKodu;
public:
  Ogretmen() {
    verdigiDersKodu=0u;
  }
  void dersVer() {
    std::cout << "Ders Veriyorum..." << std::endl;
  }
};

class Mudur:public Kisi { //Kisi sınıfından türemiş Mudur sınıfı
private:
  unsigned muduruOlduguBolumKodu;
public:
  Mudur() {
    muduruOlduguBolumKodu=0u;
  }
  void dersProgramiHazirla() {
    std::cout << "Ders Programı Hazırlıyorum..." << std::endl;
  }
};
```

Kalıtımda özellikleri devralınan sınıf olan `Kişi` sınıfına **genel sınıf** (*general class*), **taban sınıf** (*base class*) veya **ata sınıf** (*parent class*) olarak adlandırılır. Daha sonra tanımlanacak `Ogrenci`, `Ogretmen` ve `Mudur` sınıflarının ortak özellik ve davranışlarını barındırır.

Ortak özellik ve davranışları devralan ve `Kisi` sınıfından türemiş (*derived*) `Ogrenci`, `Ogretmen` ve `Mudur` sınıfları `Kisi` sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. **Özel sınıf** (*private class*), **türemiş sınıf** (*derived class*) veya **çocuk sınıf** (*child class*) olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

§2.12.2 UML Sınıf Diyagramlarında Kalıtımın Gösterilmesi

UML sınıf diyagramlarında kalıtım, çocuk sınıftan ata sınıfa doğru uzanan boş üçgen uçlu bir ok ile gösterilir.

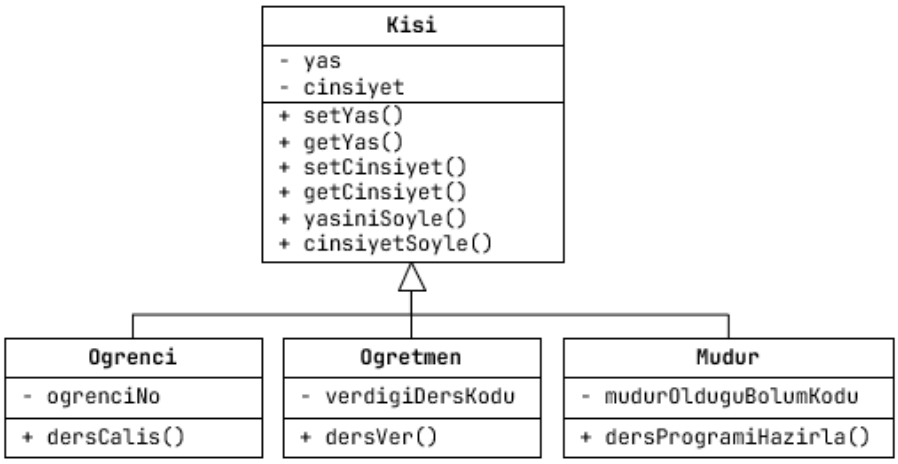
Bu gösterim, `Ogrenci`, `Ogretmen` ve `Mudur` sınıflarının `Kisi` sınıfından türediğini ifade eder.

§2.12.3 Kalıtımda Yapıcı ve Yıkıcılar

Çocuk sınıftan bir nesne üretildiğinde önce ata sınıfın yapıcısı, sonra çocuk sınıfın yapıcısı çalışır. Nesne yok edilirken ise bu sıra tersine döner. Önce çocuk sınıfın yıkıcısı, sonra ata sınıfın yıkıcısı çalışır.

```
#include <iostream>

class Ata {
```



Şekil 2.3: Katitim Uygulanmış Okul Projesi Sınıfları

```
public:
  Ata() {
    std::cout << "Ata yapici calisti" << std::endl;
  }

  ~Ata() {
    std::cout << "Ata yikici calisti" << std::endl;
  }
};

class Cocuk : public Ata {
  public:
  Cocuk() {
    std::cout << "Cocuk yapici calisti" << std::endl;
  }

  ~Cocuk() {
    std::cout << "Cocuk yikici calisti" << std::endl;
  }
};

int main() {
  Cocuk* nesne = new Cocuk();

  delete nesne;

  return 0;
}
/* Çıktı:
Ata yapici calisti
Cocuk yapici calisti
Cocuk yikici calisti
Ata yikici calisti
*/
```

Ata sınıf üzerinden çocuk sınıf nesnesi silinecekse, taban sınıfın yıkıcısı sanal (*virtual*) olarak tanımlanmalıdır:

```
#include <iostream>

class Ata {
  public:
  Ata() {
    std::cout << "Ata yapici calisti" << std::endl;
  }
```

```
  virtual ~Ata() = default; //VIRTUAL DESTRUCTOR!

};

class Cocuk : public Ata {
  public:
  Cocuk() {
    std::cout << "Cocuk yapici calisti" << std::endl;
  }

  ~Cocuk() {
    std::cout << "Cocuk yikici calisti" << std::endl;
  }
};

int main() {
  Ata* nesne2 = new Cocuk();
  delete nesne2;

  return 0;
}
/* Çıktı:
Ata yapici calisti
Cocuk yapici calisti
Cocuk yikici calisti
*/
```

eğer `Ata` sınıfının yıkıcısı sanal (`virtual`) değilse, `Cocuk` sınıfının yıkıcısı çağrılmayabilir.

J

§2.12.4 Çocuk Sınıftan Nesne Üretilmesi

Kod 2.12.1 ile verilen kalıtım örneğindeki Çocuk sınıflardan nesne üretildiğinde, nesne hem ata sınıftan gelen özelliklere hem de kendi tanımladığı özelliklere sahip olur.

```
int main() {
  Ogrenci ilhan;
  Ogretmen mehmet;
  Mudur abdullah;

  ilhan.yasiniSoyle();
  ilhan.cinsiyetSoyle();
  ilhan.dersCalis();

  mehmet.yasiniSoyle();
  mehmet.cinsiyetSoyle();
  mehmet.dersVer();

  abdullah.yasiniSoyle();
  abdullah.cinsiyetSoyle();
  abdullah.dersProgramiHazirla();
}
```

Bu örnekte `ilhan` nesnesi, `Kisi` sınıfından gelen `yasiniSoyle()` yöntemini ve `Ogrenci` sınıfına ait `dersCalis()` yöntemini kullanabilir.

§2.12.5 Kalıtımda Erişim Belirleyiciler

Kalıtım sırasında ata sınıftaki üyelerin çocuk sınıfa nasıl aktarılacağı erişim belirleyicilere bağlıdır. Genel mantık özet olarak aşağıdaki tabloda verilmiştir:

```
class Kisi {
private:
  std::string kimlikNo;
```

Üyelere Erişim	public	protected	private
Aynı Sınıf İçinde	var	var	var
Türemiş Sınıfta	var	var	yok
Sınıf Dışından	var	yok	yok

Tablo 2.1: Kalıtımda Sınıf Üyelerine Erişim

```
protected:
  std::string ad;

public:
  int yas;
};

class Ogrenci : public Kisi {
public:
  void bilgiGuncelle() {
    // this->kimlikNo = "12345678901";
    // HATA: kimlikNo private olduğu için erişilemez

    this->ad = "Ali";   // protected olduğu için erişilebilir
    this->yas = 20;     // public olduğu için erişilebilir
  }

  void bilgiYazdir() {
    std::cout << "Ad: " << this->ad << std::endl;
    std::cout << "Yas: " << this->yas << std::endl;
  }
};

int main() {
  Ogrenci ogrenci;

  ogrenci.bilgiGuncelle();
  ogrenci.bilgiYazdir();

  // ogrenci.kimlikNo = "123";
  // HATA: private

  // ogrenci.ad = "Veli";
  // HATA: protected, sınıf dışından erişilemez

  ogrenci.yas = 25;
  // public olduğu için sınıf dışından erişilebilir

  std::cout << "Yeni yas: " << ogrenci.yas << std::endl;

  return 0;
}
/* Çıktı:
Ad: Ali
Yas: 20
Yeni yas: 25
*/
```

türemiş sınıflar, taban sınıftan mirası üç farklı şekilde alabilirler;

1. **Umuma açık kalıtım** (*public inheritance*): Taban sınıfın umuma açık üyelerini türetilmiş sınıfta umuma açık hale getirir ve taban sınıfın korumalı (protected) üyeleri de türetilmiş sınıfta korumalı olarak kalır.
2. **Mahrem kalıtım** (*private inheritance*): Taban sınıfın umuma açık ve korumalı üyelerini türetilmiş sınıfta mahrem hale getirir.

3. **Korumalı kalıtım** (*protected inheritance*): Taban sınıfın umuma açık ve korumalı üyelerini türetilmiş sınıfta korumalı hale getirir.

Aşağıda bu durumu açıklayan sınıf örneği verilmiştir;

```
class Base { //taban sınıf
public:
  /* Burada tanımlanan üyeler umuma açıktır. Yani bu üyelere sınıf dışı ve içinden eriş
     ilebilir. */
  int x;
protected:
  /* burada tanımlı üyelere, üye fonksiyonları ve türeyen sınıftan türemiş sınıflardan
     erişilir. Bir başka deyişle miras alan sınıflar tarafından burada tanımlanan ü
     yelere erişilir. */
  int y;
private:
  /* burada tanımlı üyelere,  üye fonksiyonları (ve arkadaşlar) erişir.*/
  int z;
};

class PublicDerived: public Base { //public inheritance
  // Bu sınıfta x alanı(field), public olmuştur.
  // Bu sınıfta y alanı(field), protected olmuştur.
  // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Public-Derived
};

class PrivateDerived: private Base { //private inheritance
  // Bu sınıfta x alanı(field), private olmuştur.
  // Bu sınıfta y alanı(field), private olmuştur.
  // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Private-Derived
};

class ProtectedDerived: protected Base { //protected inheritance
  // Bu sınıfta x alanı(field), protected olmuştur.
  // Bu sınıfta y alanı(field), protected olmuştur.
  // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Protected-Derived
};
```

§2.12.6 Çoklu Kalıtım

Çoklu kalıtım, bir sınıfın birden fazla ata sınıftan türemesidir. Aşağıdaki örnekte hem `Yazici` hem de `Tarayici` sınıfından özellik ve davranışları devralan `CokFonksiyonluCihaz` sınıfı yer almaktadır:

```
#include <iostream>

class Yazici {
  public:
  void yazdir() {
    std::cout << "Yazdiriliyor..." << std::endl;
  }
};

class Tarayici {
  public:
  void tara() {
    std::cout << "Taranıyor..." << std::endl;
  }
};

class CokFonksiyonluCihaz: public Yazici, public Tarayici {
  public:
  void faksGonder() {
    std::cout << "Faks gonderiliyor..." << std::endl;
  }
};
```

```
int main() {
  CokFonksiyonluCihaz cf;
  cf.yazdir();
  cf.tara();
  cf.faksGonder();

  return 0;
}
```

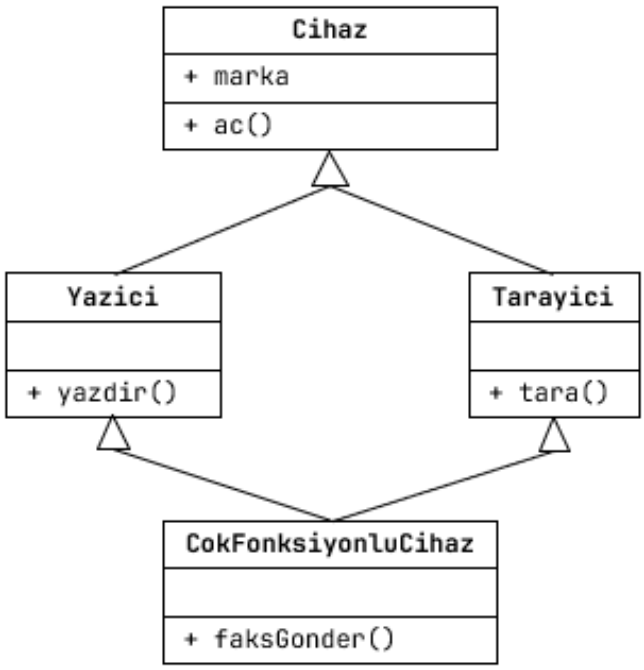
Modern nesne yönelimli diller olan Java ve C# dillerinde, **çoklu kalıtım** (*multiple inheritance*) sınıf tasarımında kullanılmaz! Çünkü mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından belirsizlik ve karmaşıklık oluşur.

Çoklu Kalıtım

de bir türemiş sınıfın birden fazla ata sınıftan miras alması istenmez. Ancak roller yani **arayüzler** de sınıf olarak kodlandığından sanki çoklu kalıtım varmış gibi algılanmaktadır. Java ve C# gibi nesne yönelimli dillerde bu arayüzler `interface` anahtar kelimesiyle tanımlandığından **birden çok sınıftan miras alma engellenmiştir**.

§2.12.7 Elmas Problemi

Çoklu kalıtımda ortaya çıkabilecek önemli sorunlardan biri **elmas problemidir** (*diamond problem*). Bu problem, iki farklı ata sınıfın aynı üst sınıftan türemesi ve çocuk sınıfın bu iki sınıftan birlikte türemesi durumunda ortaya çıkar.



Şekil 2.4: Çoklu Kalıtımda Elmas Problemi

Aşağıda bu sınıfların kodlaması verilmiştir:

```
#include <iostream>

class Cihaz {
  public:
  std::string marka;

  void ac() {
    std::cout << "Cihaz aciliyor..." << std::endl;
  }
```

```
};

class Yazici : public Cihaz {
  public:
  void yazdir() {
    std::cout << "Yazdiriliyor..." << std::endl;
  }
};

class Tarayici : public Cihaz {
  public:
  void tara() {
    std::cout << "Taranıyor..." << std::endl;
  }
};

class CokFonksiyonluCihaz : public Yazici, public Tarayici {
  public:
  void faksGonder() {
    std::cout << "Faks gonderiliyor..." << std::endl;
  }
};

int main() {
  CokFonksiyonluCihaz cf;

  cf.yazdir();
  cf.tara();
  cf.faksGonder();

  // cf.ac(); // HATA: ac() belirsizdir!
  // Yazici tarafindaki Cihaz::ac mi?
  // Tarayici tarafindaki Cihaz::ac mi?

  return 0;
}
```

Burada `cf.ac();` satırı hata verir. Çünkü `CokFonksiyonluCihaz` içinde iki tane `Cihaz` alt nesnesi vardır. Dolayısıyla derleyici, `cf.Yazici::ac();` mı çağıracak? veya `cf.Tarayici::ac();` mı çağıracak? bilemez. Belirsizliği elle çözmek için hangi yoldaki `Cihaz` metodunu çağıracağını elle belirtilmelidir.

```
int main() {
  CokFonksiyonluCihaz cf;

  cf.Yazici::ac(); // Yazici sınıfındaki ac() yöntemini kullan!
  cf.Tarayici::ac(); // Tarayici sınıfındaki ac() yöntemini kullan!

  return 0;
}
```

Bu çalışır. Ama hâlâ iki ayrı Cihaz kopyası vardır. Örneğin:

```
cf.Yazici::marka = "HP";
cf.Tarayici::marka = "Canon";

std::cout << cf.Yazici::marka << std::endl;
std::cout << cf.Tarayici::marka << std::endl;
/* Çıktı:
HP
Canon
*/
```

Bu da `CokFonksiyonluCihaz` içinde iki ayrı marka alanı olduğunu gösterir. C++'ta elmas problemi genellikle **sanal kalıtım** (*virtual inheritance*) ile çözülür.

§2.12.8 Sanal Kalıtım

Elmas problemini çözmek için **sanal** (*virtual*) ata sınıf kullanılır. Sanal ata sınıf, ortak ata sınıfın yalnızca bir kez devralınmasını sağlar.

```
class Cihaz {
public:
  std::string marka;

  void ac() {
    std::cout << "Cihaz aciliyor..." << std::endl;
  }
};

class Yazici : virtual public Cihaz  //SANAL KALITIM
{
public:
  void yazdir() {
    std::cout << "Yazdiriliyor..." << std::endl;
  }
};

class Tarayici : virtual public Cihaz //SANAL KALITIM
{
public:
  void tara() {
    std::cout << "Taranıyor..." << std::endl;
  }
};

class CokFonksiyonluCihaz : public Yazici, public Tarayici {
public:
  void faksGonder() {
    std::cout << "Faks gonderiliyor..." << std::endl;
  }
};

int main() {
  CokFonksiyonluCihaz cf;

  cf.marka = "HP";
  cf.ac();

  cf.yazdir();
  cf.tara();
  cf.faksGonder();

  std::cout << "Marka: " << cf.marka << std::endl;

  return 0;
}
/* Çıktı:
Cihaz aciliyor...
Yazdiriliyor...
Taranıyor...
Faks gonderiliyor...
Marka: HP
*/
```

Burada `cf.ac();` kodu sorunsuz çalışır. Çünkü `CokFonksiyonluCihaz` içinde yalnızca bir tane `Cihaz` nesnesi vardır. Kısaca sanal kalıtım kullanıldığında ortak temel sınıfının nesnesini, en alttaki sınıf yani `CokFonksiyonluCihaz` oluşturur.

Çoklu kalıtımda bir başka problem ortaya çıkabilir. Örneğin hem `Yazici` hem `Tarayici` içinde aynı isimde bir yöntem (mesela `durum()`) olsaydı. Bu durumda `cf.durum();` yöntemi çağrılırsa hata

alınacaktı. Bunu önlemek için aşağıdaki düzelteme yapılabilir.

```
class CokFonksiyonluCihaz : public Yazici, public Tarayici {
  public:
  void durum() {
    Yazici::durum();
    Tarayici::durum();
  }
};
```

2.13 Arkadaş Fonksiyon ve Sınıflar

C++ gibi bazı programlama dillerinde **arkadaş fonksiyon** (*friend function*), bir sınıfa ait olmamasına rağmen o sınıfın özel (*private*) ve korumalı (*protected*) üyelerine erişim izni verilen fonksiyondur. Arkadaş fonksiyonlar dikkatli kullanılmalıdır. Çünkü veri gizleme ilkesini zayıflatabilirler. Arkadaş yöntemlerin bildirimleri (*declaration*), sınıf içerisinde yer almasına rağmen, sınıfın üyesi sayılmazlar.

```
#include <iostream>

class Karmasik {
private:
  float gercek, sanal;  //mahrem üyeler

public:
  Karmasik(float pGercek=0.0, float pSanal=0.0) {
    this->gercek=pGercek;
    this->sanal=pSanal;
  }
  friend void yaz(Karmasik pKarmasik); //ARKADAS FONKSIYON BİLDİRİMİ
};

void yaz(Karmasik pKarmasik) {
  std::cout << pKarmasik.gercek << "+" <<pKarmasik.sanal <<"i" << std::endl;
}

int main(void) {
  Karmasik c1(1.0,2.0);
  yaz(c1); // 1+2i
  Karmasik c2(3.0,5.0);
  yaz(c2); // 3+5i
}
```

Benzer şekilde, akış (*stream*) sınıfı işleçleri çok yüklemesi buna örnek verilebilir. Nesne verilerinin akıştan çıkarılmasında **ayıklama işleci** (`extraction operator`) (`>>`) kullanılabilir. Benzer şekilde nesne verilerinin akışlara göndermek için **ekleme işleci** (`insertion operator`) (`<<`) kullanılabilir.

```
#include <iostream>

class Karmasik {
  float gercek, sanal; // Varsayılan olarak private
public:
  // Parametreli yapıcı (Modern ilklendirme listesi kullanımı)
  Karmasik(float pGercek = 0.0f, float pSanal = 0.0f) : gercek(pGercek), sanal(pSanal)
      {}

  // Ekleme (Output Stream) İşleci
  friend std::ostream& operator<<(std::ostream& output, const Karmasik& pKarmasik) {
    output << pKarmasik.gercek << "+" << pKarmasik.sanal << "i";
    return output;
  }

  // Ayıklama (Input Stream) İşleci
  friend std::istream& operator>>(std::istream& input, Karmasik& pKarmasik) {
```

```
    // Not: Ayıklama işlecinde nesne const olamaz çünkü değerlerini değiştireceğiz.
    input >> pKarmasik.gercek >> pKarmasik.sanal;
    return input;
  }
};

int main(void) {
  Karmasik c1(1.0f, 2.0f);
  std::cout << "c1 nesnesi: " << c1 << std::endl; // 1+2i
  Karmasik c2;
  std::cout << "Lutfen gercek ve sanal kismi giriniz (orn: 3.5 4.2): ";
  std::cin >> c2; // Kullanıcıdan veri alma
  std::cout << "Girilen c2 nesnesi: " << c2 << std::endl;

  return 0;
}
```

Ayrıca **arkadaş sınıf** (*friend class*), başka bir sınıfın özel ve korumalı üyelerine erişim izni verilen sınıftır. Bu özellik güçlüdür ancak kapsüllemeyi zayıflatabileceği için kontrollü kullanılmalıdır. Aşağıda `Karmasik` sınıfına arkadaş olan `KarmasikIslemleri` sınıfı örneği verilmiştir.

```
#include <iostream>

class Karmasik {
  private:
  float gercek;
  float sanal;

  public:
  Karmasik(float pGercek = 0.0f, float pSanal = 0.0f)
  : gercek(pGercek), sanal(pSanal) {}

  friend class KarmasikYazici;
};

class KarmasikYazici {
  public:
  void yazdir(const Karmasik& sayi) {
    std::cout << sayi.gercek << "+" << sayi.sanal << "i" << std::endl;
  }

  void detayliYazdir(const Karmasik& sayi) {
    std::cout << "Gercek kisim: " << sayi.gercek << std::endl;
    std::cout << "Sanal kisim : " << sayi.sanal << std::endl;
  }
};

int main() {
  Karmasik c1(3.5f, 2.5f);

  KarmasikYazici yazici;

  std::cout << "Karmasik sayi: ";
  yazici.yazdir(c1);

  std::cout << "Detayli gosterim:" << std::endl;
  yazici.detayliYazdir(c1);

  return 0;
}
/* Cıktı:
Karmasik sayi: 3.5+2.5i
Detayli gosterim:
Gercek kisim: 3.5
Sanal kisim : 2.5
*/
```

Bu örnekte kritik satır `friend class KarmasikYazici;` satırıdır. Bu ifade sayesinde `KarmasikYazici` sınıfının tüm üye fonksiyonları, `Karmasik` sınıfının `private` üyeleri olan `float gercek;` ve `float sanal;` alanlarına doğrudan erişebilir.

Arkadaş Kullanımı

`friend` ile "bu özel sınıfa içeriye girme izni veriyorum" der. C++'ta `friend` vardır çünkü:
- ◇ Daha düşük seviyeli kontrol sağlar,
- ◇ İşleç aşırı yükleme gibi durumlarda kullanışlıdır,
- ◇ Sınıflar arası özel erişime izin verir.

2.14 Soyut ve Somut Sınıf

Soyut sınıfın (*abstract class*) bir örneği (*instance*) olamaz. Yani bu sınıftan bir nesne imal edilemez. Bu sınıfın en az bir saf sanal yöntemi olur. Saf sanal yöntemler (*pure virtual method*), hangi davranışın gösterileceğini belirler ama bu davranışların nasıl gösterileceğini anlatmaz. Saf sanal yöntem için gövde tanımlanmaz ve sonunda `= 0;` bulunur. Bu yöntemler, soyut yöntem (*abstract method*) olarak da adlandırılır.

```
#include <iostream>
#include <string>

class SoyutKisi { //Soyut Kisi Sınıfı
public:
  virtual void raporYaz() = 0;  // Bu soyut yöntem, bu sınıfı soyut yapar.

  std::string getAdi() {
    return adi;
  }
  void setAdi(std::string pAdi) {
    if (pAdi!="") this->adi=pAdi;
  }

  SoyutKisi(std::string pAdi): adi(pAdi) { }
private:
  std::string adi;
};

int main() {
  // SoyutKisi soyutkisi; //HATA: Soyut sınıftan nene imal edilemez!
}
```

Somut sınıf (*concrete class*),nesne imal edilebilir yani örneği (*instance*) olabilir. Bu sınıflarda davranış ve durumlar gövdesiyle eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
#include <iostream>
#include <string>

class SoyutKisi { //Soyut Kisi Sınıfı
public:
  virtual void raporYaz() = 0;  // Bu saf sanal yöntem sınıfı soyut yapar.
  std::string getAdi() {
    return adi;
  }
  void setAdi(std::string pAdi) {
    if (pAdi!="") this->adi=pAdi;
  }
  SoyutKisi(std::string pAdi): adi(pAdi) { }
private:
```

```
  std::string adi;
};

class SomutKisi: public SoyutKisi { // Ortak davranışlar SoyutKisi sınıfından devralını
    yor.
public:
  void raporYaz() override {  // sanal yöntem geçersiz kılınır.
    std::cout << getAdi() << ": Rapor Yazıyor..." << std::endl;
  }
  // Bu sınıfta sanal yöntem olmadığından nesne imal edilebilir.
  SomutKisi (std::string pAdi):SoyutKisi(pAdi) { }
};

int main() {
  SomutKisi ilhan("Ilhan");
  ilhan.raporYaz(); // "Ilhan: Rapor Yazıyor..."
}
```

2.15 Statik Sınıf Üyeleri

Çoğu durumda program çalışmak üzere belleğe yüklendiğinde doğrudan icra edilebilek yönteme ve bir veriye veya nesneye ihtiyaç duyarız. İşte bu veri ve yöntemler **statik** (*static*) olarak tanımlanır. Bu yüzden statik yöntemler, program çalışmaya başladığında var olur ve program bitinceye kadar erişilebilirdirler. Ayrıca yalnızca sınıfın diğer statik üyeleri veya başka statik fonksiyonlarla çalışabilirler.

C++ Dilinde **statik yöntem** (*static method*), imal edilecek herhangi bir nesneye değil, sınıfın kendisine ait olan özel bir yöntem türüdür. Bu yöntemler, sınıfın bir örneği olan nesne imal edilmesine gerek kalmadan, sadece sınıf kimliğini kullanarak doğrudan çağrılabilirler. Bu tür yöntemlere **yardımcı yöntem** (*utility method*) adı da verilir.

Statik yöntemlere benzer şekilde statik alanlar (*static field*) da tanımlanabilir. Statik olan bu veriler veri bellekte (*data segment*) saklanırlar ve statik yöntemler gibi kullanılır. Statik alanlar genellikle bir sınıfın tüm örnekleri arasında paylaşılması gereken kaynakları veya sayaçları yönetmek için kullanılır. Statik bir alana sınıf içerisinde veya yapısında ilk değer veremeyiz! Bu nedenle sınıf dışında veri tipi belirterek başlatmalıyız!

```
#include <iostream>

class YardimciSinif {
public:
  static int genelSayac;
  static void programciAdi() {
    std::cout << "Programcı Adı: Ilhan OZKAN" << std::endl;
    std::cout << genelSayac << std::endl;
  }
};

/* statik sınıf alanını sınıf sışında veri tipi belirterek başlatma */
int YardimciSinif::genelSayac=100;

int main(void) {
  YardimciSinif::programciAdi(); /* static üye yöntemi çağırma */
  YardimciSinif::genelSayac--; /* static üye alana erişim */
  YardimciSinif::programciAdi();
}
```

İleride anlatılacak olan **tasarım desenlerinden** (*design pattern*) **tekil nesne deseni** (*singleton pattern*) statik üyelere örnek olarak verilebilir. Bu desende sınıftan yalnızca bir nesne/örnek imal edilen ve bu tekil nesneye evrensel erişim sağlayan bir tasarım desenidir. Burada statik üyeler, sınıfın tek bir paylaşılan örneğinin bakımına izin verdikleri için bu deseni uygulamak için mükemmeldir.

2.16 const Sınıf Üyeleri

Sınıfların `const` **yöntemleri** (*const method*), veri üyelerinin yani durumları (*state*) tutan alanları değiştirme izni verilmeyen fonksiyonlardır.

Üye yöntemler gibi bir sınıfın nesneleri de `const` olarak bildirilebilir. `const` olarak bildirilen bir nesne değiştirilemez ve bu nedenle, bu fonksiyonlar nesneyi değiştirmemeyi garantilediğinden, yalnızca `const` üye yöntemlerini çağırabilir.

Bir `const` nesnesi **değiştirilemez** (*immutable*) nesne olarak adlandırılır. `const` nesnelerinin veri üyesini değiştirmeye yönelik herhangi bir girişim, derleme zamanı hatasıyla (*compile-time error*) sonuçlanır.

```
#include <iostream>

class Sinif {
private:
  int alan;
public:
  Sinif() {
    alan=-1;
  }
  int getAlan() const {
    return alan;
  }
  void setAlan(int pAlan) { // Bu yöntem const tanımlanırsa hata alınır!
    alan=pAlan;  // çünkü alan değişkenini değişitir!
  }
};

int main() {
  Sinif* nesne=new Sinif();
  std::cout<<"nesne.alan:" << nesne->getAlan() << std::endl;
  nesne->setAlan(12);
  std::cout<<"nesne.alan:" << nesne->getAlan() << std::endl;
  delete nesne;
  const Sinif* constNesne=new Sinif();
  //constNesne->setAlan(9); // Hata: const nesne (immutable object) değiştirilemez!
  delete constNesne;
}
```

2.17 mutable Depolama Sınıfı

Bazen, `const` ile sabit olarak tanımlanmış nesnenin bir veya daha fazla veri üyesini değiştirme gereksinimi doğabilir. İşte bu durumda bu üyeler `mutable` anahtar kelimesiyle tanımlanır.

```
#include <iostream>

class Kisi {
public:
  unsigned long tcno;
  mutable int okulno; // mutable alan
  Kisi(){
    tcno = 43233445505UL;
    okulno = -1;
  }
};

int main(){
  const Kisi ogrenci1; // ogrenci1 nesnesi const olarak tanımlandı
  ogrenci1.okulno = 2000; // sabit olan nesnenin okulno değiştiriliyor.
  std:: cout << ogrenci1.okulno << std::endl;
  //ogrenci1.tcno = -1; //HATA: sabit nesnenin alanı değiştirilemez.
  /* assignment of member 'Kisi::tcno' in read-only object*/
}
```

2.18 Geçersiz Kılma

Geçersiz kılma (*overriding*), nesne yönelimli programlamada ata sınıfta önceden tanımlanmış bir yöntemi çocuk sınıfta yeniden tanımlamasına olanak tanıyan bir kavramıdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir. C++ dili iki tür fonksiyon geçersiz kılmayı destekler; Birincisi derleme zamanı geçersiz kılma yani **gizleme**dir (*hiding*), diğeri ise çalışma zamanı **geçersiz kılma**dır (*overriding*).

Derleme zamanında geçersiz kılma aşağıdaki şekilde yapılır;

```
#include <iostream>

class Ata {
public:
  void yap() {
    std::cout << "Baba şu şekilde yapar ..." << std::endl;
  }
};

class Cocuk : public Ata {
public:
  void yap() { //HIDING
    std::cout << "Çocuk şu şekilde yapar..." << std::endl;
  }
};

int main() {
  Ata ata;
  ata.yap(); // Baba şu şekilde yapar ...
  Cocuk cocuk;
  cocuk.yap(); // Çocuk şu şekilde yapar...
}
```

Her iki sınıf ta da yöntemin kimliği aynıdır. Ata sınıftan imal edilen nesne ata sınıfın yöntemini, çocuk sınıftan imal edilen nesne ise çocuk sınıfın yöntemini icra eder. Çalışma zamanında geçersiz kılma aşağıdaki gibi yapılır;

```
#include <iostream>

class Ata {
  public:
  virtual void yap() { // Ata sınıfın yöntemi "virtual" olarak tanımlanır
    std::cout << "Baba şu şekilde yapar ..." << std::endl;
  }
};

class Cocuk : public Ata {
  public:
  void yap() override { // Ata sınıftaki yöntem geçersiz kılınır "override"!
    std::cout << "Çocuk şu şekilde yapar..." << std::endl;
  }
};

int main() {
  Cocuk cocuk;
  cocuk.yap(); // Çocuk şu şekilde yapar...

  Ata* ataPtr=&cocuk; //çocuk nesnesi, ata sınıf nesnesi gibi kullanılıyor
  ataPtr->yap(); // Çocuk şu şekilde yapar...

  Ata& ataPtr2=cocuk; //çocuk nesnesi, ata sınıf nesnesi gibi kullanılıyor
  ataPtr2.yap(); // Çocuk şu şekilde yapar...
}
```

Buradaki her iki sınıfta da yöntemin kimliği aynıdır. Geçersiz kılınacak yöntem, taban sınıfın yön-

temini türemiş sınıfta değiştirilebileceğini belirtmek için `virtual sıfatı` (*virtual specifier*) kullanılır. Türemiş sınıfta ise geçersiz kılacağı yöntemi `override` belirleyicisi (*override specifier*) ile geçersiz kılarak yeniden tanımlar.

2.19 Arayüzler

Arayüzler (*interface*) nesnelerin sahip oldukları rollerdir (*role*). Roller bir grup davranışı (*behavior*) barındırır. roller, sınıf olarak kodlanır. Ancak Java ve C# gibi saf nesne yönelimli dillerde arayüzler, `interface` saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav yapma, değerlendirme, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Arayüz Gerçekleştirme

Java, C# gibi nesne yönelimli dillerin aksine C++ Dilinde, **çoklu kalıtım** (*multiple inheritance*) desteklenir. Bir sınıfın arayüzleri miras almasına **arayüz gerçekleştirme** (*interface realization*) veya **arayüz uygulama** (*interface imlementation*) adı verilir. Java, C# gibi nesne yönelimli dillerde de birden fazla arayüzü bir sınıf gerçekleştirebilir.

Arayüzler, soyut (*abstract*) sınıflarla tanımlanır ve veri soyutlaması (*data abstraction*) yapılmaz. Bir sınıfın soyut olabilmesi için en az bir **saf sanal yönteme** (*pure virtual method*) sahip olmalıdır. Soyut sınıflardan nesne imal edilemez! Soyut sınıflardaki sanal yöntemler türeyen sınıfta yeniden tanımlanır ve mutlaka geçersiz kılınır (*override*).

```
#include <iostream>
#include <string>

class IBaba { //Baba Rolündeki davranışlar:
public:
  virtual void cocugaBak() = 0; // saf sanal yöntem-pure virtual method
  virtual void hanimaYardimEt() = 0; // saf sanal yöntem-pure virtual method
  /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınmıf olur. Bu
      nedenle bu sınıftan nesne imal edilemez! */
  virtual ~IBaba() = default;  //sanal yıkıcı
};

class IMuzisyen { //Müzisyen Rolündeki Davranışlar:
public:
  virtual void gitarCal() = 0; // saf sanal yöntem-pure virtual method
  virtual void piyanoCal() = 0; // saf sanal yöntem-pure virtual method
  /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınmıf olur. Bu
      nedenle bu sınıftan nesne imal edilemez! */
  virtual ~IMuzisyen() = default;  //sanal yıkıcı
};

class IOgretmen { //Öğretmen Davranışları:
public:
  virtual void ogret() = 0; // saf sanal yöntem-pure virtual method
  virtual void dersNotuHazirla() = 0; // saf sanal yöntem-pure virtual method
  /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınmıf olur. Bu
      nedenle bu sınıftan nesne imal edilemez! */
  virtual ~IMuzisyen() = default;  //sanal yıkıcı
};

class Kisi:  public IBaba, public IMuzisyen, public IOgretmen {
  /* (interface realization/implementation): Gerçekleştirilen IBaba, IMuzisyen ve
      IOgretmen arayüzlerinde saf sanal yöntemler olduğundan bu Kişi sınıfında hepsi ge
      çersiz kılınmalıdır (override). */
```

```
public:
  std::string adi;
  std::string soyadi;

  void adiniSoyle() {
    std::cout << adi << " " << soyadi << std::endl;
  }

  // IBaba aracılığıyla miras alınan davranışlar:
  void cocugaBak() override {
    std::cout << "Çocuğa Bakıyorum..." << std::endl;
  }
  void hanimaYardimEt() override {
    std::cout << "Hanıma Yardım Ediyorum..." << std::endl;
  }

  // IMuzisyen aracılığıyla miras alınan davranışlar:
  void gitarCal() override {
    std::cout << "Gitar Çalıyorum..." << std::endl;
  }
  void piyanoCal() override {
    std::cout << "Piyano Çalıyorum..." << std::endl;
  }

  // IOgretmen aracılığıyla miras alınan davranışlar:
  void ogret() override {
    std::cout << "Öğretiyorum..." << std::endl;
  }
  void dersNotuHazirla() override {
    std::cout << "Ders notu hazırlıyorum..." << std::endl;
  }
};

int main() {
  Kisi ilhan;
  ilhan.adi = "İlhan";
  ilhan.soyadi = "ÖZKAN";

  //imal edilen nesne tüm arayüzdeki davranışları gösterebilir:
  ilhan.adiniSoyle(); // İlhan ÖZKAN
  ilhan.ogret(); // Öğretiyorum...
  ilhan.gitarCal(); // Gitar Çalıyorum...
  ilhan.hanimaYardimEt(); // Hanıma Yardım Ediyorum...

  //imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
  IBaba& baba=ilhan;
  baba.cocugaBak(); // Çocuğa Bakıyorum...
  baba.hanimaYardimEt(); // Hanıma Yardım Ediyorum...

  IMuzisyen& muzisyen = ilhan;
  muzisyen.gitarCal(); // Gitar Çalıyorum...
  muzisyen.piyanoCal(); // Piyano Çalıyorum...

  IOgretmen& ogretmen = ilhan;
  ogretmen.dersNotuHazirla(); // Ders notu hazırlıyorum...
  ogretmen.ogret(); // Öğretiyorum...
}
```

2.20 Çok Biçimlilik

Çok biçimlilik (*polymorphism*), ata-çocuk ilişkisi olan birden çok nesnenin olduğu bir ortamda, **verilen aynı iletiye (*message*) karşı, nesnelerin farklı davranış (*behavior*) göstermeleridir** (*same message-different behavior*).

Çok Biçimliliğin Uygulanması

Çok biçimliliğin kodlamada uygulanabilmesi (*implementation*) için üç şey gereklidir;

1. **Kalıtım** (*inheritance*): Ortak davranışın tanımlandığı taban sınıf ve bu davranışın değiştiği türeyen sınıflar.
2. **Sanal Yöntem** (*virtual method*): Nesnelere aynı iletiyi verebilmek için ortak davranışı taban sınıfta tanımlayan sanal yönteme sahip olmalıdır.
3. **Geçersiz Kılma** (*override*): Türemiş sınıfta devralınan davranışı kendine uyarlayan ve taban sınıftaki sanal yöntemi geçersiz kılan yeni bir yöntem.

Şimdi çok biçimliliğe bir örnek verelim;

```
#include <iostream>
#include <string>

class SoyutKisi { //Soyut Kisi Sınıfı
public:
  virtual ~SoyutKisi() = default; //SANAL YIKICI!
  virtual void raporYaz() = 0;
  /* saf sanal yöntem: Bu yöntem, Kişi sınıfını SOYUT yapar. Ayrıca;  raporYaz(): Tü
      reyen sınıflardan imal edilecek nesnelere aynı mesajı vermek için tanımlanan
      ortak davranıştır. */
  std::string getAdi() {
    return adi;
  }
  void setAdi(std::string pAdi) {
    if (pAdi!="") this->adi=pAdi;
  }
  SoyutKisi(std::string pAdi): adi(pAdi) { }

private:
    std::string adi;
};

class Ogrenci: public SoyutKisi {  //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
  void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
    std::cout << getAdi() <<":Öğrenci Ödevilerine İlişkin Rapor Yazıyor..." << std::
        endl;
  }
  Ogrenci(std::string pOgrenciAdi):SoyutKisi(pOgrenciAdi) { }
};

class Ogretmen: public SoyutKisi {  //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
  void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
    std::cout << getAdi() <<":Ogretmen Verdiği Derslere İlişkin Rapor Yazıyor..."
    << std::endl;
  }
  Ogretmen(std::string pOgretmenAdi):SoyutKisi(pOgretmenAdi) {  }
};

class Mudur: public SoyutKisi {  //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
  void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
    std::cout << getAdi() <<":Müdür Akademik Takvime İlişkin Rapor Yazıyor..."
    << std::endl;
  }
  Mudur(std::string pMudurAdi):SoyutKisi(pMudurAdi) { }
};

int main() {
  /* Modern C++'ta çıplak diziler yerine std::vector ve smart pointer kullanılır. std::
      unique_ptr sayesinde 'delete' yazmanıza gerek kalmaz, bellek sızıntısı önlenir. İ
      leride anlatılacaktır. */
```

```
  SoyutKisi* kisi[3];
  kisi[0]=new Ogrenci("İlhan ÖZKAN");
  kisi[1]=new Ogretmen("Hasan YILMAZ");
  kisi[2]=new Mudur("Recep AY");
  for (int i=0; i<3; i++)
    kisi[i]->raporYaz();
  /*
  Her kişiye aynı "raporYaz()" mesajı veriliyor->
  Karşılığında farklı davranışlar sergileniyor.
  */
  delete kisi[0];
  delete kisi[1];
  delete kisi[2];
}
/* Program Çıktısı:
İlhan ÖZKAN:Öğrenci Ödevilerine İlişkin Rapor Yazıyor...
Hasan YILMAZ:Ogretmen Verdiği Derslere İlişkin Rapor Yazıyor...
Recep AY:Müdür Akademik Takvime İlişkin Rapor Yazıyor...
*/
```

Kalıtım kullanılan yapılarda ata sınıf üzerinden çocuk sınıf nesnesi siliniyorsa, ata sınıfın yıkıcı fonksiyonunun sanal olması gerekir. Aksi halde çocuk sınıfın yıkıcı fonksiyonu çalışmayabilir ve kaynak sızıntısı oluşabilir. Bu örnekte ata sınıfta sanal yıkıcı (*virtual destructor*) olmazsa, `delete kisi[i]` yazıldığında türemiş sınıfların (Ogrenci, Mudur vb.) özel alanları temizlenmeyebilir.

Çok Biçimlilikte Hangi Yöntem Ne Zaman Çalıştırılır?

Her sanal (*virtual*) yönteme sahip nesnenin gizli bir `vptr` göstericisi taşır. `vptr` nesne başına, `vtable` sınıf başınadır ve fonksiyon adreslerinden oluşur. Çalışma zamanında bu **sanal yöntemler tablosu** (*virtual method table*) üzerinden doğru fonksiyon adresine dallanılır.

Çok biçimlilik bize çalışma anında (*run-time*) üstünlük sağlar. Program çalıştırıldığında her `kisi` nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir. Yazılımcı, nesnenin tipine bakmaksızın, nesnelerden aynı davranışı göstermesini ister. Burada gösterilecek davranışın, taban sınıfın davranışı mı? olduğu veya türeyen sınıfın davranışı mı? olduğuna çalışma anında (*run-time*) karar verilir. İşte bu karar verme işlemine **geç bağlama** (*late binding*) veya **dinamik bağlama** (*dynamic binding*) adı verilir. Esnekliğin istenmediği bazı durumlarda ise karar verme işlemi derleme anında (*compile-time*) yapılır. Buna ise **statik bağlama** (*static binding*) adı verilir. İkisi arasındaki seçim programcı tarafından yapılır.

Yazılım yapılacak sistemin, başlangıç koşullarına bağlı tasarım kararlarının tümü bilinemez. Ayrıca **sistemin genişlemesi için gerekli tüm kodlamanın bitirilmesi beklenemez**. İşte bunu gerçekleştiren **dinamik bağlama**, yazılım mimarisinin daha esnek (mevcut bileşenin sistemde yeniden yapılandırmasının kolayca yapılması) ve genişleyebilir (yeni bileşenlerin kolayca sisteme eklenmesi) olmasını sağlar.

2.21 Üçlü, Beşli ve Sıfır Kuralı

Bu kurallar, C++'ta bir sınıfın kaynak (bellek, dosya, bağlantı) yönetimini nasıl tasarlayacağınızı anlatan pratik kılavuzlardır.

§2.21.1 Üçlü Kural (Rule of Three)

Bir sınıf için şu üç özel üye fonksiyondan birini kendiniz yazmanız gerekiyorsa, büyük ihtimalle üçünü de yazmanız gerekir:

1. Yıkıcı (`Sinif()`)
2. Kopyalama yapıcısı (`Sinif(const Sinif&)`)
3. Kopyalama atama işleci (`Sinif& operator=(const Sinif&)`)

Sebep şu: Sınıf ham gösterici ile bir kaynak tutuyorsa yıkıcıda `delete` yazarsınız. Derleyicinin

ürettiği örtük kopyalama yapıcısı ve atama işleci ise göstericiyi olduğu gibi (sığ kopya) kopyalar. Sonuçta iki nesne aynı belleği gösterir, ikisi de yok edilirken aynı bellek iki kez silinir (çift silme, tanımsız davranış).

```
class Arac {
  Motor* motor;
public:
  Arac() : motor(new Motor("D4D", 125)) {}
  ~Arac() { delete motor; }                        // 1. yıkıcı

  Arac(const Arac& d) : motor(new Motor(*d.motor)) {}   // 2. derin kopya

  Arac& operator=(const Arac& d) {                 // 3. derin atama
    if (this != &d) {
      Motor* yeni = new Motor(*d.motor);
      delete motor;
      motor = yeni;
    }
    return *this;
  }
};
```

§2.21.2 Beşli Kural (Rule of Five)

C++11 ile taşıma (*move*) anlamı geldi. Geçici nesnelerden kaynağı kopyalamak yerine "çalmak" için iki üye daha eklenir:

4. Taşıma yapıcısı (`Sinif(Sinif&&) noexcept`)
5. Taşıma atama işleci (`Sinif& operator=(Sinif&&) noexcept`)

Yani kaynak yöneten bir sınıfta bu beşini bilinçli olarak ya yazarsınız, ya `= default` / `= delete` ile açıkça belirtirsiniz. Bu üyelerden birini tanımlamak, derleyicinin bazılarını artık otomatik üretmemesine yol açar, o yüzden hepsini düşünmek gerekir.

§2.21.3 Sıfır Kuralı (Rule of Zero)

Modern C++'ın tercih ettiği yaklaşım budur: Kaynak yönetimini kendiniz yazmayın, sınıflarınız bu beş fonksiyonun hiçbirini tanımlamasın. Kaynağı `std::unique_ptr`, `std::shared_ptr`, `std::string`, `std::vector` gibi RAII sınıflarına bırakırsanız derleyicinin ürettiği varsayılanlar zaten doğru çalışır.

```
class Arac {
private:
  Motor motor; // en basit hali: değer (value) üye
  // ya da: std::unique_ptr<Motor> motor;
public:
  Arac() : motor("D4D", 125) {}
  // yıkıcı, kopyalama, taşıma: yazmaya gerek yok
};
```

3. Bölüm
İstisnalar, Esnek Tasarım ve SOLID İlkeleri

3.1 Hatalar ve İstisnalar

Bu bölümde hata durumlarının yapısal programlamada ve nesne yönelimli dillerde nasıl yönetildiği, C++ örnekleriyle anlatılmaktadır. İstisna yönetimi `try`, `catch` ve `throw` sözcükleriyle yapılır.

§3.1.1 Yapısal Programlamada Hata Yönetimi

Yapısal programlamada hata yönetimindeki asıl sorun, hata denetim kodunun işlevsel kodla iç içe geçmesidir. Hata genellikle fonksiyonun dönüş değeriyle (*hata kodu*) bildirilir ve çağıran kod her çağrıdan sonra bu değeri denetlemek zorundadır. Denetim unutulursa hata fark edilmeden ilerler; hatalı bir durumla karşılaşıldığında ise çoğu zaman programdan çıkılır. İstisna mekanizması bu sorunu büyük ölçüde azaltır: hata bildirimi dönüş değerinden ayrılır ve hata, onu işleyebilecek koda kadar otomatik olarak taşınır. Aşağıda bir C programı örneği verilmiştir;

```c
#include <stdio.h>

int faktoriyel(int sayi) {
  if (sayi < 0) return -1;  // Aslında sıfırdan küçük sayının faktöriyeli -1 değil, tan
      ımsızdır.
  if (sayi <= 1) return 1;
  return sayi * faktoriyel(sayi - 1);
}

int main() {
  int okunan, deger, hata = 0;
  do {
    printf("Bir Sayı Giriniz:");
    scanf("%d", &okunan);
    deger = faktoriyel(okunan);
    printf("%d nin faktöriyeli: %d\n", okunan, deger);
    if (deger == -1) {
      hata = 100;
      break; // exit(100);
    }
  } while (okunan != 0);
  return hata;
}
/* Program Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
*/
```

Bu örnekte `faktoriyel` fonksiyonu hata durumunda `-1` değerini geri döndürmektedir. Bu yaklaşımın iki zayıflığı vardır:

1. Hata değeri ile gerçek sonuç aynı dönüş değeri içinde taşınır (*in-band signaling*). Burada `-1` geçerli bir faktöriyel olamayacağı için çalışır; ancak her fonksiyon için hataya ayrılabilecek böyle bir değer bulmak mümkün değildir.

2. Denetim çağıranın sorumluluğundadır. Örnekte `printf()` satırı denetimden önce çalıştığı için hatalı sonuç ekrana yazdırılmıştır. Denetim hiç yazılmasaydı `-1` bir sonraki hesaplamada sessizce kullanılacaktı.

§3.1.2 Nesne Yönelimli Programlamada İstisna Yönetimi

Nesne yönelimli programlamada **istisna** (*exception*), programın yürütülmesi sırasında normal akışı bozan beklenmeyen bir durumdur. İstisna oluştuğunda durumu tanımlayan bir **istisna nesnesi** (*exception object*) oluşturulur ve **fırlatılır**. Çalışma zamanı sistemi (*run-time*) bu nesneyi işleyebilecek uygun bir kod bloğu arar. İstisnalar iki ana kaynaktan doğar:

1. **Programcı kaynaklı hatalar**: Geçersiz argüman, boş (`null`) bir nesneye erişim, dizi sınırının aşılması gibi, kodun doğru yazılmasıyla önlenebilecek durumlar.
2. **Çevre kaynaklı durumlar**: Dosyanın bulunamaması, ağ bağlantısının kopması, diskin dolması, belleğin yetersiz kalması gibi, programın kontrolü dışındaki durumlar.

İstisnalar Eşzamanlıdır

C++ ve Java'da istisnalar yalnızca programın kendi akışı içinde, bir `throw` ifadesiyle ortaya çıkar. Bu anlamda **eşzamanlıdır** (*synchronous*). Klavye kesmesi (*interrupt*) gibi eşzamansız (*asynchronous*) olaylar istisna olarak fırlatılmaz; işletim sistemi bunları sinyal veya kesme olarak ele alır. Bir sistem çağrısının başarısız olması (örneğin diskten okuma hatası) ise çağrıyı yapan kodda, istisna veya dönüş değeri olarak bildirilir.

İstisnai durumları yönetmek için `throw`, `try` ve `catch` anahtar sözcükleri kullanılır. İstisnaları işlemek için genel yapı aşağıda verilmiştir;

```
try {
  /*
  ...
  İstisnai Durumun Ortaya Çıkabileceği Kod...
  ...
  */
  throw SomeExceptionType("İstisnai Durum Açıklaması");
  /* throw an exception: istisnai bir duruma düşme */
} catch (ExceptionName1 e1) {
  /* catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için
      kullanılır. */
}
```

Bu kodlar şu şekilde açıklanabilir;

- ◇ **`try` talimatı**: İçerisinde istisnai bir duruma düşülebileceğini tahmin ettiğimiz bir kod bloğunu temsil eder. Bu bloğu bir veya daha fazla `catch` bloğu takip eder.
- ◇ **`catch` talimatı**: `try` bloğunda bir istisnai duruma düşüldüğünde yürütülecek bir kod bloğunu temsil eder. İstisnayı işlemek için kullanılan kod, `catch` bloğunun içine yazılır.
- ◇ **`throw` talimatı**: Bir istisna nesnesi oluşturup fırlatır (*throw an exception*).
- ◇ **İstisna sonrası süreç**: Program bir `throw` ifadesiyle karşılaştığında normal akış hemen kesilir. Çalışma zamanı sistemi önce içinde bulunulan `try` bloğunun `catch` bloklarında, bulunamazsa çağıran fonksiyonlarda sırayla yukarı doğru eşleşen bir `catch` bloğu arar. Bir `try` bloğu içinde birden fazla `throw` ifadesi bulunabilir; ancak ilk fırlatılan istisna işlendiğinde `try` bloğunun kalanı çalıştırılmaz.

Bu yeni bakış açısıyla yapısal programlamadaki faktöriyel örneğini aşağıdaki gibi yazabiliriz;

```
#include <iostream>

// Faktöriyel fonksiyonu: Hata durumunda istisna nesnesi fırlatır
int faktoriyel(int sayi) {
  if (sayi < 0) {
    throw "Negatif sayilarin faktoriyeli hesaplanamaz!";
    // Metin olarak istisna nesnesi oluşturduk ve fırlattık.
```

```
  }
  if (sayi <= 1) return 1;
  return sayi * faktoriyel(sayi - 1);
}

int main() {
  int okunan;

  do {
    std::cout << "Bir Sayi Giriniz (Cikis icin 0): ";
    std::cin >> okunan;
    if (okunan == 0) break;
    try { // Hata riski olan kodu try bloğu içine alıyoruz!
      int sonuc = faktoriyel(okunan);
      std::cout << okunan << " sayisinin faktoriyeli: " << sonuc << std::endl;
    }
    catch (const char* e) // Metin olarak oluşturulmuş istisna nesnelerini yakalıyoruz.
    {   // Yakalanan hatayı işliyoruz.
      std::cerr << "HATA: " << e << std::endl;
      return -1;
    }
  } while (okunan != 0);
}
/* Program Çıktısı:
Bir Sayi Giriniz (Cikis icin 0): -5
HATA: Negatif sayilarin faktoriyeli hesaplanamaz!
*/
```

İstisna işleme yapısal programdakinden farklıdır. Şöyle ki;

- ◇ Hata işleme kodu, fonksiyonel koddan yani işin gereği yazılan koddan ayrılır.
- ◇ Fonksiyonlar ve yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir fonksiyon veya yöntemde birden çok istisnai duruma düşülebilir; ancak bunlardan bazılarını `catch` bloğunda işlemeyi seçebiliriz. Geri kalanlarını yöntemi çağıran yere bırakabiliriz. Böylece istisnayı kimin işleyeceğini programcı seçebilir.
- ◇ hem temel tiplerden (`int`, `float`, `char`,...) hem de nesnelerden (`std::string`,...) istisna nesnesi oluşturulup fırlatılabilir.

Gerçek Kodda `const char*` Fırlatmayın: C++ örneğinde `const char*` fırlatılmasının amacı, C++'ta her tipin fırlatılabildiğini göstermektir. Gerçek projelerde temel tipler veya metinler yerine `std::exception` sınıfından türeyen nesneler fırlatılmalıdır; çünkü tüm hatalar tek bir tipte (`const std::exception&`) yakalanabilir ve hatanın açıklaması nesnenin içinde taşınır. Sonraki başlıklarda bu yaklaşım anlatılmaktadır.

Taşma ve Sıfıra Bölme: C++'ta işaretli tam sayı taşması **tanımsız davranıştır** (*undefined behavior*). Yukarıdaki C++ programı `13` girildiğinde istisna fırlatmadan yanlış sonuç yazdırır:

```
Bir Sayi Giriniz (Cikis icin 0): 13
13 sayisinin faktoriyeli: 1932053504
```

Tam sayıyı sıfıra bölmek de C++'ta tanımsız davranıştır; çoğu sistemde program istisna fırlatmadan çöker. Bu nedenle C++ örneklerinde bu denetimler elle yapılıp istisna fırlatılır. Kayan noktalı sayılarda ise (Bölüm 1'deki IEEE 754 anlatımı) `10.0 / 0` istisna değil, `Infinity` üretir.

§3.1.3 Standart İstisna Tipleri

Aşağıda sıfıra bölme hatasını işleyen bir C++ program örneği verilmiştir. Örnekte standart istisna tiplerinden biri olan `runtime_error` istisnası oluşturulmuştur. Bu tip istisna tipleri için projemize `<stdexcept>` başlığı dahil edilmelidir.

```
#include <iostream>
#include <stdexcept> // runtime_error için gerekli
```

```cpp
int main() {
  std::cout << "Burasi her zaman icra edilir." << std::endl;
  try {
    // try BLOĞU: İstisna oluşma ihtimali olan kod bloğu
    int bolunen = 10;
    int bolen = 0;
    int sonuc;
    if (bolen == 0) {
      // runtime_error standart istisnası fırlatıldı.
      throw std::runtime_error("Sifira Bolmeye Izin Verilmez!");

      // throw satırından sonraki kodlar asla çalışmaz
      std::cout << "Burasi hicbir zaman icra edilmez." << std::endl;
    }
    sonuc = bolunen / bolen;
    std::cout << "Bolme Sonucu: " << sonuc << std::endl;
  } catch (const std::exception& e) {
    // catch BLOĞU: Hata yakalandığında bu blok çalışır
    std::cout << "Istisna Yakalandi: " << e.what() << std::endl;
  }
  std::cout << "Burasi da her zaman icra edilir." << std::endl;
}
/* Program Çıktısı:
Burasi her zaman icra edilir.
Istisna Yakalandi: Sifira Bolmeye Izin Verilmez!
Burasi da her zaman icra edilir.
*/
```

Yukarıdaki örnekte `try` bloğunda izin verilmeyen bölme işleminde istisnai bir durum fark ediliyor, istisna nesnesi oluşturuluyor ve `throw` ile **istisna fırlatılıyor** (*throw an exception*). `catch` bloğunda ise **istisna nesnesi yakalanarak (*catch an exception*)** işleniyor. Örnekte standart tipte istisna nesnesi kullanılmıştır. Bu tiplerin hepsi `std::exception` sınıfından türetilmiştir.

İstisna Sınıfı	Üst Kategori	Açıklama
`std::logic_error`	`std::exception`	Programın mantıksal kurgusundaki hatalar (kodun doğru yazılmasıyla önlenebilir).
`std::invalid_argument`	`logic_error`	Fonksiyona gönderilen argümanın geçersiz olması.
`std::domain_error`	`logic_error`	Matematiksel bir işlemin tanım kümesi dışında kalması (örn. negatif sayının karekökü).
`std::length_error`	`logic_error`	Nesne boyutunun (örn. `std::string`) izin verilen maksimum sınırı aşması.
`std::out_of_range`	`logic_error`	İndis veya anahtarın geçerli aralığın dışında olması (`at()` yöntemi gibi).
`std::runtime_error`	`std::exception`	Sadece program çalışırken tespit edilebilen, tahmin edilmesi zor hatalar.
`std::range_error`	`runtime_error`	Bir hesaplamanın sonucunun hedef tipte temsil edilememesi.
`std::overflow_error`	`runtime_error`	Aritmetik bir işlemin sonucunun veri tipi sınırlarını aşması (üstten taşma).
`std::underflow_error`	`runtime_error`	Çok küçük bir sonucun veri tipi tarafından temsil edilememesi (alttan taşma).
`std::bad_alloc`	`std::exception`	`new` operatörü ile bellek tahsisi yapılamaması (bellek yetersizliği).
`std::bad_cast`	`std::exception`	`dynamic_cast` ile yapılan referans dönüşümünün başarısız olması.

Tablo 3.1: C++ Standart İstisna Hiyerarşisi ve Kullanım Alanları

3.1.3.1 Belli Bir İstisnayı Yakalama

Bir `try` bloğunda farklı tipte istisnalar fırlatılabilir. Ama tüm `catch` talimatlarıyla bu istisnalar yakalanmayabilir. Ayrıca tüm istisna nesnelerinin tipleri de önceden bilinmeyebilir. Bu durumda varsayılan `catch` kullanılır. Aşağıda buna ilişkin C++ örneği verilmiştir;

```
#include <iostream>

int main() {
  try {
    // DİKKAT: throw ile ilk istisna nesnesi oluştuğunda program hemen uygun 'catch'
        bloğuna atlar.
    // Alttaki throw satırlarına asla ulaşılamaz.

    throw 10; // Bu satır çalışır ve 'int' catch bloğuna gider.

    // Aşağıdaki satırlar ölü koddur (unreachable code):
    throw "Hata";
    throw 'A';
  }
  catch (int excpInt) {
    std::cout << "Tam Sayi Istisnasi Yakalandi: " << excpInt << std::endl;
  }
  catch (const char* excpStr) {
    // Modern C++'ta string literalleri 'const char*' olarak yakalanmalıdır.
    std::cout << "Metin Istisnasi Yakalandi: " << excpStr << std::endl;
  }
  catch (char excpChar) {
    std::cout << "Karakter Istisnasi Yakalandi: " << excpChar << std::endl;
  }
  catch (...) {
    // Default catch: Yukarıdakilere uymayan her şey buraya düşer.
    std::cout << "Diger Catchlerde islenmeyen bir istisna burada islendi." << std::endl
        ;
  }
}
/* Program Çıktısı:
Tam Sayi Istisnasi Yakalandi: 10
*/
```

catch Sırası

`catch` blokları yukarıdan aşağıya sırayla denenir ve ilk uyan blok çalışır. Bu nedenle **önce en özel, en son en genel** istisna tipi yazılmalıdır. Genel tipi önce yazarsanız özel tip hiçbir zaman çalışmaz. C++ derleyicisi bu durumda yalnızca uyarı verir ve program genel bloğu çalıştırır.

3.1.3.2 Sınıf İçinde İstisna Kullanımı

Aşağıda `Kisi` sınıfında `yas` özelliğine ilişkin istisna içeren bir C++ kod örneği verilmiştir. Geçersiz bir yaş değeri, çağıran koda bir istisna olarak bildirilmektedir. Geçersiz argüman için uygun standart istisna `std::invalid_argument` sınıfıdır.

```
#include <iostream>
#include <stdexcept>
#include <string>

class Kisi {
private:
  int yas;
public:
  Kisi() : yas(15) {} // Yapıcı (Constructor)
  int getYas() const { // Nesneyi değiştirmediği için 'const' eklemek iyi bir pratiktir
    return yas;
  }
  void setYas(int pYas) {
```

```
    if (pYas < 0)
    throw std::invalid_argument("Gecersiz Yas Degeri: <0!");
    if (pYas > 120)
    throw std::invalid_argument("Cok Buyuk Yas Degeri: >120!");
    yas = pYas;
  }
};

int main() {
  Kisi ali;
  try {
    // 1. Geçerli bir değer atama
    ali.setYas(12);
    std::cout << "Guncel Yas: " << ali.getYas() << std::endl;
    // 2. İstisna yaratacak değer (Hata fırlatıldığı an try bloğu terk edilir)
    ali.setYas(-1);
    // --- BU SATIRDAN SONRASI İCRA EDİLMEZ ---
    std::cout << "Bu satira asla ulasilamayacak: " << ali.getYas() << std::endl;
    ali.setYas(130);
    std::cout << ali.getYas() << std::endl;
  }
  catch (const std::exception& istisna) {
    // Yakalanan istisna nesnesinin içeriğini yazdırıyoruz
    std::cout << "Istisnai Bir Durum Yakalandi!" << std::endl;
    std::cout << "Durum Aciklamasi: " << istisna.what() << std::endl;
  }
}
/* Program Çıktısı:
Guncel Yas: 12
Istisnai Bir Durum Yakalandi!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/
```

§3.1.4 İstisna Nesnesi ve Hata Mesajı: what()

C++'ta `std::out_of_range`, `std::bad_alloc` gibi tüm standart istisna sınıfları `std::exception` sınıfından türetilmiştir. Bu sınıfın içinde `what()` adında sanal bir yöntem bulunur. Bu yöntemin görevi, **hatayı açıklayan bir karakter dizisi** (`const char*`) döndürmektir. Bu yöntem `catch` bloğu içinde istisna nesnesi üzerinden çağrılır.

```
#include <iostream>
#include <vector>
#include <stdexcept> // Standart istisnalar için

int main() {
  std::vector<int> sayilar = {10, 20, 30};
  try {
    // Vektörün sınırları dışında bir elemana erişmeye çalışıyoruz (indis 5 yok)
    // .at() fonksiyonu hata oluşunca std::out_of_range fırlatır.
    std::cout << sayilar.at(5) << std::endl;
  }
  catch (const std::exception& e) {
    // 'e' nesnesi üzerinden what() çağrılır:
    std::cout << "Hata Yakalandi: " << e.what() << std::endl;
  }
}
/* Program Çıktısı:
Hata Yakalandi: vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
*/
```

Standart, `what()` içeriğini belirtmez; mesaj derleyiciye ve standart kütüphaneye göre değişir (yukarıdaki çıktı GCC'ye aittir).

`std::exception` hiyerarşisini kullanmalıyız! Çünkü;

- ◇ **Hiyerarşi**: Tüm hataları tek bir `catch` bloğunda `const std::exception& e` ile yakalayabilirsiniz. Arka planda çok biçimlilik çalışır.
- ◇ **Bilgi Paylaşımı**: Hatanın nedenini kodun her yerinde manuel çıktıyla yazdırmak yerine, istisna nesnesinin içinde taşırsınız.
- ◇ **Standartlara Uyumluluk**: Standart kütüphaneler (C++'ta STL) bu yapıyı kullanır. Sizin kodunuzun da bu yapıya uyması, kodun profesyonelliğini artırır.

catch ile İstisna Yakalama

C++'ta `catch (std::exception e)` yerine `catch (const std::exception& e)` kullanın! Bu, nesnenin kopyalanmasını önler ve **nesne dilimleme** (*object slicing*) problemini engeller. Önce en özel istisna sınıflarını, en son genel `std::exception` sınıfını yakalayın.

§3.1.5 noexcept Anahtar Kelimesi

C++'ta *noexcept* anahtar kelimesi, hem bir belirleyici (*noexcept specifier*) hem de bir işleç (*noexcept operator*) olarak görev yapar. C++11 ile gelmiştir ve özellikle taşıma işlemlerinde performans ve istisna güvenliği açısından önemlidir.

3.1.5.1 noexcept İşleci

`noexcept` işleci (*noexcept operator*), bir ifadenin (*expression*) istisna fırlatıp fırlatmayacağını **derleme zamanında** (*compile-time*) kontrol eder. İfade çalıştırılmaz, yalnızca incelenir. Sonuç olarak `bool` bir değer döndürür:

1. `true`: Eğer ifade istisna fırlatmayacağı garanti edilmişse.
2. `false`: Eğer ifade istisna fırlatabilirse.

Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
#include <stdexcept>

void fonksiyon() { throw std::runtime_error("oops"); }
void bosfonksiyon() {} // Belirteç kullanılmadığı için potansiyel tehlike!
struct yapi {};

int main() {
  // 1. fonksiyon() içinde throw var, derleyici bunun fırlatabileceğini biliyor.
  std::cout << noexcept(fonksiyon()) << std::endl;      // Çıktı: 0
  // 2. bosfonksiyon() boş olsa bile 'noexcept' olarak işaretlenmediği için
  // derleyici her ihtimale karşı istisna fırlatabilir kabul eder.
  std::cout << noexcept(bosfonksiyon()) << std::endl;   // Çıktı: 0
  // 3. Temel aritmetik işlemler istisna fırlatmaz.
  std::cout << noexcept(1 + 1) << std::endl;            // Çıktı: 1
  // 4. Basit bir struct'ın varsayılan yapıcısı (constructor) istisna fırlatmaz.
  std::cout << noexcept(yapi()) << std::endl;           // Çıktı: 1
}
```

3.1.5.2 noexcept Belirleyicisi

`noexcept` belirleyicisi (*noexcept specifier*), bir fonksiyonun istisna fırlatmayacağını derleyiciye **söz vererek** bildirmek için fonksiyon imzasında kullanılır. Bu sözü verirseniz `noexcept` işleci bu fonksiyon için `true` döndürür.

```
#include <iostream>

void guvenliFonksiyon() noexcept {
  // Bu fonksiyonun hata fırlatmayacağı garanti edildi.
}

int main() {
  std::cout << noexcept(guvenliFonksiyon()); // Çıktı: 1
```

```
}
```

Söz tutulmazsa ne olur? `noexcept` bir fonksiyondan istisna çıkarsa istisna hiçbir `catch` bloğuna ulaşmaz; çalışma zamanı `std::terminate` çağırarak programı sonlandırır:

```
#include <iostream>
#include <stdexcept>

void tehlikeli() { throw std::runtime_error("oops"); }

// SÖZ VERİLDİ: bu fonksiyondan istisna çıkmayacak...
void sozVerdim() noexcept {
  tehlikeli(); // ...ama çıkıyor: söz bozuldu!
}

int main() {
  try {
    sozVerdim();
  } catch (...) {
    // Buraya hiç gelinmez: istisna catch'e ulaşamadan program sonlandırılır.
    std::cout << "Yakalandi" << std::endl;
  }
}
/* Program Çıktısı:
terminate called after throwing an instance of 'std::runtime_error'
what():  oops
*/
```

Bu nedenle `noexcept` yalnızca gerçekten istisna fırlatmayacağından emin olduğunuz fonksiyonlarda kullanılmalıdır. Taşıma yapıcıları ve taşıma atama işleçleri genellikle `noexcept` bildirilir; çünkü `std::vector` gibi kapsayıcılar büyürken elemanları yalnızca taşıma işlemi `noexcept` ise taşır, değilse güvenlik için kopyalar.

§3.1.6 Kullanıcı Tanımlı İstisna Tipleri

Her zaman standart istisna nesneleri kullanmak zorunda değiliz. Kendi hata durumlarımızı anlatan istisna sınıfları tanımlamak, hataların daha anlamlı ve gruplanabilir olmasını sağlar. C++'ta bu amaçla `<exception>` başlığını projemize dahil ederek `std::exception` sınıfından kendi sınıfımızı türetebilir ve `what()` yöntemini kendi hata mesajımızla geçersiz kılabiliriz (*override*).

```
#include <iostream>
#include <exception>
#include <string>

// Özel istisna sınıfı: Maksimum yaş sınırı için
struct YasMaxException : public std::exception {
  // what() fonksiyonunu geçersiz kılıyoruz (override)
  // 'noexcept' bu fonksiyonun kendisinin hata fırlatmayacağını garanti eder
  const char* what() const noexcept override {
    return "Cok Buyuk Yas Degeri: >120!";
  }
};

// Özel istisna sınıfı: Minimum yaş sınırı için
struct YasMinException : public std::exception {
  const char* what() const noexcept override {
    return "Gecersiz Yas Degeri: <0!";
  }
};

class Kisi {
private:
  int yas;
public:
  Kisi() : yas(15) {}
```

```
  int getYas() const { return yas; }
  void setYas(int pYas) {
    if (pYas < 0)
      throw YasMinException(); // Özel istisna nesnesi fırlatılıyor
    else if (pYas > 120)
      throw YasMaxException();
    yas = pYas;
  }
};

int main() {
  Kisi ali;
  try {
    ali.setYas(12);
    std::cout << ali.getYas() << std::endl;
    // Hata burada fırlatılacak
    ali.setYas(-1);
    // Bu satırlar icra edilmez
    std::cout << ali.getYas() << std::endl;
    ali.setYas(130);
  }
  catch (const std::exception& istisna)
  {
    // Çok biçimlilik sayesinde her iki özel hata da burada yakalanır
    std::cout << "Istisnai Bir Durum Yakalandi!" << std::endl;
    std::cout << "Durum Aciklamasi: " << istisna.what() << std::endl;
  }
}
/* Program Çıktısı:
12
Istisnai Bir Durum Yakalandi!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/
```

`what()` yöntemini elle yazmak yerine daha pratik yol, `std::runtime_error` gibi hazır bir sınıftan türetmektir. Böylece mesaj taban sınıfta saklanır ve istisna nesnesi hatalı değer gibi ek bilgiyi de taşıyabilir. Aşağıdaki örnekte ayrıca ortak bir taban sınıf (`YasException`) tanımlanmıştır; böylece her iki yaş hatası da tek bir `catch` ile yakalanabilir.

```
#include <iostream>
#include <stdexcept>
#include <string>

// what() yöntemini elle yazmak yerine std::runtime_error'dan türetiyoruz:
// mesaj taban sınıfta saklanır, ek bilgi (hatalı değer) de taşınabilir.
class YasException : public std::runtime_error {
public:
  YasException(const std::string& mesaj, int deger) : std::runtime_error(mesaj), deger_
      (deger) {}
  int getDeger() const { return deger_; }
private:
  int deger_;
};

class YasMinException : public YasException {
public:
  explicit YasMinException(int deger) : YasException("Gecersiz Yas Degeri: <0!", deger)
       {}
};

class YasMaxException : public YasException {
public:
  explicit YasMaxException(int deger) : YasException("Cok Buyuk Yas Degeri: >120!",
      deger) {}
};
```

```
void yasKontrol(int yas) {
  if (yas < 0) throw YasMinException(yas);
  if (yas > 120) throw YasMaxException(yas);
}

int main() {
  try {
    yasKontrol(130);
  } catch (const YasException& e) {
    std::cout << "Durum Aciklamasi: " << e.what() << std::endl;
    std::cout << "Hatali Deger: " << e.getDeger() << std::endl;
  }
}
/* Program Çıktısı:
Durum Aciklamasi: Cok Buyuk Yas Degeri: >120!
Hatali Deger: 130
*/
```

§3.1.7 İstisnayı Yeniden Fırlatma ve Zincirleme

Bir yöntem istisnayı yakalayıp (örneğin kaydını tutup) çağırana iletmeye devam etmek isteyebilir. C++'ta bunun için `catch` bloğu içinde argümansız `throw;` yazılır. Bu ifade **aynı** istisna nesnesini kopyalamadan yukarıya taşır. `throw e;` yazmak ise nesneyi kopyalar ve türemiş sınıf bilgisini kaybettirebilir (dilimleme).

```
#include <iostream>
#include <stdexcept>

void dosyaOku() {
  throw std::runtime_error("Dosya okunamadi");
}

void kayitYukle() {
  try {
    dosyaOku();
  } catch (const std::exception& e) {
    std::cout << "[kayitYukle] Hata kaydedildi: " << e.what() << std::endl;
    throw; // Aynı istisna nesnesi, kopyalanmadan bir üst düzeye taşınır
  }
}

int main() {
  try {
    kayitYukle();
  } catch (const std::exception& e) {
    std::cout << "[main] Yakalandi: " << e.what() << std::endl;
  }
}
/* Program Çıktısı:
[kayitYukle] Hata kaydedildi: Dosya okunamadi
[main] Yakalandi: Dosya okunamadi
*/
```

§3.1.8 Yığın Çözülmesi ve Kaynak Yönetimi

Yığın çözülmesi (*stack unwinding*), C++'ın bellek yönetimi ve hata güvenliği (*exception safety*) konusundaki en güçlü özelliklerinden biridir. İstisnalar fırlatıldığında arka planda gerçekleşen bu süreci anlamak, C++ ile güvenli kod yazmak için şarttır.

Bir fonksiyon içinde `throw` ile istisna fırlatıldığında, çalışma zamanı sistemi bu istisnayı yakalayacak uygun bir `catch` bloğu aramaya başlar. Bu arama sırasında, istisnanın fırlatıldığı noktadan `catch` bloğuna kadar olan tüm yerel nesnelerin yıkıcıları (*destructor*) otomatik olarak çağrılır. İşte bu temizlik sürecine **yığın çözülmesi** (*stack unwinding*) denir.

Süreç şöyle işler;

1. **İstisna Fırlatılır**: Bir fonksiyonda `throw` satırı çalışır.
2. **Geriye Doğru Tarama**: Program o anki fonksiyondan çıkar ve çağıran (*caller*) fonksiyona geri döner.
3. **Otomatik Temizlik**: Çıkılan her fonksiyondaki yerel (yığın üzerindeki) nesnelerin yıkıcıları (`~ClassName`) sistem tarafından, oluşturulma sırasının tersine çağrılır.
4. **Catch Bloğu Bulunur**: Uygun bir `catch` bloğuna ulaşılana kadar bu süreç "yukarı doğru" devam eder. Eğer `main()` fonksiyonuna kadar uygun bir blok bulunamazsa `std::terminate` çağrılır ve program sonlandırılır. (Bu durumda yığın çözülmesinin hiç yapılıp yapılmayacağı da derleyiciye bırakılmıştır.)

Bunu aşağıdaki örneği inceleyerek anlayabiliriz;

```
#include <iostream>
#include <stdexcept>
#include <string>

class Kaynak {
  std::string ad;
  public:
  Kaynak(std::string n) : ad(n) { std::cout << ad << " acildi.\n"; }
  ~Kaynak() { std::cout << ad << " otomatik olarak KAPATILDI (Yikici calisti).\n"; }
};

void fonksiyonB() {
  Kaynak b_kaynagi("FonksiyonB Kaynagi");
  std::cout << "FonksiyonB icinde hata firlatiliyor...\n";
  throw std::runtime_error("Kritik Hata!"); // Hata burada firlatilir
  std::cout << "Bu satir asla calismaz.\n";
}

void fonksiyonA() {
  Kaynak a_kaynagi("FonksiyonA Kaynagi");
  fonksiyonB();
  std::cout << "FonksiyonA devam ediyor...\n"; // Buraya asla gelinmez
}

int main() {
  try {
    fonksiyonA();
  }
  catch (const std::exception& e) {
    std::cout << "\nCatch Blogu Yakaladi: " << e.what() << "\n\n";
  }
  return 0;
}
/* Program Çıktısı:
FonksiyonA Kaynagi acildi.
FonksiyonB Kaynagi acildi.
FonksiyonB icinde hata firlatiliyor...
FonksiyonB Kaynagi otomatik olarak KAPATILDI (Yikici calisti).
FonksiyonA Kaynagi otomatik olarak KAPATILDI (Yikici calisti).

Catch Blogu Yakaladi: Kritik Hata!
*/
```

Yığın çözülmesi, C++ dilindeki "Kaynak Edinimi İlk Değer Atamasıdır!" yani RAII (*Resource Acquisition Is Initialization*) prensibinin temelidir. Eğer bu özellik olmasaydı:

- ◇ Açık kalan dosyalar kapanmazdı.
- ◇ Kilitlenmiş (*mutex*) veriler kilitli kalırdı (*deadlock*).
- ◇ Bellek sızıntıları (*memory leaks*) kaçınılmaz olurdu.

Yıkıcılar Hata Fırlatmamalı

Yığın çözülmesi sırasında bir nesnenin yıkıcısı (`~ClassName`) çalışırken, o yıkıcının içinden **yeni bir istisna dışarı çıkmamalıdır!** Bir istisna işlenirken (yığın çözülmesi sürerken) ikinci bir istisna yıkıcıdan dışarı çıkarsa C++ çalışma zamanı ne yapacağını bilemez ve programı anında sonlandırır (`std::terminate`). C++11'den itibaren yıkıcılar varsayılan olarak `noexcept` kabul edilir; yani yıkıcıdan çıkan her istisna doğrudan programı sonlandırır. Bu nedenle **yıkıcılar her zaman güvenli olmalı ve gerekirse hataları kendi içinde ele alıp yutmalıdır**.

İstisnalar sadece bir hata bildirme yöntemi değildir; aynı zamanda C++'ta yığındaki nesnelerin güvenli bir şekilde yok edilmesinin garantisidir. `throw` ile `catch` arasındaki mesafe ne kadar uzak olursa olsun, aradaki tüm yerel nesneler usulünce yok edilir.

§3.1.9 İstisna Kullanımında İyi Uygulamalar

- **İstisnayı olağan akış denetimi için kullanmayın.** İstisna, *istisnai* durumlar içindir. Bir döngüyü bitirmek için istisna fırlatmak hem okunaksız hem yavaştır. Hata olağan ve beklenen bir sonuçsa dönüş değeri daha uygundur (C++'ta `std::optional`, Java'da `Optional`).
- **En özel istisnayı yakalayın.** `catch (...)` (C++) veya `catch (Exception e)` (Java) yalnızca en üst katmanda, programın çökmesini önlemek ve hatayı kaydetmek için kullanılmalıdır.
- **İstisnayı yutmayın.** Boş bir `catch` bloğu hatayı sessizce kaybettirir. İstisnayı işleyemiyorsanız yeniden fırlatın veya en azından kaydedin.
- **Anlamlı mesaj yazın ve nedeni koruyun.** Mesaj, hatanın *ne olduğunu* ve gerekirse hatalı değeri içermelidir. Java'da alt düzey istisnayı zincirleyin.
- **Temizliği otomatikleştirin.** C++'ta RAII ve akıllı göstericiler, Java'da `try-with-resources` kullanın; temizliği elle `catch` bloklarına yazmayın.
- **Yapıcıda doğrulama yapın.** Yapıcının dönüş değeri olmadığı için geçersiz parametreyi bildirmenin yolu istisnadır. Yapıcı istisna fırlatırsa nesne hiç oluşmamış sayılır (C++'ta o ana kadar oluşturulmuş üye nesneler yok edilir, Java'da nesne referansı hiç atanmaz). Böylece geçersiz durumda bir nesne asla var olmaz.
- **İstisna güvenliğini düşünün (C++).** Bir fonksiyon istisna fırlattığında üç düzeyde garanti verebilir: *temel garanti* (kaynak sızmaz, nesne geçerli durumda kalır), *güçlü garanti* (işlem ya tamamen yapılır ya da hiç yapılmamış gibi geri alınır) ve *istisna fırlatmama garantisi* (`noexcept`).

3.2 Unified Modeling Language

§3.2.1 UML Nedir

UML(*Unified Modeling Language*), nesneye dayalı yazılım geliştirme ile birlikte gelişen bir modelleme aracıdır. Bu nedenle 2. bölümde açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye dayalı problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML dilini, basit bir dil, gösterim ya da söz dizimi gibi algılamalıyız. UML dilinin, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Buradaki ana elemanlar, **model elemanı** (*model element*) olarak adlandırılır. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (*software engineering*) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi söz dizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler. Doksanlı yılların orta-

larında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- Booch yönteminin tasarım (*design*) ve kodlama (*implementation*) yönleri oldukça iyi idi. *Grady Booch*, Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch yöntemi güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- OMT(*Object Modeling Technique*) tekniği, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- OOSE (*Object Oriented Software Engineering*) modeli **use-case** olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılarak *Grady Booch*'un çalıştığı Rational şirketinde bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan "Unified Method" oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*'ın use-case'ler konusundaki fikirleriyle birlikte Rational şirketi, bugün UML olarak adlandırılan, yeni bir "Unified Method" geliştirdi. Üç kişiden oluşan bu grup "Three Amigos" olarak bilinir.

1989 yılında kurulan OMG (*Object Management Group*) adlıbir şirketler birliği UML'in 1997'de standartlaştırılmasını sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir gösterim (*notation*) sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri soyut (*abstract*) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir. Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir soyutlamadır (*abstraction*). Etki alanı (*domain*) ise problemin yaşandığı gerçek dünyayı ifade eder.

UML

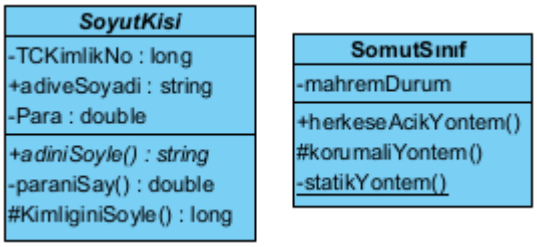
Model, birbirlerine ileti (*message*) gönderen nesnelerden (*object*) oluşur. Bu modelde nesneler canlı (*alive*) olarak düşünülmelidir. Nesneler bir şeyler bilir (*know*) ve bildikleriyle bir şeyler yaparlar (*do*). Modelde nesneler, nesne tanımında verilen özellik (*property*) ve alanları (*field*) gösteren niteliklere (*attribute*) sahiptirler. Nesnelerin gösterdikleri davranış (*behavior*) ya da geliştirdiği yöntemler (*method*) ise işlem (*operation*) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin durumunu (*state*) belirler.

UML, on dört tür diyagrama sahiptir ve bu kitapta **sınıf diyagramlarına** (*class diagram*) yer verilecektir.

§3.2.2 Sınıf Diyagramları

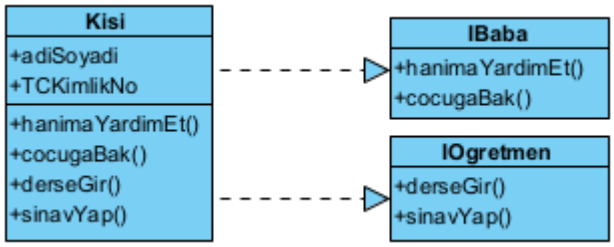
Sınıf diyagramları (*class diagram*), yazılımı geliştirilecek olan sisteme ait sınıfları (*class*) ve bu sınıflar arasındaki ilişkiyi (*relationship*) gösterir. Sınıf diyagramları durağandır (*static*). Yani sadece sınıfların birbiriyle etkileşimini (*what interacts*) gösterir, bu etkileşimler sonucunda ne olduğunu (*what happens*) göstermez. Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde nitelikler (*attribute*), en alttaki ve sonuncusunda ise ve işlemler (*operation*) yer alır.

Sınıf isminin italik yazılması, sınıfın **soyut sınıf** (*abstract class*) olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin **soyut yöntem** (*abstract method*) olduğunu gösterir. Sınıflardaki erişim belirleyicilerin (*access modifier*), özel karakterlerle birbirinden ayrılır. Burada (`-`) karakteri mahrem (*private*), (`+`) karakteri umuma açık



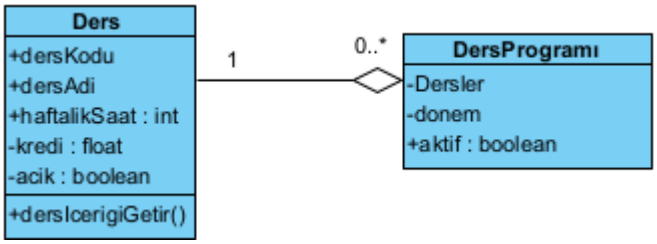
Şekil 3.1: Örnek Bir Sınıf Diyagramı

(*public*), (#) karakteri ise korumalı (*protected*) olduğunu anlatmaktadır. Statik (*static*) üyeler altı çizili olarak gösterilir.



Şekil 3.2: Sınıf Diyagramlarında Arayüz Gerçekleştirme

Arayüz gerçekleştirme (*interface realization*) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen arayüzü gösterir. Çemberler, arayüzleri (*interface*) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır.



Şekil 3.3: Sınıf Diyagramlarında Sınırlama ve Çokluk

Çokluk (*multiplicity*), ilişkide bir sınıfın örnek (*instance*) sayısını gösterir. İlişkiyi gösteren çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **ortaklık** (*association*), **bileşim** (*composition*) ve **biriktirme** (*aggregation*) ilişkilerinde kullanılır. Çokluk örnekleri:

- `0..1`: Sıfır ya da 1 örnek.
- `0..*` veya `*`: Örnek sayısı sınırsızdır.
- `1`: Yalnızca 1 örnek
- `1..*`: En az bir örnek

3.3 Sınıflar Arası İlişkiler

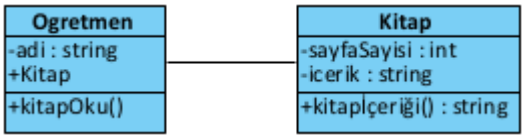
Nesne yönelimli tasarımda sınıflar arasındaki ilişkiler, sistemdeki nesnelerin birbirleriyle nasıl haberleştiğini, birbirlerini nasıl kullandığını ve yaşam döngülerinin birbirine bağlı olup olmadığını gösterir. UML sınıf diyagramlarında bu ilişkiler farklı çizgi türleri ve oklarla ifade edilir.

Bu bölümde `Ogretmen` ve `Kitap` örnekleri temel alınarak **ortaklık** (*association*), **biriktirme** (*aggregation*), **bileşim** (*composition*), **bağımlılık** (*dependency*), **genelleştirme** *generalization* ve gerçekleştirme

(*realization*) ilişkileri açıklanmıştır.

§3.3.1 Ortaklık (Association)

Ortaklık (*association*) ilişkisi, bir sınıfın başka bir sınıfı bilmesi, ona erişmesi veya onu kullanması durumudur. Bu ilişkide bir sınıf, diğer sınıfa ait bir nesneye referans veya gösterici (*pointer*) tutabilir. Ancak ortaklık ilişkisi, tek başına sahiplik veya bütün-parça anlamına gelmez. Yani ortaklıkta önemli olan nokta şudur: Bir nesne, diğer nesneyi kullanır; fakat onun sahibi olmak zorunda değildir. Ortaklık ilişkisi UML sınıf diyagramında düz çizgi ile gösterilir (Şekil 3.4):



Şekil 3.4: Ortaklık İlişkisi

Aşağıdaki örnekte `Ogretmen`, bir `Kitap` nesnesiyle ortaklık ilişkisine sahiptir. `Kitap` nesnesi `Ogretmen` sınıfı dışında oluşturulur. `Ogretmen` bu kitabı yalnızca okuma davranışı için kullanır. Burada öğretmenin kitabın sahibi olduğu veya kitabı kendi yaşam döngüsüne kattığı söylenemez.

```
#include <iostream>
#include <string>

class Kitap {
  public:
  Kitap(int pSayfaSayisi, std::string pIcerik)
  : sayfaSayisi(pSayfaSayisi), icerik(pIcerik) { }

  std::string kitapIcerigi() const {
    return icerik;
  }

  private:
  int sayfaSayisi;
  std::string icerik;
};

class Ogretmen {
  private:
  std::string adi;
  Kitap* ptrKitap; // Ogretmen, Kitap nesnesini bilir; sahibi degildir.

  public:
  Ogretmen(std::string pAdi)
  : adi(pAdi), ptrKitap(nullptr) { }

  void kitapAta(Kitap* pKitap) {
    ptrKitap = pKitap;
  }

  void kitapOku() const {
    if (ptrKitap == nullptr) {
      std::cout << adi << " su anda herhangi bir kitap okumuyor."
      << std::endl;
      return;
    }

    std::cout << adi << " su kitabi okuyor: "
    << ptrKitap->kitapIcerigi()
    << std::endl;
  }
};
```

```
int main() {
  Kitap kitap1(150, "C++ ile Nesne Yonelimli Programlama");
  Kitap kitap2(200, "Veri Yapilari ve Algoritmalar");

  Ogretmen ilhan("Ilhan");

  ilhan.kitapOku();

  ilhan.kitapAta(&kitap1);
  ilhan.kitapOku();

  ilhan.kitapAta(&kitap2);
  ilhan.kitapOku();

  return 0;
}
```

Bu örnekteki ortaklık ilişkisinde:

- `Ogretmen`, `Kitap` nesnesini bilir ve kullanır.
- `Kitap`, `Ogretmen` dışında oluşturulur.
- `Ogretmen`, `Kitap` nesnesinin sahibi değildir.
- `Ogretmen`, `Kitap` nesnesini oluşturmaz ve silmez.
- İlişki geçici veya değiştirilebilir olabilir.
- `Ogretmen` nesnesi yok edilse bile `Kitap` nesnesi yaşamaya devam eder.

Bu nedenle ortaklık ilişkisini kısaca "bilir ve kullanır" ilişkisi olarak düşünebiliriz. Ortaklıkta bütün-parça vurgusu yoktur. `Ogretmen` ile `Kitap` arasında yalnızca kullanım ilişkisi vardır.

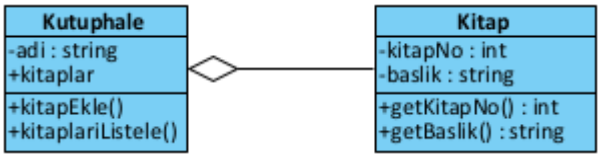
Ortaklık İçin Temel Soru

"Bu sınıf diğer sınıfı yalnızca biliyor veya kullanıyor mu?" sorusunun cevabı evet ise ilişki büyük olasılıkla ortaklık (*association*) ilişkisidir.

§3.3.2 Biriktirme (Aggregation)

Biriktirme (*aggregation*), zayıf sahiplik veya zayıf bütün-parça ilişkisi olarak düşünülebilir. Bu ilişkide bir sınıf, başka sınıflardan oluşan parçaları bünyesinde toplar. Ancak bu parçaların yaşam döngüsü bütüne sıkı biçimde bağlı değildir.

Başka bir ifadeyle, biriktirme ilişkisinde bir "bütün" nesne ve bu bütünün kullandığı "parça" nesneler vardır. Fakat parça nesneler, bütün nesne yok olduğunda zorunlu olarak yok olmak zorunda değildir. Parçalar bağımsız olarak da var olabilir. Biriktirme, UML sınıf diyagramında boş elmas ile gösterilir (Şekil 3.5):



Şekil 3.5: Biriktirme İlişkisi

Biriktirme ilişkisini daha net göstermek için `Kutuphane` ve `Kitap` örneğini ele alalım. Bir kütüphane birçok kitabı bünyesinde bulundurabilir. Ancak kitaplar kütüphane nesnesinin içinde oluşturulmak zorunda değildir. Bir kitap, kütüphaneden çıkarılsa bile başka bir kütüphaneye eklenebilir veya bağımsız olarak var olabilir.

```
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
```

```
class Kitap {
  public:
  Kitap(std::string pKitapNo, std::string pBaslik)
  : kitapNo(pKitapNo), baslik(pBaslik) {
    if (pKitapNo.empty() || pBaslik.empty()) {
      throw std::invalid_argument("Kitap no ve baslik bos olamaz.");
    }
  }

  std::string getKitapNo() const {
    return kitapNo;
  }

  std::string getBaslik() const {
    return baslik;
  }

  private:
  std::string kitapNo;
  std::string baslik;
};

class Kutuphane {
  private:
  std::string adi;
  std::vector<Kitap*> kitaplar; // Kutuphane kitaplari biriktirir; sahibi degildir.

  public:
  Kutuphane(std::string pAdi) : adi(pAdi) { }

  void kitapEkle(Kitap* kitap) {
    if (kitap == nullptr) {
      throw std::invalid_argument("Kitap nullptr olamaz.");
    }

    kitaplar.push_back(kitap);
  }

  void kitaplariListele() const {
    std::cout << adi << " kutuphanesindeki kitaplar:" << std::endl;

    for (const Kitap* kitap : kitaplar) {
      std::cout << "- " << kitap->getKitapNo()
      << " / " << kitap->getBaslik()
      << std::endl;
    }
  }
};

int main() {
  Kitap kitap1("K-1", "C++ ile Nesne Yonelimli Programlama");
  Kitap kitap2("K-2", "Veri Yapilari ve Algoritmalar");

  Kutuphane kutuphane("Merkez Kutuphane");

  kutuphane.kitapEkle(&kitap1);
  kutuphane.kitapEkle(&kitap2);

  kutuphane.kitaplariListele();

  return 0;
}
```

Bu örnekteki biriktirme ilişkisinde:

- ◇ `Kutuphane`, birden fazla `Kitap` nesnesini bünyesinde toplar.
- ◇ `Kitap` nesneleri `Kutuphane` dışında oluşturulur.
- ◇ `Kutuphane`, `Kitap` nesnelerinin yaşam döngüsünü yönetmez.
- ◇ `Kutuphane` yok edilse bile `Kitap` nesneleri yaşamaya devam edebilir.
- ◇ Kitaplar başka bir kütüphaneye aktarılabilir.
- ◇ İlişkide zayıf bütün-parça anlamı vardır.

Bu nedenle biriktirme ilişkisini kısaca "bünyesinde toplar ama yaşam döngüsünü yönetmez" ilişkisi olarak düşünebiliriz.

Biriktirme İçin Temel Soru

"Bu sınıf başka nesneleri bir bütünün parçaları gibi bünyesinde topluyor mu?, fakat onların yaşam döngüsünü yönetmiyor mu?" sorusunun cevabı evet ise ilişki büyük olasılıkla biriktirme (*aggregation*) ilişkisidir.

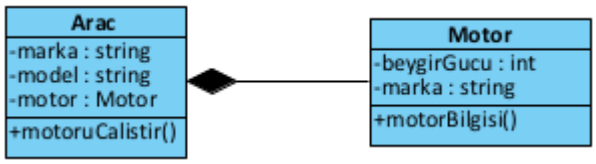
Ortaklık ve biriktirme ilişkileri birbirine benzese de aralarında kavramsal bir fark vardır. Ortaklıkta bir sınıf diğerini yalnızca bilir veya kullanır. Biriktirmede ise bir bütün-parça anlamı vardır; ancak bu bütün-parça ilişkisi zayıftır.

Özellik	Ortaklık (Association)	Biriktirme (Aggregation)
Temel anlam	Bilir veya kullanır.	Bünyesinde toplar.
Bütün-parça ilişkisi	Zorunlu değildir.	Vardır, fakat zayıftır.
Yaşam döngüsü bağı	Yoktur.	Parça, bütünden bağımsız yaşayabilir.
Sahiplik	Genellikle sahiplik yoktur.	Zayıf sahiplik vardır.
UML gösterimi	Düz çizgi.	Boş elmaslı çizgi.
Örnek	`Ogretmen` bir `Kitap` okur.	`Kutuphane` birçok `Kitap` barındırır.

Özetle, her biriktirme aynı zamanda bir tür ortaklık gibi düşünülebilir; çünkü bütün nesne parça nesneleri bilir. Ancak her ortaklık bir biriktirme değildir. Bir ilişkinin biriktirme sayılabilmesi için zayıf da olsa bütün-parça anlamı taşıması gerekir.

§3.3.3 Bileşim (Composition)

Bileşim (*composition*), güçlü sahiplik ilişkisidir. Bu ilişkide bütün nesne, parça nesnenin sahibidir. Parça nesnenin yaşam döngüsü bütün nesneye bağlıdır. Bileşim, UML sınıf diyagramında dolu elmas ile gösterilir (Şekil 3.6):



Şekil 3.6: Bileşim İlişkisi

Aşağıdaki örnekte `Motor`, `Arac` nesnesinin bir parçası olarak oluşturulur. `Arac` yok olduğunda `Motor` da yok olur.

```
#include <iostream>
#include <string>

class Motor {
public:
  std::string motorBilgisi() {
    return marka + " motor, " + std::to_string(beygirGucu) + " HP";
  }

  Motor(std::string pMarka, int pBeygirGucu) : marka(pMarka), beygirGucu(pBeygirGucu) {
      }
```

```
private:
  std::string marka;
  int beygirGucu;
};

class Arac {
private:
  std::string marka;
  std::string model;
  Motor* ptrMotor;

public:
  Arac(std::string pMarka, std::string pModel) : marka(pMarka), model(pModel) {

    // Motor nesnesi, Arac nesnesi ile birlikte oluşturuluyor.
    ptrMotor = new Motor("D4D", 120);
  }

  void motoruCalistir() {
    std::cout << marka << " " << model << " çalıştırılıyor..." << std::endl;
    std::cout << "Kullanılan motor: " << ptrMotor->motorBilgisi() << std::endl;
    std::cout << "Motor çalıştı..." << std::endl;
  }

  // YIKICI:
  // Arac yok edilirken sahip olduğu Motor nesnesi de silinir.
  ~Arac() {
    delete ptrMotor;
  }
};

int main() {
  Arac* arac = new Arac("Toyota", "Corolla");

  arac->motoruCalistir();

  delete arac;// Arac silininces, ona ait Motor da silinir.

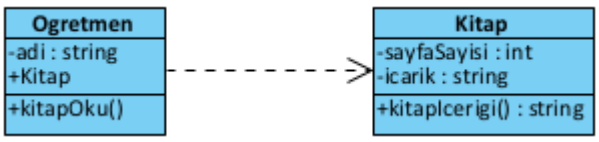
  return 0;
}
```

Bu örnekteki bileşim ilişkisinde:

- ◇ `Motor`, `Arac` nesnesinin içinde oluşturulur.
- ◇ `Motor` nesnesi `Arac`'tan bağımsız değildir.
- ◇ `Arac` yok olduğunda içindeki `Motor` da yok olur.
- ◇ Bu ilişki güçlü bir **sahiplik** (*ownership*) ilişkisidir.

§3.3.4 Bağımlılık (Dependency)

Bağımlılık (*Dependency*), bir sınıfın başka bir sınıfı geçici olarak kullanmasıdır. Bu kullanım genellikle yöntem parametresi, yerel değişken veya dönüş tipi üzerinden gerçekleşir. Bağımlılık ilişkisi, ortaklık (*association*) ilişkisine göre daha zayıf bir ilişkidir. Bağımlılık, UML sınıf diyagramında kesik çizgili ok ile gösterilir (Şekil 3.7):



Şekil 3.7: Bağımlılık İlişkisi

Aşağıdaki örnekte `Ogretmen`, `Kitap` nesnesini sınıf içinde saklamaz. Yalnızca `kitapOku()` yöntemine gelen parametre olarak kullanır.

```
#include <iostream>
#include <string>

class Kitap {
public:
  Kitap(int pSayfaSayisi, std::string pIcerik) : sayfaSayisi(pSayfaSayisi), icerik(
      pIcerik) { }

  std::string kitapIcerigi() const {
    return icerik;
  }

private:
  int sayfaSayisi;
  std::string icerik;
};

class Ogretmen {
private:
  std::string adi;

public:
  Ogretmen(std::string pAdi) : adi(pAdi) { }

  void kitapOku(const Kitap& kitap) const {
    std::cout << adi << " su kitabi okuyor: "
    << kitap.kitapIcerigi()
    << std::endl;
  }
};

int main() {
  Kitap kitap1(150, "C++ ile Nesne Yonelimli Programlama");

  Ogretmen ilhan("Ilhan");

  ilhan.kitapOku(kitap1);

  return 0;
}
```

Bu örnekteki bağımlılık ilişkisinde:

- ◇ `Ogretmen`, `Kitap` nesnesini alan değişkeni olarak tutmaz.
- ◇ `Kitap`, yalnızca yöntem parametresi olarak kullanılır. Buna **parametre olarak bağımlılık** (*dependency as a parameter*) adı da verilir. `Kitap`, parametre değil de bu yöntemin içinde imal edilip kullanılıyorsa buna **örnekleme olarak bağımlılık** (*dependency as an instantiation*) adı verilir.
- ◇ Kullanım geçicidir.
- ◇ Sahiplik veya yaşam döngüsü bağı yoktur.

§3.3.5 Genelleştirme (Generalization)

Genelleştirme (*generalization*), kalıtım ilişkisidir. Bir alt sınıfın, üst sınıfın özelliklerini ve davranışlarını devralması anlamına gelir. bu ilişki `public inheritance` ile kurulur. Genelleştirme, UML sınıf diyagramında içi boş üçgen ok ile gösterilir (Şekil 3.8).

Aşağıdaki örnekte `Ogretmen`, `Kisi` sınıfından türetilmiştir. Yani `Ogretmen` bir tür `Kisi`'dir.

```
#include <iostream>
#include <string>
```



Şekil 3.8: Genelleştirme İlişkisi

```
class Kisi {
protected:
  std::string adi;

public:
  Kisi(std::string pAdi) : adi(pAdi) { }

  void kendiniTanit() const {
    std::cout << "Benim adim " << adi << std::endl;
  }
};

class Ogretmen : public Kisi {
public:
  Ogretmen(std::string pAdi) : Kisi(pAdi) { }

  void dersAnlat() const {
    std::cout << adi << " ders anlatiyor." << std::endl;
  }
};

int main() {
  Ogretmen ilhan("Ilhan");

  ilhan.kendiniTanit();
  ilhan.dersAnlat();

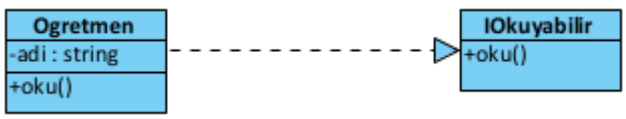
  return 0;
}
```

Bu örnekteki genelleştirme ilişkisinde:

- ◇ `Ogretmen`, `Kisi` sınıfından türetilmiştir.
- ◇ `Ogretmen`, `Kisi` sınıfının üyelerini devralır.
- ◇ Bu ilişki **bir türüdür** (*is-a*) ilişkisi olarak ifade edilir.
- ◇ `Ogretmen` bir tür `Kisi`'dir.

§3.3.6 Gerçekleştirme (Realization)

Gerçekleştirme (*realization*), bir sınıfın bir arayüzü gerçekleştirmesi durumudur. Doğrudan `interface` anahtar sözcüğü yoktur; ancak yalnızca saf sanal fonksiyonlardan oluşan soyut sınıflar arayüz gibi kullanılabilir. Gerçekleştirme, UML sınıf diyagramında kesik çizgili ve içi boş üçgen ok ile gösterilir (Şekil 3.9).



Şekil 3.9: Gerçekleştirme İlişkisi

Aşağıdaki örnekte `IOkuyabilir` bir arayüz gibi tasarlanmıştır. `Ogretmen` sınıfı bu arayüzdeki `oku()` davranışını gerçekleştirir.

```
#include <iostream>
#include <string>
```

```cpp
class IOkuyabilir {
public:
  virtual void oku() const = 0;

  virtual ~IOkuyabilir() { }
};

class Ogretmen : public IOkuyabilir {
private:
  std::string adi;

public:
  Ogretmen(std::string pAdi) : adi(pAdi) { }

  void oku() const override {
    std::cout << adi << " kitap okuyor." << std::endl;
  }
};

int main() {
  Ogretmen ilhan("Ilhan");

  ilhan.oku();

  return 0;
}
```

Bu örnekteki gerçekleştirme ilişkisinde:

- ◇ `IOkuyabilir`, arayüz gibi davranan soyut bir sınıftır. Rol davranışlarını içerir.
- ◇ `Ogretmen`, `IOkuyabilir` arayüzünü gerçekleştirir.
- ◇ `Ogretmen`, `oku()` fonksiyonunu uygulamak zorundadır.
- ◇ Bu ilişki **yapabilir** (*can-do*) veya **uygular** (*implements*) ilişkisi olarak düşünülebilir.

§3.3.7 İlişkilerin Özeti

İlişki Türü	Anlamı
Ortaklık (*association*)	A, B'yi bilir veya kullanır.
Biriktirme (*aggregation*)	A, B'yi zayıf şekilde sahiplenir; B bağımsız yaşayabilir.
Bileşim (*composition*)	A, B'nin sahibidir; B'nin yaşam döngüsü A'ya bağlıdır.
Bağımlılık (*dependency*)	A, B'yi geçici olarak kullanır.
Genelleştirme (*generalization*)	A, B'den türemiştir; A bir tür B'dir.
Gerçekleştirme (*realization*)	A, B arayüzünü gerçekleştirir.

Ortaklık (*association*) ve biriktirme (*aggregation*) arasındaki fark çoğu kaynakta yoruma açık olabilir. Uygulamada kritik ayrım, nesnenin yaşam döngüsünün kimin tarafından yönetildiğidir. Eğer parça nesnenin yaşam döngüsü bütün nesneye sıkı biçimde bağlıysa ilişki bileşim (*composition*) olarak değerlendirilir.

3.4 Sınıf Tasarlama (SOLID) İlkeleri

SOLID, nesne yönelimli tasarımın beş temel ilkesinin baş harflerinden oluşur. İlkeleri *Robert C. Martin* 2000'li yılların başında bir araya getirmiş, SOLID kısaltması ise *Michael Feathers*'a atfedilmiştir. Bu ilkelerin hiçbiri bir "kanun" değildir; kodun zamanla değiştirilmesini ucuzlatmaya yarayan tasarım pusulalarıdır.

Her ilkeyi aynı yöntemle ele alacağız: önce ilkeyi ihlal eden **kötü tasarımı** görüyoruz, sorunun tam olarak nerede çıktığını belirliyoruz, ardından **iyi tasarımla** düzeltiyoruz. Böylece her ilkenin **neyi önlediği** açıkça görülür. Bütün örnekler çalıştırılabilir tam programlardır; çıktıları listelerin altında verilmiştir.

§3.4.1 Tek Sorumluluk İlkesi (SRP)

Tek Sorumluluk İlkesi (*Single Responsibility Principle*): Bir sınıfın değişmesi için **yalnızca bir nedeni** olmalıdır.

Hem aşçı, hem muhasebeci, hem de kurye olarak çalışan bir kişiyi düşünün. Menü değişirse, vergi mevzuatı değişirse veya teslimat bölgesi değişirse hep aynı kişinin işi değişir; biri diğerinin işini bozabilir. Sınıflar için de durum aynıdır.

Kötü tasarım: `Fatura` sınıfı hem KDV hesaplıyor, hem veritabanına kaydediyor, hem de çıktı üretiyor.

```
#include <iostream>
#include <string>

// KOTU: Fatura sinifi uc farkli isten sorumlu
class Fatura {
public:
  Fatura(std::string musteri, int tutarTL) : musteri_(musteri), tutarTL_(tutarTL) {}

  // 1. Sorumluluk: is kurali (KDV hesabi)
  int kdvliToplam() const { return tutarTL_ * 120 / 100; }

  // 2. Sorumluluk: veritabani islemleri
  void kaydet() const {
    std::cout << "[Veritabani] " << musteri_ << " kaydedildi\n";
  }

  // 3. Sorumluluk: ekran/yazici ciktisi
  void yazdir() const {
    std::cout << "FATURA: " << musteri_ << " - "
    << kdvliToplam() << " TL\n";
  }

private:
  std::string musteri_;
  int tutarTL_;
};

int main() {
  Fatura fatura("Ayse Yilmaz", 1000);
  fatura.kaydet();
  fatura.yazdir();
}
/* Çıktı:
  [Veritabani] Ayse Yilmaz kaydedildi
FATURA: Ayse Yilmaz - 1200 TL
*/
```

Bu örnekteki **sorun nerede?** `Fatura` sınıfının üç ayrı değişme nedeni var:

- ◇ Muhasebe KDV oranını değiştirirse `kdvliToplam()` değişir.
- ◇ Veritabanı teknolojisi değişirse `kaydet()` değişir.
- ◇ Fatura tasarımı değişirse `yazdir()` değişir.

Üç ekip aynı dosyada çalışır, biri diğerinin kodunu yanlışlıkla bozabilir.

İyi tasarım: Her sorumluluk kendi sınıfına taşınır.

```
#include <iostream>
#include <string>

// IYI: Her sinifin degismek icin tek bir nedeni var

// Degisme nedeni: is kurali (ornegin KDV orani)
class Fatura {
```

```
public:
  Fatura(std::string musteri, int tutarTL) : musteri_(musteri), tutarTL_(tutarTL) {}

  const std::string& musteri() const { return musteri_; }
  int kdvliToplam() const { return tutarTL_ * 120 / 100; }

private:
  std::string musteri_;
  int tutarTL_;
};

// Degisme nedeni: veritabani/depolama teknolojisi
class FaturaDeposu {
public:
  void kaydet(const Fatura& fatura) const {
    std::cout << "[Veritabani] " << fatura.musteri()
    << " kaydedildi\n";
  }
};

// Degisme nedeni: faturanin gorunumu
class FaturaYazici {
public:
  void yazdir(const Fatura& fatura) const {
    std::cout << "FATURA: " << fatura.musteri() << " - "
    << fatura.kdvliToplam() << " TL\n";
  }
};

int main() {
  Fatura fatura("Ayse Yilmaz", 1000);

  FaturaDeposu depo;
  FaturaYazici yazici;

  depo.kaydet(fatura);
  yazici.yazdir(fatura);
}
```

Yeni durumda **kazanç:** Veritabanını değiştirmek yalnızca `FaturaDeposu` sınıfını, fatura görünümünü değiştirmek yalnızca `FaturaYazici` sınıfını etkiler. Her sınıf ayrı ayrı test edilebilir.

Nasıl Fark Ederim?

Sınıfı tek cümleyle "*bu sınıf X yapar ve Y yapar*" diye anlatıyorsanız veya sınıfın adı `Yonetici`, `Islemci` gibi belirsizse, büyük ihtimalle birden fazla sorumluluğu vardır. Örneklerde para birimi sadelik için tam lira (`int`) olarak tutulmuştur; gerçek uygulamalarda tutarlar kuruş cinsinden tam sayı veya özel bir `Para` sınıfıyla saklanmalıdır.

§3.4.2 Açık-Kapalı İlkesi (OCP)

Açık-Kapalı İlkesi (*Open-Closed Principle*): Sınıflar **genişlemeye açık**, **değişikliğe kapalı** olmalıdır. Bu ilke *Bertrand Meyer*'in 1988 tarihli *Object-Oriented Software Construction* kitabına dayanır.

Bir çok prizli uzatma kablosu gibi düşünün: yeni bir cihaz taktığınızda kabloyu açıp içindeki telleri değiştirmezsiniz, sadece boş prize takarsınız. Yazılımda da yeni bir özellik eklerken çalışan kodu açıp değiştirmek istemeyiz, çünkü her değişiklik yeni hata riski taşır.

Kötü tasarım: Müşteri türüne göre indirim hesaplayan sınıf `if-else` zinciriyle yazılmış.

```
#include <iostream>
#include <string>

// KOTU: Her yeni musteri turu icin bu sinifin kodu degistirilmeli
```

```cpp
class IndirimHesaplayici {
public:
  int hesapla(const std::string& musteriTuru, int tutarTL) const {
    if (musteriTuru == "ogrenci") {
      return tutarTL * 10 / 100;
    } else if (musteriTuru == "emekli") {
      return tutarTL * 15 / 100;
    }
    // "bayram" indirimi gelirse buraya yeni bir else-if eklenecek...
    return 0;
  }
};

int main() {
  IndirimHesaplayici hesaplayici;
  std::cout << "Ogrenci indirimi: "
    << hesaplayici.hesapla("ogrenci", 200) << " TL\n";
    std::cout << "Emekli indirimi : "
    << hesaplayici.hesapla("emekli", 200) << " TL\n";
}
/* Çıktı:
Ogrenci indirimi: 20 TL
Emekli indirimi : 30 TL
*/
```

Bu örnekteki **sorun nerede?** Bayram indirimi, personel indirimi, kurumsal indirim gibi her yeni tür için `IndirimHesaplayici` sınıfını açıp yeni bir `else-if` eklemek zorundayız. Çalışan ve test edilmiş kod her seferinde değişir; zincir uzadıkça hata riski artar.

İyi tasarım: İndirim kuralı bir arayüz olur, her tür kendi sınıfında yaşar.

```cpp
#include <iostream>

// IYI: Yeni indirim turu eklemek icin mevcut kodu degistirmiyoruz

class IndirimKurali {
public:
  virtual ~IndirimKurali() = default;
  virtual int hesapla(int tutarTL) const = 0;
};

class OgrenciIndirimi : public IndirimKurali {
public:
  int hesapla(int tutarTL) const override { return tutarTL * 10 / 100; }
};

class EmekliIndirimi : public IndirimKurali {
public:
  int hesapla(int tutarTL) const override { return tutarTL * 15 / 100; }
};

// Sonradan eklendi: Kasa sinifina HIC dokunulmadi
class BayramIndirimi : public IndirimKurali {
public:
  int hesapla(int tutarTL) const override { return tutarTL * 20 / 100; }
};

// Degisikliklere kapali, yeni kurallara acik
class Kasa {
public:
  int odenecekTutar(int tutarTL, const IndirimKurali& kural) const {
    return tutarTL - kural.hesapla(tutarTL);
  }
};

int main() {
```

```
  Kasa kasa;
  OgrenciIndirimi ogrenci;
  EmekliIndirimi emekli;
  BayramIndirimi bayram;

  std::cout << "Ogrenci : " << kasa.odenecekTutar(200, ogrenci) << " TL\n";
  std::cout << "Emekli  : " << kasa.odenecekTutar(200, emekli) << " TL\n";
  std::cout << "Bayram  : " << kasa.odenecekTutar(200, bayram) << " TL\n";
}
/* Çıktı:
Ogrenci : 180 TL
Emekli  : 170 TL
Bayram  : 160 TL
*/
```

Yeni durumda **kazanç:** `BayramIndirimi` sonradan eklendi, ancak `Kasa` ve diğer sınıflara tek satır dokunulmadı. Yeni bir indirim türü eklemek, yeni bir sınıf yazmak demektir.

Nasıl Fark Ederim?

Aynı tür `if-else` veya `switch` zinciri kodun farklı yerlerinde tekrar ediyor ve "yeni bir tür gelince hepsine bakmam gerekecek" diyorsanız, OCP ihlal ediliyor olabilir. Çözüm genellikle çok biçimliliktir (polimorfizm).

§3.4.3 Liskov İkame İlkesi (LSP)

Liskov İkame İlkesi (*Liskov Substitution Principle*): Türemiş sınıfın nesnesi, taban sınıf nesnesinin kullanıldığı **her yerde**, programın doğruluğunu bozmadan kullanılabilmelidir. İlke *Barbara Liskov*'un çalışmalarına (1987, Liskov–Wing 1994) dayanır.

Basit anlatımla: alt sınıf, üst sınıfın verdiği **sözü tutmalıdır**. "Her `Kus` uçar" diyen bir sınıf hiyerarşisinde, uçamayan bir `Penguen` bu sözü bozar. Burada kalıtım yalnızca "bir türüdür" (*is-a*) cümlesinin doğru olmasına değil, **davranışın** da uyumlu olmasına bağlıdır.

Kötü tasarım: `Penguen`, `Kus`'tan türüyor ve `uc()` çağrıldığında hata fırlatıyor.

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <vector>

// KOTU: Penguen, Kus'un soz verdigi davranisi yerine getiremiyor
class Kus {
public:
  virtual ~Kus() = default;
  virtual void uc() const { std::cout << "Kus ucuyor\n"; }
};

class Serce : public Kus {};

class Penguen : public Kus {
public:
  void uc() const override {
    throw std::logic_error("Penguen ucamaz!");
  }
};

// "Her Kus ucabilir" varsayimiyla yazilmis bir fonksiyon
void kuslariUcur(const std::vector<std::unique_ptr<Kus>>& kuslar) {
  for (const auto& kus : kuslar) {
    kus->uc();
  }
}
```

```
int main() {
  std::vector<std::unique_ptr<Kus>> kuslar;
  kuslar.push_back(std::make_unique<Serce>());
  kuslar.push_back(std::make_unique<Penguen>());   // Kus yerine geciyor...

  try {
    kuslariUcur(kuslar);
  } catch (const std::logic_error& hata) {
    std::cout << "CALISMA ZAMANI HATASI: " << hata.what() << "\n";
  }
}
/*Çıktı:
Kus ucuyor
CALISMA ZAMANI HATASI: Penguen ucamaz!
*/
```

Bu örnekteki **sorun nerede?** `kuslariUcur()` fonksiyonu doğru yazılmıştır: "her `Kus` uçar" sözüne güvenir. Ancak listeye bir `Penguen` girince program **çalışma zamanında** çöker. Hata, derleyici tarafından değil, kullanıcı tarafından bulunur. Bu, LSP ihlallerinin en tehlikeli yanıdır: tespit edilmesi zordur.

İyi tasarım: Uçma yeteneği, yalnızca uçabilen kuşların ortak sözleşmesine (`UcanKus`) taşınır.

```
#include <iostream>
#include <memory>
#include <vector>

// IYI: Ucabilme yetenegi, ucabilen kuslara ozgu bir sozlesme

class Kus {                        // Tum kuslarin ortak davranisi
public:
  virtual ~Kus() = default;
  virtual void ye() const { std::cout << "Kus yem yiyor\n"; }
};

class UcanKus : public Kus {       // Sadece ucabilen kuslar
public:
  virtual void uc() const = 0;
};

class Serce : public UcanKus {
public:
  void uc() const override { std::cout << "Serce ucuyor\n"; }
};

class Penguen : public Kus {
  public:
  void yuz() const { std::cout << "Penguen yuzuyor\n"; }
};

// Artik "ucabilen" olmayan bir kus bu fonksiyona hic gonderilemez
void kuslariUcur(const std::vector<std::unique_ptr<UcanKus>>& kuslar) {
  for (const auto& kus : kuslar) {
    kus->uc();
  }
}

int main() {
  std::vector<std::unique_ptr<UcanKus>> ucanKuslar;
  ucanKuslar.push_back(std::make_unique<Serce>());
  // ucanKuslar.push_back(std::make_unique<Penguen>());  // DERLEME HATASI!

  kuslariUcur(ucanKuslar);

  Penguen penguen;
  penguen.ye();
```

```
  penguen.yuz();
}
/*
Serce ucuyor
Kus yem yiyor
Penguen yuzuyor
*/
```

Yeni durumda **kazanç:** Yanlış kullanım artık **derleme zamanında** yakalanır: `Penguen` nesnesini `UcanKus` listesine eklemeye çalışmak derleme hatasıdır. Alt sınıfı üst sınıfın yerine koymak her zaman güvenlidir.

Nasıl Fark Ederim?

Bir alt sınıf üst sınıfın yöntemini "desteklenmiyor" diye hata fırlatarak, boş bırakarak ya da `if (dynamic_cast<...>)` ile tür sorarak geçersiz kılıyorsa LSP ihlal ediliyordur. Bir başka klasik örnek, `Kare` sınıfının `Dikdortgen`'den türetilmesidir: genişliği değiştirince yüksekliğin de değişmesi, dikdörtgenden beklenen davranışı bozar.

§3.4.4 Arayüzleri Ayırma İlkesi (ISP)

Arayüzleri Ayırma İlkesi (*Interface Segregation Principle*): İstemciler, **kullanmadıkları yöntemlere** bağımlı olmaya zorlanmamalıdır. Büyük, "her şeyi yapan" arayüzler yerine küçük ve role özel arayüzler tercih edilmelidir.

Bir restoranın menüsünde yalnızca içecek siparişi veren müşteriye, yemek ve tatlı seçeneklerini de "zorunlu doldurulacak" alanlar olarak sunmak mantıksızdır. Arayüzler de bu şekilde kesilmelidir.

Kötü tasarım: Tek bir şişman arayüz, yazdırma, tarama ve faks çekmeyi birlikte tanımlıyor.

```
#include <iostream>
#include <stdexcept>

// KOTU: "Sisman" arayuz - her cihaz her seyi yapmak zorunda
class ICokFonksiyonluCihaz {
public:
  virtual ~ICokFonksiyonluCihaz() = default;
  virtual void yazdir() = 0;
  virtual void tara() = 0;
  virtual void faksCek() = 0;
};

class BasitYazici : public ICokFonksiyonluCihaz {
public:
  void yazdir() override { std::cout << "Basit yazici: yaziliyor\n"; }

  // Kullanmadigi halde uygulamak zorunda kaldigi yontemler:
  void tara() override {
    throw std::logic_error("Bu cihaz tarama yapamaz!");
  }
  void faksCek() override {
    throw std::logic_error("Bu cihaz faks cekemez!");
  }
};

int main() {
  BasitYazici yazici;
  yazici.yazdir();
  try {
    yazici.tara();
  } catch (const std::logic_error& hata) {
    std::cout << "HATA: " << hata.what() << "\n";
  }
}
```

```
/* Çıktı:
Basit yazici: yaziliyor
HATA: Bu cihaz tarama yapamaz!
*/
```

Bu örnekte **sorun nerede?** `BasitYazici` yalnızca yazdırabilir. Ancak arayüz hepsini zorunlu kıldığı için tarama ve faks yöntemlerine hata fırlatan sahte gövdeler yazmak zorunda kalır. Bu hem gereksiz koddur hem de bir LSP ihlalidir ("bir cihaz taranabilir" sözü tutulmuyor). Arayüze yeni bir yöntem eklendiğinde, onu kullanmayan tüm sınıflar da değişmek zorunda kalır.

İyi tasarım: Arayüz, rollere göre bölünür. Her sınıf yalnızca gerçekten yaptığı rolleri gerçekleştirir.

```
#include <iostream>

// IYI: Kucuk, role ozel arayuzler
class IYazdirabilir {
public:
  virtual ~IYazdirabilir() = default;
  virtual void yazdir() = 0;
};

class ITarayabilir {
public:
  virtual ~ITarayabilir() = default;
  virtual void tara() = 0;
};

class IFakslayabilir {
public:
  virtual ~IFakslayabilir() = default;
  virtual void faksCek() = 0;
};

// Sadece yazdirabilir: bos veya hata firlatan yontem yok
class BasitYazici : public IYazdirabilir {
public:
  void yazdir() override { std::cout << "Basit yazici: yaziliyor\n"; }
};

// Uc rolu birden gerceklestirir
class CokFonksiyonluYazici : public IYazdirabilir,  public ITarayabilir, public
    IFakslayabilir {
public:
  void yazdir() override { std::cout << "Coklu cihaz: yaziliyor\n"; }
  void tara() override { std::cout << "Coklu cihaz: taraniyor\n"; }
  void faksCek() override { std::cout << "Coklu cihaz: faks cekiliyor\n"; }
};

// Istemci, yalnizca ihtiyaci olan role bagimli
void belgeYazdir(IYazdirabilir& cihaz) { cihaz.yazdir(); }
void belgeTara(ITarayabilir& cihaz) { cihaz.tara(); }

int main() {
  BasitYazici basit;
  CokFonksiyonluYazici coklu;

  belgeYazdir(basit);
  belgeYazdir(coklu);
  belgeTara(coklu);
  // belgeTara(basit);   // DERLEME HATASI: BasitYazici taranamaz
}
/* Çıktı:
Basit yazici: yaziliyor
Coklu cihaz: yaziliyor
```

```
Coklu cihaz: taraniyor
*/
```

Yeni durumda **kazanç:** `belgeYazdir()` fonksiyonu yalnızca `IYazdirabilir` rolüne bağımlıdır; taramayı bilmez. `BasitYazici` nesnesini `belgeTara()` fonksiyonuna vermek artık derleme hatasıdır.

§3.4.5 Bağımlılıkları Tersine Çevirme İlkesi (DIP)

Bağımlılıkları Tersine Çevirme İlkesi (*Dependency Inversion Principle*) iki parçadan oluşur:

1. Üst düzey modüller, alt düzey modüllere bağımlı olmamalıdır. **İkisi de soyutlamalara** bağımlı olmalıdır.
2. Soyutlamalar ayrıntılara bağımlı olmamalı, ayrıntılar soyutlamalara bağımlı olmalıdır.

Duvara sabitlenmiş, elektrik kablosu gömülü bir lamba düşünün: ampulü ya da kaynağı değiştirmek için duvarı yıkmanız gerekir. Oysa prizli bir lambayı istediğiniz prize takabilirsiniz. Priz, lamba (üst düzey) ile elektrik kaynağı (alt düzey) arasındaki **soyutlamadır**.

Kötü tasarım: `SiparisServisi`, e-posta gönderen somut sınıfı kendi içinde tutuyor.

```
#include <iostream>
#include <string>

class EpostaGonderici {
public:
  void gonder(const std::string& mesaj) const {
    std::cout << "[E-POSTA] " << mesaj << "\n";
  }
};

// KOTU: Ust duzey sinif, somut (alt duzey) sinifa dogrudan bagimli
class SiparisServisi {
public:
  void siparisVer(const std::string& urun) {
    std::cout << urun << " siparisi alindi\n";
    eposta_.gonder(urun + " siparisiniz alindi");
  }

private:
  EpostaGonderici eposta_;    // SMS'e gecmek icin bu sinif degismeli;
  // test ederken gercek e-posta gider!
};

int main() {
  SiparisServisi servis;
  servis.siparisVer("Kitap");
}
/* Çıktı:
Kitap siparisi alindi
[E-POSTA] Kitap siparisiniz alindi
*/
```

Sorun nerede?

- ◇ Bildirimi SMS ile göndermek istersek `SiparisServisi` sınıfını değiştirmek zorundayız.
- ◇ Sınıfı test ederken **gerçekten e-posta gider**; sahte bir gönderici kullanamayız.
- ◇ İş kuralı (sipariş alma) ile altyapı ayrıntısı (e-posta) birbirine kilitlenmiştir.

İyi tasarım: İki sınıf bir arayüz (`IBildirimGonderici`) üzerinden konuşur. Hangi göndericinin kullanılacağı dışarıdan verilir.

```
#include <iostream>
#include <string>

// IYI: Ust duzey sinif soyutlamaya (arayuze) bagimli
```

```cpp
class IBildirimGonderici {
public:
  virtual ~IBildirimGonderici() = default;
  virtual void gonder(const std::string& mesaj) = 0;
};

class EpostaGonderici : public IBildirimGonderici {
public:
  void gonder(const std::string& mesaj) override {
    std::cout << "[E-POSTA] " << mesaj << "\n";
  }
};

class SmsGonderici : public IBildirimGonderici {
public:
  void gonder(const std::string& mesaj) override {
    std::cout << "[SMS] " << mesaj << "\n";
  }
};

// Test icin sahte gonderici: gercekten hicbir sey gondermez
class SahteGonderici : public IBildirimGonderici {
public:
  void gonder(const std::string& mesaj) override { sonMesaj_ = mesaj; }
  const std::string& sonMesaj() const { return sonMesaj_; }

private:
  std::string sonMesaj_;
};

class SiparisServisi {
public:
  // Bagimlilik disaridan verilir (bagimlilik enjeksiyonu)
  explicit SiparisServisi(IBildirimGonderici& gonderici)  : gonderici_(gonderici) {}

  void siparisVer(const std::string& urun) {
    std::cout << urun << " siparisi alindi\n";
    gonderici_.gonder(urun + " siparisiniz alindi");
  }

private:
  IBildirimGonderici& gonderici_;
};

int main() {
  EpostaGonderici eposta;
  SmsGonderici sms;
  SahteGonderici sahte;

  SiparisServisi epostaliServis(eposta);   // SiparisServisi kodu degismedi
  SiparisServisi smsliServis(sms);
  SiparisServisi testServisi(sahte);

  epostaliServis.siparisVer("Kitap");
  smsliServis.siparisVer("Kalem");

  testServisi.siparisVer("Defter");
  if (sahte.sonMesaj() == "Defter siparisiniz alindi") {
    std::cout << "TEST BASARILI\n";
  }
}
/* Çıktı:
Kitap siparisi alindi
[E-POSTA] Kitap siparisiniz alindi
Kalem siparisi alindi
```

```
[SMS] Kalem siparisiniz alindi
Defter siparisi alindi
TEST BASARILI
*/
```

Yeni durumda **kazanç:** `SiparisServisi` sınıfının kodu değişmeden e-posta, SMS veya test amaçlı sahte gönderici kullanılabildi. Üç `SiparisServisi` nesnesinin davranışı yalnızca yapıcıya verilen nesneyle belirlendi.

Üç Ayrı Kavram

- **DIP** (*Dependency Inversion Principle*): Bir *tasarım ilkesidir*: "soyutlamaya bağımlı ol".
- **Bağımlılık enjeksiyonu** (*Dependency Injection*, DI): Bu ilkeyi uygulamanın bir *tekniğidir*: bağımlılığı yapıcıya veya yönteme dışarıdan vermek (yukarıdaki `SiparisServisi` yapıcısı).
- **Denetimin Tersine Çevrilmesi** (*Inversion of Control*, IoC): Nesnelerin oluşturulmasını ve bağlanmasını bir çerçeveye (*framework*) bırakan daha geniş bir yaklaşımdır.

Örnekteki yapıcıya referans verme yöntemi, nesnelerin `main()` içinde yaşadığı varsayımına dayanır. Nesnenin sahipliğini devralmak gerekiyorsa `std::unique_ptr` kullanılır.

§3.4.6 İlkelerin Özeti

SOLID ilkeleri her sınıfı mümkün olan en küçük parçalara bölmenizi gerektirmez. Gereksiz soyutlamalar kodu okunmaz hâle getirir (sınıf patlaması). Yalnızca **gerçekten değişen** noktalarda uygulayın: "bu kısım yarın değişecek mi?" sorusu iyi bir rehberdir. Basit ve değişmeyecek bir kodu arayüzlerle süslemeyin.

İlke	İhlalin Belirtisi	Çözüm
SRP	Sınıfın birden fazla değişme nedeni var; "ve" ile anlatılıyor.	Sorumlulukları ayrı sınıflara böl.
OCP	Yeni tür eklenince mevcut `if-else`/`switch` zincirleri değişiyor.	Arayüz ve çok biçimlilik kullan.
LSP	Alt sınıf, yöntemi "desteklenmiyor" diye hata fırlatıyor veya boş bırakıyor.	Hiyerarşiyi yeniden kur; yeteneği ayrı soyutlamaya taşı.
ISP	Sınıf, kullanmadığı arayüz yöntemlerini uygulamak zorunda kalıyor.	Büyük arayüzü role özel küçük arayüzlere böl.
DIP	Üst düzey sınıf, somut sınıfı kendi içinde `new` ile oluşturuyor; test edilemiyor.	Arayüze bağımlı ol, bağımlılığı dışarıdan ver.

Tablo 3.2: SOLID İlkelerinin Özeti

4. Bölüm
Şablonlar

4.1 Şablon Nedir?

Şablonlar (*template*), veri tiplerine bağlı kalmasan çalışan fonksiyonlar, sınıflar, algoritmalar yazmak için jenerik programlama (*generic programming*) ile oluşturulan yapılardır.

Jenerik programlama, kodun belirli bir veri tipine (`int`, `float`, `string` vb.) bağlı kalmadan, veri tipinden bağımsız bir şekilde yazılmasına olanak tanıyan bir yazılım geliştirme yaklaşımıdır. En temel amacı, aynı mantığa sahip işlemleri farklı veri tipleri için tekrar tekrar yazmak yerine, bir kez **şablon** (*template*) olarak hazırlayıp her tipte güvenle kullanabilmektir. Yani;

- ◇ Bir kez yazılan bir algoritma (örneğin sıralama veya arama), hem sayılar hem de metinler için çalışabilir. Böylece aynı kodu yeniden kullanılabiliriz (*reusing*).
- ◇ Kod derleme zamanında (*compile-time*) kontrol edilir. Hatalar program çalışmadan yakalanır. Böylece tip güvenliği (*type safety*) sağlanmış olur.
- ◇ Çalışma zamanında (*run-time*) bir tip kontrolü yapılmaz; derleyici her tip için özel bir kod üretir, bu yüzden çok hızlıdır. Böylece performans yakalamış oluruz.

Jenerik programlamada bir fonksiyon veya sınıf yazarken, tip kısmına gerçek bir isim (örneğin `int`) yerine bir yer tutucu (*placeholder*) koyarsınız. Genellikle bu yer tutucu için `T` harfi kullanılır.

Örnek olarak iki değeri karşılaştırıp büyüğünü bulan bir jenerik fonksiyon düşünelim. Jenerik olmayan kodda her veri tipi için bir fonksiyon yazmayı gerektirir;

```
intMax(int a, int b) {
  //...
}
doubleMax(double a, double b) {
  //...
}
```

Jeneric olan kodda ise tek bir şablon fonksiyon yazarsınız.

```
template <typename T> T Max(T a, T b) {
  return (a>=b)?a:b;
}
```

Derleyici, siz içine ne koyarsanız (`int` veya `double`) ona uygun gerçek kodu arka planda sizin için oluşturur. Eğer jenerik programlama olmasaydı, her yeni veri tipi için aynı fonksiyonları baştan yazmak zorunda kalırdık. Bu da hem çok fazla **kod kalabalığına** (*code bloat*) hem de hata yapma riskinin artmasına neden olurdu.

4.2 Foksiyon Şablonları

Jenerik programlamanın temel taşı olan fonksiyon şablonları (*function templates*), aynı mantığı yeniden yazmadan bir fonksiyonun farklı veri tipleri üzerinde çalışmasını sağlayan bir fonksiyon planı tanımlar. Bu plan aşağıdaki şekilde tanımlanır;

```
template <typename değişken-kimliği> foksiyon-bildirimi;
```

Aşağıdaki kodda, iki değerin yerini değiştiren (*swap*) bir fonksiyonu hem `int` hem `double` hem de `string` için tek bir şablonla nasıl çalıştırdığımızı görebilirsiniz.

```
#include <iostream>
#include <string>

// 'typename T' ifadesi, T'nin herhangi bir veri tipi olabileceğini belirtir.
template <typename T>
void DegerDegistir(T &a, T &b) {
  T gecici = a;
  a = b;
  b = gecici;
}

int main() {
  // 1. Tam Sayılarla Kullanım
  int x = 10, y = 20;
  DegerDegistir(x, y); // Derleyici T'yi 'int' olarak anlar.
  std::cout << "Tam Sayilar: x=" << x << ", y=" << y << std::endl;

  // 2. Ondalıklı Sayılarla Kullanım
  double d1 = 1.5, d2 = 9.9;
  DegerDegistir(d1, d2); // Derleyici T'yi 'double' olarak anlar.
  std::cout << "Ondalikli: d1=" << d1 << ", d2=" << d2 << std::endl;

  // 3. Metinlerle Kullanım
  std::string s1 = "Dunya", s2 = "Merhaba";
  DegerDegistir(s1, s2); // Derleyici T'yi 'string' olarak anlar.
  std::cout << "Metinler: s1=" << s1 << ", s2=" << s2 << std::endl;
}
/*Program Çıktısı:
Tam Sayilar: x=20, y=10
Ondalikli: d1=9.9, d2=1.5
Metinler: s1=Merhaba, s2=Dunya
*/
```

Kod 4.1: Fonksiyon Şablonu Örneği

4.3 Değişken Sayıda Argüman Alabilen Fonksiyon Şablonları

Değişken sayıda argüman alabilen fonksiyon şablonları (*variadic function template*), herhangi bir sayıda (sıfır veya daha fazla) argüman alabilen sınıf veya fonksiyon şablonlarıdır. C++ dilinde şablonlar, yalnızca bildirim sırasında belirtilmesi gereken sabit sayıda parametreye sahip olabilir. Ancak *Douglas Gregor* ve *Jaakko Järvi* tarafından ortaya koyulan **değişken sayıda argüman alabilen şablonlar** (*variadic template*) bunun üstesinden gelmeye yardımcı olur.

C++11 ile hayatımıza giren ve bir fonksiyona sınırsız sayıda ve farklı türde argüman göndermemize olanak tanıyan oldukça güçlü bir jenerik programlama özelliğidir. Değişken sayıda argüman alabilen şablonlar (*variadic şablonlar*) bir **parametre paketi** (*parameter pack*) kullanarak her şeyi kabul edebilir. Temel söz diziminde üç nokta (`...`) karakterleri bulunur;

```
template <typename... Args>
void fonksiyon(Args... args) {
  // Args: Şablon parametre paketi (tipler)
  // args: Fonksiyon parametre paketi (değerler)
}
```

Bu söz diziminde; `typename... Args` demek buraya sıfır veya daha fazla tip gelebilir demektir. `Args... args` ise bu tiplerden oluşan paketlenmiş değerleri al demektir.

Değişken sayıda argüman alabilen şablonlar (*variadic şablonlar*), özyinelemeli mantıkla çalışır. Paketin içindeki ilk elemanı alırız, geri kalan paketi tekrar aynı fonksiyona göndeririz. Paket boşal-

dığında ise durmak için bir **sonlandırıcı fonksiyon** (*base case*) yazarız. Örnek olarak argüman sayısı belli olmayan bir yazdır fonksiyonunu örnek verebiliriz;

```
#include <iostream>

// 1. Sonlandırıcı (Base Case): Paket boşaldığında bu çalışır.
void Yazdir() {
  std::cout << std::endl;
}

// 2. Variadic Şablon:
template <typename T, typename... Args>
void Yazdir(T ilk, Args... geriKalan) {
  std::cout << ilk << " "; // İlk elemanı yazdır

  // Paketi "açarak" (unpacking) geri kalanını tekrar gönderiyoruz
  Yazdir(geriKalan...);
}

int main() {
  Yazdir(1, 2.5, "Merhaba", 'A', 42);
  // Adım 1: 1 yazdırılır, (2.5, "Merhaba", 'A', 42) gönderilir.
  // Adım 2: 2.5 yazdırılır, ("Merhaba", 'A', 42) gönderilir.
  // ... en son boş paket Yazdir() fonksiyonuna düşer.
}
/*Program Çıktısı:
1 2.5 Merhaba A 42
*/
```

Kod 4.2: Değişken Sayıda Argüman Alan Fonksiyon Şablonu Örneği

Argüman paketinin içinde kaç argümanın olduğunu **`sizeof...` işlecini** (*sizeof... operator*) kullanarak öğrenebiliriz. Ayrıca C++17 öncesinde yukarıdaki gibi özyinelemeli (*recursive*) yazmak zorundaydık. C++17 ile gelen **katlama ifadeleri** (*fold expressions*) sayesinde artık döngüye veya **sonlandırıcı fonksiyona** (*base case*) gerek kalmadan paketi tek satırda açabiliriz:

```
#include <iostream>

template<typename... Args>
void YeniYazdir(Args... args) {
  std::cout << "Gelen argüman sayisi: " << sizeof...(args) << std::endl;
  // Tüm argümanların arasına << koyarak tek seferde yazdır (Fold Expression)
  (std::cout << ... << args) << std::endl;
}

int main() {
  YeniYazdir(1, 2.5, "Merhaba", 'A', 42);
  // Adım 1: 1 yazdırılır, (2.5, "Merhaba", 'A', 42) gönderilir.
  // Adım 2: 2.5 yazdırılır, ("Merhaba", 'A', 42) gönderilir.
  // ... en son boş paket Yazdir() fonksiyonuna düşer.
}
/*Program Çıktısı:
Gelen argüman sayisi: 5
12.5MerhabaA42
*/
```

Kod 4.3: Katlama İfadeleri

4.4 Katlama İfadeleri

Katlama ifadeleri (*fold expressions*), C++17 standardı ile gelen ve değişken sayıda parametre alan şablonlar ile çalışmayı inanılmaz derecede kolaylaştıran bir özelliktir. C++11 ve C++14'te, bir parametre paketindeki (*parameter pack*) tüm elemanları toplamak veya çarpmak için özyinelemeli (*recursive*) fonksiyonlar yazmak zorundaydık. Katlama ifadeleri sayesinde artık tek bir satırda bu işlemi yapabiliyoruz.

Parametre Paketini katlamaya şu örnek verilebilir; Örneğin, elinizde `1, 2, 3, 4` gibi bir paket varsa, `+` işleciyle bunu `(((1 + 2) + 3) + 4)` haline getirir. Dört Farklı Katlama Türü vardır. Aşağıdaki tabloda `args` parametre paketini, `op` ise `+` ve `*` gibi ikili (iki işleneni olan) işlecleri temsil eder;

Katlama Türü	Yazım Şekli	Açılımı (Örnek)
Unary Right Fold	`(args op ...)`	`(a1 op (a2 op (a3 op a4)))`
Unary Left Fold	`(... op args)`	`(((a1 op a2) op a3) op a4)`
Binary Right Fold	`(args op ... op init)`	`(a1 op (a2 op (a3 op (a4 op init))))`
Binary Left Fold	`(init op ... op args)`	`((((init op a1) op a2) op a3) op a4)`

Tablo 4.1: Katlama Türleri

Katlama ifadeleri şunlar için kullanmalıyız;

- ◇ **Kod Kısalığı**: Özyinelemeli fonksiyon şablonları yazmanıza ve "durma koşulu" (*base case*) belirlemenize gerek kalmaz.
- ◇ **Derleme Hızı**: Derleyici bu ifadeleri doğrudan açtığı için özyinelemeli şablonlara göre daha hızlı derlenir.
- ◇ **Okunabilirlik**: Niyetinizi (tüm paketi bir operatörle birleştirmek) koda çok daha net yansıtır.

Katlama İfadelerini Destekleyen İşleçler

Neredeyse tüm ikili operatörler desteklenir: `+ - * / % ^ & | = < > << >> == != <= >= && || , .* ->*`

Değişken sayıda sayıyı toplama **Unary Left Fold** katlama ile eskiden sayfalarca süren bu işlem artık tek satırdır;

```
#include <iostream>

template<typename... Args>
auto toplam(Args... args) {
  return (... + args); // Unary Left Fold
}

int main() {
  std::cout << toplam(1, 2, 3, 4, 5); // Çıktı: 15
  return 0;
}
```

Kod 4.4: Tekli Katlama İfadeleri

Hepsini ekrana yazdırmak için virgül işleci (*comma operator*) katlama ifadeleri ile kullanıldığında, her bir argüman için bir işlem yapılmasını sağlar:

```
#include <iostream>

template<typename... Args>
void hepsiniYazdir(Args... args) {
  (std::cout << ... << args) << std::endl; // Binary Left Fold (std::cout başlangıç değ
      eri)
}

int main() {
  hepsiniYazdir("C++ ", 17, " Fold ", "Expressions ", "Harika!");
}
```

Kod 4.5: İkili Katlama İfadeleri

Aşağıdaki örnekte, gönderilen tüm argümanların `true` olup olmadığını kontrol eden bir fonksiyon örneği vardır;

```
#include <iostream>
```

```cpp
#include <vector>

// Hepsi doğru mu? (Logical AND Fold)
template<typename... Args>
bool hepsiDogruMu(Args... args) {
  return (... && args); // Unary Left Fold: (a1 && a2 && a3 ...)
}

// En az biri doğru mu? (Logical OR Fold)
template<typename... Args>
bool enAzBiriDogruMu(Args... args) {
  return (... || args); // Unary Left Fold: (a1 || a2 || a3 ...)
}

int main() {
  bool sonuc1 = hepsiDogruMu(true, 1, (5 > 2), true);
  std::cout << "Hepsi dogru mu? " << std::boolalpha << sonuc1 << std::endl; // true
  bool sonuc2 = hepsiDogruMu(true, 0, true);
  std::cout << "Hepsi dogru mu? " << sonuc2 << std::endl; // false (cunku 0, false
      kabul edilir)
  bool sonuc3 = enAzBiriDogruMu(false, 0, (10 < 2), true);
  std::cout << "En az biri dogru mu? " << sonuc3 << std::endl; // true
}
```

Kod 4.6: Katlama İfadeleri Örneği

Katlama için iki durum meydana gelebilir;

1. **Kısa Devre Yapma** (*short-circuiting*): Katlama ifadeleri, mantıksal operatörlerin doğal davranışını korur. Örneğin (`... && args`) ifadesinde, paket içindeki ilk eleman `false` ise, derleyici diğer elemanlara bakmaz ve sonucu anında false döner. Bu performans için kritiktir.
2. **Boş Parametre Paketi Durumu**: Eğer fonksiyona hiç parametre göndermezseniz, mantıksal katlamlar varsayılan değerler döner: `&& (AND)` için boş paket `true` döner. `|| (OR)` için boş paket `false` döner.
3. Virgül için boş paket geçersizdir (`void`).

4.5 Sınıf Şablonları

Sınıf şablonları (*class templates*), fonksiyon şablonlarında olduğu gibi, bir sınıfı belirli bir veri tipine bağlı kalmadan, tipten bağımsız bir kalıp olarak tanımlamanızı sağlar. Bir sınıf şablonu yazdığınızda, aslında bir sınıf değil, o sınıfın nasıl oluşturulacağına dair bir reçete yazmış olursunuz. Temel söz dizimi aşağıda verilmiştir;

```cpp
template <typename T>
class SablonSinif {
private:
  T icerik; // T tipinde bir veri saklar
public:
  SablonSinif(T pIcerik) : icerik(pIcerik) {}

  T getIcerik() const {
    return icerik;
  }

  void setIcerik(T pYeniIcerik) {
    icerik = pYeniIcerik;
  }
};
```

Bir sınıf şablonundan nesne üretirken, hangi veri tipini şablona parametre vereceğinizi açılı ayraçlar (`< >`) içinde belirtirsiniz.

```cpp
#include <iostream>
```

```cpp
template <typename T>
class SablonSinif {
private:
  T icerik; // T tipinde bir veri saklar
public:
  SablonSinif(T pIcerik) : icerik(pIcerik) {}

  T getIcerik() const {
    return icerik;
  }

  void setIcerik(T pYeniIcerik) {
    icerik = pYeniIcerik;
  }
};

int main() {
  // T yerine 'int' geçiyoruz.
  SablonSinif<int> tamSayiSinifi(42);
  // T yerine 'string' geçiyoruz.
  SablonSinif<std::string> metinSinifi("Merhaba C++");

  std::cout << tamSayiSinifi.getIcerik() << std::endl;
  std::cout << metinSinifi.getIcerik() << std::endl;
}
/*Program Çıktısı:
42
Merhaba C++
*/
```

Kod 4.7: Sınıf Şablonu Örneği

Eğer bir şablon sınıfın yöntemini sınıfın dışında tanımlamak isterseniz, her seferinde şablon ifadesini tekrar etmeniz gerekir:

```cpp
#include <iostream>

template <typename T>
class SablonSinif {
private:
  T icerik; // T tipinde bir veri saklar
public:
  SablonSinif(T pIcerik) : icerik(pIcerik) {}

  T getIcerik() const;
  void setIcerik(T pYeniIcerik);
};

template <typename T>
T SablonSinif<T>::getIcerik() const {
  return icerik;
}

template <typename T>
void SablonSinif<T>::setIcerik(T pYeniIcerik) {
  icerik = pYeniIcerik;
}

int main() {
  // T yerine 'int' geçiyoruz
  SablonSinif<int> tamSayiSinifi(42);
  // T yerine 'string' geçiyoruz
  SablonSinif<std::string> metinSinifi("Merhaba C++");

  std::cout << tamSayiSinifi.getIcerik() << std::endl;
```

```
  std::cout << metinSinifi.getIcerik() << std::endl;
}
/*Program Çıktısı:
42
Merhaba C++
*/
```

Kod 4.8: Sınıf Şablonunda Sınıf Dışında Yöntem Tanımlama Örneği

Sınıf şablonunda sadece bir veri tipi değil, birden fazla farklı veri tipini de aynı anda kullanabilirsiniz. Örnek olarak `Cift<T1, T2>` şablonu aşağıda verilmiştir:

```
#include <iostream>
#include <string>

template <typename T1, typename T2>
class Cift {
public:
  T1 birincil;
  T2 ikincil;

  // Yapıcı (Constructor)
  Cift(T1 a, T2 b) : birincil(a), ikincil(b) {}
  void ciftYaz() {
    // T1 ve T2 tiptir, değişkenler ise 'birincil' ve 'ikincil'dir.
    std::cout << birincil << ": " << ikincil << std::endl;
  }
};

int main() {
  // Örnek 1: Yaş bilgisi (string ve int)
  Cift<std::string, int> yasVerisi("Yasim", 45);
  yasVerisi.ciftYaz();
  // Örnek 2: Cinsiyet bilgisi (string ve string)
  Cift<std::string, std::string> cinsiyetVerisi("Cinsiyetim", "ERKEK");
  cinsiyetVerisi.ciftYaz();
}
/*Program Çıktısı:
Yasim: 45
Cinsiyetim: ERKEK
*/
```

Kod 4.9: Sınıf Şablonunda Farklı Veri Tiplerini Kullanma

4.6 Faydalı Şablonlar

C++ dünyasındaki `<utility>` başlığı "isviçre çakısı" rolüne sahiptir. Bu başlık büyük veri yapıları sunmaz; bunun yerine algoritmaların ve sınıfların birbiriyle konuşmasını sağlayan, performansı artıran küçük ama hayati araçlar sunar. Bu araçlar tek başlarına birer dev olmasalar da, taşıma semantiği (*move semantics*) ve genel programlama (*generic programming*) gibi modern C++ kavramlarının temelini oluştururlar.

§4.6.1 std::pair

En eski ve en temel bileşenlerden biridir. İki farklı veri tipini tek bir birim olarak paketlemenizi sağlar. Genellikle bir fonksiyondan iki değer döndürmek gerektiğinde tercih edilir.

```
#include <iostream>
#include <utility>
#include <string>

int main() {
  // İkilinin oluşturulması
  std::pair<int, std::string> ogrenci(101, "Ali Yilmaz");
  // Elemanlara erişim (.first ve .second)
  std::cout << "ID: " << ogrenci.first << " Isim: " << ogrenci.second << std::endl;
```

```
  // std::make_pair ile tip belirtmeden oluşturma
  auto urun = std::make_pair("Defter", 25.50);
}
```

Kod 4.10: std::pair Şablonu

§4.6.2 std::move

C++11 ile gelen en devrimsel araçlardan biridir. Bir nesnenin içeriğini kopyalamak yerine, o içeriği başka bir nesneye "taşımanızı" sağlar. Özellikle büyük vektörler veya metinlerle çalışırken performansı dramatik şekilde artırır. `std::move()` aslında bir şeyi taşımaz; sadece nesneyi "taşınabilir" (*rvalue*) olarak işaretler. Asıl taşıma işlemini sınıfın taşıma yapıcısı (*move constructor*) metodu yapar.

```
#include <vector>
#include <string>
#include <utility>

int main() {
  std::string metin = "Bu cok buyuk bir veri kumesidir...";
  std::vector<std::string> liste;
  // Kopyalama yapmak yerine veriyi listeye taşıyoruz:
  liste.push_back(std::move(metin));
  // Bu noktadan sonra 'metin' değişkeni boştur veya belirsiz bir durumdadır.
}
```

Kod 4.11: std::move Şablonu

§4.6.3 std::swap

İki değişkenin değerlerini yer değiştirmek için kullanılır. `<utility>` içindeki `std::swap()`, eğer nesne destekliyorsa otomatik olarak taşıma semantiğini kullanır, bu da onu son derece hızlı kılar.

```
int x = 10, y = 20;
std::swap(x, y); // x=20, y=10 olur.
```

Kod 4.12: std::swap Şablonu

§4.6.4 std::forward

Şablon (*template*) yazarken, bir fonksiyonun aldığı parametreleri başka bir fonksiyona "olduğu gibi" (lvalue ise lvalue, rvalue ise rvalue olarak) aktarmak için kullanılır. Özellikle jenerik programlama (*generic programming*) yaparken vazgeçilmezdir.

```
template <typename T>
void araciFonksiyon(T&& arg) {
  // arg'ı gercekIslem fonksiyonuna geldiği haliyle aktarır
  gercekIslem(std::forward<T>(arg));
}
```

Kod 4.13: std::forward Şablonu

§4.6.5 Değişmez Değerler ve Diğerleri

C++17 ile gelen `std::as_const`, bir değişkeni geçici olarak salt okunur (*const*) hale getirir. Özellikle yanlışlıkla veri değiştirilmesini önlemek için fonksiyonlara parametre gönderirken kullanılır.

C++14 ile birlikte gelen `std::exchange`, bir değişkenin değerini yenisiyle değiştirirken, eski değerini döndürür. Özellikle **yıkıcı** (*destructor*) veya **taşıma atama işleci** (*move assignment*) yazarken çok kullanışlıdır.

```
int eski_deger = std::exchange(guncel_deger, 0); // guncel_deger'i 0 yapar, eskiyi dönd
    ürür.
```

Kod 4.14: std::exchange Şablonu

4.7 Akıllı Göstericiler

§4.7.1 Kaynak Edinimi İlk Değer Atamasıdır! Prensibi

Modern C++'ın bel kemiği sayılabilecek en önemli konulardan birine geldik. **RAII** (*Resource Acquisition Is Initialization*), yani "Kaynak Edinimi İlk Değer Atamasıdır" prensibi, C++'ı diğer dillerden ayıran ve bellek yönetimini güvenli kılan temel felsefedir. RAII, bir kaynağın (bellek, dosya göstericisi, ağ bağlantısı, mutex kilidi vb.) ömrünü bir nesnenin ömrüne bağlama tekniğidir. Bu prensip sayesinde C++'ta **çöp toplayıcı** (*garbage collector*) mekanizmasına ihtiyaç duyulmadan bellek sızıntıları önlenir.

RAII Prensibi

RAII'nin temel işleyişi yapıcı (*constructor*) ve yıkıcıların (*destructor*) gücüne dayanır. İki ana adıma dayanır: **Yapıcıda, kaynak (örneğin `new` ile ayrılan bellek veya açılan bir dosya) nesne oluşturulduğu anda edinilir. Yıkıcıda ise nesne kapsam (*scope*) dışına çıktığı anda kaynak otomatik olarak serbest bırakılır.**

Geleneksel programlamada en büyük risk, kaynağı serbest bırakmayı unutmaktır;

```
void riskliFonksiyon() {
  FILE* dosya = fopen("data.txt", "r");
  // ... işlemler yapılırken bir hata oluştu ve 'return' edildi ...
  if (hataOlustu) return; // Dosya kapatılmadı! (Sızıntı)

  fclose(dosya);
}
```

Kod 4.15: Riskli Kaynak Kullanımı

RAII Çözümü, **kendi kendini temizleyen sınıf yazmaktır**. Aynı kodu RAII ile aynı işlemi güvenli hale getirelim:

```
#include <iostream>
#include <fstream>
#include <string>

class DosyaYonetici {
private:
  std::ifstream dosya;
public:
  // Kaynak yapıcıda edinilir (Acquisition is Initialization)
  DosyaYonetici(std::string dosyaAdi) {
    dosya.open(dosyaAdi);
    std::cout << "Dosya acildi.\n";
  }
  // Kaynak yıkıcıda serbest bırakılır
  ~DosyaYonetici() {
    if (dosya.is_open()) {
      dosya.close();
      std::cout << "Dosya otomatik kapatildi.\n";
    }
  }
  void oku() { /* ... okuma islemleri ... */ }
};

void güvenliFonksiyon() {
  DosyaYonetici yönetici("test.txt");
  yönetici.oku();
  // Fonksiyon burada bitse de, hata fırlatsa da 'yönetici' nesnesi
  // kapsam dışı kalacağı için yıkıcısı çalışır ve dosya kapanır.
}
```

Kod 4.16: RAII ile Riskin Çözümü

C++'ta bir fonksiyon bittiğinde, o fonksiyon içindeki yerel nesnelerin yıkıcıları garantili olarak çağrılır. RAII bu garantiyi kullanarak kaynakları temizler.

Modern C++'ta kendi RAII sınıflarınızı yazabileceğiniz gibi, Standart Kütüphane (STL) zaten birçok hazır RAII aracı sunar:

- ◇ **Akıllı göstericiler** (`std::unique_ptr`, `std::shared_ptr`): Dinamik belleği (*heap*) RAII prensibiyle yönetir. `delete` yazmanıza gerek kalmaz.
- ◇ **Konteynerler** (`std::vector`, `std::string`): İçlerindeki dizileri otomatik büyütür ve yok ederler.
- ◇ **Kilit Yönetimi** (`std::lock_guard`): Multi-thread programlamada bir mutex'i kilitler ve kapsam bitince kilidi otomatik açar.

C++ dünyasında **akıllı göstericiler** (*samart pointers*), bellek yönetimini otomatik olarak yapan ve **bellek sızıntısı** (*memory leak*) gibi korkulu rüyaları ortadan kaldıran modern bir sarmalayıcı (*wrapper*) şablonlardır. Geleneksel C++ dilinde `new` ile tahsis edilen bir belleği `delete` ile silmeyi unutmak programın şişmesine neden olurken; Akıllı göstericiler, nesnenin ömrü dolduğunda belleği kendi kendine serbest bırakır. Göstericilerin işlendiği **??** başlığında, geleneksel göstericilerde şu üç temel risk vardır:

- ◇ Bellek Sızıntısı: `delete` satırı unutulursa bellek geri verilmez.
- ◇ Erken Silme: Bellek silindikten sonra hala o adrese erişmeye çalışan **sarkan gösericiler** (*dangling pointer*) programı çökertir.
- ◇ Çift Silme : Aynı adresi yanlışlıkla **iki kez serbest bırakmaya** (*double free*) çalışmak.

Göstericiler `<memory>` başlık dosyasında yer alır. Kullanmak için projemize dahil etmemiz gerekir. Üç ana akıllı gösterici bulunur ve aşağıda başlıklar halinde verilmiştir.

§4.7.2 std::unique_ptr ile Tekil Sahiplik

`std::unique_ptr` ile **Akıllı gösterici nesnesinin sadece tek bir sahibi olabilir**. Kopyalanamaz, sadece `std::move` fonksiyonu ile taşınabilir. Akıllı gösterici nesnenin ömrü tek bir kapsam (*scope*) içindeyse veya sahiplik net bir şekilde devredilecekse kullanılır. Bildiğimiz göstericiler kadar hızlıdır, ek yük getirmez.

```
#include <iostream>
#include <memory> // Akıllı göstericiler için

int main() {
  std::unique_ptr<int> ptr1 = std::make_unique<int>(10);
  // std::unique_ptr<int> p2 = p1; // HATA! Kopyalanamaz.
  std::unique_ptr<int> ptr2 = std::move(ptr1); // p1 artık boş, p2 sahibi.
}
```

Kod 4.17: std::unique_ptr ile Tekil Kaynak Sahipliği

§4.7.3 std::shared_ptr ile Paylaşılan Sahiplik

`std::shared_ptr` ile bir gösterici nesnenin birden fazla sahibi olabilir. Arka planda bir **referans sayacı** (*reference counter*) tutar. Yeni bir `shared_ptr` nesneye bağlandığında sayaç artar. Bir işaretçi yok olduğunda sayaç azalır. Sayaç sıfır olduğunda bellek temizlenir. Sayaç yönetimi nedeniyle çok küçük bir çalışma zamanı yükü vardır.

```
#include <iostream>
#include <memory> // Akıllı göstericiler için

int main() {
  auto s1 = std::make_shared<int>(100);
  {
    auto s2 = s1; // Sayaç 2 oldu.
    auto s3 = s1; // Sayaç 3 oldu.
  } // s2,s3 kapsam dışı, sayaç 1'e düştü.
  // main bitince sayaç 0 olur ve bellek silinir.
```

```
}
```

Kod 4.18: std::shared_ptr ile Paylaşılan Kaynak Sahipliği

§4.7.4 std::weak_ptr ile Zayıf Takipçi

`std::shared_ptr` tarafından yönetilen bir nesneye erişim sağlar ancak sayacı artırmaz. **Döngüsel bağımlılık** (*circular dependency*) sorununu çözmek için kullanılır. İki nesne birbirini `shared_ptr` ile tutarsa sayaç asla sıfırlanmaz; `weak_ptr` bu düğümü çözer. Aşağıdaki senaryoda bir `Anne` ve bir `Cocuk` nesnesi var. Anne çocuğunu, çocuk da annesini `shared_ptr` ile işaret ediyor. Döngüsel bağımlılık oluşmuştur.

```
#include <iostream>
#include <memory>

struct Cocuk; // Forward declaration: Cocuk diye bir tanımlama olacak!
struct Anne {
  std::shared_ptr<Cocuk> cocukPtr;
  ~Anne() { std::cout << "Anne yok edildi.\n"; }
};

struct Cocuk {
  std::shared_ptr<Anne> annePtr;
  ~Cocuk() { std::cout << "Cocuk yok edildi.\n"; }
};

void hataOlustur() {
  auto anne = std::make_shared<Anne>();
  auto cocuk = std::make_shared<Cocuk>();

  anne->cocukPtr = cocuk; // Anne çocuğu sahipleniyor (Sayaç: 1)
  cocuk->annePtr = anne; // Çocuk anneyi sahipleniyor (Sayaç: 1)
} // Fonksiyon biterken sayaçlar 0'a inmeliydi ama birbirlerini tuttukları için 1'de
    kalırlar! Bellek sızıntısı olur.
```

Kod 4.19: Paylaşılan Kaynak Örneği

`hataOlustur()` fonksiyonu bittiğinde ekranda hiçbir "yok edildi" yazısı görmezsiniz ancak bellek sızıntısı olmuştur. `weak_ptr`, bir nesneyi "gözlemler" ama onun sahibi olmaz (referans sayacını artırmaz). Döngüyü kırmak için çocuk nesnesinin annesini `weak_ptr` ile tutması gerekir.

```
#include <iostream>
#include <memory>

struct Cocuk; // Forward declaration: Cocuk diye bir tanımlama olacak!
struct Anne {
  std::shared_ptr<Cocuk> cocukPtr;
  ~Anne() { std::cout << "Anne yok edildi.\n"; }
};

struct Cocuk {
  std::weak_ptr<Anne> annePtr; // shared_ptr yerine weak_ptr kullandık!
  ~Cocuk() { std::cout << "Cocuk yok edildi.\n"; }
  void anneminAdiniDeSoyle() {
    // weak_ptr doğrudan kullanılamaz, önce shared_ptr'a dönüştürülmeli (lock)
    if (auto sPtr = annePtr.lock()) {
      std::cout << "Annem hala yasiyor.\n";
    } else {
      std::cout << "Annem bellekten silinmis.\n";
    }
  }
};

int main() {
  {
    auto anne = std::make_shared<Anne>();
```

```
    auto cocuk = std::make_shared<Cocuk>();

    anne->cocukPtr = cocuk;
    cocuk->annePtr = anne; // weak_ptr olduğu için Anne'nin sayacı artmaz!
  } // Kapsam biterken Anne silinir, Anne silindiği için Cocuk da silinir.
  std::cout << "Program sonu.\n";
}
/*Program Çıktısı:
Anne yok edildi.
Cocuk yok edildi.
Program sonu.
*/
```

Kod 4.20: Paylaşılan Kaynakta Zayıf Takipçi Örneği

Make Fonksiyonları

Örnek kodlarda akıllı gösterici imal etmekte kullanılan `make_` fonksiyonlarının kullanımı çok önemlidir.
Akıllı gösterici oluştururken doğrudan `new` kullanmak yerine `std::make_unique` ve `std::make_shared` kullanılması önerilir. Nesne oluşturulurken istisna (*exception*) oluşursa bellek sızıntısını önler. Nesne ve sayaç bloğu için tek bir bellek tahsisi `make_shared` ile yapılarak daha hızlı çalışır.

§4.7.5 Akıllı Gösterici Örnekleri

Dinamik değişken kullanımınında new ve delete işleçleriyle yapılan manuel denetim yerine aşağıdaki yeni nesil akıllı göstericileri kullanabiliriz;

```
#include <iostream>
#include <memory> // std::unique_ptr ve std::make_unique için şart

int main() {
  // 1. TEKİL DEĞİŞKEN (Modern Yöntem)
  // make_unique hem belleği tahsis eder hem de değeri içine koyar.
  std::unique_ptr<double> dPtr = std::make_unique<double>(3.14);
  std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr.get() << std::endl;

  // 2. TEKİL DEĞİŞKEN (Farklı İlklendirme)
  auto dPtr2 = std::make_unique<double>(3.1415);
  std::cout << "Double Deger 2: " << *dPtr2 << " | Adres: " << dPtr2.get() << std::endl
      ;
  // NOT: Burada 'delete dPtr' yazmıyoruz! Kapsam (main) bitince otomatik silinirler.
  // 3. DİNAMİK DİZİ (Modern Yöntem)
  int boyut = 5;
  // unique_ptr dizi olduğunu <double[]> ifadesinden anlar.
  std::unique_ptr<double[]> diziPtr = std::make_unique<double[]>(boyut);

  diziPtr[0] = 10.5;
  diziPtr[1] = 20.7;
  diziPtr[2] = 30.9;
  // Atanmayan 3. ve 4. indisler varsayılan olarak 0.0 ile doldurulur (make_unique ö
      zelliği).
  std::cout << "\n--- Akilli Dizi Elemanlari ---" << std::endl;
  for (int i = 0; i < boyut; i++) {
    std::cout << i << ". eleman: " << diziPtr[i] << " | Adres: " << &diziPtr[i] << std
        ::endl;
  }
  // KRİTİK: delete[] yazmanıza gerek yok. unique_ptr dizi olduğunu bildiği için
  // arka planda doğru silme işlemini (delete[]) kendisi yapar.
}
/*Program Çıktısı:
Double Deger: 3.14 | Adres: 0x2712e2b0
Double Deger 2: 3.1415 | Adres: 0x2712e6e0
```

```
--- Akilli Dizi Elemanlari ---
0. eleman: 10.5 | Adres: 0x2712e700
1. eleman: 20.7 | Adres: 0x2712e708
2. eleman: 30.9 | Adres: 0x2712e710
3. eleman: 0 | Adres: 0x2712e718
4. eleman: 0 | Adres: 0x2712e720
*/
```

Kod 4.21: Akıllı Göstericiler Örneği

Benzer şekilde dinamik nesne imalatını anlatan örneğini aşağıdaki gibi yazabiliriz;

```
#include <iostream>
#include <memory> // std::unique_ptr ve std::make_unique için gerekli

class Karmasik {
public:
  float gercek, sanal;
  // 1. Varsayılan Yapıcı
  Karmasik() : gercek(0.0f), sanal(0.0f) {
    std::cout << "Varsayilan yapici cagrildi" << std::endl;
  }
  // 2. Parametreli Yapıcı
  Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {
    std::cout << "Iki Parametreli yapici cagrildi" << std::endl;
  }
  // 3. Devredilen Yapıcı (Delegating Constructor)
  Karmasik(float pGercek) : Karmasik(pGercek, 0.0f) {
    std::cout << "Tek parametreli yapici cagrildi (Iki parametreliye devredildi)" <<
        std::endl;
  }
  // Yıkıcı (Destructor): Belleğin ne zaman silindiğini görmek için ekledik
  ~Karmasik() {
    std::cout << "Nesne bellekten siliniyor: " << gercek << " + " << sanal << "i" <<
        std::endl;
  }
  void yaz() const {
    std::cout << gercek << " + " << sanal << "i" << std::endl;
  }
};

int main() {
  std::cout << "--- Nesne 1 (Default) ---" << std::endl;
  // std::make_unique parametresiz çağrıldığında varsayılan yapıcıyı kullanır.
  auto k1 = std::make_unique<Karmasik>();
  k1->yaz();
  std::cout << "\n--- Nesne 2 (Parametreli) ---" << std::endl;
  // make_unique içine gönderilen argümanlar uygun yapıcıya iletilir.
  auto k2 = std::make_unique<Karmasik>(2.0f, 3.0f);
  k2->yaz();
  std::cout << "\n--- Nesne 3 (Tek Parametreli) ---" << std::endl;
  auto k3 = std::make_unique<Karmasik>(4.0f);
  k3->yaz();
  std::cout << "\n--- Program Sonu ---" << std::endl;
  // NOT: Burada 'delete' yazmıyoruz!  Program (veya kapsam) sona erdiğinde akıllı iş
      aretçiler otomatik olarak yıkıcıyı (~Karmasik) çağırır.
}
/*Program Çıktısı:
--- Nesne 1 (Default) ---
Varsayilan yapici cagrildi
0 + 0i

--- Nesne 2 (Parametreli) ---
Iki Parametreli yapici cagrildi
2 + 3i
```

```
--- Nesne 3 (Tek Parametreli) ---
Iki Parametreli yapici cagrildi
Tek parametreli yapici cagrildi (Iki parametreliye devredildi)
4 + 0i

--- Program Sonu ---
Nesne bellekten siliniyor: 4 + 0i
Nesne bellekten siliniyor: 2 + 3i
Nesne bellekten siliniyor: 0 + 0i
*/
```

Kod 4.22: Dinamik Nesne İmalatı Örneği

Akıllı göstericiler ile yapılan güvenli kod örneklerine tasarım desenleri başlığında bolca örnek bulabilirsiniz.

4.8 Standart Şablon Kütüphanesi

Standart şablon kütüphanesi STL(*Standart Template Library*), C++ dilinin en güçlü kütüphanesidir. Jenerik programlama (*generic programming*) ile birçok algoritma programcıya bu kütüphane ile sunulur. Şablon (*template*) kavramını anlamışsanız bu kütüphaneyi iyi kavrarsınız.

Standart şablon kütüphanesi, vektörler, listeler, kuyruklar ve yığınlar gibi popüler ve yaygın olarak kullanılan güçlü bir şablon sınıfları kümesidir. Bu kütüphane algoritmaları ve veri yapılarını içeren şablonlarla genel amaçlı sınıflar ve fonksiyonlar sağlar. Bu şablonlar başlıkta incelebilir;

1. **Konteynerler**: Belirli bir türdeki nesne kümeleri yani koleksiyonlarını yönetmek için kullanılan veri yapıları şablonudur. `deque`, `list`, `vector`, `map` gibi çeşitli farklı konteyner türleri vardır. Konteynerler, tuttukları verilerden bağımsız olarak hazırlanmış şablon şeklinde dinamik veri yapılarıdır (*dynamic data structure*).
2. **Algoritmalar**: Konteynerler üzerinde etki eder. Konteynerlerin içeriklerinin başlatılmasını, sıralanmasını, aranmasını ve dönüştürülmesini gerçekleştireceğiniz araçları sağlarlar.
3. **Yineleyiciler**: Koleksiyonlarının elemanları arasında gezinmek için kullanılır. Gezinme, koleksiyonların kapsayıcılarında veya kapsayıcıların alt kümeleri olabilir.
4. **Fonksiyon Nesneleri**: Bir fonksiyon sanki bir fonksiyonmuş gibi çağrılabilen bir nesnedir. Normal fonksiyonlar gibi çağrılabilir. Bu yetenek, **fonksiyon çağrı işleci** (*function call operator*) `()` yani fonksiyon parantezlerinin aşırı yüklenmesiyle elde edilir. Bu kelimelerin kısaltılmış hali olan **functor** olarak adlandırılırlar.

4.9 Konteyner Şablonları

Konteyner şablonları bize nesne kümelerini depolamak ve yönetmek için kullanılan veri yapılarıdır. Dinamik veri yapıları yapısal programlama sürecinde çokça geliştirilen ve artık standart hale gelen koleksiyonlardır. Üç başlıkta incelenebilirler;

1. **Ardışık konteynerler** (*sequential container*), elemanları belirli bir doğrusal düzende depolarlar. Elemanları ve sırasını ve düzenlemesini yönetmek için işlevlere doğrudan erişim sağlarlar;
    - ◇ `array` (Dizi): Bunlar sabit boyutlu eleman koleksiyonlarıdır.
    - ◇ `vector` (Vektör): Gerektiğinde boyutunu değiştirebilen dinamik bir dizidir.
    - ◇ `deque` (İki Uçlu Kuyruk): Her iki uçtan hızlı ekleme ve silmeye izin veren çift uçlu kuyruk.
    - ◇ `list` (Liste): Çift yönlü yinelemeye izin veren çift bağlantılı liste.
    - ◇ `forward_list` (İleri Liste): Verimli ekleme ve silmeye izin veren ancak yalnızca bir yönde gezinmeye izin veren tek bağlantılı listedir.
    - ◇ `string` (Metin): Diğer ardışık konteynerlere benzeyen dinamik bir karakter dizisidir.
2. **İlişkili konteynerler** (*associated container*), elemanları anahtarlara (*key*) göre hızlı bir şekilde geri almaya izin veren sıralı bir düzende depolar. Bu konteynerler, her bir elemanın ben-

zersiz bir anahtarla ilişkilendirildiği bir anahtar-değer (*key-value*) çifti yapısında çalışır. Bu anahtar, karşılık gelen değere hızlı erişim için kullanılır;
    - ◇ `set` (Küme): Belirli bir düzende sıralanmış benzersiz elemanların bir koleksiyonudur.
    - ◇ `map` (Harita): Anahtarların benzersiz olduğu anahtar-değer çiftlerinin bir koleksiyonudur.
    - ◇ `multiset` (Çoklu Küme): Bir kümeye benzer, ancak yinelenen elemanların izin verir.
    - ◇ `multimap` (Çoklu Harita): Bir haritaya benzer, ancak yinelenen anahtarlara izin verir.
3. **Sıralanmamış ilişkisel konteynerler** (*unordered associated container*), elemanları sırasız bir şekilde depolar ve anahtarlara (key) dayalı olarak verimli erişim, ekleme ve silmeye olanak tanır. Elemanların sıralı bir düzenini korumak yerine verileri düzenlemek için karma (*hash*) kullanırlar;
    - ◇ `unordered_set` (Sıralanmamış Küme): Belirli bir sırası olmayan, benzersiz elemanlardan oluşan bir koleksiyondur.
    - ◇ `unordered_map` (Sıralanmamış Harita): Belirli bir sırası olmayan, anahtar-değer çiftlerinden oluşan bir koleksiyondur.
    - ◇ `unordered_multiset` (Sıralanmamış Çoklu Küme): Belirli bir sırası olmayan, yinelenen elemanlara izin verir.
    - ◇ `unordered_multimap` (Sıralanmamış Çoklu Harita): Belirli bir sırası olmayan, yinelenen anahtarlara izin verir.

Konteyner adaptörleri (*container adapter*), mevcut konteynerler için farklı bir arayüz sağlar. Temel konteynerlerin davranışlarını belirli kullanım durumlarına uyacak şekilde adapte ederler;

- ◇ `stack` (Yığın): Son Giren İlk Çıkar **LIFO** (*last in, first out*) ilkesini izleyen bir veri yapısıdır.
- ◇ `queue` (Kuyruk): İlk Giren İlk Çıkar **FIFO** (*first in, first out*) ilkesini izleyen bir veri yapısıdır.
- ◇ `priority_queue` (Öncelikli Kuyruk): Elemanların önceliğe göre kaldırıldığı özel bir kuyruk türüdür.

§4.9.1 std::array

Dizileri, **??** başlığından biliyoruz. Ancak bu şablon her türlü veri tipiyle oluşturulabilir. Bu sınıf şablonunun iki parametresi vardır. `class T`, Dizi elemanlarının veri tipini belirtir. `std::size_t N` ise dizide kaç eleman olduğunu belirtir.

4.9.1.1 Diziyi Başlatma

Dizinin sayılarla ifade edilen (*scalar*) veri tiplerini barındırması halinde aşağıdaki şekilde ilk değer verilebilir ya da başlatılabilir);

```
//1. Dizilere ilk değer verme yöntemini kullanarak
std::array<int, 3> a { 0, 1, 2 };
// buna eşdeğer ifade:
std::array<int, 3> a = { 0, 1, 2 };
//2.Kopya yapıcı (copy constructor) kullanarak
std::array<int, 3> a { 0, 1, 2 };
std::array<int, 3> a2(a);
// buna eşdeğer ifade
std::array<int, 3> a2 = a;
//3.Tasıma yapıcısı (move constructor) kullanarak;
std::array<int, 3> a = std::array<int, 3>{ 0, 1, 2 };
```

Dizinin sayılarla ifade edilmeyen (*non scalar*) veri tiplerini barındırması halinde aşağıdaki şekilde ilk değer verilebilir ya da başlatılabilir);

```
struct A { int values[3]; };
//4.Tırnaklı parantez ile ilk değer verilebilir
std::array<A, 2> a{ 0, 1, 2, 3, 4, 5 };
// buna eşdeğer ifade:
std::array<A, 2> a = { 0, 1, 2, 3, 4, 5 };
//5.Tırnaklı parantezleri alt elemanlarda kullanarak;
```

```
  std::array<A, 2> a{ A{ 0, 1, 2 }, A{ 3, 4, 5 } };
  // buna eşdeğer ifade:
  std::array<A, 2> a = { A{ 0, 1, 2 }, A{ 3, 4, 5 } };
  //6.Tamamıyla tırnaklı parantez kullanarak;
  std::array<A, 2> a{{ { 0, 1, 2 }, { 3, 4, 5 } }};
  // buna eşdeğer ifade:
  std::array<A, 2> a = {{ { 0, 1, 2 }, { 3, 4, 5 } }};
  //7.Kopya yapıcı (copy constructor) kullanarak;
  std::array<A, 2> a{ 1, 2, 3 };
  std::array<A, 2> a2(a);
  // buna eşdeğer ifade:
  std::array<A, 2> a2 = a;
  //8.Taşıma yapıcısı (move constructor) kullanarak;
  std::array<A, 2> a = std::array<A, 2>{ 0, 1, 2, 3, 4, 5 };
```

4.9.1.2 Dizi Elemanlarına Erişim

1. `at(pos)` fonksiyonu ile: `pos` konumundaki elemana sınır denetimiyle bir referans döndürür. `pos`, konteynerin aralığında değilse, `std::out_of_range` türünde bir istisna atılır.

```
#include <array>

int main() {
  std::array<int, 3> arr;
  arr.at(0) = 3; //0. Elemana 3 atandı
  arr.at(1) = 4; //1. Elemana 4 atandı
  arr.at(2) = 6; //2. Elemana 6 atandı
  int a = arr.at(0); // a değişkenine 0. Dizi elemanı atandı
  int b = arr.at(1); // a değişkenine 1. Dizi elemanı atandı
  int c = arr.at(2); // a değişkenine 2. Dizi elemanı atandı
}
```

2. `operator[pos]` işleci ile: pos konumundaki elemana sınır denetimi olmadan bir başvuru döndürür. `pos`, konteynerin aralığında değilse, bir çalışma zamanı hatası (*segmentation violation*) oluşabilir. Bu yöntem, klasik dizilere eşdeğer eleman erişimi sağlar ve bu nedenle `at(pos)`'tan daha verimlidir.

```
#include <array>

int main() {
  std::array<int, 3> arr;
  arr[0] = 3;
  arr[1] = 4;
  arr[2] = 6;
  int a = arr[0];
  int b = arr[1];
  int c = arr[2];
}
```

3. `std::get<pos>` ile: `std::array` sınıfının üyesi olmayan bu fonksiyon, sınır denetimi olmaksızın derleme zamanı sabit konumu `pos`'taki elemana bir referans döndürür. `pos`, konteynerin aralığında değilse, bir çalışma zamanı hatası (*segmentation violation*) oluşabilir.

```
#include <array>

int main() {
  std::array<int, 3> arr;
  std::get<0>(arr) = 3;
  std::get<1>(arr) = 4;
  std::get<2>(arr) = 6;
  int a = std::get<0>(arr);
  int b = std::get<1>(arr);
  int c = std::get<2>(arr);
}
```

4. `front()` fonksiyonu ile: Konteynerdeki ilk elemana bir referans döndürür. Boş bir konteynerde

`front()`'u çağırmak tanımsızdır.

```
#include <array>

int main()  {
  std::array<int, 3> arr{ 2, 4, 6 };
  int a = arr.front();
}
```

5. `back()` fonksiyonu ile: Konteynerdeki son elemana bir referans döndürür. Boş bir konteynerde `back()`'u çağırmak tanımsızdır.

```
#include <array>

int main()  {
  std::array<int, 3> arr{ 2, 4, 6 };
  int a = arr.back();
}
```

6. `data()` fonksiyonu ile: Eleman depolaması olarak hizmet eden altta yatan diziye gösterici döndürür.

```
#include <iostream>
#include <array>

void yaz(const int* p, std::size_t size){
  for (std::size_t i = 0; i < size; ++i)
    std::cout << p[i] << ' ';
}

int main (){
  std::array<int, 3> a { 0, 1, 2 };
  yaz(a.data(), 3);
}
```

4.9.1.3 Dizi Elemanları Üzerinde Yineleme

```
#include <iostream>
#include <array>

int main() {
  std::array<int, 3> arr = { 1, 2, 3 };
  for (auto i : arr)
    std::cout << i << std::endl;
}
```

4.9.1.4 Dizi Boyutunu Kontrol Etme

```
#include <iostream>
#include <array>

int main() {
  std::array<int, 3> arr = { 1, 2, 3 };
  std::cout << arr.size() << std::endl;
}
```

4.9.1.5 Dizi Elemanlarını Tek Seferde Değiştirme

```
#include <iostream>
#include <array>

int main() {
  std::array<int, 3> arr = { 1, 2, 3 };
  arr.fill(100);
}
```

§4.9.2 std::vector

Bir vektör, otomatik olarak işlenen depolamaya sahip dinamik bir dizidir. Bir vektördeki elemanlara, bir dizideki elemanlar kadar verimli bir şekilde erişilebilir; avantajı, vektörlerin boyut olarak dinamik olarak değişebilmesidir. Depolama açısından vektör verileri (genellikle) dinamik olarak tahsis edilmiş belleğe yerleştirilir, bu nedenle bazı küçük ek yükler gerektirir; bunun tersine `std::array` koleksiyonunda, beyan edilen boyuta göre otomatik depolama kullanır ve bu nedenle herhangi bir ek yüke sahip değildir.

Vertörler `std:array` gibi başlatılabilir.

4.9.2.1 Vektör Elemanlarına Erişim

Vektör elemanlarına `[]` işleci ve `at()` fonksiyonu ile erişebiliriz.

```
#include <iostream>
#include <vector>

int main() {
  std::vector<int> v{ 1, 2, 3 };
  // [] işleci:
  int a = v[1];
  v[1] = 4;
  // at() fonksiyonu:
  int b = v.at(2);
  v.at(2) = 5;
  //int c = v.at(3); // Hata 4. Eleman Yok!
  for (std::size_t i = 0; i < v.size(); ++i) {
    v[i] = 1;
  }
}
```

Ayrıca `front()` ve `back()` fonksiyonları ile de vektör elemanlarına erişilebilir;

```
  std::vector<int> v{ 4, 5, 6 };
  int a = v.front();
  v.front() = 3;
  int b = v.back(); // v.back() eşdeğerdir: v[v.size() - 1]
  v.back() = 7;
```

4.9.2.2 Vektöre Eleman Ekleme ve Çıkarma

Vektöre eleman ekleme `push_back()`, vektörden eleman çıkarma `pop_back()` ve boş olduğunu kontrol etme `empty()` fonksiyonları ile yapılabilir;

```
#include <iostream>
#include <vector>

int main () {
  std::vector<int> v;
  int toplam (0);
  for (int i=1;i<=10;i++) v.push_back(i); //vektör oluştutulup elemanlar ekleniyor
  while (!v.empty()) //vektörün boş olup olmadığı kontrol edilip ilerleniyor
  {
    toplam += v.back();
    v.pop_back(); //son elemanı vektörden çıkar
  }
  std::cout << "toplam: " << toplam << std::endl;
}
```

4.9.2.3 Vektör Verisi

Vektörde `data()` yöntemi, `std::vector`'un elemanlarını dahili olarak depolamak için kullandığı ham belleğe bir gösterici döndürür. Bu gösterici, genellikle vektör verilerini C tarzı bir dizi bekleyen eski koda geçirirken kullanılır.

```
  std::vector<int> v{ 1, 2, 3, 4 };
  int* p = v.data();
  *p = 4; // v[0]=4
  ++p;
  *p = 3; // v[1]=3
  p[1] = 2;
  *(p + 2) = 1;
```

Aşağıda bir diziye benzeyen ancak büyümesi durumunda kendi depolama gereksinimlerini otomatik olarak karşılayan vektör konteynerini kullanan program örneği verilmiştir;

```
#include <iostream>
#include <vector>

int main() {
  std::vector<double> vec; // int değerler tutacak std::vector konteyneri
  int i;
  std::cout << "std::vector Uzunluğu = " << vec.size() << std::endl;
  for(i = 0; i < 5; i++) { // 5 değer vektöre koyuluyor.
    vec.push_back(i*1.5);
  }
  std::cout << "Vektörün Son Uzunluğu = " << vec.size() << std::endl;
  for(i = 0; i < 5; i++) {// vektör elemanlarına erişiliyor
    std::cout << "vec[" << i << "] = " << vec[i] << ",";
  }
  std::cout << std::endl;
  // yineleyiciler üzerinden vektör elemanlarına erişim:
  std::vector<double>::iterator v = vec.begin();
  while( v != vec.end()) {
    std::cout  << *v << ",";
    v++;
  }
  std::cout << std::endl;
}
/*Program Çıktısı:
std::vector Uzunluğu = 0
Vektörün Son Uzunluğu = 5
vec[0] = 0,vec[1] = 1.5,vec[2] = 3,vec[3] = 4.5,vec[4] = 6,
0,1.5,3,4.5,6,
*/
```

§4.9.3 std::map, std::multimap ve std::pair

Haritalar, benzersiz anahtarlara sahip **anahtar-değer** (*key-value*) çiftleri (*pair*) içeren sıralı bir ilişkisel konteynerlerdir. Anahtarlar, `compare()` karşılaştırma işlevi kullanılarak sıralanır. Arama, kaldırma ve ekleme işlemleri olan search(), `insert()`, `delete()` için logaritmik karmaşıklığa sahiptir.

- ◇ `std::map` veya `std::multimap`'ten herhangi birini kullanmak için <map> başlık dosyası projeye dahil edilmelidir.
- ◇ `std::map` ve `std::multimap` her ikisi de elemanlarının anahtarların artan sırasına göre sıralanmış halde tutar.
- ◇ `std::multimap` durumunda, aynı anahtarın değerleri için sıralama yapılmaz.
- ◇ `std::map` ve `std::multimap` arasındaki temel fark, `std::map`'in aynı anahtar için yinelenen değerlere izin vermemesidir.

4.9.3.1 Çiftleri Oluşturma

Çiftler aşağıdaki gibi oluşturulup karşılaştırılabilir;

```
  std::pair<int, int> p1 = std::make_pair(1, 2);
  std::pair<int, int> p2 = std::make_pair(2, 2);
  if (p1 == p2)
  std::cout << "eşit";
  else
```

```
  std::cout << "eşit değil!";
```

4.9.3.2 Çiftleri Sıralama

Çiftler bir konteyner içindeyken de küçük işleci ile sıralanabilir;

```
#include <iostream>
#include <utility>
#include <vector>
#include <algorithm>
#include <string>

int main() {
  std::vector<std::pair<int, std::string>> v = { {2, "Ali"}, {2, "Mustafa"}, {1, "İlhan
      "} };
  std::sort(v.begin(), v.end());
  for(const auto& p: v)
    std::cout << "(" << p.first << "," << p.second << ") ";
}
/*Program Çıktısı:
(1,İlhan) (2,Ali) (2,Mustafa)
*/
```

4.9.3.3 Haritalarda Yineleme

Haritalarda elemanlar anahtar değer ikilisidir ve aşağıdaki şekilde yineleyiciler ile erişilir;

```
#include <iostream>
#include <map>

int main() {
  std::map < int, std::string > siralama {
    std::make_pair(2, "ahmet"),
    std::make_pair(1, "ali") };
  siralama[1]="mustafa";
  std::cout << siralama.at(2) << std::endl; // ahmet
  auto it = siralama.begin();
  std::cout << it->first << " : " << it->second << std::endl; // "1 : mustafa"
  it++;
  std::cout << it->first << " : " << it->second << std::endl; // "2 : ahmet"
  std::map < std::string , int> notlar {
    std::make_pair("ahmet",100),
    std::make_pair("ali",90),
    std::make_pair("mustafa",85)
  };
  notlar["mustafa"]=95;
  std::cout << notlar.at("ali") << std::endl; // 90
  auto it2 = notlar.rbegin(); //tersten
  std::cout << it2->first << " : " << it2->second << std::endl; // "mustafa: 95"
  it2++;
  std::cout << it2->first << " : " << it2->second << std::endl; // "ali: 90"
}
```

4.9.3.4 std::multimap Örneği

```
#include <iostream>
#include <map>

int main() {
  std::multimap < int, std::string > mmp {
    std::make_pair(2, "ahmet"),
    std::make_pair(1, "ali"),
    std::make_pair(2, "mustafa")

  };
  auto it = mmp.begin();
```

```
  std::cout << it->first << " : " << it->second << std::endl; //"1 : ali"
  it++;
  std::cout << it->first << " : " << it->second << std::endl; //"2 : ahmet"
  it++;
  std::cout << it->first << " : " << it->second << std::endl; //"2 : mustafa"
  std::map < int, std::string > mp {
    std::make_pair(2, "ahmet"),
    std::make_pair(1, "ali"),
    std::make_pair(2, "mustafa")
    // Aynı anathardan var!
  };
  auto it2 = mp.rbegin(); //tersten
  std::cout << it2->first << " : " << it2->second << std::endl; //"2 : ahmet"
  it2++;
  std::cout << it2->first << " : " << it2->second << std::endl; //"1 : ali"
}
```

4.9.3.5 Haritaya Çift Ekleme

Haritaya anahtar değerler aşağıdaki şekilde eklenebilir;

```
  std::map< std::string, size_t > meyvesayisi;
  meyvesayisi.insert({"üzüm", 20});
  meyvesayisi.insert(make_pair("portakal", 30));
  meyvesayisi.insert(pair<std::string, size_t>("muz", 40));
  meyvesayisi.insert(map<std::string, size_t>::value_type("çilek", 50));
```

`insert()` fonksiyonu bir yineleyici ve bir `bool` değerden oluşan bir çift döndürür. Döndürülen değer `true` ise haritaya çift eklenmiştir. Değilse eklenmeye çalışılan çift zaten haritada vardır.

```
  auto success = meyvesayisi.insert({"üzüm", 20});
  if (!success.second) { // zaten üzüm var
    success.first->second += 20; // üzüm müktarı değiştirlebilir
  }
  meyvesayisi["karpuz"]=10; //haridata olmayan karpuz eklenmişir.
  meyvesayisi.insert({{"şeftali", 1}, {"kayısı", 1}, {"limon", 1}, {"mango", 7}});
```

Konteynerlere başlatma listeleriyle de eleman eklenebilir;

```
  std::map< std::string, size_t > meyvelistesi{ {"zeytin", 0}, {"elma", 0}, {"kavun",
      0}};
  meyvesayisi.insert(meyvelistesi.begin(), meyvelistesi.end());
```

Haritaya eklenecek çiftler yukarıdaki örneklerde olduğu gibi `make_pair()` ve `emplace()` yöntemleriyle hazırlanabilir;

```
  meyvesayisi.emplace("Harnup", 200);
```

Haritalarda bir anahtarın ilk oluşumunun yineleyicisini almak için `find()` fonksiyonu kullanılabilir. Anahtar mevcut değilse `end()` döndürür;

```
  std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
  auto it = mmp.find(6);
  if(it!=mmp.end())
  std::cout << it->first << ", " << it->second << std::endl; // 6, 5
  else
  std::cout << "Aranan Anahtar Bulunamadı!" << std::endl;
  it = mmp.find(66);
  if(it!=mmp.end())
  std::cout << it->first << ", " << it->second << std::endl;
  else
  std::cout << "Aranan anahtar bulunamadı!" << std::endl;
```

4.9.3.6 Haritada Çift Arama

Haritalarda bir anahtarın mevcut olduğunu bulmanın bir başka yolu da `count()` fonksiyonunu kullanmaktır. Bu fonksiyon, belirli bir anahtarla ilişkili değer sayısını sayar;

```
std::map< int , int > mp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
if(mp.count(3) > 0)
std::cout << "Anahtar bulundu!" << std::endl;
else
std::cout << "Anahtar Bulunamadı!" << std::endl;
```

4.9.3.7 Haritaları Başlatma

Haritaları tanımlama ve ilk değer verme;

```
std::multimap < int, std::string > mmp { std::make_pair(2, "C Lang"),
  std::make_pair(1, "CPP Lang"),
  std::make_pair(2, "Java Lang") };
std::map < int, std::string > mp { std::make_pair(2, "C lang"),
  std::make_pair(1, "CPP Lang"),
  std::make_pair(2, "Java Lang") // Eklenmeyecek!
};
// Yineleyici kullanarak;
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6,
    7} };
auto it = mmp.begin();
std::advance(it,3); //baştan üçüncü çifte git {6, 5}
std::map< int, int > mp(it, mmp.end()); // {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6, 7}
//std::pair dizisi kullanarak;
std::pair< int, int > arr[10];
arr[0] = {1, 3};
arr[1] = {1, 5};
arr[2] = {2, 5};
arr[3] = {0, 1};
std::map< int, int > mp(arr,arr+4); //{0 , 1}, {1, 3}, {2, 5}
//std::pair vektörü kullanarak
std::vector< std::pair<int, int> > v{ {1, 5}, {5, 1}, {3, 6}, {3, 2} };
std::multimap< int, int > mp(v.begin(), v.end()); // {1, 5}, {3, 6}, {3, 2}, {5, 1}
```

Haritaların boş olup olmamasına bağlı olarak `true` veya `false` döndüren bir üye `empty()` fonksiyonuna sahiptir. Ayrıca sahip olunan eleman sayısı da `size()` üye fonksiyonuna da sahiptir.

```
std::map<std::string , int> rank {{"facebook.com", 1} ,{"google.com", 2}, {"youtube.
    com", 3}};
if(!rank.empty())
std::cout << "Hatitadadaki eleman sayısı: " << rank.size() << std::endl;
else
std::cout << "Harita Boş!" << std::endl;
```

4.9.3.8 Karma Harita ve Düzenlenmemiş Harita

Bir de **karma harita** (*hash map*) vardır. **Düzenlenmemiş harita** (*unordered map*) olarak da adlandırılan bu haritalar, normal bir haritaya benzer şekilde anahtar-değer çiftlerini depolar. Ancak elemanları anahtara göre sıralamaz. Bunun yerine, anahtar için bir **karma** (*hash*) değer, ihtiyaç duyulan anahtar-değer çiftlerine hızlı bir şekilde erişmek için kullanılır;

```
std::unordered_map<std::string, size_t> meyveSayisi;
```

4.9.3.9 Haritadan Çift Silme

Haritadaki elemanlar aşağıdaki şekilde silinebilir;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
mmp.clear(); //hepsini sil
auto it = mmp.begin();
std::advance(it,3); // baştan üçüncü elemana git {6, 5}
```

```
  mmp.erase(it); // {1, 2}, {3, 4}, {8, 9}, {3, 4}, {6, 7}
```

Belli anahtarlara sahip elemanlar `erase()` üye fonksiyonu ile silinebilir;

```
  std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
  mmp.erase(6); // {1, 2}, {3, 4}, {3, 4}, {8, 9}
```

§4.9.4 std::list ve std::forwardlist

Konteynerin herhangi bir yerinden öğelerin hızlı bir şekilde eklenmesini ve kaldırılmasını istiyorsak ileri liste (`std::forward_list`) iyi bir seçimdir. Hızlı rastgele erişim desteklenmez. Bu konteyner, `std::list` ile karşılaştırıldığında, çift yönlü yineleme gerekmediğinde daha fazla alan tasarrufu sağlar.

```
#include <forward_list>
#include <list>
#include <string>
#include <iostream>

template<typename T> std::ostream& operator<<(std::ostream& s, const std::forward_list<
    T>& v) {
  s << "Forward List";
  s.put('[');
  char comma[3] = {'\0', ' ', '\0'};
  for (const auto& e : v) {
    s << comma << e;
    comma[0] = ',';
  }
  return s << ']';
}

template<typename T> std::ostream& operator<<(std::ostream& s, const std::list<T>& v) {
  s << "List";
  s.put('[');
  char comma[3] = {'\0', ' ', '\0'};
  for (const auto& e : v) {
    s << comma << e;
    comma[0] = ',';
  }
  return s << ']';
}

int main() {
  std::forward_list<std::string> words1 {"İlhan", "Ahmet", "Mehmet", "Mustafa", "
      Abdullah"};
  std::cout << "İsimler1: " << words1 << '\n';
  std::list<std::string> words2 {"İlhan", "Ahmet", "Mehmet", "Mustafa", "Abdullah"};
  std::cout << "İsimler2: " << words2 << '\n';
}
/* Program Çıktısı:
İsimler1: Forward List[İlhan, Ahmet, Mehmet, Mustafa, Abdullah]
İsimler2: List[İlhan, Ahmet, Mehmet, Mustafa, Abdullah]
*/
```

§4.9.5 std::set ve std::multiset

Küme (`std::set`), elemanları sıralanmış ve benzersiz olan bir tür konteynerdir. Kümede aynı eleman iki defa bulunmaz! Ancak `std::multiset` birden fazla eleman aynı elemana sahip olabilir.

```
#include <iostream>
#include <set>
#include <stdlib.h>

struct custom_compare final {
  bool operator() (const std::string& left, const std::string& right) const
```

```
  {
    int nLeft = atoi(left.c_str());
    int nRight = atoi(right.c_str());
    return nLeft < nRight;
  }
};

int main () {
  std::set<std::string> sut({"1", "2", "5", "23", "6", "290"});
  std::cout << "Ön tanımlı sıralama std::set<std::string> :" << std::endl;
  for (auto &&data: sut)
    std::cout << data << ", ";
  std::set<std::string, custom_compare> sut_custom({"1", "2", "5", "23", "6", "290"},
      custom_compare{});
  std::cout << std::endl << "Özel Sıralama :" << std::endl;
  for (auto &&data : sut_custom)
    std::cout << data << ", ";
  auto compare_via_lamda = [](auto &&lhs, auto &&rhs){ return lhs > rhs; };
  using set_via_lamda = std::set<std::string, decltype(compare_via_lamda)>;
  set_via_lamda sut_reverse_via_lamda({"1", "2", "5", "23", "6", "290"},
      compare_via_lamda);
  std::cout << std::endl << "Lamda Sıralaması :" << std::endl;
  for (auto &&data : sut_reverse_via_lamda)
    std::cout << data << ", ";
  return 0;
}
/*g++ main.cpp -std=C++14 ile derlenen Program Çıktısı:
Ön tanımlı sıralama std::set<std::string> :
1, 2, 23, 290, 5, 6,
Özel Sıralama :
1, 2, 5, 6, 23, 290,
Lamda Sıralaması :
6, 5, 290, 23, 2, 1,
*/
```

§4.9.6 std::optional

Değeri mevcut olabilen veya olmayabilen bir veri tipi temsil etmek için kullanılan `std::optional` olarak tanımlanan değişkenler isteğe bağlı/belki tipleri olarak da bilinir.

C++17'den önce, `nullptr` değerine sahip göstericilere sahip olmak genellikle bir değerin yokluğunu temsil ediyordu. Bu, dinamik olarak tahsis edilmiş ve zaten göstericiler tarafından yönetilen büyük nesneler için iyi bir çözümdür. Ancak, bu çözüm nadiren göstericiler tarafından dinamik olarak tahsis edilen veya yönetilen `int` gibi küçük veya ilkel türler için iyi çalışmaz. Çünkü bu veri tipinde tanımlanan bir değişken her zaman değeri vardır. `std::optional` bu yaygın soruna uygulanabilir bir çözüm sağlar.

```
#include <iostream>
#include <optional>
#include <string>

struct Hayvan {
  std::string adi;
};

struct Person {
  std::string adi;
  std::optional<Hayvan> evcilhayvan;
};

int main() {
  Person person;
  person.adi = "Ali";
  if (person.evcilhayvan) {
    std::cout << person.adi << " Adlı Kişinin " << person.evcilhayvan->adi
```

```
    << "Evcil Hayvanı Var" << std::endl;
  }
  else {
    std::cout << person.adi << " adlı kişinin evcil hayvanı yok!" << std::endl;
  }
}
/* g++ -std=c++17 main.cpp olarak derlenir. Program Çıktısı:
Ali adlı kişinin evcil hayvanı yok!
*/
```

Fonksiyondan geri dönüş değeri olarak da `std::optional` kullanılabilir;

```
std::optional<float> divide(float a, float b) {
  if (b!=0.f) return a/b;
  return {};
}
```

`std:optional` olarak tanımlı bir değeri işlemek için `value_or()` kullanılır;

```
void print_name( std::ostream& os, std::optional<std::string> const& name ) {
  std::cout << "Name is: " << name.value_or("<name missing>") << '\n';
}
```

§4.9.7 std::function ve std::bind

Herhangi bir fonksiyona kılıf çekmek için kullanılır. C tipi fonksiyon göstericisi kullanmak yerine C++ dilinde `std::function` kullanılır.

```
#include <iostream>
#include <functional>

std::function<void(int , const std::string&)> myFuncObj;
void theFunc(int i, const std::string& s) {
  std::cout << s << ": " << i << std::endl;
}

int main(int argc, char *argv[]) {
  myFuncObj = theFunc;
  myFuncObj(10, "hello world");
}
```

Argümanlarla bir fonksiyonu geri çağırmamız gereken bir durumda `std::bind` ile kullanılan `std::function` aşağıda gösterildiği gibi çok güçlü bir tasarım yapısı verir.

```
#include <iostream>
#include <functional>

class Ocak {
public:
  std::function<void(int, const std::string&)> YumurtaKaynadi = nullptr;
  void YumurtaKaynat() {
    if (YumurtaKaynadi) {
      YumurtaKaynadi(10, "Yumurta Pişti");
    }
  }
};

class Kisi {
public:
  Kisi() {
    auto yumurtaPisinceHaberVer = std::bind(&Kisi::eventHandler, this,
    std::placeholders::_1,
    std::placeholders::_2);
    ocakNesnesi.YumurtaKaynadi = yumurtaPisinceHaberVer;
  }
  void eventHandler(int i, const std::string& s) {
```

```
    std::cout << i << " dakikada "<< s <<std::endl;
  }
  void OcaktaYumurtaKaynat() {
    ocakNesnesi.YumurtaKaynat();
  }
  Ocak ocakNesnesi;
};

int main(int argc, char *argv[]) {
  Kisi ali;
  ali.OcaktaYumurtaKaynat();
}
/*Program Çıktısı:
10 dakikada Yumurta Pişti
*/
```

Aşağıda `std:function` ile çeşitli fonksiyonların çağrılmasını gösteren bir program verilmiştir;

```
#include <iostream>
#include <functional> //std::function, std::placeholders
#include <vector>  //std:vector

double c_function(int x, float y, double z) {
  double res = x + y + z;
  std::cout << "c_function çağrıldı: " << x << "+" << y << "+" << z << "=" << res  <<
      std::endl;
  return res;
}

struct yapi {
  double yapi_function(int x, float y, double z) {
    double res = x + y + z;
    std::cout << "yapi::yapi_function çağrıldı: " << x << "+" << y << "+" << z << "="
        << res << std::endl;
    return res;
  }
  double farkli_yapi_function(int x, double z, float y, long xx) {
    double res = x + y + z + xx;
    std::cout << "farkli_yapi_function çağrıldı: "  << x << "+" << y << "+" << z << "+"
         << xx << "=" << res
    << std::endl;
    return res;
  }
  double operator()(int x, float y, double z) {
    double res = x + y + z;
    std::cout << "yapi::operator() çağrıldı: " << x << "+" << y << "+" << z << "=" <<
        res << std::endl;
    return res;
  }
};

int main(void) {
  using function_type = std::function<double(int, float, double)>;
  yapi ornekyapi;
  std::vector<function_type> bindings;
  function_type var1 = c_function;
  bindings.push_back(var1);
  function_type var2 = std::bind(&yapi::yapi_function, ornekyapi, std::placeholders::_1
      , std::placeholders::_2, std::placeholders::_3);
  bindings.push_back(var2);
  function_type var3 = std::bind(&yapi::farkli_yapi_function, ornekyapi, std::
      placeholders::_1, std::placeholders::_3, std::placeholders::_2, 11l);
  bindings.push_back(var3);
  function_type var4 = ornekyapi; //operator()
  bindings.push_back(var4);
  function_type var5 = [](int x, float y, double z) {
```

```
    double res = x + y + z;
    std::cout << "lamda çağrıldı: " << x << "+" << y << "+" << z << "=" << res << std::
        endl;
    return res;
  };
  bindings.push_back(var5);
  std::cout << "Vektöre saklana fonksiyonlar test ediliyor: x = 1, y = 2, z = 3" << std
      ::endl;
  for (auto f : bindings)
    f(1, 2, 3);
}
/*Program Çıktısı:
Vektöre saklana fonksiyonlar test ediliyor: x = 1, y = 2, z = 3
c_function çağrıldı: 1+2+3=6
yapi::yapi_function çağrıldı: 1+2+3=6
farkli_yapi_function çağrıldı: 1+2+3+11=17
yapi::operator() çağrıldı: 1+2+3=6
lamda çağrıldı: 1+2+3=6
*/
```

§4.9.8 std::tuple

Farklı tiplerdeki değerler de dahil olmak üzere herhangi bir sayıda değeri tek bir dönüş nesnesine toplamak için `std::tuple` kullanılır.

```
std::tuple<int, int, int, int> foo(int a, int b) { // or auto (C++14)
  return std::make_tuple(a + b, a - b, a * b, a / b);
  /*dört farklı int değeri geri döndürülüyor */
}
```

C++ 2017 uyarlamasından sonra tırnaklı parantez olan başlatma listesi kullanılır;

```
std::tuple<int, int, int, int> foo(int a, int b) {
  return {a + b, a - b, a * b, a / b};
}
```

Döndürülen tuple'dan değerleri almak zahmetlidir ve `std::get` şablon fonksiyonunun kullanımını gerektirir:

```
auto mrvs = foo(5, 12);
auto add = std::get<0>(mrvs);
auto sub = std::get<1>(mrvs);
auto mul = std::get<2>(mrvs);
auto div = std::get<3>(mrvs);
```

Eğer tipler fonksiyon dönmeden önce bildirilebiliyorsa, o zaman `std::tie` bir tuple'ı mevcut değişkenlere açmak için kullanılabilir:

```
int add, sub, mul, div;
std::tie(add, sub, mul, div) = foo(5, 12);
```

`std::tie` ile döndürülen değerlerden birine ihtiyaç duyulmuyorsa `std::ignore` kullanılabilir:

```
std::tie(add, sub, std::ignore, div) = foo(5, 12);
```

Yapılandırılmış bağlamalar `std::tie`'dan kaçınmak için kullanılabilir:

```
auto [add, sub, mul, div] = foo(5,12);
```

§4.9.9 std::string

C dilinde metinler (*string*), bellekte son karakteri **boş karakter** (*NULL Character*) olan `'\0'` ile sonlanan karakter dizisi olarak saklanırlar.

Modern C++ dilinde, `std::string` şablonu metinler için kullanılır. C dilinden kalan karakter

dizisi yaklaşımını kullanmak çok daha düşük seviyelidir ve bellek yönetimi tamamen yazılımcının sorumluluğundadır. C++ `std::string` sınıfı, `std` isim uzayının bir parçasıdır ve 1998'de standartlaştırılmıştır.

Metinleri tutan değişkenler aşağıdaki şekilde tanımlanabilir;

```
std::string metin1 = "Merhaba";
std::string metin2("İlhan");
std::string metin3={'M','e','r','h','a','b','a'}; //Sonunda '\0' karakteri olamadan
    karakter dizisi
std::string metin4=metin2; //önceden tanımlı bir metinden
std::string metin5(metin3); //önceden tanımlı bir metinden yapıcı ile
```

Metinler, belirlenen bir ayraç metni ile belirteç (*token*) denilen küçük parçalara `strtok()` fonksiyonu ile ayrılabilir;

```
std::string str{ "Pijamalı hasta yağız şoföre çabucak güvendi." };
std::vector<std::string> tokens;
for (auto i = strtok(&str[0], " "); i != NULL; i = strtok(NULL, " "))
tokens.push_back(i);
```

Bir `std::string` yapısı verilerine `const char*` göstericisi ile erişim sağlamak için `c_str()` üye yöntemi kullanılır;

```
//C++14 ile derlenmeli
#include <iostream>
#include <string>
#include <memory>    // std::unique_ptr ve std::make_unique için
#include <algorithm> // std::copy için

int main()  {
  // --- 1. KISIM: Doğrudan Gösterici Erişimi ---
  std::string str("This is a string.");
  const char* cstr_ptr = str.c_str(); // cstr_ptr bu metni gösterir: "This is a string
      .\0"
  const char* data_ptr = str.data();  // data_ptr bu metni gösterir: "This is a string
      .\0"
  std::cout << "Original c_str: " << cstr_ptr << "\n";
  // --- 2. KISIM: Nesneden Bağımsız Güvenli Kopya Oluşturma ---
  // unique_ptr ile heap üzerinde str.size() + 1 kadar yer ayırıyoruz ('\0' dahil)
  std::unique_ptr<char[]> cstr_copy = std::make_unique<char[]>(str.size() + 1);
  // std::copy için hedef pointer adresi 'cstr_copy.get()' olarak geçilmelidir
  std::copy(str.data(), str.data() + str.size(), cstr_copy.get());
  cstr_copy[str.size()] = '\0'; // NULL karakter sonuna eklenir
  std::cout << "Kopyalanan metin: " << cstr_copy.get() << std::endl;
}
```

Buradan da anlaşılacağı üzere `std::string` şablonu arka planında metinler karakter dizileri şeklinde tutulur. `std:string` altında yatan `const char*` üzerinden işlem yapmak yerine C++17 ile birlikte gelen sabit metin şablonu olan `std::string_view` üzerinden işlemler yapılır. Değiştirilemeyen metin verileri gerektiren işlevler için üstünlük sağlar;

```
//1)Alt metin elde etmek için kopya oluşturarak;
std::string str = "Çoook uzun biiiir metiiiiiin";
//'string::substr' returns a new string (expensive if the string is long)
std::cout << str.substr(15, 10) << '\n';
//2)Hiçbir kopya oluşturmadan;
std::string_view view = str;
// string_view::substr returns a new string_view
std::cout << view.substr(15, 10) << '\n';
```

std::string	Karakter Dizisi
`std::string`, akış (*stream*) olarak temsil edilebilen nesneleri tanımlar.	`'\0'` olarak kodlanan boş karakter (*NULL Character*), karakterlerden oluşan bir karakter dizisini sonlandırır.
`std::string` dizi bozulması meydana gelmez çünkü dizeler nesneler olarak temsil edilir.	Bir dizinin tip ve boyut kaybına dizinin bozulması (*decay of array*) denir. Bu durum genellikle diziyi değer veya gösterici ile fonksiyona geçirdiğimizde ortaya çıkar. Karakter dizisinde böyle bir durum ortaya çıkabilir.
Bir `std::string` sınıfı, metinleri işlemek için çok sayıda yöntem sağlar.	Karakter dizileri, metinleri işlemek için C++ dilinde yerleşik fonksiyonlar sunmaz.
`std::string` metinlerine bellek dinamik olarak tahsis edilir.	Karakter dizisine boyutu kadar veri bellekte (*data segment*) veya yığın bellekte (*stack segment*) yer statik olarak tahsis edilir.

Tablo 4.2: std::string ile Karakter Dizilerinin Karşılaştırması

4.10 Algoritma Şablonları

Standart Şablon Kütüphanesi'ndeki algoritmalar (*algotrithm*), konteynerler üzerinde işlem yapmak için özel olarak tasarlanmış büyük bir fonksiyon kümesidir. Bu fonksiyonlar, konteynerlerin iç yapılarını bilmeye gerek kalmadan konteynerler arasında geçiş yapmak için kullanılan yineleyiciler (*iterator*) kullanılarak uygulanır.
Sıra değiştirmeyen algoritmalar;

- ◇ `for_each`: Bir aralıktaki her bir elemanı bulmak için bir fonksiyonda kullanılır.
- ◇ `count`: Bir aralıktaki değerin oluşum sayısını sayar.
- ◇ `find`: Bir aralıktaki değerin ilk oluşumunu bulur.
- ◇ `find_if`: Bir yordamı karşılayan ilk elemanı bulur.
- ◇ `find_if_not`: Bir yordamı karşılamayan ilk elemanı bulur.
- ◇ `equal`: İki aralığın eşit olup olmadığını kontrol eder.
- ◇ `search`: Bir dizi içinde bir alt diziyi arar.

Sıra değiştiren algoritmalar;

- ◇ `copy`: Elemanları bir aralıktan diğerine kopyalar.
- ◇ `copy_if`: Bir yordamı karşılayan elemanları başka bir aralığa kopyalar.
- ◇ `copy_n`: Belirli sayıda elemanı bir aralıktan diğerine kopyalar.
- ◇ `move`: Elemanları bir aralıktan diğerine taşır.
- ◇ `transform`: Bir aralığa bir fonksiyon uygular ve sonucu depolar.
- ◇ `remove`: Belirli bir değere sahip elemanları bir aralıktan kaldırır.
- ◇ `remove_if`: Bir yordamı karşılayan elemanları kaldırır.
- ◇ `unique`: Ardışık yinelenen elemanları kaldırır.
- ◇ `reverse`: Bir aralıktaki elemanların sırasını tersine çevirir.
- ◇ `swap`: Elemanları değiştirir.

Sıralama algoritmaları;

- ◇ `sort`: Bir aralıktaki elemanları sıralar.
- ◇ `stable_sort`: Eşdeğer elemanların göreli sırasını koruyarak elemanları sıralar.
- ◇ `partial_sort`: Bir aralığın bir kısmını sıralar.
- ◇ `nth_element`: Aralığı, n'inci elemanın son konumunda olacak şekilde bölümlere ayırır.

Arama algoritmaları;

- ◇ `binary_search`: Sıralanmış bir aralıkta bir elemanın bulunup bulunmadığını kontrol eder.
- ◇ `lower_bound`: Sırayı korumak için bir elemanın eklenebileceği ilk konumu arar.
- ◇ `upper_bound`: Belirtilen değerden büyük olan ilk elemanın konumunu arar.
- ◇ `equal_range`: Eşit elemanların aralığını döndürür.

Öbek (*heap*) algoritmaları;

- ◇ `make_heap`: Bir aralıktan bir öbek oluşturur.
- ◇ `push_heap`: Bir öbeğe bir eleman ekler.
- ◇ `pop_heap`: Bir öbekten en büyük eleman kaldırır.
- ◇ `sort_heap`: Bir öbekteki elemanları sıralar.

Küme algoritmaları;

- ◇ `set_union`: İki kümenin birleşimini hesaplar.
- ◇ `set_intersection`: İki kümenin kesişimini hesaplar.
- ◇ `set_difference`: İki küme arasındaki farkı hesaplar.
- ◇ `set_symmetric_difference`: İki küme arasındaki simetrik farkı hesaplar.

Sayısal algoritmalar;

- ◇ `accumulate`: bir aralığın toplamını (veya diğer işlemleri) hesaplar.
- ◇ `inner_product`: İki aralığın iç çarpımını hesaplar.
- ◇ `adjacent_difference`: Bitişik elemanlar arasındaki farkları hesaplar.
- ◇ `partial_sum`: bir aralığın kısmi toplamlarını hesaplar.

Diğer algoritmalar;

- ◇ `generate`: Bir fonksiyon tarafından üretilen değerlerle bir aralığı doldurur.
- ◇ `shuffle`: Bir aralıktaki elemanları rastgele karıştırır.
- ◇ `partition`: Elemanları bir öngörüye göre iki gruba ayırır.

Bu algoritmalar genellikle `<algorithm>` başlığı altında toplanır ve yineleyiciler (*iterator*) aracılığıyla veri yapıları (`vector`, `list`, `array`) üzerinde işlem yapar. Aşağıda birkaçına örnek verilmiştir;

1. `std::sort` (Sıralama Algoritması) :En sık kullanılan şablonlardan biridir. Verilen aralıktaki elemanları (varsayılan olarak küçükten büyüğe) sıralar.

```
#include <iostream>
#include <vector>
#include <algorithm> // Algoritma şablonları için

int main() {
  std::vector<int> sayilar = {50, 10, 40, 20, 30};
  // Şablon her türlü karşılaştırılabilir tip için çalışır
  std::sort(sayilar.begin(), sayilar.end());
  for(int n : sayilar)
    std::cout << n << " "; // Çıktı: 10 20 30 40 50
}
```

2. `std::find` (Arama Algoritması): Bir veri kümesi içinde belirli bir değeri arar. Eğer değeri bulursa o elemana ait bir yineleyici, bulamazsa sonu (`end()`) döndürür.

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
  std::vector<std::string> isimler = {"Ali", "Veli", "Ayse"};
  // "Veli" ismini arıyoruz
  auto it = std::find(isimler.begin(), isimler.end(), "Veli");
  if (it != isimler.end()) {
    std::cout << "Bulundu: " << *it << std::endl;
  } else {
    std::cout << "Bulunamadi!" << std::endl;
  }
}
```

3. `std::for_each` (Her Eleman İçin İşlem): Belirli bir aralıktaki her elemana, sizin belirlediğiniz bir fonksiyonu veya lamda ifadesini uygular.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
  std::vector<int> v = {1, 2, 3, 4, 5};
  // Her elemanın karesini ekrana yazdıran bir lambda şablonu
  std::for_each(v.begin(), v.end(), [](int &n) {
    n *= n;
    std::cout << n << " ";
  }); // Çıktı: 1 4 9 16 25
}
```

4. `std::accumulate` (Biriktirme/Toplam Algoritması): Bir aralıktaki tüm değerleri toplamak (veya başka bir işleme sokmak) için kullanılır. `<numeric>` başlığı altında bulunur.

```cpp
#include <iostream>
#include <vector>
#include <numeric> // accumulate için gerekli

int main() {
  std::vector<double> fiyatlar = {10.5, 20.0, 5.25};
  // 0.0 başlangıç değeri üzerine tüm elemanları ekler
  double toplam = std::accumulate(fiyatlar.begin(), fiyatlar.end(), 0.0);
  std::cout << "Toplam Fiyat: " << toplam << std::endl; // 35.75
}
```

4.11 Yineleme Şablonları

Standart Şablon Kütüphanesi'ndeki yineleyiciler (*iterator*), bir konteyner içindeki elemanlara gösterici görevi gören ve verilere erişmek ve bunları düzenlemek için tekdüze bir arayüz sağlayan nesnelerdir. Algoritmalar ve konteynerler arasında bir köprü görevi görürler;

- ◇ Giriş Yineleyicileri (*input iterator*): Elemanlara salt okunur erişime izin verirler.
- ◇ Çıkış Yineleyicileri (*output iterator*): Elemanlara salt yazılır erişime izin verirler.
- ◇ İleri Yineleyiciler (*forward iterator*): Artırılabilen okuma ve yazmaya izin verirler.
- ◇ Çift Yönlü Yineleyiciler (*bidirectional iterators*): Artırılabilir ve azaltılabilirler.
- ◇ Rastgele Erişim Yineleyicileri (*random access iterator*): Aritmetik işlemleri desteklerler ve elemanlara doğrudan erişebilirler.

Yineleme nesneleri, diziler gibi birden çok aynı nesneyi tutan veri yapılarında elemanlar üzerinde hareket etmeyi sağlayan nesnelerdir.

Fonksiyon	Açıklama
`begin()`	Bu fonksiyon, metnin başlangıcına işaret eden bir yineleyici (`iterator`) nesne döndürür.
`end()`	Bu fonksiyon, metnin sonunu işaret eden bir yineleyici nesne döndürür.
`rbegin()`	Bu fonksiyon, metnin başlangıcına işaret eden ters yineleyici (`reverse_iterator`) nesne döndürür.
`rend()`	Bu fonksiyon, metnin sonunu işaret eden ters yineleyici nesne döndürür.
`cbegin()`	Bu fonksiyon, metnin başlangıcına işaret eden bir sabit yineleyici (`const_iterator`) döndürür.
`cend()`	Bu fonksiyon metnin sonunu işaret eden bir sabit yineleyici (`const_iterator`) döndürür.
`crbegin()`	Bu fonksiyon metnin sonunu işaret eden bir sabit bir ters yineleyici (`const_reverse_iterator`) döndürür.
`crend()`	Bu fonksiyon,metnin başlangıcına işaret eden bir ters yineleyici (`const_reverse_iterator`) döndürür.

Tablo 4.3: Yineleme Fonksiyonları

Yineleme şablonlarına ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
#include <string>

int main() {
  std::string::iterator itr; //string yineleme nesnesi
  std::string::reverse_iterator rit; //string ters yineleme nesnesi
  std::string metin = "Merhaba Ilhan.";
  itr = metin.begin();
  std::cout << "Yineleyicinin işaret ettiği karakter: " << *itr<< std::endl;
  itr = metin.end() - 1;
  std::cout << "Yineleyicinin işaret ettiği son metin karakteri: " << *itr << std::endl
      ;
  rit = metin.rbegin();
  std::cout << "Ters yineleyicinin işaret ettiği karakter:: " << *rit << std::endl;
  rit = metin.rend() - 1;
  std::cout << "Ters yineleyicinin işaret ettiği on metin karakteri:: " << *rit << std
      ::endl;
}
/*Program çalıştığında:
Yineleyicinin işaret ettiği karakter: M
Yineleyicinin işaret ettiği son metin karakteri: .
Ters yineleyicinin işaret ettiği karakter:: .
Ters yineleyicinin işaret ettiği on metin karakteri:: M
*/
```

4.12 Fonksiyon Nesneleri

Fonksiyon nesnesi (*function object*), durum saklayabilen ve aşırı yüklenmiş fonksiyon işlecine (*function operator*) `()` sahip nesnelerdir. Bu nesnelere kısaltma olarak *functor* adı verilir. Bu işleci aşırı yüklemiş nesneler, bir fonksiyonmuş gibi çağrılabilirler ve bir fonksiyon veya fonksiyon göstericisiymiş gibi ele alınırlar.

Aşağıda bir dizinin elemanlarını 1 artıran bir dönüşüm programı verilmiştir;

```
#include <bits/stdc++.h> //transform()

int artirim(int x) { return (x+1); }

int main() {
  int dizi[] = {1, 2, 3, 4, 5};
  int adet = sizeof(dizi)/sizeof(dizi[0]);
  /* Aşağıdaki dönüşüm fonksiyonu ile dizi elemanları 1 artırılıyor */
  std::transform(dizi, dizi+adet, dizi, artirim);
  for (int i=0; i<adet; i++)
    std::cout << dizi[i] <<" ";
}
/* Program Çıktısı:
2 3 4 5 6
*/
```

Aşağıda bir fonksiyon nesnesi üzerinden aynı işlemi yapan bir program örneği verilmiştir;

```
#include <bits/stdc++.h> // transform

class artirim { // Functor
private:
  int sayi;
public:
  artirim(int n) : sayi(n) { } //Yapıcı işlenecek parametreyi alır
  /* Aşağıda verilen işleç önyüklemesi ile sayi kadar artırım işlemi yapılacaktır. */
  int operator () (int pMiktar) const {
    return sayi + pMiktar;
  }
};
```

```
int main() {
  int dizi[] = {1, 2, 3, 4, 5};
  int adet = sizeof(dizi)/sizeof(dizi[0]);
  int miktar = 5;
  std::transform(dizi, dizi+adet, dizi, artirim(miktar));
  for (int i=0; i<adet; i++)
    std::cout << dizi[i] << " ";
}
/*Program Çıktısı:
6 7 8 9 10
*/
```

Aritmetik functor (*aritmetic functor*), aritmetik operatörler temel matematiksel işlemleri gerçekleştirmek için kullanılır. Toplama (+), Çıkarma (-), Çarpma (*), Bölme (/), Kalan (%), Negatif (-) işleç işlevlerini yerine getirirler. Karşılaştırma fonksiyon nesneleri (*comparision functors*), özellikle konteynerlerde sıralama veya arama yapmak için değerleri karşılaştırmak amacıyla kullanılır. Küçüktür (<), Büyüktür (>), Küçüktür veya Eşittir (<=), Büyük veya Eşit (>=), Eşit (=), Eşit Değil (!=) işleç işlevlerini yerine getirirler. Mantıksal fonksiyon nesneleri (*logical functors*), Boole mantığını içeren senaryolar için mantıksal işlemleri gerçekleştirir. Mantıksal VE (&&), Mantıksal VEYA (||) ve Mantıksal DEĞİL (!) işleç işlevlerini yerine getirirler. Bit düzeyi fonksiyon nesneleri (*bitwise functors*), bit düzeyi işlemleri gerçekleştirir. Bit düzeyi VE (&), VEYA (|) ve ÖZEL VEYA (^) işleç işlevlerini yerine getirirler.

5. Bölüm
Tasarım Kalıpları

5.1 Kalıp Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır bileşenlerin (*component*) yeniden kullanımı (*reusing*) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde tecrübelerin (*experience*) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **Kalıp** (*pattern*) kavramı ortaya çıkmıştır. kalıpların aksine **Anti-Kalıp** (*anti-pattern*) ise **başlangıçta çekici görünen** ancak uygulama sürecinde **verimsiz olduğu kanıtlanmış yaygın hatalı çözümlerdir**.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. **kalıplar, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır**. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla kalıplar kullanılır. Kalıp kavramının temelleri Mimar *Christopher Alexander*'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır[1].

1987 yılında uluslararası "*Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar*" konferansına kadar kalıplarla ilgili bir çalışma ortaya çıkmamıştır[2]. Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı kalıplarla ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan "*Tasarım kalıpları: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri*" kitabı, tasarım kalıpların yazılımda kullanılmasında dönüm noktası olmuştur[3]. Yazılımlarda kullanılan kalıplar yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari kalıplar (*architectural pattern*)
2. Analiz kalıpları (*analysis pattern*)
3. Tasarım kalıpları (*design pattern*)
4. Kodlama kalıpları (*implementation pattern*)
5. Test kalıpları (*test pattern*)
6. Çözüm kalıpları (*solution pattern*)
7. Veri kalıpları (*data pattern*)

Bu kalıpların en önemlisi ve ilk gelişenlerden birisi, **tasarım kalıplarıdır** (*design pattern*). Tasarım kalıpları aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım kalıpları ise üç ana başlıkta toplanmaktadırlar.

1. Nesne imali ile ilgili kalıplar (*creational patterns*)
2. Yapısal kalıplar (*structural patterns*)
3. Davranışlarla ilgili kalıplar (*behavioral patterns*)

kalıpların çok sık kullanıldığı programlama yöntemine günümüzde **kalıp yönelimli program-**

[1] http://gee.cs.oswego.edu/dl/ca/ca/ca.html
[2] OOPSLA, Object Oriented Programming, Systems, Languages, and Applications
[3] Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

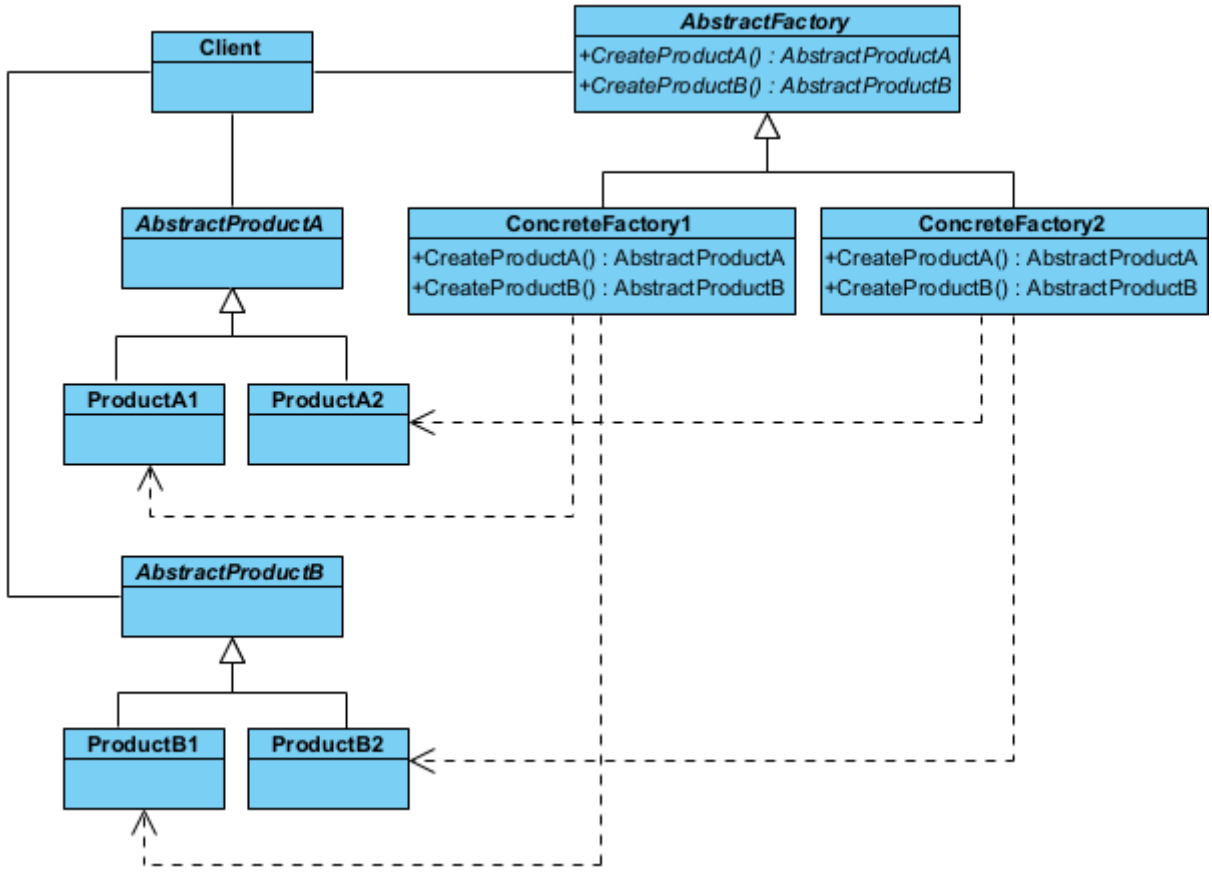
lama (*pattern oriented programming*) adı verilmektedir[4]. İzleyen sayfalarda verilen kalıplara ait UML diyagramlarının birçoğu *Erich Gamma*'nın **Design Pattern CD** yayınından alınmıştır[5]. Bölümün devamında verilecek tasarım kalıplarının UML diyagramları visual-paradigm adresinden alınmıştır[6].

5.2 Nesne İmalatı ile İlgili Kalıplar

Nesne imalatı ile ilgili kalıplar (*creational patterns*), nesne (*object*) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu kalıplarda nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

§5.2.1 Soyut Fabrika

Soyut fabrika Kalıında (*abstract factory pattern*) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (*concrete*) sınıflarını değil soyut (*abstract*) sınıfları kullanılarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.



Şekil 5.1: Soyut Fabrika kalıbı UML Diyagramı

Bu tasarım kalıbında, birbirleriyle ilişkili veya bağımlı nesne ailelerini, somut sınıflarını belirtmeden oluşturmak için kullanılan imalatla ilgili (*creational*) bir kalıptır. "Fabrikaların fabrikası" olarak da adlandırılır. Eğer sisteminizin farklı ürün aileleriyle (örneğin; Windows uyumlu pencere ailesi ve Mac uyumlu pencere ailesi) çalışması gerekiyorsa ve bu ailelerin birbirine karışmasını istemiyorsanız bu kalıbı kullanırsınız.

- ◇ Soyut Fabrika Arayüzü (`AbstractFactory`), ilgili tüm ürünleri oluşturacak yöntemlerin imzalarını (şablonunu) tanımlar. "Bir buton oluştur" ve "Bir pencere oluştur" gibi soyut görev-

[4] www.mathcs.sjsu.edu/faculty/pearce/patterns.html
[5] Design Pattern CD, Addison-Wesley,1998
[6] https://circle.visual-paradigm.com/category/uml-diagrams/gof-design/

leri belirler.
- ◇ Somut Fabrikalar (`ConcreteFactory`),belirli bir ürün ailesine ait nesneleri fiilen oluşturur. Örneğin `ModernMobilyaFabrikasi` sadece modern tarzdaki sandalyeleri ve masaları üretir.
- ◇ Soyut Ürünler (`AbstractProduct`), ürün ailesindeki her bir temel ürün türü için bir arayüz tanımlar.Tüm sandalyelerin sahip olması gereken ortak özellikleri (Örnek olarak `otur()`) belirler.
- ◇ Somut Ürünler (`ConcreteProduct`), soyut ürünlerin farklı varyasyonlarını (ailelerini) temsil eden gerçek sınıflardır. `ModernSandalye` veya `AntikaSandalye` gibi sınıflar burada yer alır.

Bu kalıbı kodlayalım;

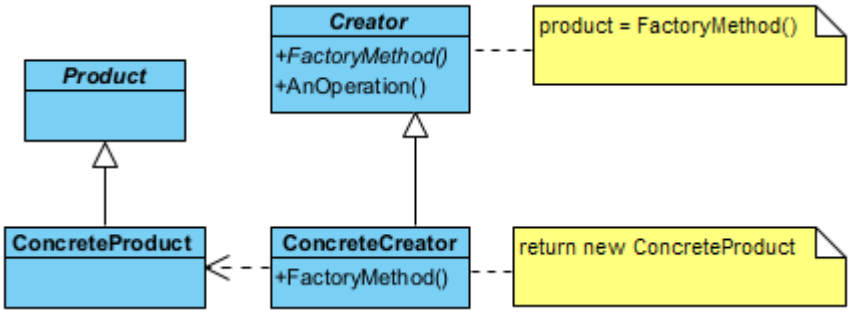
```
//C++ 14 ve üzeri ile derlenmeli
#include <iostream>
#include <string>
#include <memory> // unique_ptr için gerekli
#include <vector>
//1. Products (Ürünler)
//--- Abstract Products ---
class AbstractProductA {
public:
  AbstractProductA(std::string pAdi) : productName(pAdi) {}
  virtual ~AbstractProductA() = default; // Sanal yıkıcı (Virtual Destructor) çok ö
      nemli!  unique_ptr'nin alt sınıfları doğru silmesi için gereklidir.
  virtual std::string getUrunAdi() const { return productName; }
private:
  std::string productName;
};
class ConcreteProductA1 : public AbstractProductA {
public:
  using AbstractProductA::AbstractProductA; //Temel sınıfın TÜM yapıcılarını içeri
      aktarır!
};
class ConcreteProductA2 : public AbstractProductA {
public:
  using AbstractProductA::AbstractProductA; //Temel sınıfın TÜM yapıcılarını içeri
      aktarır!
};
// --- Abstract Product B ---
class AbstractProductB {
public:
  AbstractProductB(std::string pAdi) : productName(pAdi) {}
  virtual ~AbstractProductB() = default;
  virtual std::string getUrunAdi() const { return productName; }
  // Etkileşim için raw pointer (AbstractProductA*) kullanabiliriz. Çünkü sahiplik (
      ownership) almıyoruz, sadece gözlemliyoruz.
  void interact(const AbstractProductA* a) const {
    std::cout << this->productName << " ürünü " << a->getUrunAdi() << " ile etkileşim
        halinde..." << std::endl;
  }
private:
  std::string productName;
};
class ConcreteProductB1 : public AbstractProductB {
public:
  using AbstractProductB::AbstractProductB; //Temel sınıfın TÜM yapıcılarını içeri
      aktarır!
};

class ConcreteProductB2 : public AbstractProductB {
public:
  using AbstractProductB::AbstractProductB; //Temel sınıfın TÜM yapıcılarını içeri
      aktarır!
};
//2. Factory (Fabrika)
//--- Abstract Factory ---
```

```cpp
class AbstractFactory {
public:
  virtual ~AbstractFactory() = default;
  // unique_ptr döndürerek nesnenin sahipliğini çağırana devrediyoruz
  virtual std::unique_ptr<AbstractProductA> createProductA() = 0;
  virtual std::unique_ptr<AbstractProductB> createProductB() = 0;
};
class ConcreteFactory1 : public AbstractFactory {
public:
  std::unique_ptr<AbstractProductA> createProductA() override {
    return std::make_unique<ConcreteProductA1>("A1 ürünü");
  }
  std::unique_ptr<AbstractProductB> createProductB() override {
    return std::make_unique<ConcreteProductB1>("B1 Ürünü");
  }
};
class ConcreteFactory2 : public AbstractFactory {
public:
  std::unique_ptr<AbstractProductA> createProductA() override {
    return std::make_unique<ConcreteProductA2>("A2 ürünü");
  }
  std::unique_ptr<AbstractProductB> createProductB() override {
    return std::make_unique<ConcreteProductB2>("B2 Ürünü");
  }
};
// 3. Client (İstemci)
int main() {
  // Fabrikaları unique_ptr ile oluşturuyoruz
  auto factory1 = std::make_unique<ConcreteFactory1>();
  auto factory2 = std::make_unique<ConcreteFactory2>();
  // Ürünleri oluşturuyoruz - Bellek yönetimi artık otomatik!
  auto productA = factory1->createProductA();
  std::cout << "Fabrika 1 ürünü: " << productA->getUrunAdi() << std::endl;
  auto productB = factory2->createProductB();
  std::cout << "Fabrika 2 ürünü: " << productB->getUrunAdi() << std::endl;
  // .get() ile akıllı işaretçinin içindeki ham adresi geçici olarak kullanabiliriz
  productB->interact(productA.get());
  // delete yazmamıza gerek yok; main biterken her şey otomatik temizlenir.
}
/*Program Çıktısı:
Fabrika 1 ürünü: A1 ürünü
Fabrika 2 ürünü: B2 Ürünü
B2 Ürünü ürünü A1 ürünü ile etkileşim halinde...
*/
```

§5.2.2 Fabrika Yöntemi

Fabrika yöntemi kalıbında (*factory method pattern*) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bu kalıbın görevi, nesne üretim



Şekil 5.2: Fabrika Yöntemi kalıbı UML Diyagramı

işini alt sınıflara bırakır. Hangi nesnenin üretileceğine çalışma zamanında karar verilir.

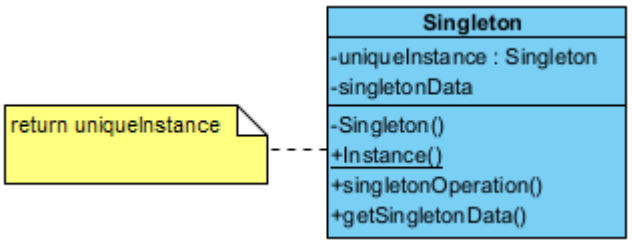
- ◇ Üretici (`Creator`), fabrika yöntemini tanımlayan soyut sınıftır.
- ◇ Ürün (`Product`), üretilecek nesnelerin ortak arayüzüdür.
- ◇ Somut Üretici (`ConcreteCreator`), ihtiyaca göre "A Ürünü" veya "B Ürünü" üreten gerçek fabrikadır.

Bu kalıbı kodlayalım;

```
//C++ 14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory> // unique_ptr için şart
//1. Products (Ürünler)
// --- Ürün Sınıfları ---
class Product {
  public:
  Product(std::string pAdi) : productName(pAdi) {}
  virtual ~Product() = default; // Alt sınıfların güvenli silinmesi için sanal yıkıcı
  virtual std::string getUrunAdi() const {
    return productName;
  }
  private:
  std::string productName;
};
class ConcreteProduct : public Product {
  public:
  using Product::Product; // Üst sınıfın yapıcısını miras al
};
//2. Creators (İmalatçılar)
// --- Creator (İmalatçı) Sınıfları ---
class Creator {
  public:
  virtual ~Creator() = default;
  virtual std::unique_ptr<Product> factoryMethod() = 0; // Sahipliği devreden Fabrika Y
      öntemi
  virtual void anOperation() = 0;
};
class ConcreteCreator : public Creator {
  public:
  std::unique_ptr<Product> factoryMethod() override {
    // C++14 kullanıyorsan make_unique, C++11 ise unique_ptr<T>(new T())
    return std::make_unique<ConcreteProduct>("Fabrika Yöntemi Ürünü");
  }
  void anOperation() override {
    std::cout << "Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir..." <<
        std::endl;
  }
};
//3. Client (İstemci)
int main() {
  // 1. Creator'ı akıllı gösterici ile oluştur
  std::unique_ptr<Creator> imalatci = std::make_unique<ConcreteCreator>();
  // 2. Ürünü oluştur (Sahiplik imalatçıdan 'urun' değişkenine geçer)
  std::unique_ptr<Product> urun = imalatci->factoryMethod();
  std::cout << "İmalatçı tarafından imal edilen ürün: " << urun->getUrunAdi() << std::
      endl;
  imalatci->anOperation();
  // Manuel 'delete' yapmaya GEREK YOK. main fonksiyonu bittiğinde urun ve imalatci
      otomatik olarak silinir.
}
/*Program Çıktısı:
İmalatçı tarafından imal edilen ürün: Fabrika Yöntemi Ürünü
Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...
*/
```

§5.2.3 Tekil Nesne

Tekil Nesne kalıbında (*singleton pattern*) amaç, bir sınıftan yalnızca bir örnek (*instance*) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez.



Şekil 5.3: Tekil kalıbı UML Diyagramı

C++11 ve sonrası modern C++'ta bu kalıp, "Meyers' Singleton" olarak bilinen çok daha zarif, güvenli ve performanslı bir yapıya dönüşmüştür. Şimdi Singleton sınıfını kodlayalım;

```
//C++11 ve üzeri ile derlenmelidir
#include <iostream>
//1. Singletone (Tekil Nesne) Sınıfı
class Singleton {
private:
  // Yapıcı yöntem gizli
  Singleton() : singletonData(10) {
    std::cout << "Nesne ilk kez olusturuldu.\n";
  }
  int singletonData;
public:
  // Kopyalama ve atama engellenir
  Singleton(const Singleton&) = delete;
  Singleton& operator=(const Singleton&) = delete;
  // En modern erişim noktası
  static Singleton& Instance() {
    // Statik yerel değişken: Sadece ilk çağrıda oluşturulur,
    // program sonlanırken otomatik silinir. Thread-safe'dir.
    static Singleton instance;
    return instance;
  }
  void anOperation() const {
    std::cout << "Modern tekil nesne gorev basinda...\n";
  }
  int getSingletonData() const { return singletonData; }
};
//2. Client (İstemci)
int main() {
  // Nesneye referans üzerinden erişmek daha güvenlidir
  Singleton& s1 = Singleton::Instance();
  Singleton& s2 = Singleton::Instance();

  if (&s1 == &s2) {
    std::cout << "s1 ve s2 AYNI nesnedir.\n";
  }
  s1.anOperation();
}
/*Program Çıktısı:
Nesne ilk kez olusturuldu.
s1 ve s2 AYNI nesnedir.
Modern tekil nesne gorev basinda...
*/
```

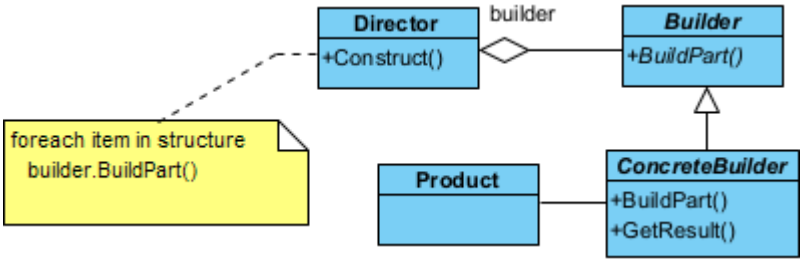
Yukarıda kodlaması yapılan bu kalıbın amacı, bir sınıftan (`Singleton`) uygulama ömrü boyunca

sadece bir adet nesne üretilmesini garanti etmektir.

- ◇ Mahrem Yapıcı (*private constructor*), dışarıdan rastgele nesne üretilmesini engeller. 7. Satırda tanımlanmıştır.
- ◇ Statik Tekil Örnek (*static unique instance*), oluşturulan tek nesneyi kendi içinde saklar. 19. Satırda tanımlanmıştır.
- ◇ Statik Erişim Yöntemi (*static access method*), nesneye her yerden erişim sağlayan kapıdır. 16. Satırda tanımlanmıştır.

§5.2.4 Kurucu

Kurucu kalıbında (*builder pattern*) amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile imalatına ilişkin kodlar birbirinden ayrılmış olur.



Şekil 5.4: Kurucu kalıbı UML Diyagramı

Bu kalıbı bir fast-food restoranındaki "Menü Hazırlama" sürecine benzetebilirsin. Hamburgerin içine turşu konulup konulmayacağı, içeceğin boyutu veya yan ürünün ne olacağı gibi adımlar tek tek belirlenir, ancak en sonunda ortaya tek bir "Menü" nesnesi çıkar.

- ◇ Ürün (`Product`), inşa edilmeye çalışılan karmaşık nesnedir. Genellikle çok sayıda parçadan veya konfigürasyondan oluşur.
- ◇ Soyut Kurucu (`Builder`), ürünün parçalarını oluşturmak için gerekli olan tüm adımları (yöntemleri) arayüz olarak tanımlar. "Ekmek koy", "Köfte ekle", "Sos dök" gibi soyut inşa adımlarını belirler.
- ◇ Somut Kurucu (`ConcreteBuilder`), soyut kurucudaki adımları fiilen gerçekleştirir ve ürünün bir örneğini saklar. "Vejetaryen Burger" veya "Double Burger" gibi farklı ürün varyasyonlarını nasıl inşa edeceğini bilir.
- ◇ Yönetici (`Director`), inşa adımlarının hangi sırayla çağrılacağını kontrol eder. İstemciden (`Client`) aldığı kurucu nesnesini kullanarak, doğru sırayla (Örn: Önce temel, sonra yan ürünler) inşa sürecini yürütür.

```cpp
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <string>
#include <memory>   // std::unique_ptr için
#include <utility>  // std::move için

//1. Ürün Sınıfı (Product)
class Product {
public:
  explicit Product(std::string name) : productName(std::move(name)) {}
  void addPart(const std::string& part) {
    parts.push_back(part);
  }
  void showParts() const {
    std::cout << productName << " Urununun parcalari:\n";
    // Modern C++: Range-based for loop ve const kullanımı
    for (const auto& part : parts) {
      std::cout << "- " << part << "\n";
```

```
    }
  }
private:
  std::string productName;
  std::vector<std::string> parts;
};

//2. Kurucu (Builder) Sınıfları
//Kurucu Arayüzü (Abstract Builder)
class Builder {
public:
  virtual ~Builder() = default; // Sanal yıkıcı şarttır
  virtual void buildPartKisim1() = 0;
  virtual void buildPartKisim2() = 0;
  virtual void buildPartKisim3() = 0;
  // Ürünün mülkiyetini (ownership) dışarıya devreden yöntem
  virtual std::unique_ptr<Product> getResult() = 0;
};

//Somut Kurucu (Concrete Builder)
class ConcreteBuilder : public Builder {
public:
  ConcreteBuilder() {
    // make_unique ile güvenli oluşturma
    product = std::make_unique<Product>("Modern C++ Kitabi");
  }
  void buildPartKisim1() override { product->addPart("Bolum 1: Akilli Gösterişciler");
     }
  void buildPartKisim2() override { product->addPart("Bolum 2: Lamda Ifadeleri"); }
  void buildPartKisim3() override { product->addPart("Bolum 3: Tasarim kalıpları"); }
  std::unique_ptr<Product> getResult() override {
    // Sahipliği dışarıya (Client'a) taşıyoruz. Builder artık buna sahip değil.
    return std::move(product);
  }
private:
  std::unique_ptr<Product> product;
};

//3. Yönetici (Director)
class Director {
public:
  // Builder'ı referans olarak almak sahiplik karmaşasını önler
  void construct(Builder& builder) {
    builder.buildPartKisim1();
    builder.buildPartKisim2();
    builder.buildPartKisim3();
  }
};

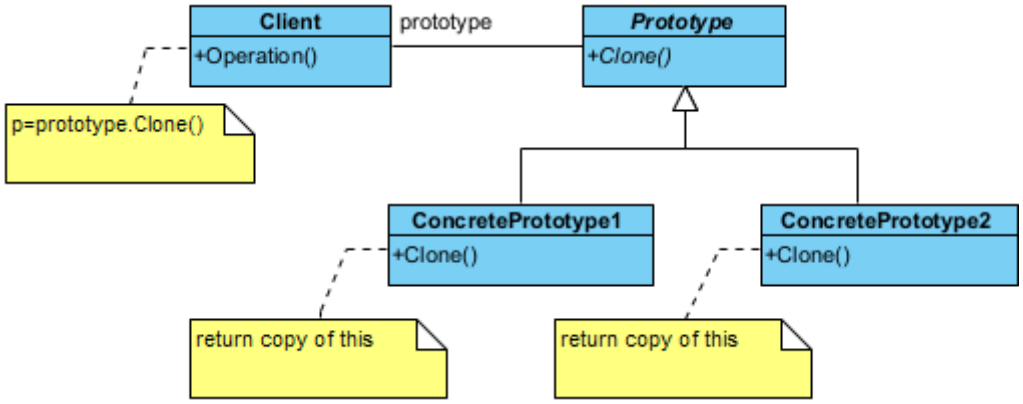
//4. Client (İstemci)
int main() {
  // Nesneleri stack üzerinde veya unique_ptr ile oluşturuyoruz
  Director director;
  auto builder = std::make_unique<ConcreteBuilder>();
  // İnşa süreci
  director.construct(*builder);
  // Ürünü teslim al (Sahiplik builder'dan main'e geçti)
  std::unique_ptr<Product> product = builder->getResult();
  if (product) {
    product->showParts();
  }
  // Manuel 'delete' yok! Kapsam bitince her şey otomatik temizlenir.
}
/*Program Çıktısı:
Modern C++ Kitabi Urununun parcalari:
- Bolum 1: Akilli Isaretciler
```

```
- Bolum 2: Lambda Ifadeleri
- Bolum 3: Tasarim kalıpları
*/
```

§5.2.5 Prototip

Prototip kalıbı (*prototype pattern*), mevcut bir nesnenin kopyasını (klonunu) oluşturmak için kullanılan imalatla ilgili bir kalıptır. Bu kalıbın temel amacı, bir nesneyi oluşturmanın maliyeti (zaman, bellek veya işlem gücü açısından) çok yüksek olduğunda, sıfırdan `new` işleci ile üretmek yerine mevcut bir örneği klonlayarak yeni nesneler elde etmektir.

- ◇ Soyut Prototip (`Prototype`), nesnenin kendisini kopyalayabilmesi için gerekli olan arayüzü tanımlar. Genellikle sadece bir `clone()` yönteminin imzasını barındırır.
- ◇ Somut Prototip (`ConcretePrototype`), soyut prototip arayüzünü uygular ve asıl kopyalama işlemini gerçekleştirir. Kendi verilerini yeni bir nesneye aktarır. C++ dünyasında bu işlem genellikle Kopya yapıcı (*copy constructor*) kullanılarak yapılır.
- ◇ İstemci (`Client`), yeni bir nesneye ihtiyaç duyduğunda, sınıfa gidip "yeni bir tane üret" demek yerine, elindeki prototip nesnesine "kendini klonla" der.



Şekil 5.5: Prototip kalıbı UML Diyagramı

Şimdi bu kalıbı kodlayalım;

```cpp
#include <iostream>
#include <memory>
#include <string>

// 1. Prototip (Prototype) Sınıfları
//--- Abstract Prototype ---
class Prototype {
protected: // Miras alan sınıfların erişebilmesi için protected
  std::string data_;
public:
  virtual ~Prototype() = default;
  // Nesneyi kopyalayıp akıllı işaretçi olarak döndürür
  virtual std::unique_ptr<Prototype> clone() const = 0;
  void showData() const {
    std::cout << "Nesne Verisi: " << data_ << std::endl;
  }
};
// --- Concrete Prototype ---
class ConcretePrototype : public Prototype {
  public:
  // Yapıcı fonksiyon (Constructor)
  ConcretePrototype(std::string data) {
    data_ = std::move(data);
  }
  // Kopyalama yapıcı (Copy Constructor) clone() için gerekli
  std::unique_ptr<Prototype> clone() const override {
```

```
    // Mevcut nesnenin (*this) kopyasını oluşturur
    return std::make_unique<ConcretePrototype>(*this);
  }
};

// 2. Client (İstemci)
void operation() {
  // 1. Orijinal nesneleri unique_ptr ile oluşturalım (Güvenli yol)
  std::unique_ptr<Prototype> object1 = std::make_unique<ConcretePrototype>("40.30");
  object1->showData();
  // 2. Klonlama işlemi
  // clone() bir unique_ptr döndürür, bu yüzden alıcı da unique_ptr olmalı
  std::unique_ptr<Prototype> clone1 = object1->clone();
  clone1->showData();
  std::unique_ptr<Prototype> object2 = std::make_unique<ConcretePrototype>("Mehmet");
  object2->showData();
  std::unique_ptr<Prototype> clone2 = object2->clone();
  clone2->showData();
  // Akıllı göstericiler (unique_ptr) sayesinde manuel 'delete' yapmaya gerek kalmaz!
}
int main() {
  operation();
}
/*Program Çıktısı:
Nesne Verisi: 40.30
Nesne Verisi: 40.30
Nesne Verisi: Mehmet
Nesne Verisi: Mehmet
*/
```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

Üstünkörü kopyalamada (*shallow copy*); C++'ta bir sınıf için özel bir kopyalama yapıcısı (*copy constructor*) yazmazsanız, derleyici varsayılan olarak bir "memberwise copy" (üye düzeyinde kopyalama) yapar. Eğer sınıfta ham işaretçiler (*raw pointers*) varsa, bu işlem teknik olarak bir bit düzeyi kopyalamadır (*bitwise copy*). Eğer nesneniz dinamik bellekten (new ile) yer ayırdıysa, hem orijinal nesne hem de kopya nesne aynı bellek adresini göstermesi bir sorundur. Nesnelerden biri yok edildiğinde (yıkıcı çalıştığında) belleği serbest bırakır. Diğeri hala o adresi kullanmaya çalıştığında sarkan gösterici (*dangling pointer*) veya çift serbest bırakma (*double free*) hatasıyla karşılaşırız. Bu, C++ geliştiricilerinin en meşhur kabuslarından biridir.

Derinlemesine kopyalamada (*deep copy*); Prototip kalıbının gerçek ruhunu yansıtan yöntemdir. Bu yöntemde, nesnenin sahip olduğu tüm kaynaklar (işaret edilen bellek alanları dahil) yeni nesne için baştan tahsis edilir. Prototip kalıbında genellikle `virtual T* clone() const;` şeklinde bir yöntem tanımlanır. Bu yöntem içerisinde `new` anahtar kelimesiyle nesnenin tam bir kopyası oluşturulur. Orijinal nesne ile kopya nesne tamamen bağımsızdır. Birinin verisini değiştirmek veya birini yok etmek diğerini etkilemez.

Tembel kopyalamada (*lazy copy*); Modern C++ dünyasında (özellikle eski `std::string` gerçekleşimlerinde ve bazı büyük kütüphanelerde) performans optimizasyonu için kullanılır. Nesne kopyalandığında başlangıçta üstünkörü kopyalama yapılır ve bir **referans sayacı** (*reference counter*) tutulur. İki nesne de aynı veriyi sadece okuyorsa (*read-only*), fazladan bellek harcamaya gerek yoktur. Nesnelerden biri veriyi değiştirmeye (write) kalkıştığı anda, gerçek bir derinlemesine kopyalama yapılır ve kopyalar birbirinden ayrılır. Buna **Copy-on-Write** (*COW*) denir. Modern C++'ta `std::shared_ptr` ve `std::make_shared` kullanımı, bu mantığın bir kısmını (kaynak paylaşımı) otomatikleştirir.

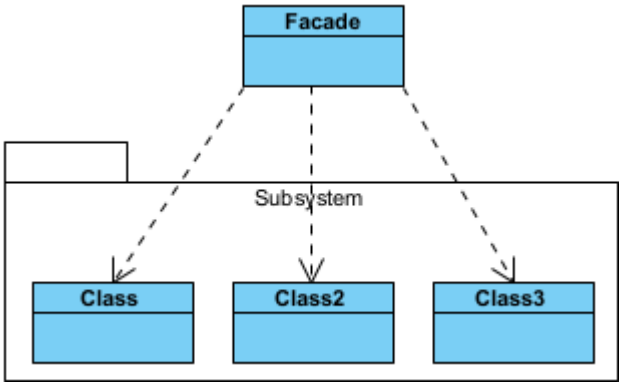
Prototip kalıbında `clone()` yöntemini yazarken, sınıfınızın içerisinde standart kütüphane nesneleri kullanıyorsanız (`std::vector` veya `std::string` gibi), C++ bunları varsayılan olarak derinlemesine kopyalama (kendi içlerindeki kopyalama mantığı sayesinde) ile kopyalar. Ancak ham gösterici (`int*`, `char*`) kullanıyorsanız, derin **kopyalamayı manuel olarak yönetmek gerekir**.

5.3 Yapısal Kalıplar

Yapısal kalıplar (*structural pattern*), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

§5.3.1 Önyüz

Önyüz kalıbında (*facade pattern*) karmaşık bir alt sistemdeki (*subsystem*) çok sayıda sınıfı ve arayüzü, kullanımı kolay tek bir üst düzey arayüz arkasında gizlemek için kullanılan yapısal bir kalıptır. Bu kalıbı bir restoranın "Garsonu" gibi düşünebilirsin. Sen sadece sipariş verirsin; garson ise mutfağın, bulaşıkhanenin ve kasanın karmaşıklığıyla senin adına ilgilenir.



Şekil 5.6: Önyüz kalıbı UML Diyagramı

- ◇ Önyüz (`Facade`), karmaşık alt sistemleri bilen ve onları yöneten "ana giriş kapısı"dır. İstemciden (`Client`) gelen basit isteği, alt sistemdeki doğru nesnelere yönlendirir ve onları uygun sırayla çalıştırır.
- ◇ Alt Sistem Sınıfları (`SubSystem`), işin asıl detaylarını yapan çok sayıda sınıftır (Örn: SesSistemi, IsikDenetleyici, Projeksiyon). `Facade` nesnesinden tamamen habersizdirler. Kendi işlerini yaparlar. `Facade` sadece bunları koordine eder.
- ◇ İstemci (`Client`), sistemi kullanan son kullanıcı veya başka bir kod parçasıdır. Alt sistemin karmaşık detaylarıyla (hangi sınıf hangi sırayla çağrılacak gibi) uğraşmaz; sadece `Facade`'in sunduğu basit yöntemleri çağırır.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <memory> // std::unique_ptr için

//1. Alt Sistemler (Subsystems)
// Bu sınıflar karmaşık iş mantığını barındırır.
class SubSystem1 {
public:
  void operation11() { std::cout << "Subsystem1: Yöntem 1 calistirildi." << std::endl;
      }
};
class SubSystem2 {
public:
  void operation22() { std::cout << "Subsystem2: Yöntem 2 calistirildi." << std::endl;
      }
};
class SubSystem3 {
public:
  void operation31() { std::cout << "Subsystem3: Yöntem 1 calistirildi." << std::endl;
      }
  void operation32() { std::cout << "Subsystem3: Yöntem 2 calistirildi." << std::endl;
      }
  void operation33() { std::cout << "Subsystem3: Yöntem 3 calistirildi." << std::endl;
```

```
    }
};

//2. Facade (Önyüz) Sınıfı
//İstemciye (Client) basit bir arayüz sunar, karmaşayı yönetir.
class Facade {
private:
  // Akıllı göstericiler ile otomatik bellek yönetimi
  std::unique_ptr<SubSystem1> subsystem1;
  std::unique_ptr<SubSystem2> subsystem2;
  std::unique_ptr<SubSystem3> subsystem3;
public:
  Facade() : subsystem1(std::make_unique<SubSystem1>()), subsystem2(std::make_unique<
      SubSystem2>()), subsystem3(std::make_unique<SubSystem3>()) {
    // Yapıcıda üye ilklendirme listesi (member initializer list) kullanımı daha
        performanslıdır.
  }

  // Yıkıcı (Destructor) yazmaya gerek yok, unique_ptr otomatik temizler.
  void executeComplexOperation() {
    std::cout << "--- Facade Karma sik Islemi Baslatiyor ---" << std::endl;
    subsystem1->operation11();
    subsystem2->operation22();
    subsystem3->operation31();
    subsystem3->operation33();
    subsystem3->operation32();
    std::cout << "--- Islem Tamamlandi ---" << std::endl;
  }
};

//3. Client (İstemci)
int main() {
  // Client tarafında Facade nesnesini doğrudan Stack üzerinde oluşturuyoruz.
  Facade facade;
  facade.executeComplexOperation();

}
/*Program Çıktısı:
--- Facade Karma sik Islemi Baslatiyor ---
Subsystem1: Yöntem 1 calistirildi.
Subsystem2: Yöntem 2 calistirildi.
Subsystem3: Yöntem 1 calistirildi.
Subsystem3: Yöntem 3 calistirildi.
Subsystem3: Yöntem 2 calistirildi.
--- Islem Tamamlandi ---
*/
```

Aşağıda bu kalıba gerçek dünya örneği olarak alt sistemleri olan bir bilgisayarın başlatılması modellenmiştir;

```
//C++17 ile derlenmelidir.
#include <iostream>
#include <string>
#include <string_view> // C++17: Bellek kopyalamayı azaltan hafif metin görünümü
#include <iomanip>     // Çıktı biçimlendirme (hex) için

// --- Sabit Tanımlamalar ---
namespace BootConstants {
  constexpr long ADDRESS = 0x7C00;
  constexpr int SECTOR = 0;
  constexpr int SECTOR_SIZE = 512;
}

//1. --- Alt Sistemler (Subsystems) ---
class CPU {
  public:
```

```
  void freeze() const {
    std::cout << "CPU: Durduruldu (Freeze)." << std::endl;
  }
  void jump(long position) const {
    std::cout << "CPU: Adrese atlandi: 0x" << std::hex << std::uppercase << position <<
        std::endl;
  }
  void execute() const {
    std::cout << "CPU: Komutlar calistiriliyor..." << std::dec << std::endl;
  }
};
class Memory {
  public:
  // const char* yerine string_view kullanarak daha güvenli veri aktarımı sağlıyoruz
  void load(long position, std::string_view data) const {
    std::cout << "Memory: 0x" << std::hex << position
    << " adresine veri yuklendi: [" << data << "]" << std::endl;
  }
};
class HardDrive {
  public:
  std::string_view read(long lba, int size) const {
    std::cout << "HardDrive: Sector " << std::dec << lba << " uzerinden "
    << size << " byte veri okundu." << std::endl;
    return "OS_KERNEL_DATA";
  }
};

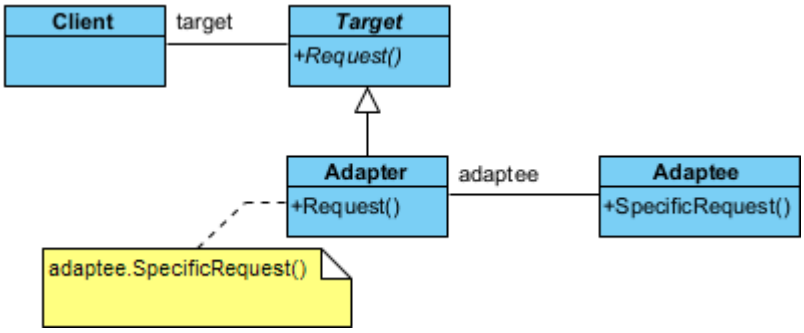
//2. --- Facade (Görünüş / Önyüz) ---
class ComputerFacade {
  private:
  // Alt sistemleri üyeler olarak tutuyoruz.
  // Eğer bu alt sistemler çok ağırsa unique_ptr kullanılabilir.
  CPU cpu_;
  Memory memory_;
  HardDrive hardDrive_;
  public:
  void start() {
    std::cout << "Bilgisayar baslatiliyor...\n" << std::string(35, '-') << std::endl;
    // Facade, karmaşık adımları bir orkestra şefi gibi yönetir:
    cpu_.freeze();
    // Diski oku ve belleğe aktar
    auto data = hardDrive_.read(BootConstants::SECTOR, BootConstants::SECTOR_SIZE);
    memory_.load(BootConstants::ADDRESS, data);
    // İşlemciyi yeni adrese yönlendir ve çalıştır
    cpu_.jump(BootConstants::ADDRESS);
    cpu_.execute();
    std::cout << std::string(35, '-') << "\nSistem hazir!" << std::endl;
  }
};

//3. --- Client (İstemci) ---
int main() {
  // İstemci (Kullanıcı) sadece "başlat" komutunu bilir.
  ComputerFacade computer;
  computer.start();
}
/*Program Çıktısı:
Bilgisayar baslatiliyor...
-----------------------------------
CPU: Durduruldu (Freeze).
HardDrive: Sector 0 uzerinden 512 byte veri okundu.
Memory: 0x7c00 adresine veri yuklendi: [OS_KERNEL_DATA]
CPU: Adrese atlandi: 0x7C00
CPU: Komutlar calistiriliyor...
-----------------------------------
```

```
Sistem hazir!
*/
```

§5.3.2 Adaptör

Adaptör kalıbı (*adapter pattern*), yabancı bir arayüzü istemcinin anlayacağı bir arayüze dönüştürür. Yani farklı arayüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu kalıbın amacı, uyumsuz iki arayüzün (*interface*) birlikte



Şekil 5.7: Adaptör kalıbı UML Diyagramı

çalışmasını sağlar. Burada yabancı arayüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin (`Client`) isteğine uygun hale getirildiği düşünülmelidir. Bir nevi elektrik prizi dönüştürücüsü görevi görür.

- ◇ İstemci (`Client`), mevcut sistemle çalışan ana kod.
- ◇ Uyumsuz Nesne (`Adaptee`), sisteme dahil edilmek istenen ama arayüzü farklı olan nesne.
- ◇ Hedef (`Target`), istemcinin (`Client`) beklediği arayüz.
- ◇ Uyarlayıcı (`Adapter`), uyumsuz nesneyi sarmalayarak, istemcinin beklediği biçime çeviren aracı sınıf.

Şimdi de kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory> // std::unique_ptr için

// 1. Adaptee (Uyarlanan Uyumsuz Arayüz): Mevcut olan ama arayüzü uyumsuz olan sınıf.
class Adaptee {
public:
  void specificRequest() const {
    std::cout << "Adaptee: Ozel istek (specificRequest) yontemi calistirildi." << std::
        endl;
  }
};

// 2. Target (Hedef): İstemcinin (Client) beklediği arayüz.
class Target {
public:
  // ÖNEMLİ: Soyut sınıflarda sanal yıkıcı (virtual destructor) şarttır!
  virtual ~Target() = default;
  virtual void request() = 0;
};

// 3. Adapter (Uyarlayıcı): Target arayüzünü Adaptee'ye bağlar.
class Adapter : public Target {
private:
  // Akıllı işaretçi ile otomatik ömür yönetimi (RAII)
  std::unique_ptr<Adaptee> adaptee;
public:
  // make_unique ile güvenli oluşturma
  Adapter() : adaptee(std::make_unique<Adaptee>()) {}
```

```
  void request() override {
    std::cout << "Adapter: Standart istek alindi, Adaptee'ye yonlendiriliyor..." << std
        ::endl;
    adaptee->specificRequest();
  }
};

// 4. Client (İstemci)
int main() {
  // Target nesnesini akıllı işaretçi ile yönetiyoruz.
  // Nesne kapsam dışına çıktığında delete işlemi otomatik yapılır.
  std::unique_ptr<Target> target = std::make_unique<Adapter>();
  target->request();
}
/*Program Çıktısı:
Adapter: Standart istek alindi, Adaptee'ye yonlendiriliyor...
Adaptee: Ozel istek (specificRequest) yontemi calistirildi.
*/
```

Bu kalıp gerçek dünya örneği olarak aşağıdaki iz kaydı (*log*) örneği verilebilir;

```
//C++ 14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory>

//1. --- Adaptee (Uyarlanan Sınıf) ---
// Değiştiremediğimiz veya eski (Legacy) olan sınıf
class OldLogger {
  public:
  void logMessage(const char* msg) {
    std::cout << "[Eski Kayit Sistemi]: " << msg << std::endl;
  }
};

//2. --- Target Interface (Hedef Arayüz) ---
// Yeni sistemin beklediği standart arayüz
class ILogger {
  public:
  virtual ~ILogger() = default;
  virtual void log(const std::string& msg) = 0;
};

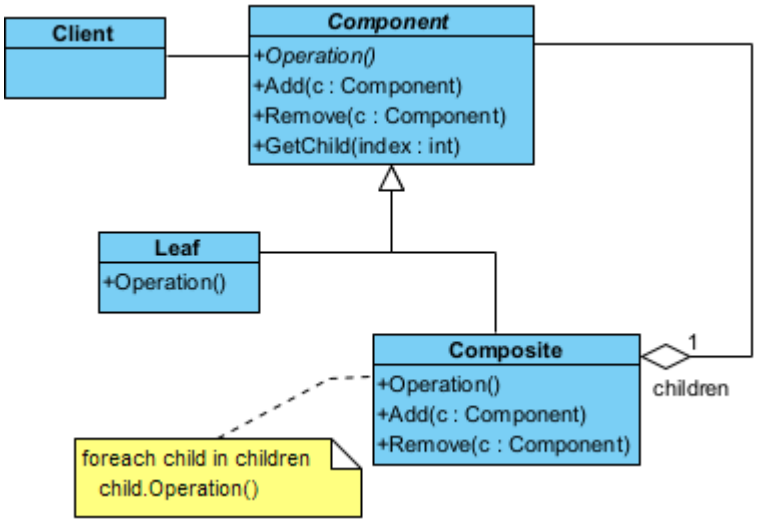
//3. --- Adapter (Adaptör Sınıfı) ---
// Eski sınıfı yeni arayüze sarmalayan dönüştürücü
class LoggerAdapter : public ILogger {
  private:
  std::unique_ptr<OldLogger> oldLogger_; // Nesne ömrünü yönetmek için akıllı işaretçi
  public:
  // Adaptör oluşturulurken eski sınıfın bir örneğini alır
  LoggerAdapter(std::unique_ptr<OldLogger> logger) : oldLogger_(std::move(logger)) {}
  // Yeni arayüz metodu çağrıldığında, arka planda eski metodu uygun parametreyle çalış
      tırır
  void log(const std::string& msg) override {
    // std::string'i eski sistemin beklediği const char*'a çeviriyoruz
    oldLogger_->logMessage(msg.c_str());
  }
};

//4. --- Client (İstemci) ---
// Sadece ILogger arayüzünü tanır
void appLog(ILogger& logger) {
  logger.log("Sistem basariyla calisti.");
}
int main() {
  // 1. Eski sistemden bir logger nesnesi oluşturalım
```

```
  auto oldSys = std::make_unique<OldLogger>();
  // 2. Bu nesneyi yeni sistemde kullanamayız çünkü metod isimleri ve tipleri farklı.
  // Bu yüzden Adaptör devreye giriyor:
  std::unique_ptr<ILogger> modernLogger =
  std::make_unique<LoggerAdapter>(std::move(oldSys));
  // 3. İstemci (appLog), arka planda eski bir sistemin çalıştığını bilmeden
  // standart log() metodunu çağırır.
  std::cout << "Modern uygulama log gönderiyor..." << std::endl;
  appLog(*modernLogger);
}
/* Programın Çıktısı:
Modern uygulama log gönderiyor...
[Eski Kayit Sistemi]: Sistem basariyla calisti.
*/
```

§5.3.3 Bileşik

Bileşik kalıbı (*composite pattern*), özyinelemeli (*recursive*) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler. Bu kalıp, her nesnenin bağımsız olarak veya aynı arayüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır.



Şekil 5.8: Bileşik kalıbı UML Diyagramı

Bu kalıbın görevi, nesneleri "parça-bütün" hiyerarşisinde (ağaç yapısı) düzenler. Tekil nesne ile nesne gruplarına aynı şekilde davranılmasını sağlar. Bir klasör örneği üzerinden ilerlersek;

- ◇ Bileşen (`Component`), hem dosyaların hem klasörlerin ortak özelliklerini tanımlayan arayüz.
- ◇ Yaprak (`Leaf`), En küçük birim (Örnek olarak Dosya).
- ◇ Bileşik (`Composite`), içinde başka birimler barındırabilen yapı (Örnek olarak Dosya içerebilen Klasör).

Şimdi de kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <list>
#include <memory>  // unique_ptr ve make_unique için
#include <string>
#include <algorithm>

//1.--- Abstract Component ---
class Component {
protected:
  Component* ebeveyn = nullptr; // Sadece gözlem amaçlı ham pointer (sahiplik içermez)
```

```
public:
  virtual ~Component() = default;
  void setEbeveyn(Component* pEbeveyn) {
    this->ebeveyn = pEbeveyn;
  }
  Component* getEbeveyn() const {
    return this->ebeveyn;
  }
  virtual bool IsComposite() const {
    return false;
  }
  // Sahipliği devralmak için unique_ptr kullanıyoruz
  virtual void addChild(std::unique_ptr<Component> component) {}
  virtual void removeChild(Component* component) {}
  virtual std::string displayOperation() const = 0;
};

//2.--- Leaf (Yaprak) ---
class Leaf : public Component {
public:
  std::string displayOperation() const override {
    return "Yaprak";
  }
};

//3.--- Composite (Dal) ---
class Composite : public Component {
private:
  // Çocukların gerçek sahibi bu listedir
  std::list<std::unique_ptr<Component>> cocuklar;
public:
  void addChild(std::unique_ptr<Component> pComponent) override {
    pComponent->setEbeveyn(this);
    this->cocuklar.push_back(move(pComponent)); // Sahiplik listeye geçer
  }
  void removeChild(Component* pComponent) override {
    // Listede bu adrese sahip olan unique_ptr'yi bul ve sil
    cocuklar.remove_if([pComponent](const std::unique_ptr<Component>& ptr) {
      return ptr.get() == pComponent;
    });
  }
  bool IsComposite() const override {
    return true;
  }
  std::string displayOperation() const override {
    std::string result;
    for (auto it = cocuklar.begin(); it != cocuklar.end(); ++it) {
      if (it != cocuklar.begin()) result += "+";
      result += (*it)->displayOperation();
    }
    return "Dal(" + result + ")";
  }
};

// 4.--- Client (İstemci) ---
int main() {
  // 1. Sadece yaprak örneği
  auto yaprak = std::make_unique<Leaf>();
  std::cout << "Sadece Yapraktan Olusan Bilesik Nesne:" ;
  std::cout << yaprak->displayOperation() << std::endl;
  // 2. Karmaşık ağaç yapısı
  auto kok = std::make_unique<Composite>();
  auto dal1 = std::make_unique<Composite>();
  // dal1'i kok'e eklemeden önce yaprakları dal1'e ekleyelim
  dal1->addChild(std::make_unique<Leaf>()); // yaprak1
  // Daha sonra silmek istediğimiz bir yaprak varsa adresini saklamalıyız
```

```
  auto yaprak2_raw = new Leaf(); // Geçici ham pointer (veya make_unique sonrası .get()
      )
  dal1->addChild(std::unique_ptr<Leaf>(yaprak2_raw));
  kok->addChild(std::move(dal1)); // dal1'in sahipliği kok'e geçti
  std::cout << "Yapraktan ve Dallardan Olusan Bilesik Nesne:" ;
  std::cout << kok->displayOperation() << std::endl;
  // 3. Silme işlemi (yaprak2_raw üzerinden güvenli bir şekilde)
  // Not: kok içindeki dal1'e erişmek için hiyerarşiyi bilmek gerekir,
  // gerçek projelerde daha dinamik yöntemler kullanılır.
  // Burada basitlik adına doğrudan kok üzerinden gösterim yapıyoruz.
  // 4. Dal2 ekleme
  auto dal2 = std::make_unique<Composite>();
  dal2->addChild(std::make_unique<Leaf>()); // yaprak3
  kok->addChild(std::move(dal2));
  std::cout << "Bir dal daha eklenmis Bilesik Nesne:" ;
  std::cout << kok->displayOperation() << std::endl;
  // Hiçbir delete işlemine gerek yok!
  // 'kok' silindiğinde tüm dallar ve yapraklar sırayla otomatik silinir.
}
/* Programın Çıktısı:
Sadece Yapraktan Olusan Bilesik Nesne:Yaprak
Yapraktan ve Dallardan Olusan Bilesik Nesne:Dal(Dal(Yaprak+Yaprak))
Bir dal daha eklenmis Bilesik Nesne:Dal(Dal(Yaprak+Yaprak)+Dal(Yaprak))
*/
```

Bu kalıba gerçek dünya örneği olarak aşağıdaki dosya sistemini gösteren örnek verilebilir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <utility>

//1. --- Component (Bileşen) ---
class FileSystemNode {
public:
  virtual ~FileSystemNode() = default;
  // Varsayılan parametreyi burada tanımlıyoruz
  virtual void showDetails(int indent = 0) const = 0;
  virtual long getSize() const = 0;
};

//2. --- Leaf (Yaprak) ---
class File : public FileSystemNode {
private:
  std::string name_;
  long size_;
public:
  File(std::string name, long size) : name_(std::move(name)), size_(size) {}
  // override kelimesiyle imzanın aynı olduğunu garantiliyoruz
  void showDetails(int indent) const override {
    std::string spacing(indent, ' ');
    std::cout << spacing << "- Dosya: " << name_ << " (" << size_ << " KB)" << std::
        endl;
  }
  long getSize() const override { return size_; }
};

//3. --- Composite (Bileşik) ---
class Directory : public FileSystemNode {
private:
  std::string name_;
  std::vector<std::shared_ptr<FileSystemNode>> children_;
public:
  Directory(std::string name) : name_(std::move(name)) {}
```

```
  void add(std::shared_ptr<FileSystemNode> node) {
    children_.push_back(node);
  }
  // Dikkat: Burada 'indent = 0' yazmaya gerek yok, base class'tan devralınır.
  void showDetails(int indent) const override {
    std::string spacing(indent, ' ');
    std::cout << spacing << "+ Klasor: " << name_ << " [Toplam: " << getSize() << " KB]
        " << std::endl;
    for (const auto& child : children_) {
      // Özyinelemeli çağrı
      child->showDetails(indent + 4);
    }
  }
  long getSize() const override {
    long total = 0;
    for (const auto& child : children_) {
      total += child->getSize();
    }
    return total;
  }
};

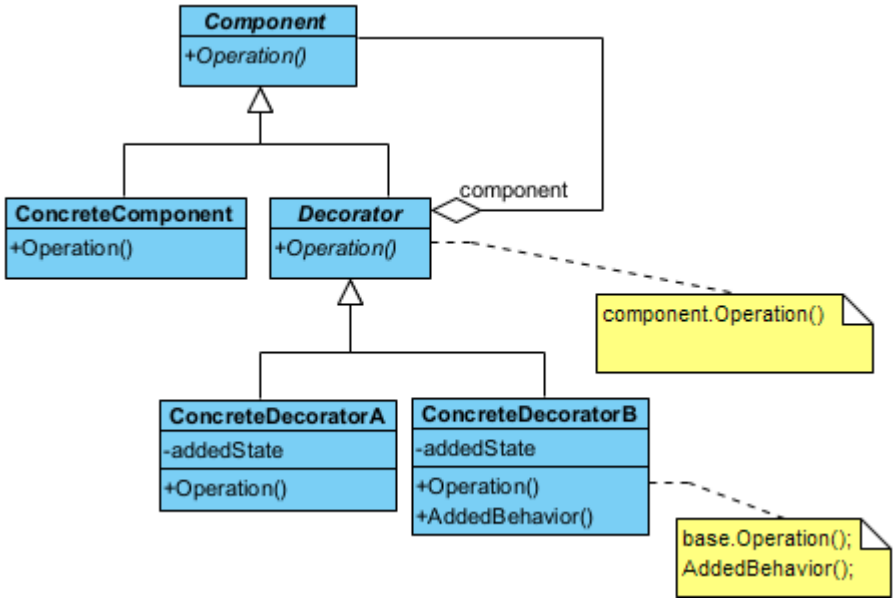
//4. --- Client (İstemci) ---
int main() {
  // Klasörleri oluşturalım
  auto root = std::make_shared<Directory>("C_Surucusu");
  auto docs = std::make_shared<Directory>("Belgelerim");
  // Dosyaları ekleyelim
  docs->add(std::make_shared<File>("Notlar.txt", 10));
  root->add(docs);
  root->add(std::make_shared<File>("Sistem.log", 500));
  // Bazı derleyiciler paylaşılan pointer (shared_ptr) üzerinden varsayılan parametreye
       erişirken sorun yaşayabilir. Açıkça 0 göndererek veya arayüzü doğru kullanarak ç
      özeriz.
  root->showDetails(0);
}
/* Programın Çıktısı:
+ Klasor: C_Surucusu [Toplam: 510 KB]
+ Klasor: Belgelerim [Toplam: 10 KB]
- Dosya: Notlar.txt (10 KB)
- Dosya: Sistem.log (500 KB)
*/
```

§5.3.4 Süsleyici

Süsleyici kalıbında (*decorator pattern*), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar.

Bu tasarım kalıbı, bir nesneye dinamik olarak (çalışma zamanında) yeni sorumluluklar veya özellikler eklemek için kullanılan yapısal bir kalıptır. Kalıtımın (*inheritance*) aksine, özellikleri sınıf hiyerarşisini karmaşıklaştırmadan, nesneleri birbirinin içine "sarmalayarak" (*wrapping*) ekler. Bunu bir Rus matruşka bebeği veya bir kahve siparişine eklenen ekstralar (süt, şeker, karamel) gibi düşünebilirsin.

- ◇ Bileşen (`Component`), hem temel nesnenin hem de süsleyicilerin uyması gereken ortak soyut arayüzü tanımlar. Temel işlevin (Örnek olarak `ucretHesapla()`) imzasını tanımlar.
- ◇ Somut Bileşen (`ConcreteComponent`), üzerine ekleme yapılacak olan "temel" nesnedir. Fonksiyonun en yalın halini gerçekleştirir (Örnek olarak Sade Kahve).
- ◇ Soyut Süsleyici (`Decorator`), süsleyicilerin soyut temel yapısını kurar. İçinde bir `Component` referansı tutar. Bu referans sayesinde hem temel nesneyi hem de başka bir süsleyiciyi sarmalayabilir.
- ◇ Somut Süsleyiciler (ConcreteDecorator), gerçek özellikleri ekleyen sınıflardır.Temel nesnenin



Şekil 5.9: Süsleyici kalıbı UML Diyagramı

metodunu çağırır ve üzerine kendi eklemesini yapar (Örnek olarak Kahve ücretine +5 TL süt bedeli eklemek).

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmeli
#include <iostream>
#include <string>
#include <memory> // std::unique_ptr için

// 1. Component (Bileşen): Ortak arayüz
class Component {
public:
  virtual ~Component() = default; // Kritik: Sanal yıkıcı
  virtual void operation() const = 0;
};


// 2. ConcreteComponent (Somut Bileşen): Temel nesne
class ConcreteComponent : public Component {
public:
  void operation() const override {
    std::cout << "Temel Urun Islemi" << std::endl;
  }
};

// 3. Decorator (Süsleyici): Sarmalayıcı temel sınıf
class Decorator : public Component {
protected:
  std::unique_ptr<Component> component; // Sahiplik dekoratördedir
public:
  Decorator(std::unique_ptr<Component> c) : component(std::move(c)) {}
  void operation() const override {
    if (component) component->operation();
  }
};

// 4. ConcreteDecorator1: Yeni Durum Ekleyen Süsleyici
class ConcreteDecorator1 : public Decorator {
private:
  int addedState;
public:
```

```
  ConcreteDecorator1(std::unique_ptr<Component> c, int state)
  : Decorator(std::move(c)), addedState(state * 100) {}
  void operation() const override {
    Decorator::operation(); // Önceki nesnenin davranışını çağır
    std::cout << "-> Decorator 1: Eklendi (State: " << addedState << ")" << std::endl;
  }
};

// 5. ConcreteDecorator2: Yeni Davranış Ekleyen Süsleyici
class ConcreteDecorator2 : public Decorator {
public:
  ConcreteDecorator2(std::unique_ptr<Component> c) : Decorator(std::move(c)) {}
  std::string addedBehavior() const {
    return "Bu metin Yeni Davranistan Geliyor...";
  }
  void operation() const override {
    Decorator::operation(); // Önceki nesnenin davranışını çağır
    std::cout << "-> Decorator 2: Yeni Davranis Tetiklendi: " << addedBehavior() << std
        ::endl;
  }
};

// 6. Client (İstemci)
int main() {
  // Nesneleri iç içe sararak (stacking) oluşturuyoruz
  std::cout << "--- Dekorator Zinciri Olusturuluyor ---" << std::endl;
  // 1. Temel ürün oluştur
  auto myProduct = std::make_unique<ConcreteComponent>();
  // 2. Ürünü Decorator 1 ile süsle
  auto decorated1 = std::make_unique<ConcreteDecorator1>(std::move(myProduct), 2);
  // 3. Ortaya çıkan nesneyi Decorator 2 ile tekrar süsle
  auto decorated2 = std::make_unique<ConcreteDecorator2>(std::move(decorated1));
  // Tek bir çağrı tüm zinciri tetikler
  decorated2->operation();
    // Her şey otomatik silinir (unique_ptr sayesinde)
}
/*Program Çıktısı:
--- Dekorator Zinciri Olusturuluyor ---
Temel Urun Islemi
-> Decorator 1: Eklendi (State: 200)
-> Decorator 2: Yeni Davranis Tetiklendi: Bu metin Yeni Davranistan Geliyor...
*/
```

Aşağıda gerçek dünya örneği olarak sipariş edilen kahveyi süsleyen bir örnek verilmiştir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory>  // unique_ptr ve move için gerekli
#include <utility> // std::move için

//1. --- Base Component (Temel Bileşen) ---
class Coffee {
public:
  virtual ~Coffee() = default;
  virtual double cost() const = 0;
  virtual std::string description() const = 0;
};

//2. --- Concrete Component (Somut Bileşen) ---
class SimpleCoffee : public Coffee {
public:
  double cost() const override { return 5.0; }
  std::string description() const override { return "Sade Kahve"; }
};
```

```cpp
//3. --- Base Decorator (Temel Süsleyici) ---
class CoffeeDecorator : public Coffee {
protected:
  std::unique_ptr<Coffee> coffee_;
public:
  CoffeeDecorator(std::unique_ptr<Coffee> coffee) : coffee_(std::move(coffee)) {}
};

//4. --- Concrete Decorators (Somut Süsleyiciler) ---
class MilkDecorator : public CoffeeDecorator {
public:
  using CoffeeDecorator::CoffeeDecorator;
  double cost() const override {
    return coffee_->cost() + 1.5; // Mevcut fiyata süt ekle
  }
  std::string description() const override {
    return coffee_->description() + ", süt";
  }
};

class SugarDecorator : public CoffeeDecorator {
public:
  using CoffeeDecorator::CoffeeDecorator;
  double cost() const override {
    return coffee_->cost() + 0.5; // Mevcut fiyata şeker ekle
  }
  std::string description() const override {
    return coffee_->description() + ", seker";
  }
};

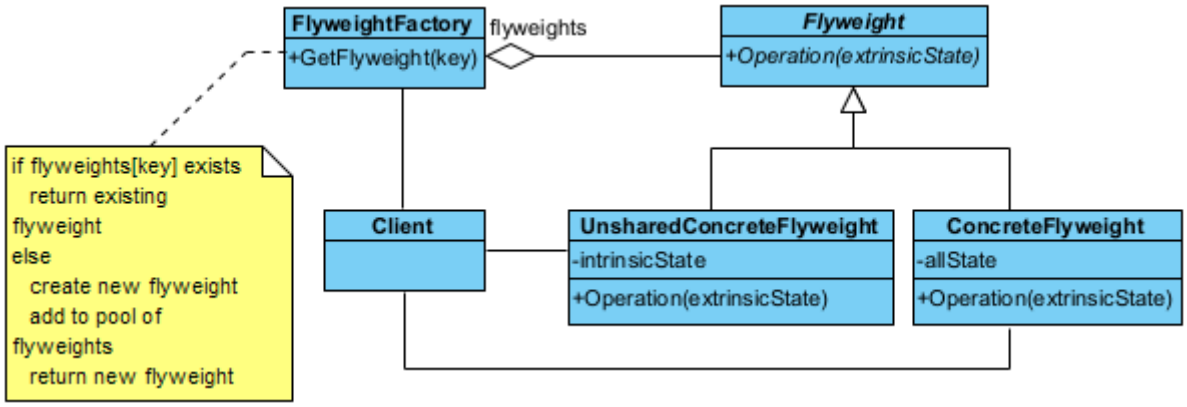
//5. --- Client (İstemci) ---
int main() {
  // 1. Sade bir kahve oluşturuyoruz
  std::unique_ptr<Coffee> kahvem = std::make_unique<SimpleCoffee>();
  // 2. Kahveye süt ekliyoruz (Sade kahveyi MilkDecorator ile sarmalıyoruz)
  kahvem = std::make_unique<MilkDecorator>(std::move(kahvem));
  // 3. Kahveye şeker ekliyoruz (Sütlü kahveyi SugarDecorator ile sarmalıyoruz)
  kahvem = std::make_unique<SugarDecorator>(std::move(kahvem));
  // Sonuçları yazdıralım
  std::cout << "Siparis: " << kahvem->description() << std::endl;
  std::cout << "Toplam Fiyat: " << kahvem->cost() << " TL" << std::endl;
}
/*
Siparis: Sade Kahve, süt, seker
Toplam Fiyat: 7 TL
*/
```

§5.3.5 Sinek Sıklet

Sinek Sıklet kalıbında (*flyweight pattern*),çok sayıda benzer nesnenin bulunduğu durumlarda bellek kullanımını minimize etmek için kullanılan yapısal (*structural*) bir kalıptır.

Bu kalıbın temel stratejisi, nesnelerin ortak verilerini (*intrinsic state*) paylaşarak bellekte tek bir kopyasını tutmak, her nesneye özgü değişen verileri (*extrinsic state*) ise dışarıdan sağlamaktır.

- ◇ Soyut Sinek Sıklet (`Flyweight`), paylaşılan nesnelerin uyması gereken arayüzü tanımlar. Nesneye özgü verilerin (*extrinsic state*) parametre olarak geçildiği yöntemin imzasını barındırır.
- ◇ Somut Sinek Sıklet (`ConcreteFlyweight`), paylaşılan veriyi (*intrinsic state*) kendi içinde saklayan nesnedir. Bu nesneler "değişmez" (*immutable*) olmalıdır. Örneğin; bir oyun için "Ağaç" modelinin 3D verisi burada bir kez tutulur.
- ◇ Sinek Sıklet Fabrikası (`FlyweightFactory`), `Flyweight` nesnelerini yönetir ve paylaşılmasını sağlar. İstendiğinde, eğer nesne daha önce oluşturulmuşsa mevcut olanı verir; yoksa yenisini oluşturup havuza ekler.



Şekil 5.10: Sinek Sıklet kalıbı UML Diyagramı

◇ Bağlam (`Context`), her nesneye özgü, sürekli değişen verileri (konum, renk gibi) tutar. Somut `ConcreteFlyweight` nesnesini bu verilerle birlikte kullanarak asıl işlemi gerçekleştirir.

Şimdi kalıbımizi kodlayalım;

```
//C++17 ve üzeri ile derlenecek
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <memory>

// 1. Flyweight Arayüzü
class Flyweight {
  public:
  virtual ~Flyweight() = default;
  virtual void operation(const std::string& extrinsicState) const = 0;
  protected:
  std::string intrinsicState; // Ortak olan, değişmez durum
  Flyweight(std::string state) : intrinsicState(std::move(state)) {}
};
// 2. Somut Paylaşılan Flyweight (Concrete Flyweight)
class ConcreteFlyweight : public Flyweight {
  public:
  explicit ConcreteFlyweight(std::string state) : Flyweight(std::move(state)) {}

  void operation(const std::string& extrinsicState) const override {
    std::cout << "[Paylasilan] Ic Durum: " << intrinsicState
    << " | Dis Durum: " << extrinsicState << std::endl;
  }
};
// 3. Flyweight Factory (Fabrika)
class FlyweightFactory {
private:
  // Hızlı erişim için map kullanıyoruz. Nesne ömrünü shared_ptr yönetir.
  std::unordered_map<std::string, std::shared_ptr<Flyweight>> flyweights;
public:
  std::shared_ptr<Flyweight> getFlyweight(const std::string& key) {
    // Eğer bu anahtara sahip nesne zaten varsa onu döndür
    if (flyweights.find(key) != flyweights.end()) {
      std::cout << "-> Fabrika: " << key << " zaten var, mevcut olan döndürülüyor.\n";
      return flyweights[key];
    }

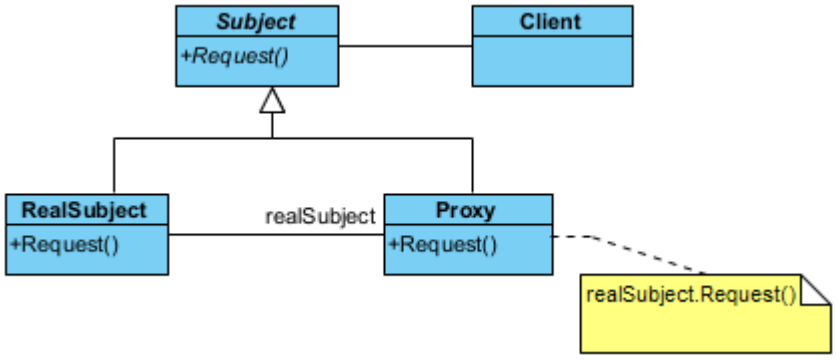
    // Yoksa yeni bir tane oluştur ve depola
    std::cout << "-> Fabrika: " << key << " bulunamadi, yeni olusturuluyor.\n";
    auto flyweight = std::make_shared<ConcreteFlyweight>(key);
    flyweights[key] = flyweight;
```

```
    return flyweight;
  }
  void listFlyweights() const {
    std::cout << "\nFabrikadaki toplam nesne sayisi: " << flyweights.size() << "\n";
    for (const auto& [key, value] : flyweights) {
      std::cout << "- " << key << "\n";
    }
  }
};
// 4. Client (İstemci)
int main() { // Client (İstemci)
  FlyweightFactory factory;
  // Aynı nesneleri tekrar tekrar isteyelim
  auto v1 = factory.getFlyweight("Binek Arac");
  auto v2 = factory.getFlyweight("Kamyon");
  auto v3 = factory.getFlyweight("Binek Arac"); // Fabrikadan mevcut olan gelecek
  // Dışsal durumları (konum, plaka gibi) işlem sırasında gönderiyoruz
  v1->operation("Konum: 41.00, 28.97");
  v2->operation("Konum: 39.93, 32.85");
  v3->operation("Konum: 40.71, -74.00");
  factory.listFlyweights();
  // shared_ptr sayesinde manuel delete gerekmez.
}
/*Program Çıktısı:
-> Fabrika: Binek Arac bulunamadi, yeni olusturuluyor.
-> Fabrika: Kamyon bulunamadi, yeni olusturuluyor.
-> Fabrika: Binek Arac zaten var, mevcut olan döndürülüyor.
[Paylasilan] Ic Durum: Binek Arac | Dis Durum: Konum: 41.00, 28.97
[Paylasilan] Ic Durum: Kamyon | Dis Durum: Konum: 39.93, 32.85
[Paylasilan] Ic Durum: Binek Arac | Dis Durum: Konum: 40.71, -74.00

Fabrikadaki toplam nesne sayisi: 2
- Kamyon
- Binek Arac
*/
```

§5.3.6 Vekil

Vekil kalıbı (*proxy pattern*), kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. Bu kalıp, başka bir nesneye erişimi kontrol etmek için kullanılan yapısal bir kalıptır. Bir nesnenin önüne "vekil" bir katman koyarak, asıl nesneye erişmeden önce güvenlik kontrolü yapabilir, nesnenin yüklenmesini geciktirebilir (*lazy loading*) veya işlem sonuçlarını önbelleğe alabilirsiniz.



Şekil 5.11: Vekil kalıbı UML Diyagramı

Bu kalıbı bir şirketteki "Yönetici Asistanı" gibi düşünebilirsin. Yöneticiyle doğrudan görüşemezsin; önce asistanla (`Proxy`) görüşürsün, asistan uygun görürse seni yöneticiye (`RealSubject`) yönlendirir.

◇ Bağlam (`Subject`), hem asıl nesnenin hem de vekilin (`Proxy`) uyması gereken ortak soyut

arayüzü tanımlar. İstemcinin (`Client`) her iki nesneye de aynı şekilde davranabilmesini sağlar.
- ◇ Gerçek Bağlam (RealSubject), asıl işi yapan, asıl mantığı barındıran nesnedir. Genellikle oluşturulması maliyetli veya erişimi kısıtlı olan nesnedir.
- ◇ Vekil (`Proxy`), gerçek nesneye olan referansı kendi içinde saklar. İstekleri gerçek nesneye iletmeden önce araya girer. Erişim kontrolü yapabilir, gerçek nesne henüz oluşturulmamışsa onu oluşturabilir veya gelen isteği reddedebilir.
- ◇ İstemci (`Client`), nesnenin doğrudan kendisine değil, vekil (`Proxy`) nesnesine ulaşır ve bu nesneye isteklerini iletir.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory> // std::unique_ptr için

// 1. Subject (Özne Arayüzü): Gerçek nesne ve vekilin ortak arayüzü
class Subject {
public:
  virtual ~Subject() = default; // Kritik: Modern C++ sanal yıkıcı
  virtual void request() const = 0;
};

// 2. RealSubject (Gerçek Özne): Asıl işi yapan nesne
class RealSubject : public Subject {
private:
  int state;
public:
  explicit RealSubject(int pState) : state(pState) {}
  void request() const override {
    std::cout << "RealSubject: Istek isleniyor (Durum: " << state << ")" << std::endl;
  }
};

// 3. Proxy (Vekil): Gerçek nesneye erişimi yönetir
class Proxy : public Subject {
private:
  // Mülkiyet vekilin içindedir (RAII)
  std::unique_ptr<RealSubject> realSubject;
  bool checkAccess() const {
    std::cout << "Proxy: Erisim yetkisi kontrol ediliyor..." << std::endl;
    return true;
  }
  void logAccess() const {
    std::cout << "Proxy: Islem gunlugu (log) kaydedildi." << std::endl;
  }
public:
  // Senaryo 1: Vekil kendi nesnesini oluşturur (Lazy Initialization örneği)
  Proxy() : realSubject(std::make_unique<RealSubject>(20)) {}
  // Senaryo 2: Mevcut bir nesnenin kopyası üzerinden vekillik yapar
  explicit Proxy(const RealSubject& source) : realSubject(std::make_unique<RealSubject
      >(source)) {}
  void request() const override {
    if (checkAccess()) {
      realSubject->request();
      logAccess();
    }
  }
};

// 4. Client (İstemci)
int main() { // Client (İstemci)
  // 1. Doğrudan erişim
  std::cout << "--- Dogrudan RealSubject Kullanimi ---" << std::endl;
  auto gercekOzne = std::make_unique<RealSubject>(10);
```

```
    gercekOzne->request();
    // 2. Kendi nesnesini yöneten vekil (Default Proxy)
    std::cout << "--- Proxy (Yeni Nesne Ile) ---" << std::endl;
    auto vekil1 = std::make_unique<Proxy>();
    vekil1->request();
    // 3. Mevcut nesneyi kopyalayan vekil
    std::cout << "--- Proxy (Mevcut Nesne Kopyasi Ile) ---" << std::endl;
    auto vekil2 = std::make_unique<Proxy>(*gercekOzne);
    vekil2->request();
    // Not: Manuel 'delete' yok! unique_ptr sayesinde her şey otomatik temizlenir.
}
/*Program Çıktısı:
--- Dogrudan RealSubject Kullanimi ---
RealSubject: Istek isleniyor (Durum: 10)
--- Proxy (Yeni Nesne Ile) ---
Proxy: Erisim yetkisi kontrol ediliyor...
RealSubject: Istek isleniyor (Durum: 20)
Proxy: Islem gunlugu (log) kaydedildi.
--- Proxy (Mevcut Nesne Kopyasi Ile) ---
Proxy: Erisim yetkisi kontrol ediliyor...
RealSubject: Istek isleniyor (Durum: 10)
Proxy: Islem gunlugu (log) kaydedildi.
*/
```

Bu kalıpla ilgili şu kavramlardan bahsetmek gerekir;

- ◇ Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa uzak vekil (*remote proxy*) kullanılır. Gerçek nesne çok ağırsa (örneğin büyük bir resim), nesneyi sadece gerçekten ihtiyaç duyulduğunda (*lazy loading*) oluşturur.
- ◇ Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa koruma vekili (*protection proxy*) olarak adlandırılır. Yetkilendirme (*authentication*) kontrolü yapar.
- ◇ Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, akıllı referans (*smart reference*) olarak adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, akıllı gösterici (*smart pointer*) olarak da adlandırılır. Nesne başka bir sunucudaysa ağ trafiğini yönetir.

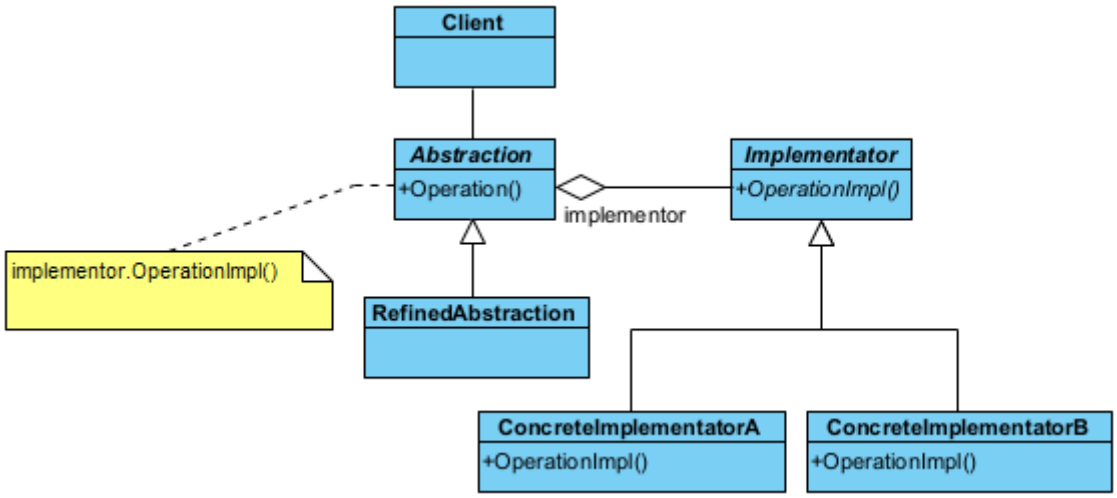
§5.3.7 Köprü

Köprü kalıbı (*bridge pattern*), işi yapan alt yüklenici veya taşeron (*implementator*) nesne ile işi yaptıran nesne (*client*) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak arayüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir. Bu kalıp, yazılımlarda oldukça çok kullanılan bir kalıpdir. Çünkü taşeronlar değişse de yeni taşeron eklense de istemci tarafında bir şey değişmez.

Bu tasarım kalıbı, bir nesnenin soyutlamasını (*abstraction*), onun uygulamasından (*implementation*) ayırarak her ikisinin de birbirinden bağımsız olarak geliştirilmesini sağlayan yapısal bir kalıptır. Bu kalıp, özellikle **sınıf patlaması** (*class explosion*) dediğimiz, her yeni özellik için onlarca alt sınıf türetmek zorunda kaldığımız durumlarda imdadımıza yetişir. Kalıtım yerine bileşimi (*composition*) kullanır.

- ◇ Soyutlama (`Abstraction`), istemcinin (`Client`) kullandığı üst düzey kontrol arayüzüdür. İçinde bir `Implementor` referansı tutar. İstemciden gelen komutları bu referansa iletir.
- ◇ Geliştirilmiş Soyutlama (`RefinedAbstraction`),soyutlamayı genişleten alt sınıflardır. Temel arayüzü kullanarak daha spesifik kontrol mantıkları ekler (Örn: Standart Kumanda -> Gelişmiş Akıllı Kumanda).
- ◇ Taşeron Arayüzü (`Implementor`), tüm somut uygulama sınıfları için ortak arayüzü tanımlar. Genellikle düşük seviyeli (ilkel) operasyonları barındırır (Örn: `cihaziAc()`, `sesAyari()`).
- ◇ Somut Taşeronlar (`ConcreteImplementor`), `Implementor` arayüzünü gerçekleyen asıl cihazlardır. Platforma veya cihaza özgü gerçek kodları barındırır (Örn: Televizyon, Radyo).

Şimdi kalıbımizi kodlayalım;



Şekil 5.12: Köprü kalıbı UML Diyagramı

```cpp
//C++14 ve üzeri ile derlenmeli
#include <iostream>
#include <memory> // Akıllı işaretçiler için
#include <utility>

//1.--- Implementor (Uygulayıcı/Taşeron Arayüzü) ---
class Implementor {
public:
  virtual ~Implementor() = default; // Güvenli temizlik için sanal yıkıcı
  virtual void operationImpl() = 0;
};

//2.--- Concrete Implementors ---
class ConcreteImplementorA : public Implementor {
public:
  void operationImpl() override {
    std::cout << "A Taseronu tarafindan yapilan is..." << std::endl;
  }
};
class ConcreteImplementorB : public Implementor {
public:
  void operationImpl() override {
    std::cout << "B Taseronu tarafindan yapilan is..." << std::endl;
  }
};

//3.--- Abstraction (Soyut Yüklenici) ---
class Abstraction {
protected:
  // Taşeronun mülkiyeti (sahipliği) yükleniciye ait olsun veya olmasın,
  // burada shared_ptr veya unique_ptr seçimi stratejiktir.
  // Taşeronların çalışma anında değişebilirliği için unique_ptr ile sahipliği yö
     netiyoruz.
  std::unique_ptr<Implementor> implementor;
public:
  Abstraction(std::unique_ptr<Implementor> pImpl) : implementor(std::move(pImpl)) {}
  virtual ~Abstraction() = default;
  // Çalışma anında taşeronu güvenli bir şekilde değiştiriyoruz
  void setImplementor(std::unique_ptr<Implementor> pImpl) {
    implementor = move(pImpl);
  }
  void operation() {
    if (implementor) {
      std::cout << "Yuklenici Taserona is yaptiriyor:" << std::endl;
```

```
        implementor->operationImpl();
    }
  }
};

//4.--- Refined Abstraction (Geliştirilmiş Soyutlama) ---
class RefinedAbstraction : public Abstraction {
public:
  using Abstraction::Abstraction; // Üst sınıfın constructor'ını kullan
};

// 5.--- Client (İstemci) ---
int main() {
  // 1. Yükleniciyi ilk taşeronla (A) başlatıyoruz
  auto yuklenici = std::make_unique<RefinedAbstraction>(std::make_unique<
      ConcreteImplementorA>());
  yuklenici->operation();
  std::cout << "--- Taseron Degistiriliyor ---" << std::endl;
  // 2. Çalıştırma anında yeni bir taşerona (B) geçiyoruz
  // Eski taşeron (A), unique_ptr'nin doğası gereği otomatik olarak hafızadan silinir.
  yuklenici->setImplementor(std::make_unique<ConcreteImplementorB>());
  yuklenici->operation();
  // Hiçbir delete işlemine gerek yok! 'yuklenici' kapsam dışına çıktığında içindeki ta
      şeronla birlikte silinecektir.
}
/*Program Çıktısı:
Yuklenici Taserona is yaptiriyor:
A Taseronu tarafindan yapilan is...
--- Taseron Degistiriliyor ---
Yuklenici Taserona is yaptiriyor:
B Taseronu tarafindan yapilan is...
*/
```

Köprü kalıbı aşağıdaki durumlarda kullanılır;

- ◇ Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalışma anında (*run-time*) taşeronlar arasında geçiş yapmayı sağlar.
- ◇ Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.
- ◇ Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- ◇ Projede aynı anda farklı birden çok taşeron tasarımı yapılabildiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum *Rumbaugh* tarafından iç içe genelleştirme (*nested generalization*) olarak adlandırılmıştır.

5.4 Davranışla İlgili Kalıplar

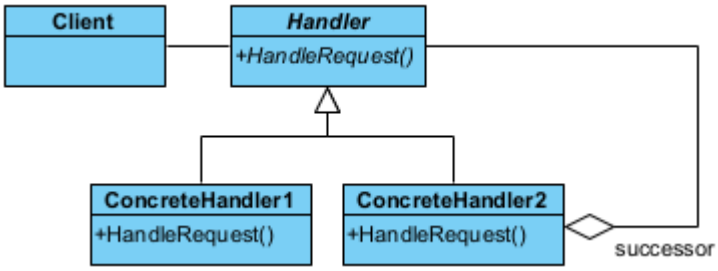
Davranışla ilgili kalıplar (*behavioral patterns*) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

§5.4.1 Sorumluluk Zinciri

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi halefine (*successor*) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda **sorumluluk zinciri kalıbı** (*chain of responsibility pattern*) kullanılır.

Bu kalıbın temel amacı, isteği gönderen ile isteği işleyen arasındaki bağı gevşetmektir (*decoupling*). İstek, zincir boyunca ilerler ve her bir işleyici ya isteği çözer ya da bir sonraki işleyiciye paslar.

- ◇ İşleyici (`Handler`), İsteklerin nasıl işleneceğine dair ortak bir arayüz tanımlar. İşleyici, bir sonraki işleyiciyi tutan bir referans barındırır ve zincirin halkalarını birbirine bağlar.
- ◇ Somut İşleyiciler (`ConcreteHandler`), kendisine gelen isteği kontrol eder. Eğer istek kendi sorumluluk alanındaysa işi bitirir. Eğer değilse (veya işin bir kısmını yapıp devam etmesi gerekiyorsa), isteği zincirdeki bir sonraki halkaya iletir.



Şekil 5.13: Sorumluluk Zinciri kalıbı UML Diyagramı

- İstemci (`Client`), zinciri oluşturur (halkaları birbirine bağlar) ve ilk isteği zincirin başına gönderir. İsteğin kim tarafından, nasıl çözüleceğiyle ilgilenmez; sadece "çözülmesini" bekler.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenecek
#include <iostream>
#include <memory>

//1. Soyut İşleyici (Handler)
class Handler {
protected:
  std::unique_ptr<Handler> successor; // Zincirin bir sonraki halkası
public:
  virtual ~Handler() = default;
  // Zincire yeni bir halka eklemek için yardımcı metod
  void setSuccessor(std::unique_ptr<Handler> next) {
    successor = std::move(next);
  }
  virtual void handleRequest(int request) = 0;
};

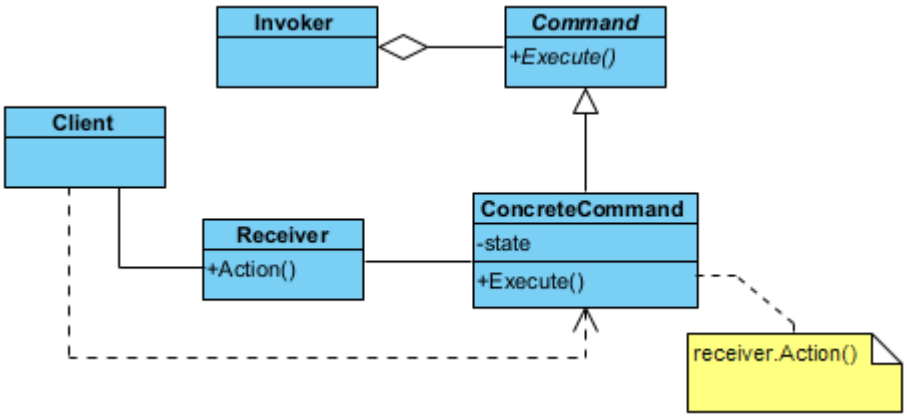
//2. Somut İşleyici 1: Küçük Talepler (Örn: Memur)
class ConcreteHandler1 : public Handler {
public:
  void handleRequest(int request) override {
    if (request < 100) { //Birinci Halka: Kendisi
      std::cout << "Memur: " << request << " birimlik talebi onayladi." << std::endl;
    } else if (successor) { //İkinci Halka: Halefi
      std::cout << "Memur: Talep limitimi asiyor, Sefe devrediliyor..." << std::endl;
      successor->handleRequest(request);
    } else {
      std::cout << "Hata: Talebi karsilayacak kimse bulunamadi!" << std::endl;
    }
  }
};

//3. Somut İşleyici 2: Büyük Talepler (Örn: Sef)
class ConcreteHandler2 : public Handler {
public:
  void handleRequest(int request) override {
    if (request >= 100 && request < 500) {
      std::cout << "Sef: " << request << " birimlik talebi onayladi." << std::endl;
    } else if (successor) {
      std::cout << "Sef: Talep limitimi asiyor, Mudure devrediliyor..." << std::endl;
      successor->handleRequest(request);
    } else {
      std::cout << "Sef: Bu kadar buyuk bir yetkim yok ve baska amir tanimli degil!" <<
           std::endl;
    }
  }
};
```

```
//4. Client (İstemci)
int main() {
  // Zinciri oluşturuyoruz
  auto memur = std::make_unique<ConcreteHandler1>();
  auto sef = std::make_unique<ConcreteHandler2>();
  // Memurun halefi Sef olsun
  memur->setSuccessor(std::move(sef));
  std::cout << "--- Talep 1 (50) ---" << std::endl;
  memur->handleRequest(50);
  std::cout << "--- Talep 2 (250) ---" << std::endl;
  memur->handleRequest(250);
  std::cout << "--- Talep 3 (1000) ---" << std::endl;
  memur->handleRequest(1000);
  // unique_ptr sayesinde tüm zincir otomatik olarak silinir.
}
/*Program Çıktısı:
--- Talep 1 (50) ---
Memur: 50 birimlik talebi onayladi.
--- Talep 2 (250) ---
Memur: Talep limitimi asiyor, Sefe devrediliyor...
Sef: 250 birimlik talebi onayladi.
--- Talep 3 (1000) ---
Memur: Talep limitimi asiyor, Sefe devrediliyor...
Sef: Bu kadar buyuk bir yetkim yok ve baska amir tanimli degil!
*/
```

§5.4.2 Komut

Bazı durumlarda bir nesne diğer bir nesneye ileti (*message*) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut kalıbı** (*command pattern*) kullanılır.



Şekil 5.14: Komut kalıbı UML Diyagramı

Bu tasarım kalıbı, bir isteği veya eylemi kendi başına bir nesneye dönüştüren davranışsal bir kalıptır. Bu dönüşüm sayesinde eylemleri parametre olarak geçebilir, kuyruğa alabilir (*queue*), iz kaydını tutabilir (*log*) veya geri alabilirsiniz (*undo*). Kısacası; "ne yapılacağı" bilgisini, "kimin yapacağı" bilgisinden tamamen ayırır.

- ◇ Komut (`Command`), tüm komutların uyması gereken soyut arayüzü tanımlar. Genellikle tek bir yöntem barındırır (Örnek olarak `execute()`). Bazı durumlarda geri alma işlemi için `undo()` yöntemini de içerir.
- ◇ Somut Komut (`ConcreteCommand`), soyut komutu gerçek bir eylemle birleştirir. Alıcı (`Receiver`) nesnesini tanır ve `execute()` çağrıldığında alıcının ilgili metodunu tetikler.
- ◇ Alıcı(Receiver), işin asıl mutfağıdır. Gerçek mantığı ve operasyonları barındırır. Komut nesnesi ona "çalış" dediğinde, o işi nasıl yapacağını bilir (Örnek olarak Işığı açmak, dosyayı kaydetmek).

- ◇ Çağırıcı(Invoker), komutun ne zaman çalışacağına karar verir. Komutu saklar ve kullanıcı bir düğmeye bastığında veya bir olay gerçekleştiğinde komutun execute() metodunu çağırır. Alıcıyı doğrudan tanımaz.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenecek
#include <iostream>
#include <string>
#include <vector>
#include <memory>

//1. Receiver (Alıcı): Asıl işi yapan sınıf
class Receiver {
public:
  void action(const std::string& pParam) const {
    std::cout << "Receiver: [" << pParam << "] komutu icin temel islemler yapiliyor.\n"
        ;
  }
  void actionA(const std::string& pParam) const {
    std::cout << "Receiver: [" << pParam << "] icin A islemi.\n";
  }
  void actionB(const std::string& pParam) const {
    std::cout << "Receiver: [" << pParam << "] icin B islemi.\n";
  }
};

//2. Command (Komut Arayüzü)
class Command {
public:
  virtual ~Command() = default;
  virtual void execute() const = 0;
};

//3. Concrete Commands (Somut Komutlar)
class SimpleCommand : public Command {
private:
  std::string state;
  Receiver& receiver; // Komut, mevcut bir alıcıya referansla bağlanır
public:
  SimpleCommand(std::string pState, Receiver& r) : state(std::move(pState)), receiver(r
      ) {}
  void execute() const override {
    receiver.action(state);
  }
};
class ComplexCommand : public Command {
private:
  std::string state;
  Receiver& receiver;
public:
  ComplexCommand(std::string pState, Receiver& r) : state(std::move(pState)), receiver(
      r) {}
  void execute() const override {
    receiver.actionA(state);
    receiver.actionB(state);
  }
};

//4. Invoker (Çağırıcı): Komutları saklar ve tetikler
class Invoker {
private:
  std::vector<std::unique_ptr<Command>> history; // Komut geçmişini saklar
public:
  void storeAndExecute(std::unique_ptr<Command> cmd) {
    cmd->execute();
```

```
    history.push_back(std::move(cmd)); // Komutu sakla (Undo işlemleri için gerekebilir
        )
  }
};

//5. Client (İstemci)
int main() {
  // Tek bir alıcı (Receiver) oluşturulur
  Receiver receiver;
  Invoker invoker;
  std::cout << "--- Komutlar Tetikleniyor ---\n";
  // Komutlar oluşturulur ve Invoker aracılığıyla çalıştırılır
  invoker.storeAndExecute(std::make_unique<SimpleCommand>("Yazdir", receiver));
  invoker.storeAndExecute(std::make_unique<ComplexCommand>("Logla ve Gönder", receiver)
      );
  // Her şey otomatik temizlenir
}
/*Program Çıktısı:
--- Komutlar Tetikleniyor ---
Receiver: [Yazdir] komutu icin temel islemler yapiliyor.
Receiver: [Logla ve Gönder] icin A islemi.
Receiver: [Logla ve Gönder] icin B islemi.
*/
```

Aşağıda komut kalıbına uygun bir ışığım kapatılıp açılması örneği verilmiştir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// --- Receiver (Alıcı) ---
class Light {
public:
  void on()  { std::cout << "Isik ACILDI." << std::endl; }
  void off() { std::cout << "Isik KAPATILDI." << std::endl; }
};

// --- Command (Soyut Komut) ---
class Command {
public:
  virtual ~Command() = default;
  virtual void execute() = 0;
  virtual void undo() = 0;
};

// --- Concrete Commands (Somut Komutlar) ---
class LightOnCommand : public Command {
private:
  Light& light_; // Referans kullanımı: Işık nesnesi var olmak zorundadır.
public:
  explicit LightOnCommand(Light& light) : light_(light) {}
  void execute() override { light_.on(); }
  void undo() override    { light_.off(); }
};
class LightOffCommand : public Command {
private:
  Light& light_;
public:
  explicit LightOffCommand(Light& light) : light_(light) {}
  void execute() override { light_.off(); }
  void undo() override    { light_.on(); }
};

// --- Invoker (Çağırıcı) ---
```

```cpp
class RemoteControl {
private:
  // Geçmişi tutan yığın (LIFO - Last In First Out)
  std::vector<std::unique_ptr<Command>> history_;
public:
  void executeCommand(std::unique_ptr<Command> cmd) {
    cmd->execute();
    history_.push_back(std::move(cmd));
  }
  void undoLast() {
    if (!history_.empty()) {
      // back() ile son komutu alıyoruz
      std::cout << "[UNDO] ";
      history_.back()->undo();
      // Komut işini bitirdi, artık hafızadan silebiliriz
      history_.pop_back();
    } else {
      std::cout << "[UYARI] Geri alinacak islem yok!" << std::endl;
    }
  }
};

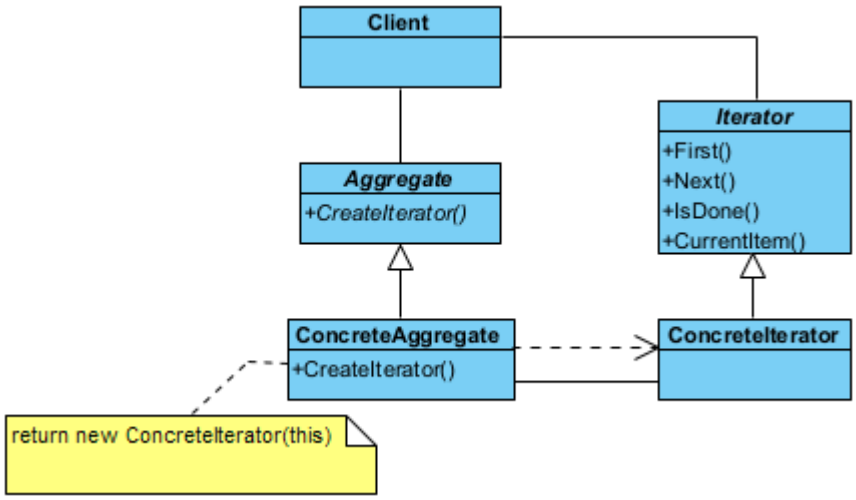
// --- Client (İstemci) ---
int main() {
  Light oturmaOdasiIsigi;
  RemoteControl kumanda;
  std::cout << "--- Islemler Basliyor ---" << std::endl;
  // Işığı açalım
  kumanda.executeCommand(std::make_unique<LightOnCommand>(oturmaOdasiIsigi));
  // Işığı kapatalım
  kumanda.executeCommand(std::make_unique<LightOffCommand>(oturmaOdasiIsigi));
  std::cout << std::endl <<"--- Geri Alma (Undo) Islemleri ---" << std::endl;
  kumanda.undoLast(); // Kapatmayı geri alır -> Işığı Açar
  kumanda.undoLast(); // Açmayı geri alır    -> Işığı Kapatır
  kumanda.undoLast(); // Liste boş uyarısı verir
}
/*Program Çıktısı:
--- Islemler Basliyor ---
Isik ACILDI.
Isik KAPATILDI.

--- Geri Alma (Undo) Islemleri ---
[UNDO] Isik ACILDI.
[UNDO] Isik KAPATILDI.
[UYARI] Geri alinacak islem yok!
*/
```

§5.4.3 Yineleyici

Yineleyici kalıbında (*iterator pattern*), birden fazla elemandan oluşan küme (*aggregate*) içindeki her bir elemana sırayla erişim (iteration) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu kalıpta, istemci tarafından veri kümesi soyut (*abstract*) olarak kullanılmış olur. Yani istemciye, veri yapısından bağımsız bir şekilde verilere erişim sağlayan bir arayüz (*interface*) sunulur. Bu tasarım kalıbı, bir veri grubunun (liste, ağaç, yığın vb.) iç yapısını dışarıya açmadan, o grubun elemanları üzerinde sırayla gezinebilmemizi sağlayan davranışsal bir kalıptır. Bu kalıp sayesinde, verilerin bellekte nasıl tutulduğunu (bir dizi mi yoksa birbirine bağlı halkalar mı?) bilmenize gerek kalmadan standart bir şekilde tüm elemanları dolaşabilirsiniz.

- ◇ Yineleyici (`Iterator`), elemanlar üzerinde gezinmek için gerekli olan standart arayüzü tanımlar. Genellikle `next()` (sonraki eleman), `hasNext()` (başka eleman var mı?) ve `current()` (mevcut eleman) gibi yöntemleri barındırır.
- ◇ Somut Yineleyici (`ConcreteIterator`), belirli bir veri yapısı (örneğin bir Vektör veya Liste)



Şekil 5.15: Yineleyici kalıbı UML Diyagramı

için gezinme mantığını uygular. O an hangi elemanda olduğunu (indis veya işaretçi bazlı) takip eder.

- ◇ Soyut Koleysiyon (`Aggregate`), koleksiyonun (*container*) kendisi için ortak bir arayüz sağlar. Kendi elemanlarını gezmek için kullanılacak olan uygun `Iterator` nesnesini oluşturan `createIterato` adlı bir yönteme sahitir.
- ◇ Somut Koleksiyon (`ConcreteAggregate`), verileri asıl saklayan sınıftır. İstendiğinde, kendi iç yapısını en verimli şekilde gezecek olan somut yineleyiciyi (`ConcreteIterator`) geri döndürür.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory> // std::unique_ptr için

//1. Soyut Iterator Arayüzü
class Iterator {
  public:
  virtual ~Iterator() = default;
  virtual double first() = 0;
  virtual double next() = 0;
  virtual bool hasNext() const = 0; // isDone yerine daha yaygın isimlendirme
  virtual double currentItem() const = 0;
};

//2. Soyut Koleksiyon (Aggregate) Arayüzü
class Aggregate {
public:
  virtual ~Aggregate() = default;
  virtual void addItem(double item) = 0;
  virtual size_t size() const = 0;
  virtual double getItem(size_t index) const = 0;
  virtual std::unique_ptr<Iterator> createIterator() = 0;
};

//3. Somut Iterator (Concrete Iterator)
class ConcreteIterator : public Iterator {
private:
  Aggregate* aggregate; // Koleksiyona sadece referans/işaretçi tutar, mülkiyet almaz
  size_t current = 0;
public:
  explicit ConcreteIterator(Aggregate* pAggregate) : aggregate(pAggregate) {}
  double first() override {
    current = 0;
```

```
    return currentItem();
  }
  double next() override {
    if (hasNext()) {
      return aggregate->getItem(current++);
    }
    return -1.0; // Veya hata fırlatılabilir
  }
  bool hasNext() const override {
    return current < aggregate->size();
  }
  double currentItem() const override {
    return aggregate->getItem(current);
  }
};

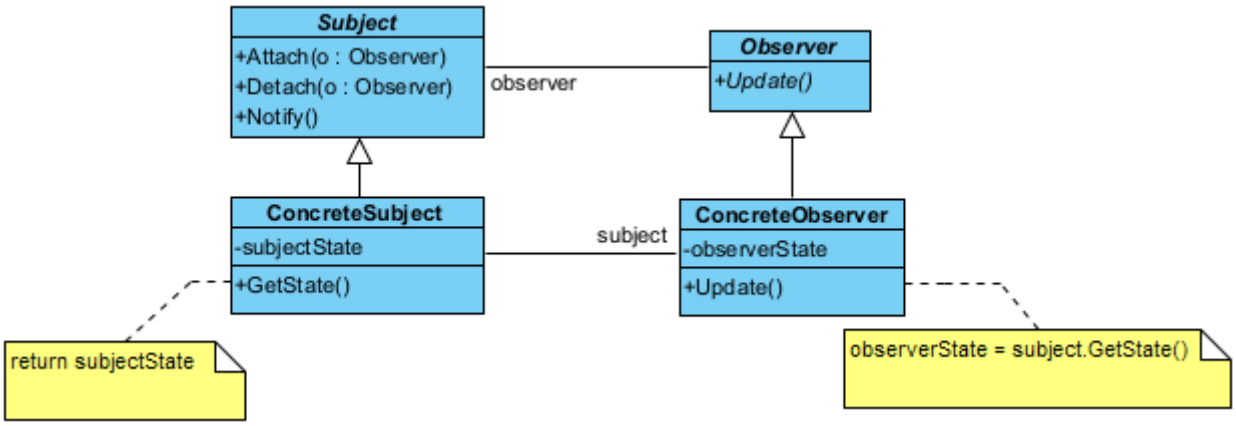
//4. Somut Koleksiyon (Concrete Aggregate)
class ConcreteAggregate : public Aggregate {
private:
  std::vector<double> items;
public:
  void addItem(double item) override {
    items.push_back(item);
  }
  size_t size() const override {
    return items.size();
  }
  double getItem(size_t index) const override {
    return items.at(index); // out_of_range kontrolü için .at() daha güvenlidir
  }
  std::unique_ptr<Iterator> createIterator() override {
    // factory method: mülkiyeti istemciye (unique_ptr ile) devreder
    return std::make_unique<ConcreteIterator>(this);
  }
};

//5. Client (İstemci)
int main() {// Client (İstemci)
  auto aggregate = std::make_unique<ConcreteAggregate>();
  aggregate->addItem(10.5);
  aggregate->addItem(20.7);
  aggregate->addItem(30.9);
  auto iterator = aggregate->createIterator();
  std::cout << "Koleksiyon elemanlari geziliyor:" << std::endl;
  // Standart döngü yapısı
  for (iterator->first(); iterator->hasNext(); ) {
    std::cout << "Deger: " << iterator->next() << std::endl;
  }
    // Manuel delete gerekmez, her şey otomatik temizlenir
}
/*Program Çıktısı:
Koleksiyon elemanlari geziliyor:
Deger: 10.5
Deger: 20.7
Deger: 30.9
*/
```

§5.4.4 Gözlemci

Gözlemci kalıbı (*observer pattern*), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar.

Bu tasarım kalıbı, nesneler arasında bire-çok (*one-to-many*) bir bağımlılık kurmak için kullanılır. Bir nesnenin durumu değiştiğinde, ona bağlı olan tüm diğer nesnelerin (abonelerin) bu değişiklikten otomatik olarak haberdar edilmesini ve güncellenmesini sağlar. Bu kalıp, modern yazılımlardaki



Şekil 5.16: Gözlemci kalıbı UML Diyagramı

"olay güdümlü" (event-driven) sistemlerin ve **MVC** (*Model-View-Controller*) mimarisinin kalbidir.

- Bağlam (`Subject`), takip edilen asıl yayıncı nesnedir. Durumu değiştikçe etrafa haber salar. İçinde bir "Gözlemci Listesi" tutar. Yeni gözlemci eklemek (`attach()`), çıkarmak (`detach()`) ve hepsine haber vermek (`notify()`) için yöntemler barındırır.
- Gözlemci (`Observer`), güncelleme mesajlarını alabilmek için gerekli olan standart abone arayüzünü tanımlar. Genellikle sadece tek bir `update()` metodu içerir.
- Somut Yayıncı (`ConcreteSubject`), gerçek veriyi saklayan sınıftır (Örnek olarak bir borsa takip sistemi veya haber sitesi). Kendi içindeki veri değiştiğinde, miras aldığı `notify()` metodunu çağırarak tüm aboneleri tetikler.
- Somut Gözlemci (`ConcreteObserver`), haberi alan ve buna göre tepki veren sınıftır. `update()` yöntemi tetiklendiğinde, yayıncıdan gelen yeni veriyi alır ve kendi içinde işler (Ekrana yazar, veritabanına kaydeder vb.).

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>

//1. Gözlemci: Observer Arayüzü
class Observer {
public:
  virtual ~Observer() = default; // Kritik: Sanal yıkıcı
  virtual void update(int state) = 0; // Durumu doğrudan parametre olarak alabilir (
      Push model)
};

//2. Bağlam/Yayıncı: Taban Sınıfı
class Subject {
  private:
  // weak_ptr kullanımı: Gözlemci silinmişse Subject onu zorla hayatta tutmaz.
  std::vector<std::weak_ptr<Observer>> observers;
  public:
  virtual ~Subject() = default;
  void attach(std::shared_ptr<Observer> observer) {
    observers.push_back(observer);
  }
  void detach(std::shared_ptr<Observer> observer) {
    observers.erase(std::remove_if(observers.begin(), observers.end(),
    [&observer](const std::weak_ptr<Observer>& wp) {
      return wp.expired() || wp.lock() == observer;
    }), observers.end());
```

```
  }
  void notify(int state) {
    for (auto it = observers.begin(); it != observers.end(); ) {
      if (auto obj = it->lock()) { // Nesne hala hayattaysa
        obj->update(state);
        ++it;
      } else { // Nesne silinmişse listeden temizle
        it = observers.erase(it);
      }
    }
  }
};

//3. Somut Bağlam/Yayıncı (Concrete Subject)
class NewsAgency : public Subject {
private:
  int newsId = 0;
public:
  void setNews(int id) {
    newsId = id;
    std::cout << "Haber Ajansi: Yeni haber yayinlandi (ID: " << id << ")\n";
    notify(newsId); // Otomatik bildirim
  }
};

//4. Somut Gözlemci (Concrete Observer)
class NewsChannel : public Observer {
private:
  std::string name;
public:
  explicit NewsChannel(std::string n) : name(std::move(n)) {}
  void update(int state) override {
    std::cout << " -> " << name << " kanali bilgiyi aldi. Yeni haber ID: " << state <<
        std::endl;
  }
};

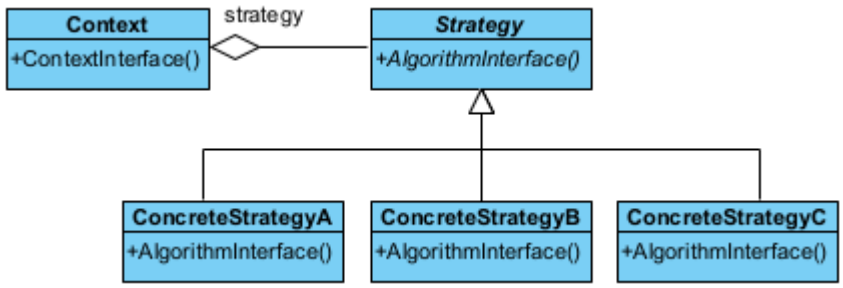
//5. Client (İstemci)
int main() { // İstemci (Client)
  // Nesneleri paylaşımlı akıllı işaretçiler ile oluşturuyoruz
  auto agency = std::make_shared<NewsAgency>();
  auto channel1 = std::make_shared<NewsChannel>("TRT Haber");
  auto channel2 = std::make_shared<NewsChannel>("NTV");
  auto channel3 = std::make_shared<NewsChannel>("CNN Turk");
  agency->attach(channel1);
  agency->attach(channel2);
  agency->attach(channel3);
  // Bir durumu değiştirelim, bildirimler otomatik gidecek
  agency->setNews(101);
  std::cout << "-------------------------------------" << std::endl;
  agency->detach(channel2); // NTV ayrılıyor
  agency->setNews(102);
  // Manuel 'delete' yok, bellek sızıntısı imkansız.
}
/*Program Çıktısı:
Haber Ajansi: Yeni haber yayinlandi (ID: 101)
-> TRT Haber kanali bilgiyi aldi. Yeni haber ID: 101
-> NTV kanali bilgiyi aldi. Yeni haber ID: 101
-> CNN Turk kanali bilgiyi aldi. Yeni haber ID: 101
-------------------------------------
Haber Ajansi: Yeni haber yayinlandi (ID: 102)
-> TRT Haber kanali bilgiyi aldi. Yeni haber ID: 102
-> CNN Turk kanali bilgiyi aldi. Yeni haber ID: 102
*/
```

Bu kalıp aşağıdaki durumlarda kullanılır;

- ◇ Bir soyutlama (*abstraction*) sonucu ortaya çıkan bağımsız iki farklı yönün (*aspect*) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- ◇ Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- ◇ Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.

§5.4.5 Strateji

Strateji kalıbı (*strategy pattern*), belirli bir davranışı gerçekleştirmek için değiştirilebilen kapsüllenmiş (*encapsulated*) algoritmalar kümesini tanımlar.



Şekil 5.17: Strateji kalıbı UML Diyagramı

Bu kalıbın görevi, bir işi yapmanın birden fazla yolu varsa, bu yolları (algoritmaları) sınıflara ayırır ve birbirinin yerine kullanılabilir hale getirir.

- ◇ Bağlam (`Context`), işi yapacak olan sınıf. Ancak işi "nasıl" yapacağını bir strateji nesnesine sorar.
- ◇ Strateji (`Strategy`): Tüm algoritmaların uyması gereken arayüzdür.
- ◇ Somut Strateji (`ConcreteStrategy`), farklı algoritmalar ile tanımlı stratejilerdir (Örnek olarak "Hızlı Sıralama", "Kabarcık Sıralama" veya "Kredi Kartıyla Öde", "Nakit Öde").

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlnmelidir
#include <iostream>
#include <memory>
#include <vector>

//1. Strateji Arayüzü
class Strategy {
public:
  virtual ~Strategy() = default; // Kritik: Sanal yıkıcı
  virtual void execute() const = 0;
};

//2. Somut Stratejiler
class ConcreteStrategyA : public Strategy {
public:
  void execute() const override {
    std::cout << "Algoritma A: Hizli ama daha fazla bellek kullaniyor.\n";
  }
};
class ConcreteStrategyB : public Strategy {
public:
  void execute() const override {
    std::cout << "Algoritma B: Yavas ama bellek dostu.\n";
  }
};
class ConcreteStrategyC : public Strategy {
public:
  void execute() const override {
    std::cout << "Algoritma C: Dengeli bir yaklasim.\n";
```

```
  }
};

//3. Baglam (Context)
class Context {
private:
  // Stratejinin mulkiyetini almaz, sadece referans/isaretci tutar (Aggregation)
  Strategy* strategy_ = nullptr;
public:
  explicit Context(Strategy* strategy) : strategy_(strategy) {}
  void setStrategy(Strategy* strategy) {
    strategy_ = strategy;
  }
  void run() const {
    if (strategy_) {
      strategy_->execute();
    } else {
      std::cout << "Hata: Henuz bir strateji belirlenmedi!\n";
    }
  }
};

//4. Client (İstemci)
int main() {
  // Stratejileri akilli isaretcilerle yonetiyoruz
  auto s1 = std::make_unique<ConcreteStrategyA>();
  auto s2 = std::make_unique<ConcreteStrategyB>();
  auto s3 = std::make_unique<ConcreteStrategyC>();
  // Baglam (Context) nesnesini olustur ve baslangic stratejisini ata
  Context context(s1.get());
  std::cout << "--- Durum 1 ---" << std::endl;
  context.run();
  // Calisma zamaninda stratejiyi degistir
  context.setStrategy(s2.get());
  std::cout << "--- Durum 2 ---" << std::endl;
  context.run();
  context.setStrategy(s3.get());
  std::cout << "--- Durum 3 ---" << std::endl;
  context.run();
  // Manuel 'delete' yok, s1-s2-s3 otomatik temizlenir.
}
/*Program Çıktısı:
--- Durum 1 ---
Algoritma A: Hizli ama daha fazla bellek kullaniyor.
--- Durum 2 ---
Algoritma B: Yavas ama bellek dostu.
--- Durum 3 ---
Algoritma C: Dengeli bir yaklasim.
*/
```

Bu kalıbın **bağlam** nesnesinin, diğer tasarım kalıplarının biri ile iç içe kullanılması halinde, bu kalıp **sıfır kalıp** (*null pattern*) olarak adlandırılır. Sıfır kalıpta, `Context` nesnesi akıllıdır ve her zaman ne yapacağını bilir. Bu kalıp aşağıdaki durumda kullanılır;

- ◇ Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- ◇ Birden çok algoritmanın olması durumunda,
- ◇ Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- ◇ Bir sınıf kendi yöntemlerinde çok fazla sayıda duruma göre seçim (*conditional choice*) kullanıyorsa.

Aşağıda bu kalıba ilişkin sıralama algoritmalarının seçildiği gerçek bir örnek verilmiştir;

```
//C++ 14 ve üzeri ile derlenmelidir.
#include <iostream>
```

```cpp
#include <vector>
#include <memory>
#include <algorithm> // std::sort için

// --- Abstract Strategy ---
class SortStrategy {
public:
  virtual ~SortStrategy() = default;
  virtual void sort(std::vector<int>& data) = 0;
};
// --- Concrete Strategy A: Quick Sort ---
class QuickSort : public SortStrategy {
public:
  void sort(std::vector<int>& data) override {
    std::cout << "QuickSort algoritmasi kullaniliyor..." << std::endl;
    // Gerçek QuickSort yerine std::sort kullanalım
    // (genelde introsort/quicksort türevidir)
    std::sort(data.begin(), data.end());
  }
};

// --- Concrete Strategy B: Merge Sort ---
class MergeSort : public SortStrategy {
public:
  void sort(std::vector<int>& data) override {
    std::cout << "MergeSort algoritmasi kullaniliyor..." << std::endl;
    // Örnek amaçlı standart stable_sort (merge sort benzeri garantiler sunar)
    std::stable_sort(data.begin(), data.end());
  }
};

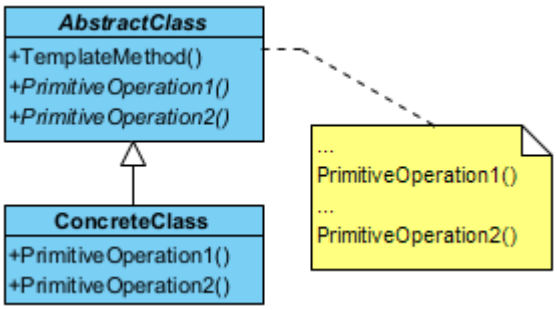
// --- Context: Sorter ---
class Sorter {
private:
  std::unique_ptr<SortStrategy> strategy_;
public:
  // Stratejiyi çalışma zamanında değiştirmemizi sağlar
  void setStrategy(std::unique_ptr<SortStrategy> strategy) {
    strategy_ = std::move(strategy);
  }
  void sort(std::vector<int>& data) {
    if (!strategy_) {
      std::cout << "Hata: Once bir strateji belirlenmelidir!" << std::endl;
      return;
    }
    strategy_->sort(data);
  }
};
// Yardımcı fonksiyon: Verileri ekrana basmak için
void printVector(const std::vector<int>& data) {
  for (int n : data)
    std::cout << n << " ";
  std::cout << std::endl;
}
int main() {
  std::vector<int> sayilar = {34, 12, 5, 78, 1, 45};
  Sorter siralayici;
  std::cout << "Orijinal liste: ";
  printVector(sayilar);
  // 1. QuickSort Stratejisini Seçelim
  siralayici.setStrategy(std::make_unique<QuickSort>());
  siralayici.sort(sayilar);
  printVector(sayilar);
  // Listeyi tekrar karıştıralım
  sayilar = {99, 2, 56, 11, 0, 7};
  std::cout << "\nYeni liste: ";
```

```
  printVector(sayilar);
  // 2. MergeSort Stratejisine Geçelim (Çalışma zamanında değişim)
  siralayici.setStrategy(std::make_unique<MergeSort>());
  siralayici.sort(sayilar);
  printVector(sayilar);
}
/* Programın Çıktısı:
Orijinal liste: 34 12 5 78 1 45
QuickSort algoritmasi kullaniliyor...
1 5 12 34 45 78

Yeni liste: 99 2 56 11 0 7
MergeSort algoritmasi kullaniliyor...
0 2 7 11 56 99
*/
```

§5.4.6 Şablon Yöntem

Şablon yöntem kalıbında (*template method pattern*), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır.



Şekil 5.18: Şablon Yöntem kalıbı UML Diyagramı

Bu kalıp, nesne yönelimli programlamada "algoritma iskeleti" oluşturmak için kullanılır. Bir işin adımlarını (sırasını) belirler ancak bu adımların bazılarının nasıl yapılacağını alt sınıflara bırakır. Kod tekrarını önlemek ve bir işlemin temel yapısını korumak için mükemmeldir.

- ◇ Soyut Sınıf (`AbstractClass`), algoritmanın ana iskeletini ve Şablon yöntemi (`TemplateMethod()`) barındırır. Değişmez adımları kendi içinde tanımlar, değişebilecek adımları ise soyut (*virtual*) olarak işaretler. Şablon yöntem genellikle `final` olarak tanımlanır ki algoritmanın sırası bozulmasın.
- ◇ Somut Sınıf (`ConcreteClass`); Soyut sınıftaki "boşlukları" doldurur. Algoritmanın değişken adımlarını kendi ihtiyacına göre özelleştirir. Algoritmanın genel akışına müdahale edemez, sadece kendisine izin verilen adımları yazar.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <memory> // std::unique_ptr için

//1. Soyut Sınıf (Abstract Class)
class AbstractClass {
public:
  // Kritik: Polimorfik sınıflarda sanal yıkıcı şarttır.
  virtual ~AbstractClass() = default;
  // Şablon Yöntem: Algoritmanın iskeletini sabitler.
  // Alt sınıflar bu sırayı değiştiremez.
  void templateMethod() {
    baseOperation();          // Ortak adım
    requiredOperation1();     // Özelleştirilebilir adım
    hook();                   // İsteğe bağlı adım (Hook)
```

```
    requiredOperation2();    // Özelleştirilebilir adım
  }
protected:
  // Temel operasyon: Tüm alt sınıflar için aynıdır.
  void baseOperation() const {
    std::cout << "Sistem: Algoritmanin ortak temel adimi calistiriliyor.\n";
  }
  // Alt sınıfların mutlaka uygulaması gereken adımlar (Saf sanal)
  virtual void requiredOperation1() = 0;
  virtual void requiredOperation2() = 0;
  // Hook (Kanca): Alt sınıflar isterse ezer, istemezse boş kalır.
  virtual void hook() {}
};

//2. Somut Sınıf (Concrete Class)
class ConcreteClass : public AbstractClass {
  protected:
  // override anahtar kelimesi okunabilirliği ve güvenliği artırır.
  void requiredOperation1() override {
    std::cout << "ConcreteClass: Adim 1 ozel olarak uygulandi.\n";
  }
  void requiredOperation2() override {
    std::cout << "ConcreteClass: Adim 2 ozel olarak uygulandi.\n";
  }
  // İsteğe bağlı: Hook'u ezebiliriz.
  void hook() override {
    std::cout << "ConcreteClass: Hook (Kanca) devreye girdi.\n";
  }
};

//3. Client (İstemci)
int main() {
  // Akıllı işaretçi kullanarak bellek yönetimini otomatiğe bağlıyoruz.
  std::unique_ptr<AbstractClass> algoritma = std::make_unique<ConcreteClass>();
  std::cout << "--- Algoritma Baslatiliyor ---\n";
  algoritma->templateMethod();
  // Kapsam dışına çıkınca bellek otomatik temizlenir.
}
/*Program Çıktısı:
--- Algoritma Baslatiliyor ---
Sistem: Algoritmanin ortak temel adimi calistiriliyor.
ConcreteClass: Adim 1 ozel olarak uygulandi.
ConcreteClass: Hook (Kanca) devreye girdi.
ConcreteClass: Adim 2 ozel olarak uygulandi.
*/
```

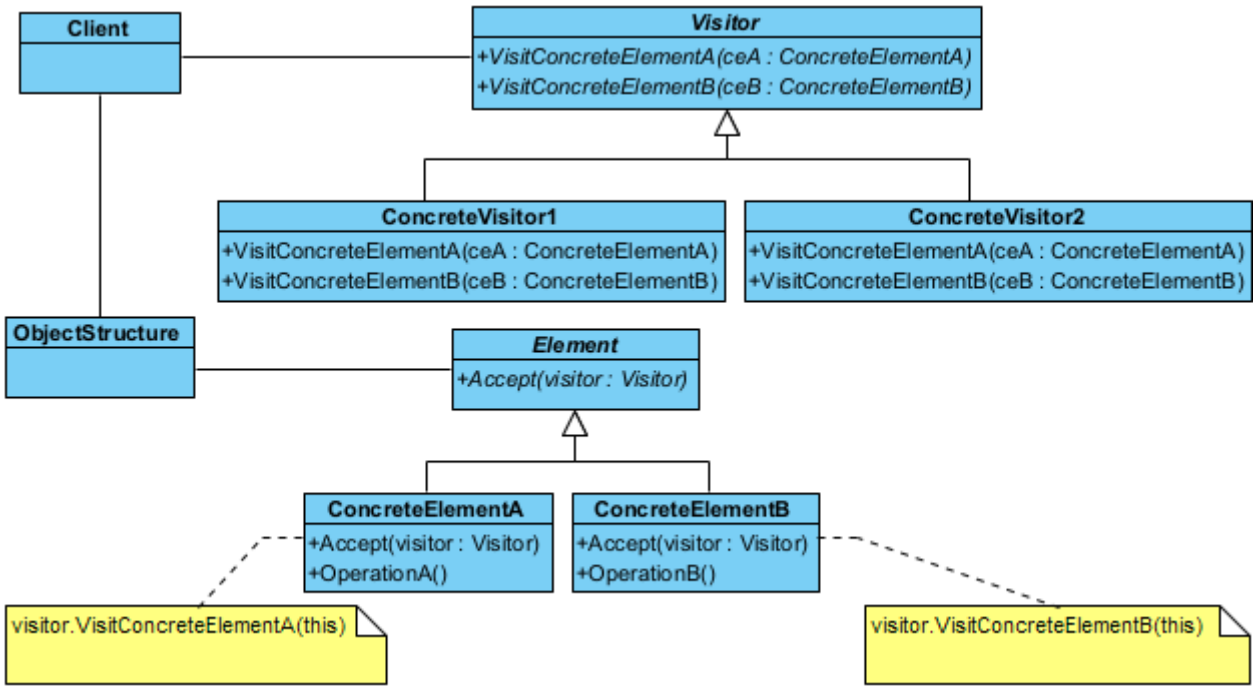
Bu kalıpta, kullanılacak algoritmalardan hangilerinin standart, hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

- ◇ İçinde "Bizi çağırmayın! Biz sizi çağıracağız." sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- ◇ Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanımlanır.
- ◇ Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- ◇ Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- ◇ Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.

§5.4.7 Ziyaretçi

Ziyaretçi kalıbı (*visitor pattern*), bir nesne grubunun (koleksiyonun) yapısını değiştirmeden, bu nesneler üzerinde uygulanacak yeni işlemleri dışarıdan eklememizi sağlayan davranışsal bir kalıptır. Bu kalıp, "Veri Yapısı" ile "Algoritma"yı birbirinden ayırır. Eğer elinizde sabit bir sınıf hiyerarşisi

varsa (örneğin bir dokümandaki paragraflar, resimler, tablolar) ve sürekli bu sınıflar üzerinde yeni işlemler (PDF'e dönüştür, HTML'e dönüştür, kelime say) tanımlamanız gerekiyorsa, her seferinde o sınıfların içine girip kod yazmak yerine ziyaretçi kalıbını kullanırsınız.



Şekil 5.19: Ziyaretçi kalıbı UML Diyagramı

- ◇ Ziyaretçi (`Visitor`), koleksiyondaki her bir somut nesne türü için bir ziyaret yöntemi (`visit()`) tanımlayan soyut bir sınıftır. "Eğer bir Paragraf nesnesine gidersem ne yapacağım?", "Eğer bir Resim nesnesine gidersem ne yapacağım?" sorularının imzasını belirler.
- ◇ Somut Ziyaretçi (`ConcreteVisitor`), gerçek operasyonu gerçekleştirir. Algoritmanın kendisidir. Örneğin ExportToPDFVisitor sınıfı, her nesne türünün PDF'e nasıl dönüştürüleceğini bilir.
- ◇ Eleman (`Element`), bir ziyaretçiyi kabul edebilmek için standart bir "kapı" açan soyut bir sınıftır. Ziyaretçi kabul etmek için `accept(Visitor* v)` yöntemini barındırır.
- ◇ Somut Elemanlar (`ConcreteElement`), veri yapısının asıl parçalarıdır. `accept()` yöntemi çağrıldığında, ziyaretçiye "Ben bir [NesneTürü]'yüm, beni ziyaret et" der (`v->visit(this)`). Buna yazılım dünyasında **çifte sevkiyat** (*double dispatch*) denir.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// 1. İleriye Dönük Bildirimler (Forward Declarations): Böyle sınıflar olacak!
class ConcreteElementA;
class ConcreteElementB;

// 2. Visitor (Ziyaretçi) Arayüzü
class Visitor {
public:
  virtual ~Visitor() = default; // Kritik: Sanal yıkıcı
  virtual void visit(ConcreteElementA* element) = 0;
  virtual void visit(ConcreteElementB* element) = 0;
};

// 3. Element (Eleman) Arayüzü
```

```
class Element {
public:
  virtual ~Element() = default;
  // "Double Dispatch" mekanizmasının kalbi: accept metodu
  virtual void accept(Visitor* visitor) = 0;
};

// 4. Somut Elemanlar
class ConcreteElementA : public Element {
public:
  void accept(Visitor* visitor) override {
    visitor->visit(this);
  }
  void operationA() const {
    std::cout << "ConcreteElementA: Ozel islem calistiriliyor.\n";
  }
};
class ConcreteElementB : public Element {
public:
  void accept(Visitor* visitor) override {
    visitor->visit(this);
  }
  void operationB() const {
    std::cout << "ConcreteElementB: Ozel islem calistiriliyor.\n";
  }
};

// 5. Somut Ziyaretçiler
class ConcreteVisitor1 : public Visitor {
public:
  void visit(ConcreteElementA* element) override {
    std::cout << "Visitor 1: ElementA ziyaret edildi.\n";
    element->operationA();
  }
  void visit(ConcreteElementB* element) override {
    std::cout << "Visitor 1: ElementB ziyaret edildi.\n";
    element->operationB();
  }
};
class ConcreteVisitor2 : public Visitor {
public:
  void visit(ConcreteElementA* element) override {
    std::cout << "Visitor 2: ElementA üzerinde farkli bir islem yapiliyor.\n";
  }
  void visit(ConcreteElementB* element) override {
    std::cout << "Visitor 2: ElementB üzerinde farkli bir islem yapiliyor.\n";
  }
};

// 6. Nesne Yapısı (Object Structure)
class ObjectStructure {
  private:
  std::vector<std::unique_ptr<Element>> elements;
  public:
  void add(std::unique_ptr<Element> e) {
    elements.push_back(std::move(e));
  }
  void accept(Visitor* v) {
    for (const auto& e : elements) {
      e->accept(v);
    }
  }
};

// 7. Client (İstemci)
int main() {
```

```
  ObjectStructure structure;
  // Elemanları ekliyoruz (Sahiplik structure nesnesine geçiyor)
  structure.add(std::make_unique<ConcreteElementA>());
  structure.add(std::make_unique<ConcreteElementB>());
  // Ziyaretçileri oluşturuyoruz
  auto v1 = std::make_unique<ConcreteVisitor1>();
  auto v2 = std::make_unique<ConcreteVisitor2>();
  std::cout << "--- Ziyaretci 1 Basliyor ---\n";
  structure.accept(v1.get());
  std::cout << "--- Ziyaretci 2 Basliyor ---\n";
  structure.accept(v2.get());
  // Her şey otomatik ve güvenli bir şekilde silinir.
}
/*Program Çıktısı:
--- Ziyaretci 1 Basliyor ---
Visitor 1: ElementA ziyaret edildi.
ConcreteElementA: Ozel islem calistiriliyor.
Visitor 1: ElementB ziyaret edildi.
ConcreteElementB: Ozel islem calistiriliyor.
--- Ziyaretci 2 Basliyor ---
Visitor 2: ElementA üzerinde farkli bir islem yapiliyor.
Visitor 2: ElementB üzerinde farkli bir islem yapiliyor.
*/
```

Bu kalıp aşağıdaki durumlarda kullanılır;

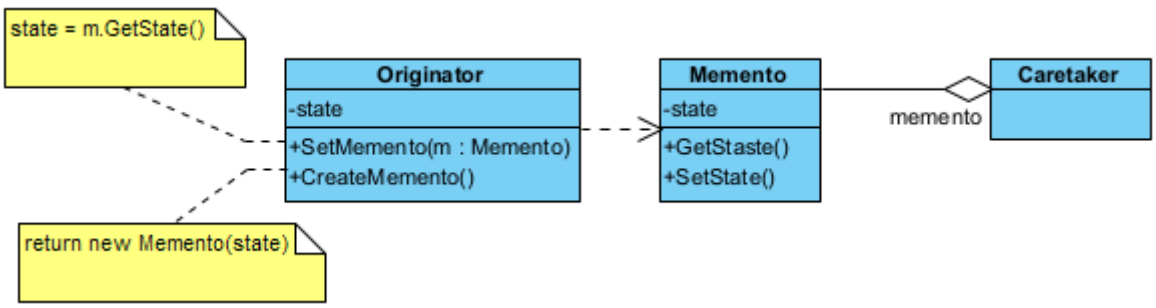
- ◇ Nesneler birden fazla olup birçok arayüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- ◇ Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- ◇ Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

Bu kalıbı kullanmanın aşağıda sıralanan üstünlükleri sağlar;

- ◇ Yeni işlemleri (*operation*) sisteme eklemeyi kolaylaştırır.
- ◇ Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- ◇ Yeni `ConcreteElement` nesnelerinin eklenmesini zorlaştırır.
- ◇ Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir. Sınıflardaki tüm durumlar (*state*) biriktirilebilir.
- ◇ Bazı durumlarda kapsülleme (*encapsulation*) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

§5.4.8 Hatıra

Hatıra kalıbı (*memento pattern*), nesnelerin bir andaki durumlarını (*state*) saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarıda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **kılıflamayı** (*encapsulation*) ihlal etmeden yapılır.



Şekil 5.20: Hatıra kalıbı UML Diyagramı

Hatıra kalıbının amacı, nesnenin iç durumunu (*state*) bozmadan saklamak ve daha sonra geri yüklemektir.

- ◇ Yaratıcı (`Originator`), durumu değişen asıl nesnedir. Kendi "anlık görüntüsünü" (*snapshot*) oluşturur ve geri yükler.
- ◇ Hatıra (`Memento`), sadece veriyi tutan pasif nesnedir. Dışarıdan değiştirilemez; sadece `Originator` içeriğini okuyabilir.
- ◇ Bakıcı (`Caretaker`), hatıraları güvenli bir yerde (liste veya yığın) saklar. İçeriği asla değiştirmez, sadece saklar ve geri verir.

Şimdi kalıbımizi kodlayalım;

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

//1. Memento: Durumu saklayan pasif nesne
class Memento {
private:
  // Sadece Originator bu veriye erişmeli (Encapsulation)
  friend class Originator;
  int state;
  explicit Memento(int pState) : state(pState) {}
  int getState() const { return state; }
};

// 2. Originator: Durumu değişen ve "hatıra" üreten asıl nesne
class Originator {
private:
  int state;
public:
  void setState(int pState) {
    state = pState;
    std::cout << "Originator: Durum degistirildi -> " << state << std::endl;
  }
  // Mevcut durumu bir Memento nesnesine paketler
  std::unique_ptr<Memento> save() {
    std::cout << "Originator: Durum yedekleniyor... (" << state << ")" << std::endl;
    return std::unique_ptr<Memento>(new Memento(state));
  }
  // Bir Memento nesnesinden durumu geri yukler
  void restore(const Memento* memento) {
    if (memento) {
      state = memento->getState();
      std::cout << "Originator: Durum geri yuklendi -> " << state << std::endl;
    }
  }
};

// 3. Caretaker: Hatıraları saklayan bekçi
class Caretaker {
private:
  std::vector<std::unique_ptr<Memento>> history;
  Originator& originator;
public:
  explicit Caretaker(Originator& o) : originator(o) {}
  void backup() {
    history.push_back(originator.save());
  }
  void undo() {
    if (history.empty()) return;
    auto memento = std::move(history.back());
    history.pop_back();
    originator.restore(memento.get());
```

```
  }
};

// 4. İstemci (Client)
int main() {
  Originator player;
  Caretaker saveManager(player);
  player.setState(100); // Bölüm 1
  saveManager.backup(); // Kayıt noktası
  player.setState(50);  // Can azaldı
  player.setState(0);   // Öldü!
  std::cout << "--- Geri Yukleniyor ---" << std::endl;
  saveManager.undo();   // Bölüm 1'e geri dön
}
/*Program Çıktısı:
Originator: Durum degistirildi -> 100
Originator: Durum yedekleniyor... (100)
Originator: Durum degistirildi -> 50
Originator: Durum degistirildi -> 0
--- Geri Yukleniyor ---
Originator: Durum geri yuklendi -> 100
*/
```

Aşağıdaki örnekte bir metin editörünün geçmiş özelliği modellenmiştir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// --- Memento ---
// Sadece saklanacak veriyi tutar.
class EditorMemento {
private:
  std::string content;
public:
  EditorMemento(std::string text) : content(text) {}
  std::string getSavedContent() const { return content; }
};

// --- Originator ---
// Durumu kaydedilecek olan asıl nesne.
class TextEditor {
private:
  std::string text;
public:
  void type(std::string newText) { text += newText; }
  std::string getText() const { return text; }
  // Mevcut durumu bir Memento içine paketler.
  std::unique_ptr<EditorMemento> save() {
    return std::make_unique<EditorMemento>(text);
  }
  // Bir Memento'dan durumu geri yükler.
  void restore(const EditorMemento& memento) {
    text = memento.getSavedContent();
  }
};

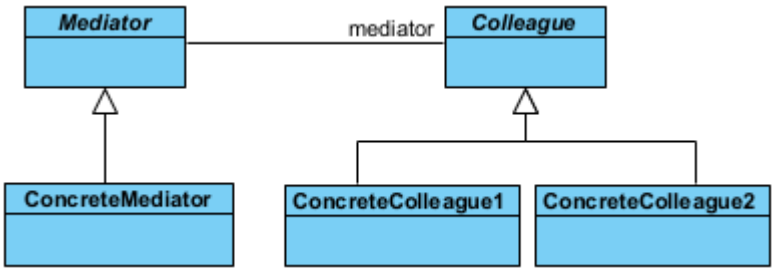
// --- Caretaker ---
// Geçmişi (Memento listesini) yönetir.
class History {
private:
  std::vector<std::unique_ptr<EditorMemento>> mementos;
public:
  void push(std::unique_ptr<EditorMemento> m) {
```

```
    mementos.push_back(std::move(m));
  }
  std::unique_ptr<EditorMemento> pop() {
    if (mementos.empty()) return nullptr;
    auto m = std::move(mementos.back());
    mementos.pop_back();
    return m;
  }
};

int main() {
  TextEditor editor;
  History history;
  // 1. Yazmaya başla ve ilk durumu kaydet
  editor.type("Merhaba ");
  history.push(editor.save());
  std::cout << "Guncel Metin: " << editor.getText() << std::endl;
  // 2. Yazmaya devam et ve ikinci durumu kaydet
  editor.type("Dunya!");
  history.push(editor.save());
  std::cout << "Guncel Metin: " << editor.getText() << std::endl;
  // 3. Bir hata yapalım
  editor.type(" Yanlis bir ekleme.");
  std::cout << "Hata Sonrasi: " << editor.getText() << std::endl;
  // 4. Geri al (Undo)
  auto undo = history.pop(); // Son kaydedilen "Merhaba Dunya!" idi, onu alalım.
  auto redo = history.pop(); // Bir önceki "Merhaba " idi.
  if (redo) {
    editor.restore(*redo);
    std::cout << "Geri Alindi: " << editor.getText() << std::endl;
  }
}
/*Program Çıktısı:
Guncel Metin: Merhaba
Guncel Metin: Merhaba Dunya!
Hata Sonrasi: Merhaba Dunya! Yanlis bir ekleme.
Geri Alindi: Merhaba
*/
```

§5.4.9 Ara Bulucu

Ara Bulucu kalıbı (*mediator pattern*), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu tasarım



Şekil 5.21: Ara Bulucu kalıbı UML Diyagramı

kalıbı, bir dizi nesne arasındaki karmaşık iletişim ağını basitleştirmek için kullanılır. Nesnelerin birbirleriyle doğrudan konuşmasını engeller ve tüm iletişimi tek bir "merkezi aracı" üzerinden yürütmelerini sağlar. Bu kalıp, özellikle nesne sayısı arttığında ortaya çıkan "örümcek ağı" gibi karmaşık bağlantıları (*tight coupling*) çözmek için birebirdir.

- ◇ Soyut Aracı (`Mediator`), nesneler arasındaki iletişim protokolünü (arayüzünü) tanımlar. "Hangi olay gerçekleştiğinde kimlere haber verilecek?" sorusunun genel çerçevesini çizer.

- ◇ Somut Aracı (`ConcreteMediator`), iletişim trafiğini yöneten asıl "santral" binasıdır. tüm meslektaşları (`Colleague`) tanır ve aralarındaki koordinasyonu sağlar. Bir bileşenden mesaj geldiğinde, mantık çerçevesinde diğerlerine iletir.
- ◇ Meslektaş (`Colleague`) Sınıflar, kendi işlevini yerine getiren bağımsız sınıflardır. Diğer sınıfların varlığından haberdar değildirler. Sadece "Aracı"ya (`Mediator`) haber verirler. "Ben düğmeye bastım, gerisini aracı halletsin" mantığıyla çalışırlar.

Şimdi kalıbımizi kodlayalım;

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>

// 1. İleriye Dönük Bildirim: İleride böyle bir sınıf olacak!
class Colleague;

// 2. Mediator (Ara bulucu) Arayüzü
class Mediator {
public:
  virtual ~Mediator() = default;
  virtual void registerColleague(std::shared_ptr<Colleague> colleague) = 0;
  virtual void relayMessage(const std::string& message, Colleague* sender) = 0;
};

// 3. Colleague (Meslektaş) Taban Sınıfı
class Colleague {
protected:
  std::string name;
  Mediator* mediator; // Döngüsel bağımlılığı önlemek için ham işaretçi veya weak_ptr
public:
  Colleague(std::string pName, Mediator* pMediator) : name(std::move(pName)), mediator(
      pMediator) {}
  virtual ~Colleague() = default;
  std::string getName() const { return name; }
  void send(const std::string& message) {
    mediator->relayMessage(message, this);
  }
  void receive(const std::string& message, const std::string& senderName) {
    std::cout << "[" << name << "] " << senderName
    << " kisisinden mesaj aldi: " << message << std::endl;
  }
};

// 4. Somut Ara bulucu (Concrete Mediator)
class ChatRoom : public Mediator {
private:
  std::vector<std::shared_ptr<Colleague>> colleagues;
public:
  void registerColleague(std::shared_ptr<Colleague> colleague) override {
    colleagues.push_back(colleague);
  }
  void relayMessage(const std::string& message, Colleague* sender) override {
    std::cout << "--- LOG: " << sender->getName() << " bir mesaj yayinladi ---" << std
        ::endl;

    for (const auto& colleague : colleagues) {
      // Mesajı gönderen kişiye geri göndermiyoruz
      if (colleague.get() != sender) {
        colleague->receive(message, sender->getName());
      }
    }
  }
};
```

```
// 5. Somut Meslektaş (Concrete Colleague)
class User : public Colleague {
public:
  using Colleague::Colleague; // Yapıcıyı miras al
};

// 6. İstemci (Client)
int main() {
  // Arabulucu nesnesi
  auto oda = std::make_shared<ChatRoom>();
  // Meslektaşlar (Kullanıcılar)
  auto ilhan = std::make_shared<User>("Ilhan", oda.get());
  auto mehmet = std::make_shared<User>("Mehmet", oda.get());
  auto ayse = std::make_shared<User>("Ayse", oda.get());
  // Kayıt işlemleri
  oda->registerColleague(ilhan);
  oda->registerColleague(mehmet);
  oda->registerColleague(ayse);
  // İletişim başlıyor
  ilhan->send("Selam millet!");
  mehmet->send("Merhabalar Ilhan.");
  ayse->send("Nasılsınız?");
  // Her şey otomatik temizlenir.
}
/*Program Çıktısı:
--- LOG: Ilhan bir mesaj yayinladi ---
[Mehmet] Ilhan kisisinden mesaj aldi: Selam millet!
[Ayse] Ilhan kisisinden mesaj aldi: Selam millet!
--- LOG: Mehmet bir mesaj yayinladi ---
[Ilhan] Mehmet kisisinden mesaj aldi: Merhabalar Ilhan.
[Ayse] Mehmet kisisinden mesaj aldi: Merhabalar Ilhan.
--- LOG: Ayse bir mesaj yayinladi ---
[Ilhan] Ayse kisisinden mesaj aldi: Nasılsınız?
[Mehmet] Ayse kisisinden mesaj aldi: Nasılsınız?
*/
```

Bu kalıpta, sistemde yalnızca bir adet ara bulucu olacaksa soyut `Mediator` nesnesi tanımlanmayabilir. Ayrıca ara bulucu ile meslektaşlar arasındaki iletişim iki türlü yapılır;

- ◇ Birisinde bu iletişim gözlemci kalıbı (*observer pattern*) ile sağlanır. `Colleague` sınıfı gözlemci kalıbındaki bağlam `Subject` sınıfı gibi davranır.
- ◇ Diğerinde ise `Mediator` nesnesine, görüşecekleri meslektaşları belirterek doğrudan ileti gönderirler.

Aşağıda havaalanındaki kule ile uçaklar arasındaki iletişimi modelleyen bir örnek verilmiştir;

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// Forward declaration
class Airplane;

// --- Mediator Arayüzü ---
class IAirTrafficControl {
public:
  virtual ~IAirTrafficControl() = default;
  virtual void sendMessage(const std::string& message, Airplane* originator) = 0;
  virtual void registerAirplane(Airplane* airplane) = 0;
};

// --- Colleague (Meslektaş/Bileşen) ---
class Airplane {
protected:
  IAirTrafficControl* mediator_;
```

```
  std::string callSign_;
public:
  Airplane(IAirTrafficControl* mediator, std::string callSign) : mediator_(mediator),
      callSign_(callSign) {}
  virtual ~Airplane() = default;
  virtual void send(const std::string& message) {
    std::cout << "[" << callSign_ << "] Kuleye mesaj gonderiyor: " << message << std::
        endl;
    mediator_->sendMessage(message, this);
  }
  virtual void receive(const std::string& message) {
    std::cout << "[" << callSign_ << "] Mesaj aldi: " << message << std::endl;
  }
  std::string getCallSign() const { return callSign_; }
};

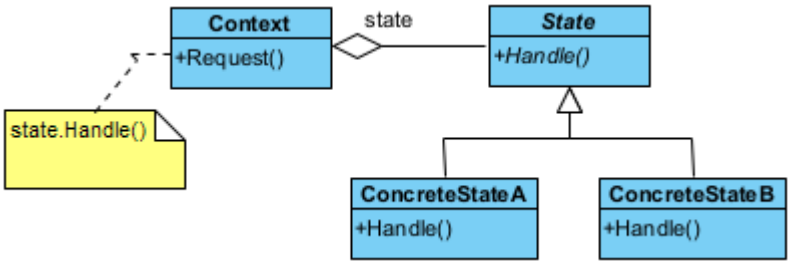
// --- Concrete Mediator (Somut Aracı) ---
class AtcTower : public IAirTrafficControl {
private:
  std::vector<Airplane*> airplanes_;
public:
  void registerAirplane(Airplane* airplane) override {
    airplanes_.push_back(airplane);
  }
  void sendMessage(const std::string& message, Airplane* originator) override {
    for (auto airplane : airplanes_) {
      // Mesajı gönderen uçak hariç herkese ilet (veya kurala göre işle)
      if (airplane != originator) {
        airplane->receive("Kule'den " + originator->getCallSign() + " bilgisi: " +
            message);
      }
    }
  }
};

int main() {
  // 1. Aracı (Kule) oluşturulur
  auto tower = std::make_unique<AtcTower>();
  // 2. Meslektaşlar (Uçaklar) oluşturulur ve kuleye bildirilir
  auto boeing = std::make_unique<Airplane>(tower.get(), "THY-123");
  auto airbus = std::make_unique<Airplane>(tower.get(), "PGS-456");
  auto cessna = std::make_unique<Airplane>(tower.get(), "OZEL-789");
  tower->registerAirplane(boeing.get());
  tower->registerAirplane(airbus.get());
  tower->registerAirplane(cessna.get());
  // 3. İletişim merkezi üzerinden yürütülür
  boeing->send("Pist 1'e inis izni istiyorum.");
  std::cout << "----------------------------------------" << std::endl;
  cessna->send("Pistten cikis yapildi.");
}
/* Program Çıktısı:
[THY-123] Kuleye mesaj gonderiyor: Pist 1'e inis izni istiyorum.
[PGS-456] Mesaj aldi: Kule'den THY-123 bilgisi: Pist 1'e inis izni istiyorum.
[OZEL-789] Mesaj aldi: Kule'den THY-123 bilgisi: Pist 1'e inis izni istiyorum.
----------------------------------------
[OZEL-789] Kuleye mesaj gonderiyor: Pistten cikis yapildi.
[THY-123] Mesaj aldi: Kule'den OZEL-789 bilgisi: Pistten cikis yapildi.
[PGS-456] Mesaj aldi: Kule'den OZEL-789 bilgisi: Pistten cikis yapildi.
*/
```

§5.4.10 Durum

Durum kalıbı (*state pattern*), nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.

Bu kalıbın amacı, nesnenin iç durumu değiştiğinde davranışını da değiştirmek. Nesne sanki



Şekil 5.22: Durum kalıbı UML Diyagramı

sınıfını değiştiriyormuş gibi görünür.

- ◇ Bağlam (`Context`), istemcinin (`Client`) etkileşime girdiği ana sınıftır. Mevcut durum nesnesini bir referans olarak tutar.
- ◇ Soyut Durum (`State`), tüm somut durumların uyması gereken arayüzü (Interface) tanımlar.
- ◇ Somut Durumlar (`ConcreteState`), Her sınıf, belirli bir durumdaki davranışı temsil eder (Örn: Oynatılıyor, Duraklatıldı). Bir durumdan diğerine geçiş mantığını genellikle bu sınıflar yönetir.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory>

// 1. State Arayüzü (Soyut Durum)
class Context; // Forward declaration
class State {
public:
  virtual ~State() = default;
  virtual void handle(Context* context) = 0;
};

// 2. Context (Bağlam)
class Context {
private:
  std::unique_ptr<State> currentState;
public:
  explicit Context(std::unique_ptr<State> state) : currentState(std::move(state)) {}
  void transitionTo(std::unique_ptr<State> state) {
    std::cout << "Context: Durum degistiriliyor...\n";
    currentState = std::move(state);
  }
  void request() {
    currentState->handle(this);
  }
};

// 3. Somut Durumlar
class ConcreteStateB : public State {
public:
  void handle(Context* context) override; // İleride tanımlanacak
};
class ConcreteStateA : public State {
public:
  void handle(Context* context) override {
    std::cout << "ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.\n";
    context->transitionTo(std::make_unique<ConcreteStateB>());
  }
};
void ConcreteStateB::handle(Context* context) {
  std::cout << "ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.\n";
```

```
  context->transitionTo(std::make_unique<ConcreteStateA>());
}

// 4. İstemci (Client)
int main() {
  // Başlangıç durumu A ile Context oluşturuluyor
  auto context = std::make_unique<Context>(std::make_unique<ConcreteStateA>());
  // İstekler gönderildikçe durumlar kendi içlerinde birbirini tetikler
  context->request(); // A -> B geçişi
  context->request(); // B -> A geçişi
  context->request(); // A -> B geçişi
  // Manuel delete gerekmez.
}
/*Program Çıktısı:
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.
Context: Durum degistiriliyor...
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
*/
```

Bu kalıbın bağlam (`Context`) nesnesinin, diğer tasarım kalıpları ile iç içe kullanılması halinde, bu kalıp, **sıfır kalıp** (*null pattern*) olarak adlandırılır. Ayrıca bu kalıpla ilgili olarak iki önemli şeyden bahsetmek gerekir:

- ◇ Sahip olunan duruma göre icra edilecek davranış, tanımlanan `State` sınıfına bağlıdır. Yani ilgili davranış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut state (`ConcreteState`) sınıfları içinde tanımlanan yöntemler, durumu kullanan bağlam (`Context`) sınıf içinde tanımlanmak zorundadır.
- ◇ Durumu kullanan bağlam (`Context`) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan özellik (*property*) içinde `set` yöntemi kullanılmak zorundadır.

Bu kalıp netice olarak, durum geçişlerini (*state transition*) açığa çıkarır. Bu kalıbı kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- ◇ **Durum geçişleri kim tarafından tanımlanmalıdır?**: Bu kalıp, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem bağlam (`Context`) nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem `State` nesnesi ve halefi (*successor*) tarafından yapılır.
- ◇ **Tablo tabanlı alternatif**: Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri tekdüze (*uniform*) tanımlanmak zorundadır.
- ◇ **`State` nesnelerini yaratma ve yok etme**: Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, State nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- ◇ **Dinamik kalıtım (*dynamic inheritance*) kullanma**: Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılamaz. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Bu kalıp için en gerçekçi örneklerden biri **otomat makineleridir** (*vending machine*). Makinenin davranışı; içinde para olup olmamasına, ürünün kalıp kalmamasına veya ürün seçilip seçilmediğine göre tamamen değişir. Eğer para yoksa "Ürün Seç" düğmesi çalışmaz; para varsa çalışır;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory>
```

```cpp
#include <string>

// --- Forward Declarations ---
class VendingMachine;

// --- Abstract State ---
class State {
public:
  virtual ~State() = default;
  virtual void insertMoney() = 0;
  virtual void ejectMoney() = 0;
  virtual void selectProduct() = 0;
  virtual void dispense() = 0;
};

// --- Concrete States ---
// 1. Para Yok Durumu
class NoMoneyState : public State {
  VendingMachine& machine;
public:
  NoMoneyState(VendingMachine& m) : machine(m) {}
  void insertMoney() override;
  void ejectMoney() override { std::cout << "Hata: Zaten para yok.\n"; }
  void selectProduct() override { std::cout << "Hata: Once para yuklemelisiniz.\n"; }
  void dispense() override { std::cout << "Hata: Odeme yapilmadan urun verilmez.\n"; }
};

// 2. Para Var Durumu
class HasMoneyState : public State {
  VendingMachine& machine;
public:
  HasMoneyState(VendingMachine& m) : machine(m) {}
  void insertMoney() override { std::cout << "Bilgi: Zaten para yuklu.\n"; }
  void ejectMoney() override;
  void selectProduct() override;
  void dispense() override { std::cout << "Bilgi: Once urun secimi yapmalisiniz.\n"; }
};

// 3. Ürün Satıldı/Teslimat Durumu
class SoldState : public State {
  VendingMachine& machine;
public:
  SoldState(VendingMachine& m) : machine(m) {}
  void insertMoney() override { std::cout << "Lutfen bekleyin: Urununuz hazirlaniyor.\n
      "; }
  void ejectMoney() override { std::cout << "Hata: Islem geri alinamaz, urun veriliyor
      .\n"; }
  void selectProduct() override { std::cout << "Hata: Islem zaten devam ediyor.\n"; }
  void dispense() override;
};

// 4. Stok Yok Durumu
class SoldOutState : public State {
public:
  void insertMoney() override { std::cout << "Hata: Stok bitti, para yukleyemezsiniz.\n
      "; }
  void ejectMoney() override { std::cout << "Hata: Para yok.\n"; }
  void selectProduct() override { std::cout << "Hata: Stok bitti.\n"; }
  void dispense() override { std::cout << "Hata: Stok yok.\n"; }
};

// --- Context: VendingMachine ---
class VendingMachine {
private:
  std::unique_ptr<State> currentState;
  int count = 0;
```

```
public:
  VendingMachine(int productCount) : count(productCount) {
    if (count > 0)
    currentState = std::make_unique<NoMoneyState>(*this);
    else
    currentState = std::make_unique<SoldOutState>();
  }
  void setState(std::unique_ptr<State> state) {
    currentState = std::move(state);
  }
  // Proxy metodlar
  void insertMoney() { currentState->insertMoney(); }
  void ejectMoney() { currentState->ejectMoney(); }
  void selectProduct() { currentState->selectProduct(); }
  void dispense() { currentState->dispense(); }
  void releaseProduct() {
    if (count > 0) {
      std::cout << ">>> URUN VERILDI. Afiyet olsun!\n";
      count--;
    }
  }
  int getCount() const { return count; }
};

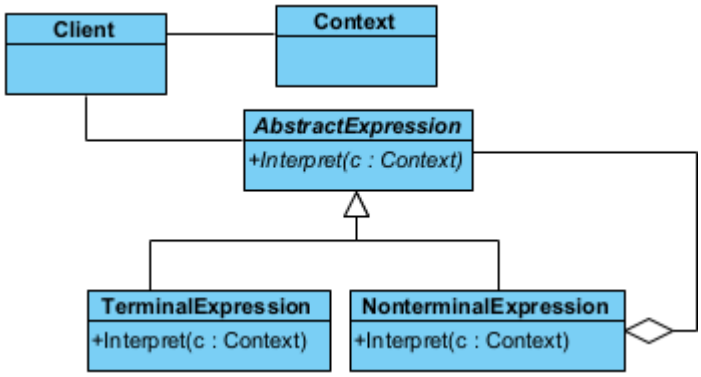
// --- Yöntem Tanımlamaları (Durum Geçiş Mantığı) ---
void NoMoneyState::insertMoney() {
  std::cout << "Para kabul edildi.\n";
  machine.setState(std::make_unique<HasMoneyState>(machine));
}
void HasMoneyState::ejectMoney() {
  std::cout << "Para iade ediliyor...\n";
  machine.setState(std::make_unique<NoMoneyState>(machine));
}
void HasMoneyState::selectProduct() {
  std::cout << "Urun secildi, hazirlaniyor...\n";
  machine.setState(std::make_unique<SoldState>(machine));
  machine.dispense(); // Otomatik teslimat tetiklemesi
}
void SoldState::dispense() {
  machine.releaseProduct();
  if (machine.getCount() > 0) {
    machine.setState(std::make_unique<NoMoneyState>(machine));
  } else {
    std::cout << "Dikkat: Son urun satildi!\n";
    machine.setState(std::make_unique<SoldOutState>());
  }
}

int main() {
  // 2 ürünlük bir otomat oluşturalım
  VendingMachine machine(2);
  std::cout << "--- Senaryo 1: Basarili Alim ---\n";
  machine.insertMoney();
  machine.selectProduct();
  std::cout << "--- Senaryo 2: Para Iadesi ---\n";
  machine.insertMoney();
  machine.ejectMoney();
  std::cout << "--- Senaryo 3: Son Urunu Alma ---\n";
  machine.insertMoney();
  machine.selectProduct();
  std::cout << "--- Senaryo 4: Stok Bittiginde Deneme ---\n";
  machine.insertMoney();
}
/* Programın Çıktısı:
--- Senaryo 1: Basarili Alim ---
Para kabul edildi.
```

```
Urun secildi, hazirlaniyor...
*/
```

§5.4.11 Yorumlayıcı

Yorumlayıcı kalıbı (*interpreter pattern*), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dil bilgisi için bir temsili ve aynı zamanda dil bilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.



Şekil 5.23: Yorumlayıcı kalıbı UML Diyagramı

Bu kalıbın amacı, belirli bir gramer yapısına sahip ifadeleri (matematiksel işlemler, SQL komutları vb.) çözümlemektir.

- ◇ Soyut İfade (`AbstractExpression`), tüm düğümlerin sahip olduğu `interpret()` yöntemini tanımlayan arayüzdür.
- ◇ Uç İfade (`TerminalExpression`), gramerin en küçük birimleridir (Sayılar veya değişkenler). Daha fazla parçalanamazlar.
- ◇ Uç Olmayan İfade (`NonTerminalExpression`), gramerin kurallarını (Toplama, Çarpma vb.) temsil eder. Genellikle içinde diğer ifadeleri (uç yada veya uç olmayan) barındırır.
- ◇ Bağlam (`Context`), yorumlama sırasında ihtiyaç duyulan küresel bilgileri (Değişken değerleri, sembol tabloları) taşır.

Şimdi kalıbımizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <map>

// 1. Context (Bağlam): Yorumlama sırasında kullanılan küresel bilgiler (Değişken değ
    erleri vb.)
class Context {
private:
  std::map<std::string, int> variables;
public:
  void setVariable(const std::string& name, int value) { variables[name] = value; }
  int getVariable(const std::string& name) const { return variables.at(name); }
};

// 2. AbstractExpression (Soyut İfade)
class AbstractExpression {
public:
  virtual ~AbstractExpression() = default;
  virtual int interpret(const Context& context) const = 0;
};

// 3. TerminalExpression (Uç İfade): Sayılar veya değişkenler gibi atomik birimler
```

```
class NumberExpression : public AbstractExpression {
private:
  int number;
public:
  explicit NumberExpression(int n) : number(n) {}
  int interpret(const Context&) const override { return number; }
};

class VariableExpression : public AbstractExpression {
private:
  std::string name;
public:
  explicit VariableExpression(std::string n) : name(std::move(n)) {}
  int interpret(const Context& context) const override { return context.getVariable(
      name); }
};

// 4. NonterminalExpression (Uç Olmayan İfade): Operatörler (+, -, *) gibi karmaşık yap
    ılar
class AddExpression : public AbstractExpression {
private:
  std::unique_ptr<AbstractExpression> left;
  std::unique_ptr<AbstractExpression> right;
public:
  AddExpression(std::unique_ptr<AbstractExpression> l, std::unique_ptr<
      AbstractExpression> r) : left(std::move(l)), right(std::move(r)) {}
  int interpret(const Context& context) const override {
    return left->interpret(context) + right->interpret(context);
  }
};

// 5. İstemci (Client)
int main() {
  Context context;
  context.setVariable("x", 10);
  context.setVariable("y", 5);
  // İfade: (x + y) + 20
  // Ağaç yapısı oluşturuluyor
  auto expression = std::make_unique<AddExpression>(
  std::make_unique<AddExpression>( std::make_unique<VariableExpression>("x"), std::
      make_unique<VariableExpression>("y") ), std::make_unique<NumberExpression>(20) );
  std::cout << "Sonuc: " << expression->interpret(context) << std::endl;
  // unique_ptr sayesinde tüm ağaç otomatik temizlenir
}
/*Program Çıktısı:
Sonuc: 35
*/
```

Bu kalıbı uygularken aşağıda verilen durumlar irdelenmelidir;

- ◇ **Soyut söz dizimin oluşturulması**: Bu Kalıp yorumlanacak dile ilişkin soyut bir söz dizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- ◇ **Yorumlayacak işlemin tanımlanması**: Eğer işlemler ortak bir yapıdaysa yeni bir `Expression` nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi kalıbı kullanılır. Programlama dillerinin çoğu soyut bir söz dizimine sahiptir ve yorumlanacak öğeler ayrı bir Visitor nesnesi tarafından yorumlanır.
- ◇ **Cümle sonlarını bitiren ortak semboller için sineksiklet kalıbı kullanılabilir**: Cümle sonlarındaki semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle sineksiklet kalıbı (*flyweight pattern*) yorumlanacak yerler için bir eleme yapar.

Aşağıda bir metindeki ifadeyi hesaplayan bir uygulama örneği verilmiştir;

```
//C++14 ve üzeri ile derlenmelidir
```

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <sstream>
#include <stack>

// --- Önceki Örnekteki Sınıflar (Aynen Kalıyor) ---
class Context {};
class Expression {
public:
  virtual ~Expression() = default;
  virtual int interpret(Context& context) const = 0;
};
class NumberExpression : public Expression {
  int number;
public:
  explicit NumberExpression(int n) : number(n) {}
  int interpret(Context&) const override { return number; }
};
class AdditionExpression : public Expression {
  std::unique_ptr<Expression> left, right;
public:
  AdditionExpression(std::unique_ptr<Expression> l, std::unique_ptr<Expression> r)
  : left(std::move(l)), right(std::move(r)) {}
  int interpret(Context& c) const override { return left->interpret(c) + right->
      interpret(c); }
};
class MultiplicationExpression : public Expression {
  std::unique_ptr<Expression> left, right;
public:
  MultiplicationExpression(std::unique_ptr<Expression> l, std::unique_ptr<Expression> r
      )
  : left(std::move(l)), right(std::move(r)) {}
  int interpret(Context& c) const override { return left->interpret(c) * right->
      interpret(c); }
};

// --- Dinamik Interpreter Sınıfı ---
class Interpreter {
public:
  int interpret(const std::string& rawExpression) {
    Context context;
    auto expressionTree = buildExpressionTree(rawExpression);
    return expressionTree->interpret(context);
  }
private:
  // String ifadeyi parçalara (tokens) ayıran yardımcı metod
  std::vector<std::string> tokenize(const std::string& str) {
    std::vector<std::string> tokens;
    std::stringstream ss(str);
    std::string temp;
    while (ss >> temp) tokens.push_back(temp);
    return tokens;
  }
  // Gerçek Dinamik Ağaç Oluşturucu (Shunting-yard benzeri basitleştirilmiş mantık)
  std::unique_ptr<Expression> buildExpressionTree(const std::string& expression) {
    auto tokens = tokenize(expression);
    // İşlem önceliği için iki yığın (stack)
    std::stack<std::unique_ptr<Expression>> nodes;
    std::stack<char> operators;
    auto applyOp = [&]() {
      auto right = std::move(nodes.top()); nodes.pop();
      auto left = std::move(nodes.top()); nodes.pop();
      char op = operators.top(); operators.pop();
      if (op == '+')
```

```
        nodes.push(std::make_unique<AdditionExpression>(std::move(left), std::move(
            right)));
      else if (op == '*')
        nodes.push(std::make_unique<MultiplicationExpression>(std::move(left), std::
            move(right)));
    };
    for (const std::string& token : tokens) {
      if (isdigit(token[0])) {
        nodes.push(std::make_unique<NumberExpression>(std::stoi(token)));
      } else {
        char currentOp = token[0];
        // İşlem önceliği kontrolü: Çarpma (+) dan önce yapılır
        while (!operators.empty() &&
        ((currentOp == '+' && (operators.top() == '+' || operators.top() == '*')) || (
            currentOp == '*' && operators.top() == '*'))) {
          applyOp();
        }
        operators.push(currentOp);
      }
    }
    while (!operators.empty()) applyOp();
    return std::move(nodes.top());
  }
};

int main() {
  Interpreter parser;
  // Farklı kombinasyonları test edelim:
  std::string expr1 = "2 + 3 * 4";      // Beklenen: 14
  std::string expr2 = "10 * 2 + 5 * 3"; // Beklenen: 35
  std::string expr3 = "1 + 2 + 3 + 4";  // Beklenen: 10
  std::cout << expr1 << " = " << parser.interpret(expr1) << std::endl;
  std::cout << expr2 << " = " << parser.interpret(expr2) << std::endl;
  std::cout << expr3 << " = " << parser.interpret(expr3) << std::endl;
}
/*Program Çıktısı:
2 + 3 * 4 = 14
10 * 2 + 5 * 3 = 35
1 + 2 + 3 + 4 = 10
*/
```

5.5 MVC Mimari Kalıbı

MVC (*Model-View-Controller*), bir yazılım projesinde kodları görevlerine göre üç ana parçaya ayıran bir mimari Kalıpdir. Temel amacı, veriyi (mantık), kullanıcı arayüzünü ve bu ikisi arasındaki iletişimi birbirinden bağımsız hale getirerek kodun yönetimini kolaylaştırmaktır.

- ◇ **Veri ve Mantık** (*model*): Uygulamanın beynidir. Veritabanı işlemleri, hesaplamalar ve kurallar burada döner. "Veri nedir?" sorusuna cevap verir.
- ◇ **Arayüz/Görünüm** (*view*): Kullanıcının gördüğü ekrandır (HTML, CSS, butonlar). Sadece veriyi gösterir, arka planda ne olduğuyla ilgilenmez. "Veri nasıl görünmeli?" sorusuna cevap verir.
- ◇ **Aracı/Kontrolcü** (*controller*): Kullanıcıdan gelen istekleri karşılar. `Model`'den veriyi alır ve hangi `View`'da gösterileceğine karar verir. "Hangi veri nereye gitmeli?" sorusuna cevap verir.

Basit bir restoran senaryosunda MVC'yi bir restorana benzetebiliriz;

- ◇ Model (Aşçı): Mutfağın içindedir. Yemeği hazırlar, malzemeleri kontrol eder. Müşteriyi görmez, sadece siparişe göre yemeği (veriyi) üretir.
- ◇ View (Yemek Masası/Sunum): Müşterinin önüne gelen tabak ve sunumdur. Müşteri yemeğin nasıl piştiğini görmez, sadece sunulan tabağı görür.
- ◇ Controller (Garson): Müşteriden siparişi alır, aşçıya iletir. Yemek piştiğinde ise tabağı alır ve müşteriye servis eder.

Bir başka örnek olarak kullanıcı profili verilebilir. Diyelim ki bir kullanıcının profil bilgilerini ekrana getireceksiniz:

- ◇ Kullanıcı (User): Profil sayfasına tıklar (istek/request).
- ◇ Controller: Bu isteği yakalar. "Kullanıcı verisi lazım" diyerek Model'e seslenir.
- ◇ Model: Veritabanına gider, "Ahmet, 25 yaşında, Ankara" bilgilerini bulur ve Controller'a geri verir.
- ◇ Controller: Aldığı bu "Ahmet" verisini alır ve "Profil_Ekrani.html" adlı View dosyasına gönderir.
- ◇ View: Gelen veriyi şık bir tasarıma yerleştirir ve kullanıcıya gösterir.

MVC kullanmalıyız çünkü;

- ◇ **Kod Karmaşasını Önle**r: Tasarımcı sadece `View` ile, yazılımcı sadece `Model`/`Controller` ile ilgilenir.
- ◇ **Bakım Kolaylığı**: Görünümü değiştirmek istediğinizde veritabanı koduna dokunmanız gerekmez.
- ◇ **Yeniden Kullanılabilirlik**: Aynı `Model` verisini (örneğin kullanıcı listesi), hem bir web sayfasında hem de bir mobil uygulamada farklı `View`'lar ile gösterebilirsiniz.

C++ nesne yönelimli bir dil olduğu için MVC yapısını sınıflar arasındaki ilişkiyle kurgulamak, mantığı anlamak açısından çok faydalıdır. Bir "Öğrenci Bilgi Sistemi" üzerinden, bileşenlerin birbirine bağımlılığını (coupling) minimize edecek şekilde bir örnek verilebilir. Öğrenci Bilgi Sisteminde her bileşen kendi sorumluluğunu üstlenir: `Model` veriyi tutar, `View` ekrana basar, `Controller` ise akışı yönetir.

1. Model (Veri ve İş Mantığı): Sadece veriyi ve bu veriye erişim yöntemlerini içerir. Görselleştirme ile hiç ilgilenmez.

```cpp
#include <string>
class OgrenciModel {
private:
  std::string isim;
  std::string numara;
public:
  OgrenciModel(std::string i, std::string n) : isim(i), numara(n) {}
  // Getter ve Setter yöntemleri
  void setIsim(std::string i) { isim = i; }
  std::string getIsim() const { return isim; }
  void setNumara(std::string n) { numara = n; }
  std::string getNumara() const { return numara; }
};
```

2. View (Görünüm): Verinin kullanıcıya nasıl sunulacağını belirler. Genellikle sadece "Yazdır" veya "Göster" gibi yöntemleri vardır.

```cpp
#include <iostream>
class OgrenciView {
public:
  void ogrenciDetaylariniBas(std::string isim, std::string numara) {
    std::cout << "--- Ogrenci Bilgileri ---" << std::endl;
    std::cout << "Ad: " << isim << std::endl;
    std::cout << "No: " << numara << std::endl;
    std::cout << "-------------------------" << std::endl;
  }
};
```

3. Controller (Denetleyici): Model ve View arasındaki köprüdür. Kullanıcıdan gelen güncellemeleri Model'e yansıtır ve View'u güncellenmeye zorlar.

```cpp
class OgrenciController {
private:
  OgrenciModel& model;
  OgrenciView& view;
```

```
public:
  OgrenciController(OgrenciModel& m, OgrenciView& v) : model(m), view(v) {}
  void setOgrenciIsim(std::string i) { model.setIsim(i); }
  std::string getOgrenciIsim() { return model.getIsim(); }
  void setOgrenciNumara(std::string n) { model.setNumara(n); }
  std::string getOgrenciNumara() { return model.getNumara(); }
  // View'u güncelleme komutu
  void gorunumuGuncelle() {
    view.ogrenciDetaylariniBas(model.getIsim(), model.getNumara());
  }
};
```

4. Sistemin Çalıştırılması:

```
int main() {
  // 1. Model'i (Veriyi) oluştur
  OgrenciModel model("Ahmet Yilmaz", "2026001");
  // 2. View'u (Arayüzü) oluştur
  OgrenciView view;
  // 3. Controller'i bağla
  OgrenciController controller(model, view);
  // İlk hali ekrana bas
  controller.gorunumuGuncelle();
  // Veriyi Controller üzerinden güncelle
  controller.setOgrenciIsim("Mehmet Demir");
  // Güncel hali tekrar bas
  std::cout << "\nVeri guncellendikten sonra:\n";
  controller.gorunumuGuncelle();
}
/* Program Çıktısı:
--- Ogrenci Bilgileri ---
Ad: Ahmet Yilmaz
No: 2026001
-------------------------

Veri guncellendikten sonra:
--- Ogrenci Bilgileri ---
Ad: Mehmet Demir
No: 2026001
-------------------------
*/
```

Buradaki mimari ile ilgili şunlar söylenebilir;

- **Bağımsızlık**: Dikkat edersen `OgrenciModel` sınıfının içinde ne bir `cout` var ne de `OgrenciView` referansı. Bu sayede yarın bir gün veriyi ekrana değil de bir dosyaya yazmak istersen (yeni bir `View`), Model koduna dokunman gerekmez.
- **Bellek Yönetimi**: Örnekte referans (&) kullandım. Ancak büyük projelerde nesne ömürlerini yönetmek için `std::shared_ptr` veya `std::unique_ptr` kullanmak daha sağlıklı olacaktır.
- **Gözlemci Tasarım kalıbı İlişkisi**: Gelişmiş MVC yapılarında, `Model` değiştiğinde `Controller`'a haber vermesi için genellikle **gözlemci tasarım kalıbı** ile birlikte kullanılır.

6. Bölüm
Temiz Kod ve Mimari Standartlar

Temiz kod, yalnızca derlenen ve çalışan kod anlamına gelmez. Temiz kod; okunabilir, anlaşılabilir, değiştirilebilir, test edilebilir ve sürdürülebilir koddur. Yazılım projelerinde asıl maliyet çoğu zaman ilk geliştirme aşamasında değil, bakım ve değişiklik aşamalarında ortaya çıkar. Bu nedenle iyi bir yazılımcı yalnızca problemi çözen kodu değil, gelecekte değiştirilebilecek ve başkaları tarafından anlaşılabilecek kodu yazmayı hedeflemelidir.

Önceki bölümlerde nesne yönelimli programlama, kalıtım, çok biçimlilik, arayüzler, istisna yönetimi, UML ve SOLID ilkeleri ele alınmıştır. Bu bölümde ise bu kavramların gerçek yazılım projelerinde nasıl temiz, düzenli ve mimari açıdan sağlıklı bir yapıya dönüştürülebileceği açıklanacaktır.

6.1 Temiz Kod Nedir?

Temiz kod, okuyan kişiye ne yaptığını açıkça anlatan koddur. Temiz kodda gereksiz karmaşıklık yoktur; sınıflar, fonksiyonlar ve değişkenler anlamlı isimlere sahiptir; hata yönetimi açık ve kontrollüdür; bağımlılıklar yönetilebilir durumdadır.

Temiz kodun temel özellikleri şunlardır:

- ◇ Okunabilir olmalıdır.
- ◇ Amacını açıkça ifade etmelidir.
- ◇ Gereksiz tekrarlardan kaçınmalıdır.
- ◇ Kolay test edilebilir olmalıdır.
- ◇ Değişikliklere karşı dayanıklı olmalıdır.
- ◇ Hata durumlarını açık biçimde yönetmelidir.

§6.1.1 Kodun Okunabilir Olması

Kod, yalnızca bilgisayar tarafından değil, insanlar tarafından da okunur. Hatta kaynak kodun ömrü boyunca insanlar tarafından okunma sayısı, bilgisayar tarafından derlenme sayısından çok daha fazladır. Bu nedenle kodun okunabilir olması, yazılım kalitesinin temel ölçütlerinden biridir.

Aşağıdaki C++ kodu çalışabilir ancak okunabilir değildir:

```
int h(int a,int b){if(a>b)return a-b;else return b-a;}
```

Kod 6.1: Okunabilir Olmayan C++ Kodu

Aynı kod daha okunabilir şekilde şöyle yazılabilir:

```
int mutlakFarkHesapla(int birinciSayi, int ikinciSayi) {
  if (birinciSayi > ikinciSayi) {
    return birinciSayi - ikinciSayi;
  }

  return ikinciSayi - birinciSayi;
}
```

Kod 6.2: Okunabilir C++ Kodu

Java dilinde benzer örnek aşağıdaki gibidir:

```
public int mutlakFarkHesapla(int birinciSayi, int ikinciSayi) {
  if (birinciSayi > ikinciSayi) {
    return birinciSayi - ikinciSayi;
  }

  return ikinciSayi - birinciSayi;
}
```

Kod 6.3: Okunabilir Java Kodu

İkinci örnekte fonksiyon adı, parametre adları ve kod biçimi daha açıktır. Kodun ne yaptığı, yorum satırına ihtiyaç duyulmadan anlaşılmaktadır.

§6.1.2 Kodun Niyetini Açık Etmesi

Temiz kod, yalnızca işlemi değil, niyeti de açık eder. Bir kod parçasını okuyan kişi, yazılımcının ne yapmak istediğini anlayabilmelidir.

Aşağıdaki C++ kodunda değişkenlerin amacı belirsizdir:

```
double x = 1000;
double y = x * 0.18;
double z = x + y;
```

Kod 6.4: Niyeti Açık Olmayan C++ Kodu

Daha açık hali şöyledir:

```
double urunFiyati = 1000;
double kdvTutari = urunFiyati * 0.18;
double toplamFiyat = urunFiyati + kdvTutari;
```

Kod 6.5: Niyeti Açık C++ Kodu

Java örneği:

```
double urunFiyati = 1000;
double kdvTutari = urunFiyati * 0.18;
double toplamFiyat = urunFiyati + kdvTutari;
```

Kod 6.6: Niyeti Açık Java Kodu

Burada amaç yalnızca hesaplama yapmak değil, hesaplamanın ne anlama geldiğini de göstermektir.

§6.1.3 Yorum Satırları Ne Zaman Kullanılmalı?

Yorum satırları, kötü yazılmış kodu açıklamak için değil, kodun kendisinin anlatamadığı bağlamı açıklamak için kullanılmalıdır. Eğer bir kod parçası ancak yorumla anlaşılabiliyorsa, öncelikle kodun kendisi iyileştirilmelidir.

Kötü yorum örneği:

```
int yas = 20; // yas degiskenine 20 degeri atandi
```

Kod 6.7: Gereksiz Yorum Kullanımı

Bu yorum gereksizdir. Kod zaten bunu açıkça söylemektedir.

Faydalı yorum örneği:

```
// Banka kurali geregi 18 yasindan kucuk kullanicilar hesap acamaz.
if (yas < 18) {
  throw std::invalid_argument("Kullanici yasi 18'den kucuk olamaz.");
}
```

Kod 6.8: Faydalı Yorum Kullanımı

Java karşılığı:

```
// Banka kurali geregi 18 yasindan kucuk kullanicilar hesap acamaz.
if (yas < 18) {
  throw new IllegalArgumentException("Kullanici yasi 18'den kucuk olamaz.");
}
```

Kod 6.9: Java Dilinde Faydalı Yorum Kullanımı

Burada yorum, kodun ne yaptığını değil, neden yaptığını açıklar.

6.2 Anlamlı İsimlendirme

İsimlendirme, temiz kodun en önemli parçalarından biridir. Değişken, fonksiyon, sınıf, arayüz ve sabit isimleri, kodun niyetini açıkça göstermelidir.

§6.2.1 Değişken İsimlendirme

Değişken isimleri, değişkenin neyi temsil ettiğini açıkça anlatmalıdır.

Kötü C++ örneği:

```
int x = 45;
int y = 60;
int z = x + y;
```

İyi C++ örneği:

```
int vizeNotu = 45;
int finalNotu = 60;
int toplamNot = vizeNotu + finalNotu;
```

Java örneği:

```
int vizeNotu = 45;
int finalNotu = 60;
int toplamNot = vizeNotu + finalNotu;
```

Tek harfli değişkenler yalnızca kısa döngülerde veya matematiksel bağlamlarda tercih edilmelidir.

§6.2.2 Fonksiyon ve Yöntem İsimlendirme

Fonksiyon ve yöntem isimleri genellikle fiil içermelidir. Çünkü fonksiyonlar bir eylem gerçekleştirir.

Kötü örnek:

```
void islem();
```

İyi örnek:

```
void faturaYazdir();
void kullaniciDogrula();
void toplamTutarHesapla();
```

Java örneği:

```
void faturaYazdir() { }
void kullaniciDogrula() { }
void toplamTutarHesapla() { }
```

Fonksiyon adı, fonksiyonun içinde ne yapıldığını mümkün olduğunca açık göstermelidir.

§6.2.3 Sınıf İsimlendirme

Sınıflar genellikle isim veya isim tamlaması ile adlandırılır. Çünkü sınıflar bir kavramı, varlığı veya sorumluluğu temsil eder.

Kötü sınıf isimleri:

```
class Manager {};
class Helper {};
class Processor {};
```

Bu isimler çok geneldir. Sınıfın ne yaptığı anlaşılamaz.

Daha iyi isimler:

```
class FaturaYazici {};
class KullaniciDogrulayici {};
class OdemeHesaplayici {};
```

Java örneği:

```
class FaturaYazici { }
class KullaniciDogrulayici { }
class OdemeHesaplayici { }
```

§6.2.4 Sabit, Enum ve Arayüz İsimlendirme

Sabitler, değişmeyen değerleri temsil eder. C++ dilinde sabitler genellikle `const` veya `constexpr` ile tanımlanır.

```
constexpr double KDV_ORANI = 0.20;
const int MAKSIMUM_DENEME_SAYISI = 3;
```

Kod 6.10: C++ Dilinde Sabit Tanımlama

Java dilinde sabitler genellikle `static final` ile tanımlanır:

```
public static final double KDV_ORANI = 0.20;
public static final int MAKSIMUM_DENEME_SAYISI = 3;
```

Kod 6.11: Java Dilinde Sabit Tanımlama

Enum isimleri de anlamlı olmalıdır.

```
enum class SiparisDurumu {
  Hazirlaniyor,
  Kargoda,
  TeslimEdildi,
  IptalEdildi
};
```

Kod 6.12: C++ Enum Örneği

Java karşılığı:

```
enum SiparisDurumu {
  HAZIRLANIYOR,
  KARGODA,
  TESLIM_EDILDI,
  IPTAL_EDILDI
}
```

Kod 6.13: Java Enum Örneği

6.3 Küçük Fonksiyonlar ve Yöntemler

Fonksiyonlar küçük, anlaşılır ve tek bir iş yapan yapılar olmalıdır. Bir fonksiyon ne kadar uzarsa, anlaşılması ve test edilmesi o kadar zorlaşır.

§6.3.1 Fonksiyonların Tek İş Yapması

Bir fonksiyon yalnızca tek bir soyutlama düzeyinde tek bir işi yapmalıdır.

Kötü C++ örneği:

```
void siparisIsle() {
  // Siparis dogrula
  // Stok kontrol et
  // Odeme al
  // Fatura olustur
  // E-posta gonder
}
```

Kod 6.14: Birden Fazla İş Yapan Fonksiyon

Daha iyi tasarım:

```
void siparisIsle() {
  siparisDogrula();
  stokKontrolEt();
  odemeAl();
  faturaOlustur();
  bildirimGonder();
}
```

Kod 6.15: Tek İş Yapan Fonksiyonlara Ayrılmış Kod

Java karşılığı:

```
void siparisIsle() {
  siparisDogrula();
  stokKontrolEt();
  odemeAl();
  faturaOlustur();
  bildirimGonder();
}
```

Bu yapıda ana yöntem süreci yönetir; detaylar ayrı yöntemlere aktarılır.

§6.3.2 Parametre Sayısını Azaltma

Çok fazla parametre alan fonksiyonlar zor okunur ve hata yapmaya açıktır.

Kötü örnek:

```
void kullaniciOlustur(std::string ad, std::string soyad, std::string email,
std::string telefon, int yas, std::string adres);
```

Daha iyi yaklaşım, parametre nesnesi kullanmaktır:

```
class KullaniciBilgisi {
  public:
  std::string ad;
  std::string soyad;
  std::string email;
  std::string telefon;
  int yas;
  std::string adres;
};

void kullaniciOlustur(const KullaniciBilgisi& bilgi);
```

Java örneği:

```
class KullaniciBilgisi {
  String ad;
  String soyad;
  String email;
```

```
  String telefon;
  int yas;
  String adres;
}

void kullaniciOlustur(KullaniciBilgisi bilgi) {
  // Kullanici olusturma islemleri
}
```

§6.3.3 Yan Etkilerden Kaçınma

Yan etki, bir fonksiyonun beklenmeyen şekilde dış durumu değiştirmesidir. Yan etkiler kodu anlamayı ve test etmeyi zorlaştırır.

Kötü C++ örneği:

```
int toplam = 0;

void ekle(int sayi) {
  toplam += sayi;
}
```

Daha iyi yaklaşım:

```
int topla(int mevcutToplam, int eklenecekSayi) {
  return mevcutToplam + eklenecekSayi;
}
```

Java örneği:

```
int topla(int mevcutToplam, int eklenecekSayi) {
  return mevcutToplam + eklenecekSayi;
}
```

§6.3.4 Erken Çıkış ve Guard Clause Kullanımı

İç içe geçmiş `if` blokları kodu karmaşıklaştırır. Geçersiz durumları fonksiyonun başında eleyerek kod daha okunabilir hale getirilebilir.

Kötü örnek:

```
void paraCek(double miktar) {
  if (miktar > 0) {
    if (miktar <= bakiye) {
      bakiye -= miktar;
    }
  }
}
```

İyi örnek:

```
void paraCek(double miktar) {
  if (miktar <= 0) {
    throw std::invalid_argument("Miktar pozitif olmalidir.");
  }

  if (miktar > bakiye) {
    throw std::runtime_error("Yetersiz bakiye.");
  }

  bakiye -= miktar;
}
```

Java karşılığı:

```
void paraCek(double miktar) {
```

```
    if (miktar <= 0) {
      throw new IllegalArgumentException("Miktar pozitif olmalidir.");
    }

    if (miktar > bakiye) {
      throw new IllegalStateException("Yetersiz bakiye.");
    }

    bakiye -= miktar;
  }
```

6.4 Küçük Sınıflar ve Sorumluluk Ayrımı

Sınıflar küçük ve tek sorumluluğa sahip olmalıdır. Bir sınıfın büyüklüğü yalnızca satır sayısıyla değil, üstlendiği sorumluluk sayısıyla ölçülmelidir.

§6.4.1 Sınıfın Boyutu Ne ile Ölçülür?

Bir sınıfın çok sayıda farklı nedenle değişmesi gerekiyorsa, o sınıf büyük ihtimalle birden fazla sorumluluk taşıyordur.

Kötü örnek:

```
class Fatura {
  public:
  void toplamHesapla();
  void veritabaninaKaydet();
  void pdfOlustur();
  void emailGonder();
};
```

Bu sınıf hem hesaplama, hem veri kaydetme, hem belge üretme, hem bildirim gönderme görevlerini üstlenmiştir.

Daha iyi tasarım:

```
class Fatura {};
class FaturaHesaplayici {};
class FaturaDeposu {};
class FaturaPdfOlusturucu {};
class FaturaBildirimServisi {};
```

Java karşılığı:

```
class Fatura { }
class FaturaHesaplayici { }
class FaturaDeposu { }
class FaturaPdfOlusturucu { }
class FaturaBildirimServisi { }
```

§6.4.2 Tanrı Sınıf Problemi

Tanrı sınıf, sistemdeki hemen hemen her işi yapan büyük sınıftır. Genellikle `Manager`, `SystemController`, `ApplicationService` gibi isimlerle ortaya çıkar.

Kötü örnek:

```
class KutuphaneManager {
  public:
  void kitapEkle();
  void uyeEkle();
  void oduncVer();
  void cezaHesapla();
  void raporOlustur();
  void emailGonder();
  void veritabaninaKaydet();
```

```
};
```

Daha iyi yaklaşım:

```
class KitapServisi {};
class UyeServisi {};
class OduncServisi {};
class CezaHesaplayici {};
class RaporServisi {};
class BildirimServisi {};
```

§6.4.3 Sınıfların Birlikte Değişen Davranışlara Göre Ayrılması

Birlikte değişen kodlar aynı sınıfta, farklı nedenlerle değişen kodlar farklı sınıflarda olmalıdır. Örneğin ceza hesaplama kuralları değiştiğinde yalnızca `CezaHesaplayici` sınıfı etkilenmelidir.

```
class CezaHesaplayici {
  public:
  double cezaHesapla(int gecikmeGunu) const {
    return gecikmeGunu * 5.0;
  }
};
```

Java karşılığı:

```
class CezaHesaplayici {
  double cezaHesapla(int gecikmeGunu) {
    return gecikmeGunu * 5.0;
  }
}
```

§6.4.4 Bağlaşım ve Uyum

Bağlaşım (coupling), sınıfların birbirine ne kadar bağımlı olduğunu gösterir. Uyum (cohesion) ise bir sınıfın içindeki üyelerin aynı amaca ne kadar hizmet ettiğini gösterir.

İyi tasarımda:

- ◇ Düşük bağlaşım olmalıdır.
- ◇ Yüksek uyum olmalıdır.

Düşük bağlaşım, değişikliklerin sistemin her yerine yayılmasını engeller. Yüksek uyum ise sınıfın tek ve açık bir amaca hizmet etmesini sağlar.

6.5 Kod Tekrarı ve Yeniden Kullanılabilirlik

Kod tekrarı, yazılım bakım maliyetini artıran en önemli problemlerden biridir. Aynı mantığın birden fazla yerde tekrar edilmesi, değişiklik gerektiğinde tüm kopyaların güncellenmesini zorunlu kılar.

§6.5.1 DRY İlkesi

DRY, "Don't Repeat Yourself" ifadesinin kısaltmasıdır. Türkçeye "Kendini Tekrar Etme" olarak çevrilebilir.

Kötü C++ örneği:

```
double ogrenciIndirimi(double tutar) {
  return tutar * 0.10;
}

double kampanyaIndirimi(double tutar) {
  return tutar * 0.10;
}
```

Eğer aynı indirim oranı iş kuralını temsil ediyorsa, tekrar azaltılmalıdır:

```
constexpr double INDIRIM_ORANI = 0.10;

double indirimHesapla(double tutar) {
  return tutar * INDIRIM_ORANI;
}
```

Java karşılığı:

```
class IndirimHesaplayici {
  private static final double INDIRIM_ORANI = 0.10;

  double indirimHesapla(double tutar) {
    return tutar * INDIRIM_ORANI;
  }
}
```

§6.5.2 Gereksiz Soyutlama Tehlikesi

Her tekrar hemen soyutlanmamalıdır. İki kod parçası şu an benziyor olabilir ancak farklı nedenlerle değişecekse aynı soyutlama altında birleştirilmemelidir.

Yanlış soyutlama, kod tekrarından daha tehlikeli olabilir. Çünkü yanlış soyutlama, farklı davranışları aynı yapıya zorlar.

§6.5.3 Kalıtım Yerine Bileşim

Kod tekrarını azaltmak için her zaman kalıtım kullanılmamalıdır. Kalıtım, sınıflar arasında güçlü bir bağ kurar. Eğer ilişki "bir türüdür" değil de "sahiptir" biçimindeyse bileşim tercih edilmelidir.

Kötü yaklaşım:

```
class Motor {};
class Araba : public Motor {};
```

Araba bir motor değildir; motoru vardır. Doğru yaklaşım:

```
class Motor {};

class Araba {
  private:
  Motor motor;
};
```

Java karşılığı:

```
class Motor { }

class Araba {
  private Motor motor = new Motor();
}
```

6.6 Hata Yönetimi ve İstisna Mimarisi

Hata yönetimi, temiz kodun önemli bir parçasıdır. Hata durumları iş kodunun arasına dağılmamalı, anlamlı istisna sınıfları ve tutarlı bir mimari ile yönetilmelidir.

§6.6.1 Hata Kodları Yerine İstisnalar

Hata kodları, fonksiyonun normal dönüş değeri ile hata bilgisini aynı kanal üzerinden taşır.

Kötü C++ örneği:

```
int paraCek(double miktar) {
```

```
  if (miktar <= 0) return -1;
  if (miktar > bakiye) return -2;

  bakiye -= miktar;
  return 0;
}
```

Daha iyi yaklaşım:

```
void paraCek(double miktar) {
  if (miktar <= 0) {
    throw std::invalid_argument("Miktar pozitif olmalidir.");
  }

  if (miktar > bakiye) {
    throw std::runtime_error("Yetersiz bakiye.");
  }

  bakiye -= miktar;
}
```

Java karşılığı:

```
void paraCek(double miktar) {
  if (miktar <= 0) {
    throw new IllegalArgumentException("Miktar pozitif olmalidir.");
  }

  if (miktar > bakiye) {
    throw new IllegalStateException("Yetersiz bakiye.");
  }

  bakiye -= miktar;
}
```

§6.6.2 Anlamlı İstisna Sınıfları

Genel istisnalar yerine alan odaklı istisnalar tercih edilmelidir.

C++ örneği:

```
class YetersizBakiyeException : public std::runtime_error {
  public:
  YetersizBakiyeException()
  : std::runtime_error("Yetersiz bakiye.") {}
};
```

Java örneği:

```
class YetersizBakiyeException extends RuntimeException {
  public YetersizBakiyeException() {
    super("Yetersiz bakiye.");
  }
}
```

§6.6.3 İstisnayı Yutmamak

Boş `catch` blokları hatanın kaybolmasına neden olur.

Kötü örnek:

```
try {
  dosyaOku();
} catch (...) {
  // Hata yutuldu
}
```

Daha iyi yaklaşım:

```
try {
  dosyaOku();
} catch (const std::exception& e) {
  std::cerr << "Dosya okuma hatasi: " << e.what() << std::endl;
  throw;
}
```

Java karşılığı:

```
try {
  dosyaOku();
} catch (IOException e) {
  System.err.println("Dosya okuma hatasi: " + e.getMessage());
  throw e;
}
```

§6.6.4 Kaynak Yönetimi

C++ dilinde kaynak yönetimi için RAII ve akıllı göstericiler kullanılmalıdır.

```
std::unique_ptr<Dosya> dosya = std::make_unique<Dosya>("veri.txt");
```

Java dilinde kaynak yönetimi için `try-with-resources` kullanılır:

```
try (BufferedReader reader = Files.newBufferedReader(Path.of("veri.txt"))) {
  String satir = reader.readLine();
}
```

§6.6.5 Katmanlara Göre Hata Yönetimi

Alt katmanda oluşan teknik hatalar, üst katmanda iş anlamı taşıyan hatalara dönüştürülebilir.

Java örneği:

```
try {
  repository.kaydet(kullanici);
} catch (SQLException e) {
  throw new KullaniciKaydedilemediException("Kullanici kaydedilemedi.", e);
}
```

C++ örneği:

```
try {
  depo.kaydet(kullanici);
} catch (const std::exception& e) {
  throw std::runtime_error("Kullanici kaydedilemedi.");
}
```

6.7 Boş Değerler, Optional ve Geçersiz Durum Yönetimi

Boş değerler, yazılım hatalarının önemli kaynaklarından biridir. Null referanslar, özellikle nesne yönelimli dillerde çalışma zamanı hatalarına neden olabilir.

§6.7.1 Null Referans Problemi

Java'da null referans üzerinden yöntem çağırmak `NullPointerException` hatasına neden olur.

```
Kullanici kullanici = null;
System.out.println(kullanici.getAd()); // NullPointerException
```

C++ dilinde benzer problem null pointer üzerinden erişimle oluşur:

```
Kullanici* kullanici = nullptr;
std::cout << kullanici->getAd(); // Tanımsız davranış
```

§6.7.2 Optional Kullanımı

Bir değerin olmayabileceği açıkça belirtilmek isteniyorsa Optional kullanılabilir.

C++ örneği:

```
#include <optional>

std::optional<Kullanici> kullaniciBul(int id) {
  if (id == 1) {
    return Kullanici("Ilhan");
  }

  return std::nullopt;
}
```

Java örneği:

```
Optional<Kullanici> kullaniciBul(int id) {
  if (id == 1) {
    return Optional.of(new Kullanici("Ilhan"));
  }

  return Optional.empty();
}
```

§6.7.3 Null Object Pattern

Null Object Pattern, null yerine zararsız davranış gösteren nesne kullanma yaklaşımıdır.

C++ örneği:

```
class IBildirim {
  public:
  virtual void gonder(std::string mesaj) = 0;
  virtual ~IBildirim() = default;
};

class BosBildirim : public IBildirim {
  public:
  void gonder(std::string mesaj) override {
    // Bilerek bos bırakıldı
  }
};
```

Java örneği:

```
interface Bildirim {
  void gonder(String mesaj);
}

class BosBildirim implements Bildirim {
  public void gonder(String mesaj) {
    // Bilerek bos bırakıldı
  }
}
```

§6.7.4 Geçersiz Nesne Oluşturmamak

Bir nesne, oluşturulduğu anda geçerli durumda olmalıdır. Yapıcılar bu nedenle doğrulama yapmalıdır.

C++ örneği:

```
class Kullanici {
  public:
  Kullanici(std::string ad) {
    if (ad.empty()) {
```

```
        throw std::invalid_argument("Ad bos olamaz.");
      }

      this->ad = ad;
    }

    private:
    std::string ad;
};
```

Java örneği:

```
class Kullanici {
  private String ad;

  public Kullanici(String ad) {
    if (ad == null || ad.isBlank()) {
      throw new IllegalArgumentException("Ad bos olamaz.");
    }

    this.ad = ad;
  }
}
```

6.8 Kapsülleme ve Değişmez Nesneler

Kapsülleme, nesnenin iç durumunu dış dünyadan koruyarak bu duruma yalnızca kontrollü yöntemler üzerinden erişilmesini sağlar. Nesne yönelimli programlamada kapsülleme yalnızca alanları `private` yapmak değildir; asıl amaç, nesnenin her zaman geçerli ve tutarlı durumda kalmasını sağlamaktır.

Değişmez nesneler ise oluşturulduktan sonra iç durumu değiştirilemeyen nesnelerdir. Bu tür nesneler daha güvenli, daha öngörülebilir ve çok iş parçacıklı ortamlarda daha kolay yönetilebilir yapılar sunar.

§6.8.1 Alanları Doğrudan Açmamak

Bir sınıfın alanlarının doğrudan dışarıya açılması, nesnenin iç durumunun kontrolsüz biçimde değiştirilmesine neden olur. Bu durum, nesnenin geçersiz durumlara düşmesine yol açabilir.

Aşağıdaki C++ örneğinde `bakiye` alanı doğrudan dışarıya açıktır:

```
class BankaHesabi {
  public:
  double bakiye;
};

int main() {
  BankaHesabi hesap;
  hesap.bakiye = -5000; // Geçersiz durum
}
```

Kod 6.16: C++ Dilinde Alanların Doğrudan Açılması

Bu tasarımda dış kod, hesabın bakiyesini negatif yapabilmektedir. Bu durum iş kurallarına aykırıdır. Daha doğru yaklaşım, alanı `private` yapmak ve davranışlar üzerinden değiştirmektir.

```
#include <stdexcept>

class BankaHesabi {
  private:
  double bakiye;

  public:
  BankaHesabi() : bakiye(0.0) {}
```

```
    void paraYatir(double miktar) {
      if (miktar <= 0) {
        throw std::invalid_argument("Yatırılacak miktar pozitif olmalıdır.");
      }

      bakiye += miktar;
    }

    void paraCek(double miktar) {
      if (miktar <= 0) {
        throw std::invalid_argument("Çekilecek miktar pozitif olmalıdır.");
      }

      if (miktar > bakiye) {
        throw std::runtime_error("Yetersiz bakiye.");
      }

      bakiye -= miktar;
    }

    double bakiyeOgren() const {
      return bakiye;
    }
  };
```

Kod 6.17: C++ Dilinde Kontrollü Erişim

Java karşılığı aşağıdaki gibidir:

```
  class BankaHesabi {
    private double bakiye;

    public BankaHesabi() {
      this.bakiye = 0.0;
    }

    public void paraYatir(double miktar) {
      if (miktar <= 0) {
        throw new IllegalArgumentException("Yatırılacak miktar pozitif olmalıdır.");
      }

      this.bakiye += miktar;
    }

    public void paraCek(double miktar) {
      if (miktar <= 0) {
        throw new IllegalArgumentException("Çekilecek miktar pozitif olmalıdır.");
      }

      if (miktar > this.bakiye) {
        throw new IllegalStateException("Yetersiz bakiye.");
      }

      this.bakiye -= miktar;
    }

    public double bakiyeOgren() {
      return this.bakiye;
    }
  }
```

Kod 6.18: Java Dilinde Kontrollü Erişim

Burada `bakiye` alanı dışarıdan doğrudan değiştirilemez. Bakiye ancak iş kurallarını uygulayan `paraYatir()` ve `paraCek()` yöntemleriyle değiştirilebilir.

§6.8.2 Değişmez Nesneler

Değişmez nesne, oluşturulduktan sonra durumu değiştirilemeyen nesnedir. Özellikle değer nesneleri (value object) için bu yaklaşım oldukça uygundur. Para, tarih aralığı, koordinat, renk, kimlik bilgisi gibi nesneler çoğu zaman değişmez olarak tasarlanabilir.

C++ dilinde değişmez nesne tasarımı için alanlar `private` yapılır ve dışarıya yalnızca okuma yöntemleri sunulur:

```
#include <string>
#include <stdexcept>

class Para {
  private:
  int kurus;
  std::string paraBirimi;

  public:
  Para(int pKurus, std::string pParaBirimi)
  : kurus(pKurus), paraBirimi(pParaBirimi) {

    if (pKurus < 0) {
      throw std::invalid_argument("Para miktarı negatif olamaz.");
    }

    if (pParaBirimi.empty()) {
      throw std::invalid_argument("Para birimi boş olamaz.");
    }
  }

  int kurusDegeri() const {
    return kurus;
  }

  std::string paraBirimiDegeri() const {
    return paraBirimi;
  }

  Para ekle(const Para& diger) const {
    if (paraBirimi != diger.paraBirimi) {
      throw std::invalid_argument("Farklı para birimleri toplanamaz.");
    }

    return Para(kurus + diger.kurus, paraBirimi);
  }
};
```

Kod 6.19: C++ Dilinde Değişmez Nesne

Bu sınıfta `ekle()` yöntemi mevcut nesneyi değiştirmez. Bunun yerine yeni bir `Para` nesnesi döndürür.

Java dilinde değişmez nesne tasarımı için alanlar `private final` yapılır:

```
class Para {
  private final int kurus;
  private final String paraBirimi;

  public Para(int kurus, String paraBirimi) {
    if (kurus < 0) {
      throw new IllegalArgumentException("Para miktarı negatif olamaz.");
    }

    if (paraBirimi == null || paraBirimi.isBlank()) {
      throw new IllegalArgumentException("Para birimi boş olamaz.");
    }
```

```
    this.kurus = kurus;
    this.paraBirimi = paraBirimi;
  }

  public int getKurus() {
    return kurus;
  }

  public String getParaBirimi() {
    return paraBirimi;
  }

  public Para ekle(Para diger) {
    if (!this.paraBirimi.equals(diger.paraBirimi)) {
      throw new IllegalArgumentException("Farklı para birimleri toplanamaz.");
    }

    return new Para(this.kurus + diger.kurus, this.paraBirimi);
  }
}
```

Kod 6.20: Java Dilinde Değişmez Nesne

Değişmez nesnelerin en önemli üstünlükleri şunlardır:

- ◇ Yan etki oluşturmazlar.
- ◇ Test edilmeleri kolaydır.
- ◇ Çok iş parçacıklı ortamlarda daha güvenlidirler.
- ◇ Nesnenin beklenmedik biçimde değişmesi engellenir.

§6.8.3 Nesne Durumunun Tutarlılığı

Bir nesne, yaşamı boyunca geçerli durumda kalmalıdır. Nesnenin geçerli kalmasını sağlayan kurallara **invariant** adı verilir.

Örneğin bir banka hesabında bakiye negatif olamaz kuralı bir invarianttır. Benzer şekilde bir öğrencinin not ortalaması 0 ile 100 arasında olmalıdır.

C++ örneği:

```
#include <stdexcept>

class Ogrenci {
  private:
  double notOrtalamasi;

  public:
  Ogrenci(double pNotOrtalamasi) {
    notOrtalamasiAyarla(pNotOrtalamasi);
  }

  void notOrtalamasiAyarla(double deger) {
    if (deger < 0 || deger > 100) {
      throw std::invalid_argument("Not ortalaması 0 ile 100 arasında olmalıdır.");
    }

    notOrtalamasi = deger;
  }

  double notOrtalamasiOgren() const {
    return notOrtalamasi;
  }
};
```

Kod 6.21: C++ Dilinde Nesne Tutarlılığı

Java örneği:

```
class Ogrenci {
  private double notOrtalamasi;

  public Ogrenci(double notOrtalamasi) {
    notOrtalamasiAyarla(notOrtalamasi);
  }

  public void notOrtalamasiAyarla(double deger) {
    if (deger < 0 || deger > 100) {
      throw new IllegalArgumentException("Not ortalaması 0 ile 100 arasında olmalıdır
          .");
    }

    this.notOrtalamasi = deger;
  }

  public double notOrtalamasiOgren() {
    return notOrtalamasi;
  }
}
```

Kod 6.22: Java Dilinde Nesne Tutarlılığı

Bu yapıda nesne, oluşturulduğu andan itibaren geçerli durumdadır. Dışarıdan geçersiz veri atanması engellenmiştir.

6.9 Katmanlı Mimari Temelleri

Katmanlı mimari, yazılım sistemini farklı sorumluluklara sahip katmanlara ayıran bir mimari yaklaşımdır. Amaç, kullanıcı arayüzü, iş kuralları, veri erişimi ve altyapı kodlarının birbirine karışmasını engellemektir.

Genel olarak bir uygulama şu katmanlardan oluşabilir:

- Sunum Katmanı
- İş Katmanı
- Veri Erişim Katmanı
- Altyapı Katmanı

Bu ayrım, sistemin daha kolay test edilmesini, değiştirilmesini ve genişletilmesini sağlar.

§6.9.1 Katmanlı Mimari Nedir?

Katmanlı mimaride her katman belirli bir sorumluluğa sahiptir. Örneğin kullanıcıdan veri almak sunum katmanının, iş kurallarını uygulamak iş katmanının, veritabanı işlemleri ise veri erişim katmanının görevidir.

Basit bir sipariş sistemi şu şekilde düşünülebilir:

- `SiparisController`: Kullanıcı isteğini alır.
- `SiparisServisi`: İş kurallarını uygular.
- `SiparisDeposu`: Veriyi kaydeder veya getirir.

C++ dilinde basit bir katmanlı yapı aşağıdaki gibi modellenebilir:

```
#include <iostream>
#include <string>

class Siparis {
  public:
  std::string urunAdi;

  explicit Siparis(std::string pUrunAdi) : urunAdi(pUrunAdi) {}
};
```

```cpp
class SiparisDeposu {
  public:
  void kaydet(const Siparis& siparis) {
    std::cout << siparis.urunAdi << " veritabanına kaydedildi." << std::endl;
  }
};

class SiparisServisi {
  private:
  SiparisDeposu& depo;

  public:
  explicit SiparisServisi(SiparisDeposu& pDepo) : depo(pDepo) {}

  void siparisOlustur(const std::string& urunAdi) {
    if (urunAdi.empty()) {
      throw std::invalid_argument("Ürün adı boş olamaz.");
    }

    Siparis siparis(urunAdi);
    depo.kaydet(siparis);
  }
};
```

Kod 6.23: C++ Dilinde Basit Katmanlı Mimari

Java örneği:

```java
class Siparis {
  private final String urunAdi;

  public Siparis(String urunAdi) {
    this.urunAdi = urunAdi;
  }

  public String getUrunAdi() {
    return urunAdi;
  }
}

class SiparisDeposu {
  public void kaydet(Siparis siparis) {
    System.out.println(siparis.getUrunAdi() + " veritabanına kaydedildi.");
  }
}

class SiparisServisi {
  private final SiparisDeposu depo;

  public SiparisServisi(SiparisDeposu depo) {
    this.depo = depo;
  }

  public void siparisOlustur(String urunAdi) {
    if (urunAdi == null || urunAdi.isBlank()) {
      throw new IllegalArgumentException("Ürün adı boş olamaz.");
    }

    Siparis siparis = new Siparis(urunAdi);
    depo.kaydet(siparis);
  }
}
```

Kod 6.24: Java Dilinde Basit Katmanlı Mimari

§6.9.2 Katmanlar Arası Bağımlılık

Katmanlar arası bağımlılık dikkatli yönetilmelidir. Sunum katmanı iş katmanını kullanabilir; iş katmanı veri erişim katmanını kullanabilir. Ancak veri erişim katmanı sunum katmanına bağımlı olmamalıdır.

Yanlış bağımlılık örneği:

```
class SiparisDeposu {
  public:
  void kaydet() {
    // Veritabanı işlemi
    // Ekrana mesaj yazdırmak veri katmanının görevi değildir.
    std::cout << "Sipariş kaydedildi." << std::endl;
  }
};
```

Bu örnekte veri erişim katmanı kullanıcı arayüzü davranışı göstermektedir. Bu temiz bir ayrım değildir.

Daha doğru yaklaşım:

```
class SiparisDeposu {
  public:
  void kaydet(const Siparis& siparis) {
    // Sadece veri kaydetme işlemi yapılır.
  }
};
```

Java karşılığı:

```
class SiparisDeposu {
  public void kaydet(Siparis siparis) {
    // Sadece veri kaydetme işlemi yapılır.
  }
}
```

§6.9.3 DTO, Entity ve Model Ayrımı

DTO, Entity ve Model kavramları çoğu projede karıştırılır.

- **DTO**: Katmanlar arasında veri taşımak için kullanılır.
- **Entity**: Veritabanındaki bir tabloyu veya kalıcı varlığı temsil eder.
- **Model**: İş alanındaki kavramı temsil eder.

Java dilinde basit bir DTO örneği:

```
class KullaniciDto {
  public String ad;
  public String email;
}
```

Kod 6.25: Java Dilinde DTO Örneği

Entity örneği:

```
class KullaniciEntity {
  private Long id;
  private String ad;
  private String email;

  // Getter ve setter yöntemleri
}
```

Kod 6.26: Java Dilinde Entity Örneği

Alan modeli örneği:

```
class Kullanici {
  private final String ad;
  private final String email;

  public Kullanici(String ad, String email) {
    if (ad == null || ad.isBlank()) {
      throw new IllegalArgumentException("Ad boş olamaz.");
    }

    if (email == null || !email.contains("@")) {
      throw new IllegalArgumentException("Geçersiz e-posta adresi.");
    }

    this.ad = ad;
    this.email = email;
  }
}
```

Kod 6.27: Java Dilinde Alan Modeli Örneği

C++ dilinde de benzer ayrım yapılabilir:

```
struct KullaniciDto {
  std::string ad;
  std::string email;
};
```

Kod 6.28: C++ Dilinde DTO Örneği

§6.9.4 Repository ve Service Kavramları

Repository, veri erişimini soyutlayan yapıdır. Service ise iş kurallarını yürüten sınıftır.

C++ örneği:

```
class IKullaniciDeposu {
  public:
  virtual void kaydet(const Kullanici& kullanici) = 0;
  virtual ~IKullaniciDeposu() = default;
};

class KullaniciServisi {
  private:
  IKullaniciDeposu& depo;

  public:
  explicit KullaniciServisi(IKullaniciDeposu& pDepo) : depo(pDepo) {}

  void kullaniciKaydet(const Kullanici& kullanici) {
    depo.kaydet(kullanici);
  }
};
```

Kod 6.29: C++ Dilinde Repository ve Service

Java örneği:

```
interface KullaniciDeposu {
  void kaydet(Kullanici kullanici);
}

class KullaniciServisi {
  private final KullaniciDeposu depo;

  public KullaniciServisi(KullaniciDeposu depo) {
    this.depo = depo;
  }
```

```
    public void kullaniciKaydet(Kullanici kullanici) {
      depo.kaydet(kullanici);
    }
  }
```

Kod 6.30: Java Dilinde Repository ve Service

Repository veri erişim ayrıntılarını gizler. Service sınıfı ise iş akışını yönetir.

6.10 Bağımlılık Yönetimi ve Bağımlılık Enjeksiyonu

Bağımlılık yönetimi, bir sınıfın ihtiyaç duyduğu diğer sınıflarla ilişkisini düzenleme işidir. Kötü yönetilen bağımlılıklar, kodun test edilmesini ve değiştirilmesini zorlaştırır.

§6.10.1 Bağımlılık Yönü

Üst düzey sınıflar, alt düzey ayrıntılara doğrudan bağımlı olmamalıdır. Bunun yerine arayüzlere bağımlı olmalıdır. Bu yaklaşım, 3. bölümde anlatılan Bağımlılıkları Tersine Çevirme İlkesi (DIP) ile ilişkilidir.

Kötü C++ örneği:

```
class EpostaGonderici {
  public:
  void gonder(std::string mesaj) {
    std::cout << mesaj << std::endl;
  }
};

class SiparisServisi {
  private:
  EpostaGonderici gonderici;

  public:
  void siparisVer() {
    gonderici.gonder("Sipariş alındı.");
  }
};
```

Bu tasarımda `SiparisServisi`, doğrudan `EpostaGonderici` sınıfına bağlıdır.

§6.10.2 Bağımlılık Enjeksiyonu (Dependency Injection)

Bağımlılık enjeksiyonu, bir sınıfın ihtiyaç duyduğu bağımlılıkların sınıf içinde oluşturulması yerine dışarıdan verilmesidir.

C++ örneği:

```
class IBildirimGonderici {
  public:
  virtual void gonder(const std::string& mesaj) = 0;
  virtual ~IBildirimGonderici() = default;
};

class EpostaGonderici : public IBildirimGonderici {
  public:
  void gonder(const std::string& mesaj) override {
    std::cout << "[E-posta] " << mesaj << std::endl;
  }
};

class SiparisServisi {
  private:
  IBildirimGonderici& gonderici;

  public:
```

```
  explicit SiparisServisi(IBildirimGonderici& pGonderici)
  : gonderici(pGonderici) {}

  void siparisVer() {
    gonderici.gonder("Sipariş alındı.");
  }
};
```

Kod 6.31: C++ Dilinde Bağımlılık Enjeksiyonu

Java örneği:

```
interface BildirimGonderici {
  void gonder(String mesaj);
}

class EpostaGonderici implements BildirimGonderici {
  public void gonder(String mesaj) {
    System.out.println("[E-posta] " + mesaj);
  }
}

class SiparisServisi {
  private final BildirimGonderici gonderici;

  public SiparisServisi(BildirimGonderici gonderici) {
    this.gonderici = gonderici;
  }

  public void siparisVer() {
    gonderici.gonder("Sipariş alındı.");
  }
}
```

Kod 6.32: Java Dilinde Bağımlılık Enjeksiyonu

Bu yaklaşım sayesinde `SiparisServisi` e-posta, SMS veya sahte bir test göndericisi ile çalışabilir.

§6.10.3 Denetimin Tersine Çevrilmesi (IoC) Kavramı

Denetimin Tersine Çevrilmesi (Inversion of Control - IoC), nesnelerin oluşturulması ve birbirine bağlanması sorumluluğunun uygulama kodundan alınarak dış bir yapıya verilmesidir.

Basit bir uygulamada bu dış yapı `main()` fonksiyonu olabilir:

```
int main() {
  EpostaGonderici eposta;
  SiparisServisi servis(eposta);

  servis.siparisVer();
}
```

Java örneği:

```
public class Main {
  public static void main(String[] args) {
    BildirimGonderici gonderici = new EpostaGonderici();
    SiparisServisi servis = new SiparisServisi(gonderici);

    servis.siparisVer();
  }
}
```

Büyük Java projelerinde Spring gibi çerçeveler bu nesne oluşturma ve bağlama işlemini otomatik olarak yapar. Ancak temel mantık aynıdır: sınıf kendi bağımlılığını üretmez, dışarıdan alır.

6.11 Test Edilebilir Kod Yazmak

Test edilebilir kod, doğruluğu otomatik testlerle kolayca kontrol edilebilen koddur. Test edilebilir kod genellikle düşük bağımlılığa, küçük fonksiyonlara ve açık sorumluluklara sahiptir.

§6.11.1 Test Edilebilirlik Nedir?

Bir kodun test edilebilir olması için şu özellikleri taşıması gerekir:

- ◇ Bağımlılıkları dışarıdan verilebilmelidir.
- ◇ Global duruma bağımlı olmamalıdır.
- ◇ Yan etkileri sınırlı olmalıdır.
- ◇ Küçük ve tek sorumluluklu parçalardan oluşmalıdır.

Kötü Java örneği:

```
class RaporServisi {
  public void raporGonder() {
    EpostaGonderici gonderici = new EpostaGonderici();
    gonderici.gonder("Rapor hazır.");
  }
}
```

Bu sınıf test edilirken gerçek e-posta gönderimi tetiklenebilir. Daha iyi yaklaşım, bağımlılığı dışarıdan almaktır.

§6.11.2 Birim Test Mantığı

Birim test, bir sınıfın veya yöntemin küçük ve izole bir parçasını test etmektir. Birim testlerde genellikle Arrange-Act-Assert düzeni kullanılır:

- ◇ Arrange: Test verileri hazırlanır.
- ◇ Act: Test edilecek işlem çağrılır.
- ◇ Assert: Sonuç doğrulanır.

Java örneği:

```
class Hesaplayici {
  int topla(int a, int b) {
    return a + b;
  }
}

class HesaplayiciTest {
  public static void main(String[] args) {
    // Arrange
    Hesaplayici hesaplayici = new Hesaplayici();

    // Act
    int sonuc = hesaplayici.topla(2, 3);

    // Assert
    if (sonuc != 5) {
      throw new RuntimeException("Test başarısız.");
    }

    System.out.println("Test başarılı.");
  }
}
```

Kod 6.33: Java Dilinde Basit Birim Test Mantığı

C++ örneği:

```
#include <cassert>
#include <iostream>
```

```cpp
class Hesaplayici {
  public:
  int topla(int a, int b) const {
    return a + b;
  }
};

int main() {
  // Arrange
  Hesaplayici hesaplayici;

  // Act
  int sonuc = hesaplayici.topla(2, 3);

  // Assert
  assert(sonuc == 5);

  std::cout << "Test başarılı." << std::endl;
}
```

Kod 6.34: C++ Dilinde Basit Birim Test Mantığı

Gerçek projelerde C++ için Google Test, Catch2; Java için JUnit gibi test çerçeveleri kullanılır.

§6.11.3 Mock ve Fake Nesneler

Testlerde gerçek bağımlılık yerine sahte nesneler kullanılabilir. Bu sayede dosya sistemi, veritabanı ve ağ gibi dış bağımlılıklar devre dışı bırakılır.

C++ örneği:

```cpp
class IBildirimGonderici {
  public:
  virtual void gonder(const std::string& mesaj) = 0;
  virtual ~IBildirimGonderici() = default;
};

class FakeBildirimGonderici : public IBildirimGonderici {
  public:
  std::string sonMesaj;

  void gonder(const std::string& mesaj) override {
    sonMesaj = mesaj;
  }
};
```

Kod 6.35: C++ Dilinde Fake Nesne

Java örneği:

```java
class FakeBildirimGonderici implements BildirimGonderici {
  String sonMesaj;

  public void gonder(String mesaj) {
    this.sonMesaj = mesaj;
  }
}
```

Kod 6.36: Java Dilinde Fake Nesne

Fake nesne, gerçek sistemle iletişim kurmaz. Sadece testin sonucu doğrulamasına yardımcı olur.

§6.11.4 Test Edilemeyen Kodun Belirtileri

Aşağıdaki özelliklere sahip kodlar genellikle zor test edilir:

◇ İçeride doğrudan `new` ile bağımlılık oluşturması.

- ◇ Statik yöntemlere aşırı bağımlı olması.
- ◇ Global değişken kullanması.
- ◇ Dosya sistemi, veritabanı veya ağa doğrudan bağımlı olması.
- ◇ Çok fazla işi tek yöntemde yapması.

Kötü C++ örneği:

```
class SiparisServisi {
  public:
  void siparisVer() {
    EpostaGonderici gonderici;
    gonderici.gonder("Sipariş alındı.");
  }
};
```

Bu yapı test sırasında farklı bir gönderici ile değiştirilemez.

6.12 Kod Biçimlendirme Standartları

Kod biçimlendirme, kodun görsel düzenini belirler. Aynı ekipte çalışan geliştiricilerin ortak stil kullanması, kodun okunabilirliğini artırır.

§6.12.1 Girintileme ve Boşluklar

Girintileme, kod bloklarının kapsamını görsel olarak göstermelidir.

Kötü örnek:

```
if (yas >= 18) {
  std::cout << "Giriş yapabilir.";
}
```

İyi örnek:

```
if (yas >= 18) {
  std::cout << "Giriş yapabilir.";
}
```

Java örneği:

```
if (yas >= 18) {
  System.out.println("Giriş yapabilir.");
}
```

§6.12.2 Dosya ve Klasör Düzeni

Dosya ve klasör düzeni, projenin anlaşılmasını kolaylaştırır.

C++ projelerinde genellikle başlık ve kaynak dosyaları ayrılır:

```
include/
Kullanici.h
Siparis.h

src/
Kullanici.cpp
Siparis.cpp
```

Java projelerinde paket yapısı kullanılır:

```
src/main/java/
com/ornek/uygulama/domain/
com/ornek/uygulama/service/
com/ornek/uygulama/repository/
```

§6.12.3 Kod Stil Rehberi

Kod stil rehberi, ekip içinde ortak yazım kurallarını tanımlar. Örneğin:

- ◇ Sınıf isimleri büyük harfle başlamalıdır.
- ◇ Fonksiyon isimleri küçük harfle başlamalıdır.
- ◇ Sabitler büyük harflerle yazılmalıdır.
- ◇ Satır uzunlukları makul sınırda tutulmalıdır.

C++ için `clang-format`, Java için `google-java-format` gibi araçlar biçimlendirmeyi otomatikleştirebilir.

§6.12.4 C++ ve Java İçin Stil Farkları

C++ ve Java topluluklarının isimlendirme gelenekleri farklıdır.

C++ örnekleri:

```
class KullaniciHesabi {};
int toplamTutarHesapla();
constexpr int MAKSIMUM_DENEME_SAYISI = 3;
```

Java örnekleri:

```
class KullaniciHesabi { }
int toplamTutarHesapla() { return 0; }
static final int MAKSIMUM_DENEME_SAYISI = 3;
```

Java'da paket isimleri genellikle küçük harfle yazılır:

```
package com.ornek.kutuphane.servis;
```

6.13 Yeniden Düzenleme (Refactoring) ve Kod Kokuları

Yeniden düzenleme, kodun dış davranışını değiştirmeden iç yapısını iyileştirme işlemidir. Amaç, kodu daha okunabilir, daha test edilebilir ve daha sürdürülebilir hale getirmektir.

§6.13.1 Refactoring Nedir?

Refactoring, çalışan kodu daha iyi hale getirme işlemidir. Bu işlem sırasında kodun kullanıcıya görünen davranışı değişmemelidir.

Örneğin uzun bir fonksiyon küçük fonksiyonlara ayrılabilir. Bu işlem davranışı değiştirmez, yalnızca kodun yapısını iyileştirir.

§6.13.2 Yaygın Refactoring Teknikleri

Yaygın refactoring teknikleri şunlardır:

- ◇ Extract Method
- ◇ Extract Class
- ◇ Rename Variable
- ◇ Move Method
- ◇ Replace Conditional with Polymorphism
- ◇ Introduce Parameter Object

Örnek olarak Extract Method tekniği:

Kötü C++ örneği:

```
void faturaYazdir(double tutar) {
  double kdv = tutar * 0.20;
  double toplam = tutar + kdv;

  std::cout << "Toplam: " << toplam << std::endl;
}
```

Daha iyi hali:

```
double kdvliToplamHesapla(double tutar) {
  return tutar + (tutar * 0.20);
}

void faturaYazdir(double tutar) {
  std::cout << "Toplam: " << kdvliToplamHesapla(tutar) << std::endl;
}
```

Java karşılığı:

```
double kdvliToplamHesapla(double tutar) {
  return tutar + (tutar * 0.20);
}

void faturaYazdir(double tutar) {
  System.out.println("Toplam: " + kdvliToplamHesapla(tutar));
}
```

§6.13.3 Kod Kokuları

Kod kokusu, kodda doğrudan hata olmayan ancak gelecekte problem çıkarma ihtimali yüksek olan tasarım işaretidir.

Yaygın kod kokuları:

- Uzun yöntemler
- Büyük sınıflar
- Tekrarlayan kod
- Uzun parametre listeleri
- Aşırı `if-else` veya `switch` kullanımı
- Tanrı sınıf
- Veri sınıfı
- Boş `catch` blokları

Aşırı koşul kullanımı örneği:

```
double indirimHesapla(std::string tur, double tutar) {
  if (tur == "ogrenci") return tutar * 0.10;
  if (tur == "emekli") return tutar * 0.15;
  if (tur == "personel") return tutar * 0.20;

  return 0;
}
```

Bu yapı OCP ihlaline yol açabilir. Çok biçimlilik ile iyileştirilebilir.

§6.13.4 Refactoring Öncesi Testin Önemi

Refactoring işleminden önce testlerin bulunması büyük önem taşır. Testler, kodun davranışının değişmediğini doğrular. Test olmadan yapılan refactoring, davranışın bozulmasına neden olabilir.

Refactoring sırasında şu adımlar izlenmelidir:

1. Önce mevcut davranış için test yazılır.
2. Küçük bir yapısal iyileştirme yapılır.
3. Testler çalıştırılır.
4. Testler başarılıysa sonraki adıma geçilir.

6.14 Performans ve Temiz Kod Dengesi

Temiz kod yazmak, performansı önemsememek anlamına gelmez. Benzer şekilde performanslı kod yazmak da kodu okunamaz hale getirmek anlamına gelmemelidir. İyi yazılım geliştirme pratiğinde

önce doğru, anlaşılır ve test edilebilir kod yazılır; ardından gerçekten ihtiyaç varsa ölçüm yapılarak performans iyileştirmesi uygulanır.

Performans ile temiz kod arasındaki dengeyi kurarken şu ilkelere dikkat edilmelidir:

- ◇ Önce doğru çalışan kod yazılmalıdır.
- ◇ Kodun okunabilirliği gereksiz yere bozulmamalıdır.
- ◇ Performans problemi varsayımla değil, ölçümle belirlenmelidir.
- ◇ Küçük kod hilelerinden önce algoritma ve veri yapısı seçimi gözden geçirilmelidir.
- ◇ Optimizasyon yapılan kod mutlaka testlerle korunmalıdır.

Birçok yazılım projesinde performans sorunları, birkaç satırlık mikro optimizasyondan değil, yanlış algoritma seçiminden, gereksiz veritabanı çağrılarından, yanlış veri yapısından veya kaynakların hatalı yönetilmesinden kaynaklanır.

§6.14.1 Erken Optimizasyon

Erken optimizasyon, gerçek bir performans problemi ölçülmeden kodu hızlandırmaya çalışmaktır. Bu yaklaşım çoğu zaman kodu karmaşıklaştırır, okunabilirliği azaltır ve hata ihtimalini artırır.

Donald Knuth'un sıkça alıntılanan sözü bu konuda yol göstericidir: "Erken optimizasyon bütün kötülüklerin köküdür." Buradaki amaç performansı önemsiz görmek değil, kanıtlanmamış performans kaygısıyla kodu bozmanın yanlış olduğunu vurgulamaktır.

Aşağıdaki C++ örneğinde basit ve okunabilir bir toplam alma işlemi verilmiştir:

```cpp
#include <vector>

int toplamHesapla(const std::vector<int>& sayilar) {
  int toplam = 0;

  for (int sayi : sayilar) {
    toplam += sayi;
  }

  return toplam;
}
```

Kod 6.37: C++ Dilinde Okunabilir Toplama

Aynı işlem, gereksiz yere karmaşık hale getirilebilir:

```cpp
#include <vector>

int toplamHesapla(const std::vector<int>& sayilar) {
  int toplam = 0;
  const int* ptr = sayilar.data();
  const int* son = ptr + sayilar.size();

  while (ptr != son) {
    toplam += *ptr;
    ++ptr;
  }

  return toplam;
}
```

Kod 6.38: Gereksiz Karmaşık Toplama

İkinci kod bazı durumlarda düşük seviyeli kontrol sağlıyor gibi görünebilir; ancak çoğu uygulama için ilk kod yeterince hızlı ve çok daha okunabilirdir. Derleyiciler çoğu zaman bu tür basit döngüleri zaten iyi optimize eder.

Java dilinde de benzer durum geçerlidir:

```java
import java.util.List;

class Hesaplayici {
  int toplamHesapla(List<Integer> sayilar) {
    int toplam = 0;

    for (int sayi : sayilar) {
      toplam += sayi;
    }

    return toplam;
  }
}
```

Kod 6.39: Java Dilinde Okunabilir Toplama

Gereksiz karmaşıklıktan kaçınılmalıdır. Örneğin okunabilir bir döngüyü yalnızca "daha hızlı olabilir" düşüncesiyle anlaşılması zor bir yapıya dönüştürmek doğru değildir.

> **Ölçmeden Optimize Etmeyin**
> Performans iyileştirmesi yapılmadan önce gerçekten bir darboğaz olup olmadığı ölçülmelidir. C++ tarafında profil araçları, Java tarafında profiler araçları kullanılabilir. Ölçüm yapılmadan yapılan optimizasyonlar çoğu zaman yanlış yere odaklanır.

Erken optimizasyondan kaçınmak için şu adımlar izlenebilir:

1. Önce doğru ve temiz kod yazılır.
2. Kod testlerle güvence altına alınır.
3. Performans ölçümü yapılır.
4. Darboğaz tespit edilir.
5. Yalnızca gerekli bölüm optimize edilir.
6. Testler yeniden çalıştırılır.

§6.14.2 Algoritmik Verimlilik

Performansın en önemli belirleyicilerinden biri algoritma ve veri yapısı seçimidir. Yanlış veri yapısı seçildiğinde, kod ne kadar mikro düzeyde optimize edilirse edilsin istenen performans elde edilemeyebilir.

Örneğin bir elemanın bir koleksiyon içinde bulunup bulunmadığını sık sık kontrol ediyorsak, liste yerine küme kullanmak daha doğru olabilir.

C++ dilinde `std::vector` içinde arama yapmak doğrusal zaman alır:

```cpp
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
  std::vector<int> sayilar = {1, 2, 3, 4, 5};

  bool varMi = std::find(sayilar.begin(), sayilar.end(), 5) != sayilar.end();

  if (varMi) {
    std::cout << "Sayi bulundu." << std::endl;
  }
}
```

Kod 6.40: C++ Dilinde Vector İçinde Arama

Bu arama işlemi `O(n)` karmaşıklığa sahiptir. Eleman sayısı arttıkça arama süresi de artar. Eğer sık arama yapılacaksa `std::unordered_set` daha uygun olabilir:

```cpp
#include <unordered_set>
#include <iostream>
```

```cpp
int main() {
  std::unordered_set<int> sayilar = {1, 2, 3, 4, 5};

  bool varMi = sayilar.find(5) != sayilar.end();

  if (varMi) {
    std::cout << "Sayi bulundu." << std::endl;
  }
}
```

Kod 6.41: C++ Dilinde unordered_set ile Arama

Java dilinde de `List` içinde arama doğrusal zaman alır:

```java
import java.util.List;

public class ListeArama {
  public static void main(String[] args) {
    List<Integer> sayilar = List.of(1, 2, 3, 4, 5);

    if (sayilar.contains(5)) {
      System.out.println("Sayi bulundu.");
    }
  }
}
```

Kod 6.42: Java Dilinde List İçinde Arama

Sık arama yapılacaksa `HashSet` tercih edilebilir:

```java
import java.util.Set;

public class KumeArama {
  public static void main(String[] args) {
    Set<Integer> sayilar = Set.of(1, 2, 3, 4, 5);

    if (sayilar.contains(5)) {
      System.out.println("Sayi bulundu.");
    }
  }
}
```

Kod 6.43: Java Dilinde HashSet ile Arama

Bu örnek, performansın çoğu zaman daha karmaşık kod yazmakla değil, doğru veri yapısını seçmekle iyileştirildiğini gösterir.

Bir başka örnek olarak iç içe döngüler verilebilir. Aşağıdaki C++ kodunda iki liste karşılaştırılır:

```cpp
#include <vector>
#include <iostream>

bool ortakElemanVarMi(const std::vector<int>& a, const std::vector<int>& b) {
  for (int x : a) {
    for (int y : b) {
      if (x == y) {
        return true;
      }
    }
  }

  return false;
}
```

Kod 6.44: C++ Dilinde Verimsiz İç İçe Döngü

Bu kodun karmaşıklığı yaklaşık olarak `O(n*m)` düzeyindedir. Daha iyi bir yaklaşım, listelerden

birini kümeye dönüştürmektir:

```cpp
#include <vector>
#include <unordered_set>

bool ortakElemanVarMi(const std::vector<int>& a, const std::vector<int>& b) {
  std::unordered_set<int> kume(a.begin(), a.end());

  for (int eleman : b) {
    if (kume.find(eleman) != kume.end()) {
      return true;
    }
  }

  return false;
}
```

Kod 6.45: C++ Dilinde Daha Verimli Ortak Eleman Arama

Java karşılığı:

```java
import java.util.HashSet;
import java.util.List;
import java.util.Set;

class OrtakElemanKontrolu {
  boolean ortakElemanVarMi(List<Integer> a, List<Integer> b) {
    Set<Integer> kume = new HashSet<>(a);

    for (int eleman : b) {
      if (kume.contains(eleman)) {
        return true;
      }
    }

    return false;
  }
}
```

Kod 6.46: Java Dilinde Daha Verimli Ortak Eleman Arama

Burada kod hem okunabilir kalmış hem de algoritmik olarak daha verimli hale gelmiştir.

§6.14.3 Bellek ve Kaynak Yönetimi

Performans yalnızca işlemci zamanı ile ilgili değildir. Bellek kullanımı, dosya tanıtıcıları, ağ bağlantıları, veritabanı bağlantıları ve kilitler gibi kaynaklar da performansı ve sistem kararlılığını etkiler.

C++ dilinde kaynak yönetimi özellikle önemlidir. Çünkü C++ dilinde dinamik olarak ayrılan bellek ve dış kaynaklar programcı tarafından dikkatli yönetilmelidir. Modern C++'ta ham göstericiler yerine RAII ve akıllı göstericiler kullanılmalıdır.

Kötü C++ örneği:

```cpp
class Rapor {
  public:
  void yazdir() {
    // Rapor yazdırma işlemi
  }
};

void raporOlustur() {
  Rapor* rapor = new Rapor();
  rapor->yazdir();

  // delete unutulursa bellek sızıntısı oluşur.
```

```
}
```

Kod 6.47: C++ Dilinde Bellek Sızıntısı Riski

Daha iyi yaklaşım `std::unique_ptr` kullanmaktır:

```
#include <memory>

class Rapor {
  public:
  void yazdir() {
    // Rapor yazdırma işlemi
  }
};

void raporOlustur() {
  std::unique_ptr<Rapor> rapor = std::make_unique<Rapor>();
  rapor->yazdir();

  // Fonksiyon bitince bellek otomatik serbest bırakılır.
}
```

Kod 6.48: C++ Dilinde unique_ptr Kullanımı

Hatta mümkünse dinamik bellek ayırmaya hiç gerek olmadan yığın üzerinde nesne oluşturulabilir:

```
class Rapor {
  public:
  void yazdir() {
    // Rapor yazdırma işlemi
  }
};

void raporOlustur() {
  Rapor rapor;
  rapor.yazdir();
}
```

Kod 6.49: C++ Dilinde Yığın Üzerinde Nesne Kullanımı

Java dilinde bellek yönetimi çöp toplayıcı tarafından yapılır. Ancak bu durum, dış kaynakların otomatik ve zamanında kapatılacağı anlamına gelmez. Dosya, ağ ve veritabanı bağlantıları gibi kaynaklar mutlaka kapatılmalıdır.

Java'da doğru yaklaşım `try-with-resources` kullanmaktır:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class DosyaOkuma {
  public static void main(String[] args) throws IOException {
    try (BufferedReader reader = Files.newBufferedReader(Path.of("veri.txt"))) {
      String satir = reader.readLine();
      System.out.println(satir);
    }
  }
}
```

Kod 6.50: Java Dilinde try-with-resources Kullanımı

Bu yapı sayesinde `BufferedReader`, bloktan çıkıldığında otomatik olarak kapatılır.

C++ dilinde dosya işlemlerinde de RAII yaklaşımı uygulanır:

```
#include <fstream>
```

```cpp
#include <iostream>
#include <string>

void dosyaOku() {
  std::ifstream dosya("veri.txt");

  if (!dosya.is_open()) {
    throw std::runtime_error("Dosya açılamadı.");
  }

  std::string satir;
  std::getline(dosya, satir);

  std::cout << satir << std::endl;

  // dosya nesnesi kapsamdan çıkınca otomatik kapanır.
}
```

Kod 6.51: C++ Dilinde RAII ile Dosya Yönetimi

Bellek ve kaynak yönetiminde temel kural şudur: Kaynağı alan nesne, kaynağı güvenli biçimde serbest bırakmalıdır. C++ tarafında RAII ve akıllı göstericiler, Java tarafında ise try-with-resources bu yaklaşımın temel araçlarıdır.

6.15 Güvenli Kodlama Standartları

Güvenli kodlama, yazılımın kötü niyetli veya hatalı girdiler karşısında güvenilir biçimde çalışmasını sağlamayı amaçlar. Temiz kod yalnızca okunabilir değil, aynı zamanda güvenli de olmalıdır.

Güvenli kodlama şu konuları kapsar:

- ◇ Kullanıcı girdilerinin doğrulanması
- ◇ Taşma ve sınır hatalarının önlenmesi
- ◇ Hassas verilerin korunması
- ◇ Hata mesajlarında gereksiz teknik bilgi sızdırılmaması
- ◇ Kaynakların güvenli yönetilmesi

§6.15.1 Girdi Doğrulama

Kullanıcıdan, dosyadan, ağdan veya dış sistemlerden gelen hiçbir veri doğrudan güvenilir kabul edilmemelidir. Bütün girdiler kullanılmadan önce doğrulanmalıdır.

C++ örneği:

```cpp
#include <iostream>
#include <stdexcept>

int yasOku(int yas) {
  if (yas < 0 || yas > 120) {
    throw std::invalid_argument("Yaş değeri 0 ile 120 arasında olmalıdır.");
  }

  return yas;
}

int main() {
  try {
    int yas = yasOku(25);
    std::cout << "Yaş: " << yas << std::endl;
  } catch (const std::exception& e) {
    std::cout << "Hata: " << e.what() << std::endl;
  }
}
```

Kod 6.52: C++ Dilinde Girdi Doğrulama

Java örneği:

```
class KullaniciDogrulayici {
  int yasDogrula(int yas) {
    if (yas < 0 || yas > 120) {
      throw new IllegalArgumentException("Yaş değeri 0 ile 120 arasında olmalıdır.");
    }

    return yas;
  }
}
```

Kod 6.53: Java Dilinde Girdi Doğrulama

E-posta gibi biçim gerektiren verilerde de temel doğrulama yapılmalıdır:

```
class EpostaDogrulayici {
  void dogrula(String eposta) {
    if (eposta == null || !eposta.contains("@")) {
      throw new IllegalArgumentException("Geçersiz e-posta adresi.");
    }
  }
}
```

Kod 6.54: Java Dilinde Basit E-posta Doğrulama

Girdi doğrulama yalnızca kullanıcı arayüzünde değil, iş katmanında da yapılmalıdır. Çünkü aynı iş kuralı farklı arayüzlerden çağrılabilir.

§6.15.2 Taşma ve Sınır Hataları

Taşma hatası, bir sayısal işlemin sonucunun veri tipinin taşıyabileceği sınırı aşmasıdır. C++ dilinde işaretli tam sayı taşması tanımsız davranışa neden olur. Java dilinde ise taşma sessizce gerçekleşir; istisna fırlatılmaz.

C++ dilinde basit taşma örneği:

```
#include <iostream>
#include <limits>

int main() {
  int maksimum = std::numeric_limits<int>::max();
  int sonuc = maksimum + 1; // Tanımsız davranış

  std::cout << sonuc << std::endl;
}
```

Kod 6.55: C++ Dilinde Taşma Riski

Güvenli yaklaşım, işlemden önce sınır kontrolü yapmaktır:

```
#include <limits>
#include <stdexcept>

int guvenliTopla(int a, int b) {
  if (b > 0 && a > std::numeric_limits<int>::max() - b) {
    throw std::overflow_error("Tam sayı taşması oluştu.");
  }

  if (b < 0 && a < std::numeric_limits<int>::min() - b) {
    throw std::underflow_error("Tam sayı alttan taşması oluştu.");
  }

  return a + b;
}
```

Kod 6.56: C++ Dilinde Güvenli Toplama

Java dilinde taşmayı denetlemek için `Math.addExact()` kullanılabilir:

```
class GuvenliAritmetik {
  int guvenliTopla(int a, int b) {
    return Math.addExact(a, b);
  }
}
```

Kod 6.57: Java Dilinde Güvenli Toplama

Eğer taşma oluşursa Java `ArithmeticException` fırlatır.

Dizi ve liste sınırları da güvenlik açısından önemlidir. C++ dilinde `operator[]` sınır kontrolü yapmaz:

```
#include <vector>
#include <iostream>

int main() {
  std::vector<int> sayilar = {1, 2, 3};

  std::cout << sayilar[10] << std::endl; // Tanımsız davranış
}
```

Kod 6.58: C++ Dilinde Sınır Kontrolsüz Erişim

Daha güvenli yaklaşım `at()` kullanmaktır:

```
#include <vector>
#include <iostream>
#include <stdexcept>

int main() {
  std::vector<int> sayilar = {1, 2, 3};

  try {
    std::cout << sayilar.at(10) << std::endl;
  } catch (const std::out_of_range& e) {
    std::cout << "İndis sınır dışında." << std::endl;
  }
}
```

Kod 6.59: C++ Dilinde Sınır Kontrollü Erişim

Java dilinde sınır aşımı otomatik olarak `IndexOutOfBoundsException` fırlatır:

```
import java.util.List;

public class SinirAsimi {
  public static void main(String[] args) {
    List<Integer> sayilar = List.of(1, 2, 3);

    System.out.println(sayilar.get(10)); // IndexOutOfBoundsException
  }
}
```

Kod 6.60: Java Dilinde Sınır Aşımı

§6.15.3 Hassas Verilerin Korunması

Parola, erişim anahtarı, kimlik numarası, kredi kartı bilgisi ve oturum anahtarı gibi veriler hassas verilerdir. Bu veriler düz metin olarak saklanmamalı, loglara yazılmamalı ve hata mesajlarında gösterilmemelidir.

Kötü Java örneği:

```
class GirisServisi {
  void girisYap(String kullaniciAdi, String parola) {
    System.out.println("Giriş denemesi: " + kullaniciAdi + " parola=" + parola);
```

```
    }
}
```

Kod 6.61: Java Dilinde Hassas Veriyi Loglama Hatası

Daha güvenli yaklaşım:

```
class GirisServisi {
  void girisYap(String kullaniciAdi, String parola) {
    System.out.println("Giriş denemesi: " + kullaniciAdi);

    // Parola loglanmaz.
  }
}
```

Kod 6.62: Java Dilinde Hassas Veriyi Gizleme

C++ örneği:

```
#include <iostream>
#include <string>

class GirisServisi {
  public:
  void girisYap(const std::string& kullaniciAdi, const std::string& parola) {
    std::cout << "Giriş denemesi: " << kullaniciAdi << std::endl;

    // Parola ekrana veya log dosyasına yazılmamalıdır.
  }
};
```

Kod 6.63: C++ Dilinde Hassas Veriyi Loglamamak

Parolalar veritabanında düz metin olarak saklanmamalıdır. Gerçek sistemlerde parola saklamak için güvenli özetleme ve tuzlama (salt) yaklaşımları kullanılır. Bu konu güvenlik mühendisliğinin ayrı bir uzmanlık alanıdır.

§6.15.4 Hata Mesajlarında Bilgi Sızdırmamak

Hata mesajları kullanıcıya gerekli bilgiyi vermeli, ancak sistemin iç yapısını açığa çıkarmamalıdır. Teknik ayrıntılar loglarda tutulabilir; kullanıcıya ise anlaşılır ve güvenli mesaj gösterilmelidir.

Kötü Java örneği:

```
try {
  kullaniciKaydet();
} catch (Exception e) {
  System.out.println("SQL Hatası: " + e.getMessage());
}
```

Kod 6.64: Java Dilinde Bilgi Sızdıran Hata Mesajı

Bu yaklaşım kullanıcıya veritabanı yapısı hakkında bilgi sızdırabilir. Daha iyi yaklaşım:

```
try {
  kullaniciKaydet();
} catch (Exception e) {
  // Teknik ayrıntı loglanır.
  System.err.println("Teknik hata: " + e.getMessage());

  // Kullanıcıya genel mesaj gösterilir.
  System.out.println("İşlem şu anda gerçekleştirilemiyor.");
}
```

Kod 6.65: Java Dilinde Güvenli Hata Mesajı

C++ örneği:

```
try {
  kullaniciKaydet();
} catch (const std::exception& e) {
  std::cerr << "Teknik hata: " << e.what() << std::endl;
  std::cout << "İşlem şu anda gerçekleştirilemiyor." << std::endl;
}
```

Kod 6.66: C++ Dilinde Güvenli Hata Mesajı

Hata mesajları ikiye ayrılmalıdır:

- ◇ **Kullanıcı mesajı**: Basit, anlaşılır ve güvenli olmalıdır.
- ◇ **Teknik log**: Geliştiriciye hata ayıklamada yardımcı olacak ayrıntıları içermelidir.

6.16 Dokümantasyon ve API Tasarımı

Dokümantasyon, yazılımın nasıl kullanılacağını, nasıl kurulacağını ve hangi sorumluluklara sahip olduğunu açıklar. Temiz kod, iyi dokümantasyon ihtiyacını azaltabilir; ancak tamamen ortadan kaldırmaz.

API tasarımı ise bir sınıfın, modülün veya servisin dış dünyaya sunduğu arayüzün tasarlanmasıdır. İyi API tasarımı, kullanıcıyı doğru kullanıma yönlendirir ve hatalı kullanımı zorlaştırır.

§6.16.1 Kod İçi Dokümantasyon

Kod içi dokümantasyon, sınıfların ve yöntemlerin nasıl kullanılacağını açıklamak için kullanılır. Ancak her satırın yorumlanması gerekmez. Kodun kendisi açık olmalı, yorumlar ise niyet, kısıt veya önemli kararları açıklamalıdır.

C++ tarafında Doxygen biçiminde açıklama yazılabilir:

```
/**
* @brief İki sayının toplamını hesaplar.
*
* @param a Birinci sayı.
* @param b İkinci sayı.
* @return İki sayının toplamı.
*/
int topla(int a, int b) {
  return a + b;
}
```

Kod 6.67: C++ Dilinde Doxygen Yorum Örneği

Java tarafında Javadoc kullanılır:

```
/**
* İki sayının toplamını hesaplar.
*
* @param a birinci sayı
* @param b ikinci sayı
* @return iki sayının toplamı
*/
int topla(int a, int b) {
  return a + b;
}
```

Kod 6.68: Java Dilinde Javadoc Örneği

Yorumlar özellikle şu durumlarda faydalıdır:

- ◇ Karmaşık iş kuralının nedenini açıklamak
- ◇ Algoritmanın varsayımlarını belirtmek
- ◇ Dış sistemlerle ilgili kısıtları açıklamak
- ◇ Güvenlik veya performans gerekçesini belirtmek

§6.16.2 Dış Dokümantasyon

Dış dokümantasyon, projenin kod dışındaki açıklamalarını kapsar. Örneğin:

- README dosyası
- Kurulum adımları
- Kullanım örnekleri
- API belgeleri
- Mimari karar kayıtları
- Sürüm notları

Bir README dosyasında genellikle şu başlıklar bulunmalıdır:

```
# Proje Adı

## Amaç
Bu proje ne yapar?

## Kurulum
Projeyi çalıştırmak için gerekli adımlar.

## Kullanım
Örnek komutlar veya kod parçaları.

## Test
Testlerin nasıl çalıştırılacağı.

## Katkı
Projeye katkı kuralları.
```

Dış dokümantasyon, projeye yeni katılan geliştiricilerin hızlı uyum sağlamasına yardımcı olur.

§6.16.3 İyi API Tasarımı

İyi API, yanlış kullanımı zor, doğru kullanımı kolay hale getirir. API tasarımında isimlendirme, parametre yapısı, hata yönetimi ve dönüş değerleri önemlidir.

Kötü C++ örneği:

```
void islem(bool detayli, bool kaydet, bool mailGonder);
```

Kod 6.69: C++ Dilinde Belirsiz API

Bu yöntem çağrıldığında parametrelerin ne anlama geldiği açık değildir:

```
islem(true, false, true);
```

Daha iyi yaklaşım, niyeti açık yöntemler veya parametre nesnesi kullanmaktır:

```
class RaporSecenekleri {
  public:
  bool detayli;
  bool kaydet;
  bool mailGonder;
};

void raporOlustur(const RaporSecenekleri& secenekler);
```

Kod 6.70: C++ Dilinde Daha Açık API

Java örneği:

```
class RaporSecenekleri {
  boolean detayli;
  boolean kaydet;
  boolean mailGonder;
}
```

```
void raporOlustur(RaporSecenekleri secenekler) {
  // Rapor oluşturma işlemi
}
```

Kod 6.71: Java Dilinde Daha Açık API

Daha iyi bir API tasarımı için:

- ◇ Yöntem isimleri açık olmalıdır.
- ◇ Boolean parametrelerden kaçınılmalıdır.
- ◇ Geçersiz nesne oluşturulmasına izin verilmemelidir.
- ◇ Hata durumları açıkça belirtilmelidir.
- ◇ Geri dönüş değerleri belirsiz olmamalıdır.

6.17 Sürüm Kontrolü ve Kod Kalitesi Süreci

Sürüm kontrolü, yazılım geliştirme sürecinde kodun geçmişini izlemeyi, farklı geliştiricilerin birlikte çalışmasını ve hatalı değişikliklerden geri dönmeyi sağlar. Günümüzde en yaygın kullanılan sürüm kontrol sistemi Git'tir.

Kod kalitesi ise yalnızca bireysel yazım alışkanlıklarına bırakılmamalıdır. Ekip içinde kod inceleme, otomatik test, statik analiz ve sürekli entegrasyon gibi süreçlerle desteklenmelidir.

§6.17.1 Git Kullanım Disiplini

Git kullanırken küçük, anlamlı ve izlenebilir commitler oluşturmak önemlidir. Bir commit tek bir mantıksal değişikliği temsil etmelidir.

Kötü commit mesajları:

```
duzeltme
son hali
update
hata giderildi
```

Daha iyi commit mesajları:

```
Kullanici yas dogrulamasi eklendi
Siparis servisine birim testler eklendi
Fatura hesaplama hatasi giderildi
Bildirim gonderici arayuzu eklendi
```

İyi bir Git disiplini için:

- ◇ Commitler küçük tutulmalıdır.
- ◇ Commit mesajları yapılan işi açıklamalıdır.
- ◇ Ana dal doğrudan bozulmamalıdır.
- ◇ Yeni özellikler ayrı dallarda geliştirilmelidir.
- ◇ Kod birleştirmeden önce testler çalıştırılmalıdır.

Basit bir Git çalışma akışı:

```
git checkout -b ozellik/kullanici-dogrulama
git add .
git commit -m "Kullanici yas dogrulamasi eklendi"
git push origin ozellik/kullanici-dogrulama
```

§6.17.2 Kod İnceleme

Kod inceleme (code review), bir geliştiricinin yazdığı kodun başka geliştiriciler tarafından incelenmesidir. Amaç kişiyi eleştirmek değil, kod kalitesini artırmaktır.

Kod incelemede dikkat edilebilecek noktalar:

- ◇ Kod okunabilir mi?
- ◇ İsimlendirmeler anlamlı mı?
- ◇ Sınıf ve yöntemler tek sorumluluğa sahip mi?
- ◇ Gereksiz tekrar var mı?
- ◇ Hata yönetimi doğru mu?
- ◇ Testler yeterli mi?
- ◇ Güvenlik riski var mı?

Kod inceleme kültüründe dikkat edilmesi gereken önemli nokta, yorumların kişiye değil koda yönelik olmasıdır. Örneğin:

- ◇ Kötü ifade: "Bunu yanlış yazmışsın."
- ◇ Daha iyi ifade: "Bu yöntemin iki sorumluluğu var gibi görünüyor; ayrı yöntemlere bölünebilir."

§6.17.3 Sürekli Entegrasyon

Sürekli entegrasyon (Continuous Integration - CI), kod değişikliklerinin otomatik olarak derlenmesi, test edilmesi ve kalite kontrollerinden geçirilmesi sürecidir.

Bir CI sürecinde genellikle şu adımlar bulunur:

1. Kod depoya gönderilir.
2. Proje otomatik olarak derlenir.
3. Birim testler çalıştırılır.
4. Statik analiz araçları çalıştırılır.
5. Kod biçimlendirme kontrol edilir.
6. Başarılıysa değişiklik birleştirilebilir.

C++ projelerinde CI sürecinde şu araçlar kullanılabilir:

- ◇ `cmake`
- ◇ `ctest`
- ◇ `clang-tidy`
- ◇ `clang-format`
- ◇ Google Test veya Catch2

Java projelerinde ise şu araçlar yaygındır:

- ◇ Maven veya Gradle
- ◇ JUnit
- ◇ SpotBugs
- ◇ Checkstyle
- ◇ PMD

Sürekli entegrasyon, kod kalitesini kişisel dikkate bırakmaz; otomatik süreçlerle güvence altına alır.

6.18 Temiz Mimari İlkelerine Giriş

Temiz mimari, yazılım sistemlerinde iş kurallarını teknik ayrıntılardan ayırmayı amaçlayan bir mimari yaklaşımdır. Buradaki temel düşünce şudur: Bir yazılım sisteminin en değerli kısmı, veritabanı, kullanıcı arayüzü, dosya sistemi veya kullanılan framework değildir; asıl değer, sistemin çözmek istediği probleme ait iş kurallarıdır.

Örneğin bir kütüphane otomasyonunda en önemli şey hangi veritabanının kullanıldığı değil, kitabın nasıl ödünç verileceği, üyenin kaç kitap alabileceği, gecikme cezasının nasıl hesaplanacağı ve kitabın ne zaman iade edilmiş sayılacağıdır. Temiz mimari, bu iş kurallarını sistemin merkezine yerleştirir ve dış teknolojilere bağımlı olmaktan korur.

Temiz mimarinin temel amacı şunlardır:

- ◇ İş kurallarını teknik ayrıntılardan ayırmak.

- ◇ Sistemi test edilebilir hale getirmek.
- ◇ Veritabanı, arayüz veya framework değişikliklerinden iş kurallarını korumak.
- ◇ Bağımlılıkların yönünü kontrol altında tutmak.
- ◇ Kodun uzun vadede sürdürülebilir olmasını sağlamak.

Temiz mimaride bağımlılıklar dış katmanlardan iç katmanlara doğru olmalıdır. Yani dış dünya, iş kurallarına bağımlı olabilir; ancak iş kuralları dış dünyaya bağımlı olmamalıdır.

§6.18.1 Mimari Neden Önemlidir?

Küçük programlarda mimari çok önemli değilmiş gibi görünebilir. Ancak sistem büyüdükçe sınıflar, modüller, katmanlar ve bağımlılıklar artar. Eğer mimari bilinçli kurulmazsa sistem zamanla değiştirilemez, test edilemez ve sürdürülemez hale gelir.

Kötü mimariye sahip sistemlerde genellikle şu sorunlar görülür:

- ◇ İş kuralları kullanıcı arayüzü kodunun içine karışır.
- ◇ Veritabanı kodu her sınıfa dağılır.
- ◇ Sınıflar doğrudan somut sınıflara bağımlı hale gelir.
- ◇ Test yazmak zorlaşır.
- ◇ Küçük bir değişiklik birçok dosyayı etkiler.
- ◇ Yeni özellik eklemek riskli hale gelir.

Aşağıdaki Java örneği, kötü mimariye sahip bir sınıfı göstermektedir:

```
class OduncController {
  public void kitapOduncVer(String uyeNo, String kitapNo) {
    // Kullanıcıdan gelen veri kontrol ediliyor
    if (uyeNo == null || kitapNo == null) {
      throw new IllegalArgumentException("Üye no ve kitap no boş olamaz.");
    }

    // Veritabanı işlemi burada yapılıyor
    System.out.println("Veritabanından üye bilgisi okundu.");
    System.out.println("Veritabanından kitap bilgisi okundu.");

    // İş kuralı burada uygulanıyor
    System.out.println("Üyenin ödünç kitap sayısı kontrol edildi.");

    // Veritabanına kayıt burada yapılıyor
    System.out.println("Ödünç verme kaydı veritabanına yazıldı.");

    // Bildirim burada gönderiliyor
    System.out.println("Üyeye e-posta gönderildi.");
  }
}
```

Kod 6.72: Java Dilinde Kötü Mimari Örneği

Bu sınıf çok fazla sorumluluğa sahiptir. Kullanıcı isteğini almak, veri doğrulamak, veritabanına erişmek, iş kuralı uygulamak ve bildirim göndermek aynı sınıfta toplanmıştır. Bu yapı hem SRP ilkesini hem de DIP ilkesini ihlal eder.

C++ dilinde benzer kötü tasarım aşağıdaki gibi gösterilebilir:

```
#include <iostream>
#include <stdexcept>
#include <string>

class OduncController {
  public:
  void kitapOduncVer(const std::string& uyeNo, const std::string& kitapNo) {
    if (uyeNo.empty() || kitapNo.empty()) {
      throw std::invalid_argument("Üye no ve kitap no boş olamaz.");
    }
```

```
    std::cout << "Veritabanından üye bilgisi okundu." << std::endl;
    std::cout << "Veritabanından kitap bilgisi okundu." << std::endl;

    std::cout << "Üyenin ödünç kitap sayısı kontrol edildi." << std::endl;

    std::cout << "Ödünç verme kaydı veritabanına yazıldı." << std::endl;

    std::cout << "Üyeye e-posta gönderildi." << std::endl;
  }
};
```

Kod 6.73: C++ Dilinde Kötü Mimari Örneği

Bu örneklerde asıl sorun, bütün sorumlulukların tek sınıfta toplanmasıdır. İyi mimaride bu işlemler ayrı sınıflara ve katmanlara dağıtılır.

§6.18.2 İş Kurallarını Merkeze Almak

Temiz mimaride iş kuralları sistemin merkezinde yer alır. İş kuralları, uygulamanın hangi problemi çözdüğünü belirler. Bu nedenle iş kuralları mümkün olduğunca dış teknolojilerden bağımsız tasarlanmalıdır.

Bir kütüphane sistemi için bazı iş kuralları şunlar olabilir:

- ◇ Üye aynı anda en fazla beş kitap ödünç alabilir.
- ◇ Mevcut olmayan kitap ödünç verilemez.
- ◇ Geciken kitaplar için günlük ceza hesaplanır.
- ◇ Pasif üyeye kitap ödünç verilemez.

Bu kuralların veritabanı, dosya sistemi veya kullanıcı arayüzü içinde gömülü olmaması gerekir. Bunun yerine ayrı iş kuralı sınıfları oluşturulmalıdır.

C++ dilinde iş kuralı sınıfı aşağıdaki gibi yazılabilir:

```
#include <stdexcept>

class OduncKurali {
  public:
  void oduncAlabilirMi(int mevcutOduncSayisi,
  bool kitapMevcutMu,
  bool uyeAktifMi) const {
    if (!uyeAktifMi) {
      throw std::runtime_error("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw std::runtime_error("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw std::runtime_error("Üye en fazla beş kitap ödünç alabilir.");
    }
  }
};
```

Kod 6.74: C++ Dilinde İş Kuralı Sınıfı

Java karşılığı:

```
class OduncKurali {
  public void oduncAlabilirMi(int mevcutOduncSayisi,
  boolean kitapMevcutMu,
  boolean uyeAktifMi) {
    if (!uyeAktifMi) {
      throw new IllegalStateException("Pasif üyeye kitap ödünç verilemez.");
    }
```

```
    if (!kitapMevcutMu) {
      throw new IllegalStateException("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw new IllegalStateException("Üye en fazla beş kitap ödünç alabilir.");
    }
  }
}
```

Kod 6.75: Java Dilinde İş Kuralı Sınıfı

Bu sınıflar veritabanı veya kullanıcı arayüzü bilmez. Yalnızca iş kuralını uygular. Böylece birim test yazmak kolaylaşır.

C++ için basit test:

```
#include <cassert>

int main() {
  OduncKurali kural;

  kural.oduncAlabilirMi(2, true, true);

  // Buraya gelindiyse kural geçerli durumda çalışmıştır.
  assert(true);
}
```

Kod 6.76: C++ Dilinde İş Kuralı Testi

Java için basit test mantığı:

```
public class OduncKuraliTest {
  public static void main(String[] args) {
    OduncKurali kural = new OduncKurali();

    kural.oduncAlabilirMi(2, true, true);

    System.out.println("Test başarılı.");
  }
}
```

Kod 6.77: Java Dilinde İş Kuralı Testi

Bu yaklaşımda iş kuralını test etmek için veritabanı, web sunucusu veya kullanıcı arayüzü gerekmez.

§6.18.3 Dış Dünyayı Soyutlamak

Dış dünya; veritabanı, dosya sistemi, e-posta servisi, web servisi, kullanıcı arayüzü ve framework gibi teknik ayrıntılardan oluşur. Temiz mimaride iş kuralları bu ayrıntılara doğrudan bağımlı olmamalıdır.

Örneğin ödünç verme servisi doğrudan `MySqlOduncDeposu` sınıfına bağımlı olursa, veritabanı değiştiğinde iş kuralı sınıfı da etkilenir. Bunun yerine iş katmanı bir arayüze bağımlı olmalıdır.

C++ dilinde bu yapı şu şekilde kurulabilir:

```
#include <iostream>
#include <string>

class Odunc {
  public:
  std::string uyeNo;
  std::string kitapNo;

  Odunc(std::string pUyeNo, std::string pKitapNo)
```

```
    : uyeNo(pUyeNo), kitapNo(pKitapNo) {}
};

class IOduncDeposu {
  public:
  virtual void kaydet(const Odunc& odunc) = 0;
  virtual ~IOduncDeposu() = default;
};

class BellekOduncDeposu : public IOduncDeposu {
  public:
  void kaydet(const Odunc& odunc) override {
    std::cout << odunc.uyeNo << " üyesine "
    << odunc.kitapNo << " kitabı ödünç verildi." << std::endl;
  }
};
```

Kod 6.78: C++ Dilinde Depo Arayüzü ile Soyutlama

Java karşılığı:

```
class Odunc {
  private final String uyeNo;
  private final String kitapNo;

  public Odunc(String uyeNo, String kitapNo) {
    this.uyeNo = uyeNo;
    this.kitapNo = kitapNo;
  }

  public String getUyeNo() {
    return uyeNo;
  }

  public String getKitapNo() {
    return kitapNo;
  }
}

interface OduncDeposu {
  void kaydet(Odunc odunc);
}

class BellekOduncDeposu implements OduncDeposu {
  public void kaydet(Odunc odunc) {
    System.out.println(odunc.getUyeNo() + " üyesine "
    + odunc.getKitapNo() + " kitabı ödünç verildi.");
  }
}
```

Kod 6.79: Java Dilinde Depo Arayüzü ile Soyutlama

Bu tasarımda iş katmanı doğrudan veritabanı ayrıntısını bilmez. Sadece `IOduncDeposu` veya `OduncDeposu` arayüzünü bilir. Gerçek veritabanı, bellek içi depo veya test için sahte depo kullanılabilir.

C++ dilinde ödünç verme servisi:

```
class OduncServisi {
  private:
  IOduncDeposu& depo;
  OduncKurali& kural;

  public:
  OduncServisi(IOduncDeposu& pDepo, OduncKurali& pKural)
  : depo(pDepo), kural(pKural) {}
```

```
    void oduncVer(const std::string& uyeNo,
    const std::string& kitapNo,
    int mevcutOduncSayisi,
    bool kitapMevcutMu,
    bool uyeAktifMi) {
      kural.oduncAlabilirMi(mevcutOduncSayisi, kitapMevcutMu, uyeAktifMi);

      Odunc odunc(uyeNo, kitapNo);
      depo.kaydet(odunc);
    }
};
```

Kod 6.80: C++ Dilinde İş Servisi

Java dilinde ödünç verme servisi:

```
class OduncServisi {
  private final OduncDeposu depo;
  private final OduncKurali kural;

  public OduncServisi(OduncDeposu depo, OduncKurali kural) {
    this.depo = depo;
    this.kural = kural;
  }

  public void oduncVer(String uyeNo,
  String kitapNo,
  int mevcutOduncSayisi,
  boolean kitapMevcutMu,
  boolean uyeAktifMi) {
    kural.oduncAlabilirMi(mevcutOduncSayisi, kitapMevcutMu, uyeAktifMi);

    Odunc odunc = new Odunc(uyeNo, kitapNo);
    depo.kaydet(odunc);
  }
}
```

Kod 6.81: Java Dilinde İş Servisi

Bu tasarımda bağımlılık yönü önemlidir. `OduncServisi`, somut veritabanı sınıfına değil, arayüze bağımlıdır. Bu, temiz mimarinin temel kurallarından biridir.

§6.18.4 Temiz Mimari Katmanları

Temiz mimari genellikle içten dışa doğru katmanlardan oluşur. En içte iş kuralları, en dışta ise teknik ayrıntılar bulunur.

Basitleştirilmiş katmanlar şu şekilde düşünülebilir:

1. **Entities**: Temel iş nesneleri ve iş kuralları.
2. **Use Cases**: Uygulama senaryoları ve iş akışları.
3. **Interface Adapters**: Dış dünya ile iç katman arasında veri dönüştüren yapılar.
4. **Frameworks and Drivers**: Veritabanı, web arayüzü, dosya sistemi ve dış servisler.

Bu katmanlar aşağıdaki gibi özetlenebilir:

Katman	Görevi
Entities	En temel iş kurallarını ve alan nesnelerini içerir.
Use Cases	Sistemin uygulama senaryolarını yönetir.
Interface Adapters	DTO, Controller, Presenter, Repository adaptörleri gibi yapıları içerir.
Frameworks and Drivers	Veritabanı, web framework, dosya sistemi ve dış servislerdir.

6.18.4.1 Entities Katmanı

Entities katmanı, sistemin en temel iş kavramlarını içerir. Bu katman mümkün olduğunca dış teknolojilerden bağımsız olmalıdır.

C++ örneği:

```cpp
#include <stdexcept>
#include <string>

class Kitap {
  private:
  std::string kitapNo;
  std::string ad;
  bool mevcut;

  public:
  Kitap(std::string pKitapNo, std::string pAd)
  : kitapNo(pKitapNo), ad(pAd), mevcut(true) {
    if (pKitapNo.empty() || pAd.empty()) {
      throw std::invalid_argument("Kitap no ve ad boş olamaz.");
    }
  }

  bool mevcutMu() const {
    return mevcut;
  }

  void oduncVerildi() {
    if (!mevcut) {
      throw std::runtime_error("Kitap zaten ödünç verilmiş.");
    }

    mevcut = false;
  }

  void iadeEdildi() {
    mevcut = true;
  }
};
```

Kod 6.82: C++ Dilinde Entity Örneği

Java örneği:

```java
class Kitap {
  private final String kitapNo;
  private final String ad;
  private boolean mevcut;

  public Kitap(String kitapNo, String ad) {
    if (kitapNo == null || kitapNo.isBlank()) {
      throw new IllegalArgumentException("Kitap no boş olamaz.");
    }

    if (ad == null || ad.isBlank()) {
      throw new IllegalArgumentException("Kitap adı boş olamaz.");
    }

    this.kitapNo = kitapNo;
    this.ad = ad;
    this.mevcut = true;
  }

  public boolean mevcutMu() {
    return mevcut;
  }
```

```
  public void oduncVerildi() {
    if (!mevcut) {
      throw new IllegalStateException("Kitap zaten ödünç verilmiş.");
    }

    this.mevcut = false;
  }

  public void iadeEdildi() {
    this.mevcut = true;
  }
}
```

Kod 6.83: Java Dilinde Entity Örneği

Burada `Kitap` sınıfı veritabanı bilmez, dosya bilmez, web bilmez. Yalnızca kendi iş kuralını bilir.

6.18.4.2 Use Cases Katmanı

Use Case katmanı, sistemin belirli bir kullanım senaryosunu temsil eder. Örneğin kitap ödünç verme, kitap iade etme, üye kaydetme gibi işlemler bu katmanda yer alabilir.

C++ örneği:

```
class KitapOduncVerUseCase {
  private:
  IOduncDeposu& oduncDeposu;
  OduncKurali& oduncKurali;

  public:
  KitapOduncVerUseCase(IOduncDeposu& pDepo, OduncKurali& pKural)
  : oduncDeposu(pDepo), oduncKurali(pKural) {}

  void calistir(const std::string& uyeNo,
  const std::string& kitapNo,
  int mevcutOduncSayisi,
  bool kitapMevcutMu,
  bool uyeAktifMi) {
    oduncKurali.oduncAlabilirMi(mevcutOduncSayisi,
    kitapMevcutMu,
    uyeAktifMi);

    Odunc odunc(uyeNo, kitapNo);
    oduncDeposu.kaydet(odunc);
  }
};
```

Kod 6.84: C++ Dilinde Use Case Örneği

Java örneği:

```
class KitapOduncVerUseCase {
  private final OduncDeposu oduncDeposu;
  private final OduncKurali oduncKurali;

  public KitapOduncVerUseCase(OduncDeposu oduncDeposu,
  OduncKurali oduncKurali) {
    this.oduncDeposu = oduncDeposu;
    this.oduncKurali = oduncKurali;
  }

  public void calistir(String uyeNo,
  String kitapNo,
  int mevcutOduncSayisi,
  boolean kitapMevcutMu,
```

```
    boolean uyeAktifMi) {
      oduncKurali.oduncAlabilirMi(mevcutOduncSayisi,
      kitapMevcutMu,
      uyeAktifMi);

      Odunc odunc = new Odunc(uyeNo, kitapNo);
      oduncDeposu.kaydet(odunc);
    }
  }
```

Kod 6.85: Java Dilinde Use Case Örneği

Bu sınıf, "kitap ödünç verme" kullanım senaryosunu temsil eder. Kullanıcı arayüzü veya veritabanı ayrıntısı içermez.

6.18.4.3 Interface Adapters Katmanı

Interface Adapters katmanı, dış dünyadan gelen veriyi iç katmanın anlayacağı biçime dönüştürür. Örneğin web isteğinden gelen veri bir DTO ile alınabilir ve Use Case sınıfına aktarılabilir.

Java DTO örneği:

```
class OduncVerRequest {
  public String uyeNo;
  public String kitapNo;
}
```

Kod 6.86: Java Dilinde DTO Örneği

Controller benzeri basit yapı:

```
class OduncController {
  private final KitapOduncVerUseCase useCase;

  public OduncController(KitapOduncVerUseCase useCase) {
    this.useCase = useCase;
  }

  public void oduncVer(OduncVerRequest request) {
    useCase.calistir(
    request.uyeNo,
    request.kitapNo,
    2,
    true,
    true
    );
  }
}
```

Kod 6.87: Java Dilinde Adapter Örneği

C++ dilinde benzer bir adapter sınıfı:

```
class OduncVerRequest {
  public:
  std::string uyeNo;
  std::string kitapNo;
};

class OduncController {
  private:
  KitapOduncVerUseCase& useCase;

  public:
  explicit OduncController(KitapOduncVerUseCase& pUseCase)
  : useCase(pUseCase) {}

  void oduncVer(const OduncVerRequest& request) {
```

```
      useCase.calistir(
      request.uyeNo,
      request.kitapNo,
      2,
      true,
      true
      );
    }
};
```

Kod 6.88: C++ Dilinde Adapter Örneği

Burada controller, dış dünyadan gelen isteği use case sınıfına iletir. İş kuralı controller içinde değildir.

6.18.4.4 Frameworks and Drivers Katmanı

Bu katman sistemin en dış katmanıdır. Veritabanı, dosya sistemi, web framework, kullanıcı arayüzü, harici servisler bu katmanda yer alır. Bu katman değişebilir olmalıdır.

Örneğin önce bellek içi depo kullanılırken daha sonra veritabanı deposuna geçilebilir.

Java örneği:

```
class BellekOduncDeposu implements OduncDeposu {
  public void kaydet(Odunc odunc) {
    System.out.println("Bellek içine kaydedildi.");
  }
}

class VeritabaniOduncDeposu implements OduncDeposu {
  public void kaydet(Odunc odunc) {
    System.out.println("Veritabanına kaydedildi.");
  }
}
```

Kod 6.89: Java Dilinde Farklı Depo Gerçekleştirmeleri

C++ örneği:

```
class VeritabaniOduncDeposu : public IOduncDeposu {
  public:
  void kaydet(const Odunc& odunc) override {
    std::cout << "Veritabanına kaydedildi." << std::endl;
  }
};
```

Kod 6.90: C++ Dilinde Farklı Depo Gerçekleştirmeleri

Use Case sınıfı bu somut sınıfları bilmez. Yalnızca arayüzü bilir. Bu sayede dış teknoloji değişse bile iş kuralı değişmez.

Temiz Mimaride Bağımlılık Kuralı
Kaynak kod bağımlılıkları her zaman iç katmanlara doğru olmalıdır. İç katmanlar dış katmanları bilmemelidir. Veritabanı, kullanıcı arayüzü veya framework değiştiğinde iş kuralları etkilenmemelidir.

6.19 Bölüm Sonu Uygulaması: Kütüphane Ödünç Verme Sistemi

Bu bölümde, temiz kod ve mimari standartlar başlığı altında anlatılan kavramları pekiştirmek için küçük bir kütüphane ödünç verme sistemi tasarlanacaktır. Amaç, önce kötü tasarlanmış ve sorumlulukları birbirine karışmış bir yapıyı görmek, ardından bu yapıyı temiz kod, SOLID ilkeleri, bağımlılık yönetimi ve temiz mimari yaklaşımıyla yeniden düzenlemektir.

Bu uygulamada şu kavramlar birlikte kullanılacaktır:

- ◇ Anlamlı isimlendirme
- ◇ Küçük sınıflar ve tek sorumluluk
- ◇ Hata yönetimi ve anlamlı istisnalar
- ◇ Bağımlılık enjeksiyonu
- ◇ Arayüzlere bağımlılık
- ◇ Test edilebilir tasarım
- ◇ Temiz mimari katmanları

Uygulama Senaryosu

Bir kütüphane sisteminde üyeler kitap ödünç alabilmektedir. Sistemin temel iş kuralları şunlardır:

- ◇ Üye numarası boş olamaz.
- ◇ Kitap numarası boş olamaz.
- ◇ Pasif üyeye kitap ödünç verilemez.
- ◇ Mevcut olmayan kitap ödünç verilemez.
- ◇ Üye aynı anda en fazla beş kitap ödünç alabilir.
- ◇ Başarılı ödünç verme işleminden sonra ödünç kaydı oluşturulur.
- ◇ İşlem başarılı olduğunda üyeye bildirim gönderilir.

Bu kurallar ilk bakışta basit görünse de, kötü tasarlanmış bir sistemde kullanıcı arayüzü, iş kuralları, veri kaydetme ve bildirim gönderme kodları kolayca birbirine karışabilir. Bu nedenle uygulama önce kötü tasarımla, daha sonra iyi tasarımla ele alınacaktır.

Kötü Tasarım: Her İşi Yapan Sınıf

Aşağıdaki örnekte bütün sorumluluklar tek bir sınıfta toplanmıştır. Bu sınıf hem doğrulama yapmakta, hem iş kuralı uygulamakta, hem ödünç kaydı oluşturmakta, hem de bildirim göndermektedir.

```
class KutuphaneManager {
  public void kitapOduncVer(String uyeNo,
  String kitapNo,
  boolean uyeAktifMi,
  boolean kitapMevcutMu,
  int mevcutOduncSayisi) {
    if (uyeNo == null || uyeNo.isBlank()) {
      throw new IllegalArgumentException("Üye no boş olamaz.");
    }

    if (kitapNo == null || kitapNo.isBlank()) {
      throw new IllegalArgumentException("Kitap no boş olamaz.");
    }

    if (!uyeAktifMi) {
      throw new IllegalStateException("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw new IllegalStateException("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw new IllegalStateException("Üye en fazla beş kitap ödünç alabilir.");
    }

    System.out.println("Ödünç kaydı veritabanına yazıldı.");
    System.out.println("Üyeye bildirim gönderildi.");
  }
}
```

Kod 6.91: Java Dilinde Kötü Tasarlanmış Kütüphane Sınıfı

C++ dilinde aynı kötü tasarım aşağıdaki gibi gösterilebilir:

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

class KutuphaneManager {
  public:
  void kitapOduncVer(const std::string& uyeNo,
  const std::string& kitapNo,
  bool uyeAktifMi,
  bool kitapMevcutMu,
  int mevcutOduncSayisi) {
    if (uyeNo.empty()) {
      throw std::invalid_argument("Üye no boş olamaz.");
    }

    if (kitapNo.empty()) {
      throw std::invalid_argument("Kitap no boş olamaz.");
    }

    if (!uyeAktifMi) {
      throw std::runtime_error("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw std::runtime_error("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw std::runtime_error("Üye en fazla beş kitap ödünç alabilir.");
    }

    std::cout << "Ödünç kaydı veritabanına yazıldı." << std::endl;
    std::cout << "Üyeye bildirim gönderildi." << std::endl;
  }
};
```

Kod 6.92: C++ Dilinde Kötü Tasarlanmış Kütüphane Sınıfı

Bu tasarımın temel sorunları şunlardır:

- ◇ `KutuphaneManager` sınıfı çok fazla sorumluluk üstlenmiştir.
- ◇ İş kuralları, veri kaydetme ve bildirim gönderme kodları aynı sınıftadır.
- ◇ Sınıfın test edilmesi zordur.
- ◇ Bildirim yöntemi değişirse bu sınıf değişmek zorundadır.
- ◇ Veri kaydetme yöntemi değişirse bu sınıf değişmek zorundadır.
- ◇ Sınıf, açık-kapalı ilkesine uygun değildir.
- ◇ Bağımlılıkları tersine çevirme ilkesi uygulanmamıştır.

Bu nedenle bu tasarım temiz kod ve mimari standartlara uygun değildir.

İyi Tasarım İçin Sorumlulukların Ayrılması

İyi tasarımda her sınıf yalnızca kendi sorumluluğunu üstlenmelidir. Bu uygulama için sorumluluklar aşağıdaki gibi ayrılabilir:

- ◇ `Odunc`: Ödünç verme kaydını temsil eder.
- ◇ `OduncKurali`: Ödünç verme iş kurallarını uygular.
- ◇ `OduncDeposu`: Ödünç kaydını saklama işlemini soyutlar.
- ◇ `BildirimGonderici`: Bildirim gönderme davranışını soyutlar.
- ◇ `OduncServisi`: Ödünç verme kullanım senaryosunu yönetir.

Bu ayrım sayesinde iş kuralları, veri erişimi ve bildirim gönderme işlemleri birbirinden ayrılmış olur.

Java ile Temiz Tasarım

Aşağıda Java dilinde temiz mimariye daha uygun bir çözüm verilmiştir.

```
class Odunc {
  private final String uyeNo;
  private final String kitapNo;

  public Odunc(String uyeNo, String kitapNo) {
    if (uyeNo == null || uyeNo.isBlank()) {
      throw new IllegalArgumentException("Üye no boş olamaz.");
    }

    if (kitapNo == null || kitapNo.isBlank()) {
      throw new IllegalArgumentException("Kitap no boş olamaz.");
    }

    this.uyeNo = uyeNo;
    this.kitapNo = kitapNo;
  }

  public String getUyeNo() {
    return uyeNo;
  }

  public String getKitapNo() {
    return kitapNo;
  }
}
```

Kod 6.93: Java Dilinde Ödünç Kaydı Sınıfı

`Odunc` sınıfı, ödünç verme kaydını temsil eden basit ve değişmez bir sınıftır. Üye numarası ve kitap numarası yapıcıda doğrulanır.

```
class OduncKurali {
  public void kontrolEt(boolean uyeAktifMi,
  boolean kitapMevcutMu,
  int mevcutOduncSayisi) {
    if (!uyeAktifMi) {
      throw new IllegalStateException("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw new IllegalStateException("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw new IllegalStateException("Üye en fazla beş kitap ödünç alabilir.");
    }
  }
}
```

Kod 6.94: Java Dilinde Ödünç Verme İş Kuralı

`OduncKurali` sınıfı yalnızca iş kurallarından sorumludur. Veritabanı veya bildirim sistemi hakkında bilgi sahibi değildir.

```
interface OduncDeposu {
  void kaydet(Odunc odunc);
}

interface BildirimGonderici {
  void gonder(String uyeNo, String mesaj);
}
```

Kod 6.95: Java Dilinde Depo ve Bildirim Arayüzleri

Bu arayüzler sayesinde servis sınıfı somut sınıflara değil, soyutlamalara bağımlı hale getirilir.

```
class BellekOduncDeposu implements OduncDeposu {
  @Override
  public void kaydet(Odunc odunc) {
    System.out.println("Ödünç kaydı oluşturuldu: "
    + odunc.getUyeNo() + " - " + odunc.getKitapNo());
  }
}

class EpostaBildirimGonderici implements BildirimGonderici {
  @Override
  public void gonder(String uyeNo, String mesaj) {
    System.out.println(uyeNo + " üyesine e-posta gönderildi: " + mesaj);
  }
}
```

Kod 6.96: Java Dilinde Somut Depo ve Bildirim Sınıfları

```
class OduncServisi {
  private final OduncDeposu oduncDeposu;
  private final BildirimGonderici bildirimGonderici;
  private final OduncKurali oduncKurali;

  public OduncServisi(OduncDeposu oduncDeposu,
  BildirimGonderici bildirimGonderici,
  OduncKurali oduncKurali) {
    this.oduncDeposu = oduncDeposu;
    this.bildirimGonderici = bildirimGonderici;
    this.oduncKurali = oduncKurali;
  }

  public void kitapOduncVer(String uyeNo,
  String kitapNo,
  boolean uyeAktifMi,
  boolean kitapMevcutMu,
  int mevcutOduncSayisi) {
    oduncKurali.kontrolEt(uyeAktifMi, kitapMevcutMu, mevcutOduncSayisi);

    Odunc odunc = new Odunc(uyeNo, kitapNo);

    oduncDeposu.kaydet(odunc);

    bildirimGonderici.gonder(uyeNo, "Kitap ödünç verme işlemi başarılı.");
  }
}
```

Kod 6.97: Java Dilinde Ödünç Verme Servisi

`OduncServisi`, ödünç verme kullanım senaryosunu yönetir. Ancak verinin nereye kaydedildiğini ve bildirimin nasıl gönderildiğini bilmez. Bu ayrıntılar arayüzlerin arkasına gizlenmiştir.

```
public class KutuphaneUygulamasi {
  public static void main(String[] args) {
    OduncDeposu depo = new BellekOduncDeposu();
    BildirimGonderici bildirim = new EpostaBildirimGonderici();
    OduncKurali kural = new OduncKurali();

    OduncServisi servis = new OduncServisi(depo, bildirim, kural);

    servis.kitapOduncVer("UYE-1", "KITAP-1", true, true, 2);
  }
}
```

Kod 6.98: Java Dilinde Uygulamanın Çalıştırılması

Bu tasarımda yeni bir bildirim türü eklemek için `OduncServisi` sınıfını değiştirmeye gerek yok-

tur. Örneğin SMS bildirimi eklemek için yalnızca yeni bir sınıf yazılır:

```
class SmsBildirimGonderici implements BildirimGonderici {
  @Override
  public void gonder(String uyeNo, String mesaj) {
    System.out.println(uyeNo + " üyesine SMS gönderildi: " + mesaj);
  }
}
```

Kod 6.99: Java Dilinde Yeni Bildirim Türü Eklemek

Bu durum açık-kapalı ilkesine uygundur.

C++ ile Temiz Tasarım

Aynı tasarım C++ dilinde de arayüzler ve referanslar kullanılarak gerçekleştirilebilir.

```
#include <iostream>
#include <stdexcept>
#include <string>

class Odunc {
  private:
  std::string uyeNo;
  std::string kitapNo;

  public:
  Odunc(std::string pUyeNo, std::string pKitapNo)
  : uyeNo(pUyeNo), kitapNo(pKitapNo) {
    if (pUyeNo.empty()) {
      throw std::invalid_argument("Üye no boş olamaz.");
    }

    if (pKitapNo.empty()) {
      throw std::invalid_argument("Kitap no boş olamaz.");
    }
  }

  std::string getUyeNo() const {
    return uyeNo;
  }

  std::string getKitapNo() const {
    return kitapNo;
  }
};
```

Kod 6.100: C++ Dilinde Ödünç Kaydı Sınıfı

```
class OduncKurali {
  public:
  void kontrolEt(bool uyeAktifMi,
  bool kitapMevcutMu,
  int mevcutOduncSayisi) const {
    if (!uyeAktifMi) {
      throw std::runtime_error("Pasif üyeye kitap ödünç verilemez.");
    }

    if (!kitapMevcutMu) {
      throw std::runtime_error("Mevcut olmayan kitap ödünç verilemez.");
    }

    if (mevcutOduncSayisi >= 5) {
      throw std::runtime_error("Üye en fazla beş kitap ödünç alabilir.");
    }
  }
};
```

Kod 6.101: C++ Dilinde Ödünç Verme İş Kuralı

```
class IOduncDeposu {
  public:
  virtual void kaydet(const Odunc& odunc) = 0;
  virtual ~IOduncDeposu() = default;
};

class IBildirimGonderici {
  public:
  virtual void gonder(const std::string& uyeNo,
  const std::string& mesaj) = 0;
  virtual ~IBildirimGonderici() = default;
};
```

Kod 6.102: C++ Dilinde Depo ve Bildirim Arayüzleri

```
class BellekOduncDeposu : public IOduncDeposu {
  public:
  void kaydet(const Odunc& odunc) override {
    std::cout << "Ödünç kaydı oluşturuldu: "
    << odunc.getUyeNo() << " - "
    << odunc.getKitapNo() << std::endl;
  }
};

class EpostaBildirimGonderici : public IBildirimGonderici {
  public:
  void gonder(const std::string& uyeNo,
  const std::string& mesaj) override {
    std::cout << uyeNo << " üyesine e-posta gönderildi: "
    << mesaj << std::endl;
  }
};
```

Kod 6.103: C++ Dilinde Somut Depo ve Bildirim Sınıfları

```
class OduncServisi {
  private:
  IOduncDeposu& oduncDeposu;
  IBildirimGonderici& bildirimGonderici;
  OduncKurali& oduncKurali;

  public:
  OduncServisi(IOduncDeposu& pOduncDeposu,
  IBildirimGonderici& pBildirimGonderici,
  OduncKurali& pOduncKurali)
  : oduncDeposu(pOduncDeposu),
  bildirimGonderici(pBildirimGonderici),
  oduncKurali(pOduncKurali) {}

  void kitapOduncVer(const std::string& uyeNo,
  const std::string& kitapNo,
  bool uyeAktifMi,
  bool kitapMevcutMu,
  int mevcutOduncSayisi) {
    oduncKurali.kontrolEt(uyeAktifMi, kitapMevcutMu, mevcutOduncSayisi);

    Odunc odunc(uyeNo, kitapNo);

    oduncDeposu.kaydet(odunc);

    bildirimGonderici.gonder(uyeNo,
    "Kitap ödünç verme işlemi başarılı.");
  }
};
```

Kod 6.104: C++ Dilinde Ödünç Verme Servisi

```
int main() {
  BellekOduncDeposu depo;
  EpostaBildirimGonderici bildirim;
  OduncKurali kural;

  OduncServisi servis(depo, bildirim, kural);

  servis.kitapOduncVer("UYE-1", "KITAP-1", true, true, 2);

  return 0;
}
```

Kod 6.105: C++ Dilinde Uygulamanın Çalıştırılması

Bu tasarımda `OduncServisi` sınıfı, somut sınıflara değil arayüzlere bağımlıdır. Böylece bildirim yöntemi veya veri saklama yöntemi değiştiğinde servis sınıfının değiştirilmesine gerek kalmaz.

Test Edilebilirlik

Temiz mimarinin önemli faydalarından biri test edilebilirliktir. Örneğin gerçek e-posta göndermek yerine test sırasında sahte bir bildirim gönderici kullanılabilir.

Java dilinde sahte bildirim gönderici:

```
class FakeBildirimGonderici implements BildirimGonderici {
  private String sonMesaj;

  @Override
  public void gonder(String uyeNo, String mesaj) {
    this.sonMesaj = mesaj;
  }

  public String getSonMesaj() {
    return sonMesaj;
  }
}
```

Kod 6.106: Java Dilinde Test İçin Fake Bildirim Gönderici

C++ dilinde sahte bildirim gönderici:

```
class FakeBildirimGonderici : public IBildirimGonderici {
  public:
  std::string sonMesaj;

  void gonder(const std::string& uyeNo,
  const std::string& mesaj) override {
    sonMesaj = mesaj;
  }
};
```

Kod 6.107: C++ Dilinde Test İçin Fake Bildirim Gönderici

Bu yapı sayesinde test sırasında gerçek dış sistemlere ihtiyaç duyulmaz.

UML Sınıf Diyagramı

Bu uygulamadaki temel sınıflar ve ilişkiler UML sınıf diyagramı ile aşağıdaki gibi düşünülebilir:

- `OduncServisi`, `OduncKurali` sınıfını kullanır.
- `OduncServisi`, `OduncDeposu` arayüzüne bağımlıdır.
- `OduncServisi`, `BildirimGonderici` arayüzüne bağımlıdır.
- `BellekOduncDeposu`, `OduncDeposu` arayüzünü gerçekleştirir.

◇ `EpostaBildirimGonderici`, `BildirimGonderici` arayüzünü gerçekleştirir.
◇ `OduncServisi`, işlem sırasında `Odunc` nesnesi oluşturur.

Bu ilişkiler, 3. bölümde anlatılan gerçekleştirme, bağımlılık ve kullanım ilişkileriyle uyumludur.

Uygulamanın SOLID İlkeleri Açısından Değerlendirilmesi

Bu tasarım SOLID ilkeleri açısından değerlendirildiğinde:

◇ **SRP**: Her sınıfın tek sorumluluğu vardır.
◇ **OCP**: Yeni depo veya bildirim türü eklemek için servis sınıfı değiştirilmez.
◇ **LSP**: Arayüzü gerçekleştiren sınıflar, arayüzün beklenen davranışını bozmaz.
◇ **ISP**: Depo ve bildirim davranışları ayrı arayüzlere bölünmüştür.
◇ **DIP**: Servis sınıfı somut sınıflara değil, soyutlamalara bağımlıdır.

Alıştırmalar

Aşağıdaki alıştırmalarla uygulamayı genişletebilirsiniz:

1. `SmsBildirimGonderici` sınıfını C++ ve Java dillerinde yazınız.
2. `VeritabaniOduncDeposu` adında yeni bir depo sınıfı ekleyiniz.
3. Üyenin gecikmiş kitabı varsa yeni kitap alamayacağı kuralını ekleyiniz.
4. Ödünç verme işlemi için özel bir `OduncException` sınıfı oluşturunuz.
5. `OduncServisi` için fake depo ve fake bildirim sınıflarıyla test yazınız.
6. Bu uygulamanın UML sınıf diyagramını çiziniz.

Bölüm Özeti

Bu uygulamada, her şeyi yapan büyük bir sınıf yerine, sorumlulukları ayrılmış, arayüzlere bağımlı, test edilebilir ve genişletilebilir bir tasarım oluşturulmuştur. Böylece temiz kodun yalnızca satırların düzenli yazılması değil, aynı zamanda doğru sorumluluk ayrımı ve doğru bağımlılık yönetimi olduğu görülmüştür.

Temiz kod, küçük ayrıntılardan başlar; ancak iyi mimariyle tamamlanır.